HEAR Configurable Accelerator Reference

for Ezairo® 7100

M-20798-009
November 2016

© SCILLC, 2016
Previous Edition © 2015
"All Rights Reserved"

ON Semiconductor®

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is a registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction 13
    1.1 Purpose 13
    1.2 Intended Audience 13
    1.3 Conventions 13
    1.4 Manual Organization 13
    1.5 Further Reading 14
2. Usage Model 15
    2.1 Introduction to Using the HEAR 15
    2.2 Model Overview 15
    2.3 Microcode Module Configuration and Usage 16
        2.3.1 Module Attributes 17
        2.3.2 Constants 17
        2.3.3 Configuration Parameters 17
        2.3.4 Address Parameters 18
        2.3.5 Parameter Restrictions 20
        2.3.6 State Variables 20
        2.3.7 Numeric Types and Data Formats 21
    2.4 HEAR Interfacing 22
        2.4.1 Signals to the HEAR 22
        2.4.2 Signals from the HEAR 23
        2.4.3 Shared Memory 23
            2.4.3.1 Microcode Configuration Reserved Memory 23
            2.4.3.2 FIFO Buffer Reserved Memory 24
            2.4.3.3 Sine/Cosine Tables 25
    2.5 System Configuration and Usage 25
        2.5.1 FIFO Controller 25
            2.5.1.1 FIFO Configuration. 26
            2.5.1.2 Software Controlled FIFOs 27
            2.5.1.3 Dual FIFOs 28
        2.5.2 Function Chain Controller 29
        2.5.3 Block Floating Point 29
            2.5.3.1 BFP Data Types 29
            2.5.3.2 BFP Usage 30
            2.5.3.3 BFP Unit Sharing 30
            2.5.3.4 BFP Implementation 30
        2.5.4 Arithmetic Flags and Control 31
            2.5.4.1 Saturation Detection 31
            2.5.4.2 Accumulator Overflow Detection 32
            2.5.4.3 Overflow Protection Control 32
        2.5.5 Run-Time Usage. 33
            2.5.5.1 Control from the CFX 33
            2.5.5.2 Reset Behavior 34
            2.5.5.3 Modification of Run-Time Parameters 34

3. Modules . . . 35
3.1 Overview . . . 35
3.1.1 Conventions . . . 35
3.2 WOLA Filterbank . . . 36
3.2.1 Analysis and Synthesis . . . 37
3.2.1.1 Analysis Implementation . . . 38
3.2.1.2 Synthesis Implementation . . . 41
3.2.1.3 Input and Output Data Types . . . 42
3.2.1.4 Analysis FIFO Configuration . . . 43
3.2.1.5 Synthesis FIFO Configuration . . . 44
3.2.1.6 Analysis and Synthesis Windows . . . 45
3.2.1.7 Initialization . . . 47
3.2.1.8 State Variables . . . 47
3.2.2 Gain Application . . . 48
3.2.2.1 Real Gains . . . 49
3.2.2.2 Complex Gains . . . 49
3.2.3 Subband Channel Mixing . . . 50
3.2.4 Subband Energy Calculation . . . 50
3.2.5 Subband Normalization . . . 51
3.2.6 WOLA Filterbank Performance . . . 52
3.2.6.1 Block Rate . . . 52
3.2.6.2 Group Delay . . . 52
3.2.6.3 Frequency Response . . . 53
3.2.6.4 Power Consumption . . . 53
3.3 Transforms . . . 54
3.3.1 Discrete Fourier Transforms . . . 54
3.3.1.1 Complex Input DFT . . . 55
3.3.1.2 Real Input DFT . . . 56
3.3.1.3 Windowed DFT . . . 57
3.4 Block Floating Point Support . . . 57
3.5 Filters . . . 57
3.5.1 Time-Domain FIR Filter . . . 57
3.5.1.1 FIR Filter FIFO Configuration . . . 58
3.5.2 Time-Domain IIR Filter . . . 59
3.5.2.1 IIR Filter FIFO Configuration . . . 60
3.5.3 Biquad IIR Filter . . . 62
3.5.3.1 Biquad_IIR Filter FIFO Configuration . . . 64
3.5.4 Subband FIR Filter . . . 65
3.5.4.1 Subband FIR Filter FIFO Configuration . . . 67
3.5.5 Subband IIR Filter . . . 68
3.5.5.1 Subband IIR Filter FIFO Configuration . . . 70
3.5.6 Attack-Release Averaging . . . 71
3.5.7 Exponential Averaging . . . 73
3.6 Vector Operations . . . 73
3.6.1 Vector Addition . . . 75

3.6.2 Complex Vector Addition . . . 77
3.6.3 Vector-Scalar Addition . . . 77
3.6.4 Complex Vector-Scalar Addition . . . 77
3.6.5 Vector Subtraction . . . 78
3.6.6 Complex Vector Subtraction . . . 78
3.6.7 Vector-Scalar Subtraction . . . 78
3.6.8 Complex Vector-Scalar Subtraction . . . 78
3.6.9 Vector Multiplication . . . 79
3.6.10 Complex Vector Multiplication . . . 79
3.6.11 Complex Conjugate Vector Multiplication . . . 79
3.6.12 Vector-Scalar Multiplication . . . 80
3.6.13 Complex Vector-Scalar Multiplication . . . 80
3.6.14 Complex Conjugate Vector-Scalar Multiplication . . . 80
3.6.15 Complex Conjugate Vector Copy . . . 80
3.6.16 Vector Magnitude . . . 80
3.6.17 Vector Squared Magnitude . . . 81
3.6.18 Inner Product . . . 82
3.6.19 Complex Inner Product . . . 82
3.6.20 Linear to Log2 Conversion . . . 82
3.6.21 Log2 to Linear Conversion . . . 83
3.6.22 Complex Number Coordinate System Conversions . . . 83
3.6.22.1 Polar-to-Rectangular Conversion . . . 83
3.6.22.2 Rectangular-to-Polar Conversion . . . 84
3.6.22.3 CORDIC Implementation . . . 84
3.6.23 Single Word Vector Copy . . . 87
3.6.24 Two Word Vector Copy . . . 87
3.6.25 Four Word Vector Copy . . . 87
3.6.26 Vector Maximum . . . 87
3.6.27 Vector-Scalar Maximum . . . 88
3.6.28 Vector Minimum . . . 88
3.6.29 Vector-Scalar Minimum . . . 88
3.6.30 Vector Bound . . . 88
3.6.31 Vector Shift . . . 88
3.6.32 Vector-Scalar Shift . . . 88
3.6.33 Vector Absolute Value . . . 88
3.6.34 L1 Norm of Vector Difference . . . 88
3.7 Signal Statistics . . . 89
3.7.1 Complex Autocorrelation . . . 90
3.7.2 Autocorrelation . . . 90
3.7.3 Average Magnitude . . . 90
3.7.4 Average Squared Magnitude . . . 91
3.7.5 Complex Correlation . . . 91
3.7.6 Correlation . . . 92

3.7.7 Mean . . . 92
3.7.8 Sum . . . 92
3.7.9 Variance . . . 93
3.7.10 Complex Variance . . . 93
3.7.11 Maximum Element . . . 93
3.7.12 Minimum Element . . . 94
4. Module Reference . . . 95
4.1 Overview . . . 95
4.2 WOLA_Analysis . . . 95
4.2.1 Constants . . . 96
4.2.2 Parameters . . . 97
4.2.3 State Variables . . . 101
4.2.4 Functions . . . 101
4.3 WOLA_Synthesis . . . 101
4.3.1 Constants . . . 103
4.3.2 Parameters . . . 104
4.3.3 State Variables . . . 107
4.3.4 Functions . . . 108
4.4 Gain_R . . . 108
4.4.1 Parameters . . . 109
4.4.2 State Variables . . . 111
4.4.3 Functions . . . 111
4.5 Gain_C . . . 111
4.5.1 Parameters . . . 112
4.5.2 State Variables . . . 114
4.5.3 Functions . . . 114
4.6 Mix_Channels . . . 114
4.6.1 Parameters . . . 115
4.6.2 Functions . . . 115
4.7 Energy . . . 116
4.7.1 Parameters . . . 116
4.7.2 Functions . . . 118
4.8 Normalize . . . 118
4.8.1 Constants . . . 119
4.8.2 Parameters . . . 119
4.8.3 Functions . . . 121
4.9 BFP_State . . . 121
4.9.1 Parameters . . . 122
4.9.2 State Variables . . . 122
4.9.3 Functions . . . 122
4.10 DFT_C_Forward . . . 122
4.10.1 Parameters . . . 123
4.10.2 Functions . . . 125
4.11 DFT_C_Inverse . . . 125

4.11.1 Parameters . . . 126
4.11.2 Functions . . . 127
4.12 DFT_R_Forward . . . 128
4.12.1 Parameters . . . 128
4.12.2 Functions . . . 130
4.13 DFT_R_Inverse . . . 130
4.13.1 Parameters . . . 131
4.13.2 Functions . . . 132
4.14 Win_DFT_R_Forward . . . 133
4.14.1 Parameters . . . 133
4.14.2 Functions . . . 136
4.15 Win_DFT_R_Inverse . . . 136
4.15.1 Parameters . . . 137
4.15.2 Functions . . . 139
4.16 Attack_Release . . . 139
4.16.1 Constants . . . 140
4.16.2 Parameters . . . 140
4.16.3 Functions . . . 143
4.17 FIR_R . . . 144
4.17.1 Constants . . . 145
4.17.2 Parameters . . . 145
4.17.3 Functions . . . 148
4.18 IIR_R . . . 148
4.18.1 Constants . . . 149
4.18.2 Parameters . . . 150
4.18.3 Functions . . . 153
4.19 Biquad_IIR . . . 153
4.19.1 Parameters . . . 154
4.19.2 Functions . . . 156
4.20 sFIR_C . . . 157
4.20.1 Parameters . . . 157
4.20.2 Functions . . . 159
4.21 sIIR_C . . . 159
4.21.1 Parameters . . . 160
4.21.2 Functions . . . 163
4.22 Exp_Average . . . 163
4.22.1 Constants . . . 164
4.22.2 Parameters . . . 164
4.22.3 Functions . . . 166
4.23 vAdd_C . . . 167
4.23.1 Constants . . . 168
4.23.2 Parameters . . . 168
4.23.3 Functions . . . 170
4.24 vAdd_R . . . 170

4.24.1 Constants . . . 171
4.24.2 Parameters . . . 171
4.24.3 Functions . . . 174
4.25 vAddScalar_C . . . 174
4.25.1 Constants . . . 175
4.25.2 Parameters . . . 175
4.25.3 Functions . . . 177
4.26 vAddScalar_R . . . 177
4.26.1 Constants . . . 178
4.26.2 Parameters . . . 179
4.26.3 Functions . . . 181
4.27 vSub_C . . . 181
4.27.1 Constants . . . 182
4.27.2 Parameters . . . 182
4.27.3 Functions . . . 184
4.28 vSub_R . . . 185
4.28.1 Constants . . . 186
4.28.2 Parameters . . . 186
4.28.3 Functions . . . 188
4.29 vSubScalar_C . . . 188
4.29.1 Constants . . . 189
4.29.2 Parameters . . . 190
4.29.3 Functions . . . 192
4.30 vSubScalar_R . . . 192
4.30.1 Constants . . . 193
4.30.2 Parameters . . . 194
4.30.3 Functions . . . 196
4.31 vMult_C . . . 196
4.31.1 Constants . . . 197
4.31.2 Parameters . . . 197
4.31.3 Functions . . . 200
4.32 vMult_R . . . 200
4.32.1 Constants . . . 201
4.32.2 Parameters . . . 201
4.32.3 Functions . . . 204
4.33 vMultConj . . . 204
4.33.1 Constants . . . 205
4.33.2 Parameters . . . 206
4.33.3 Functions . . . 208
4.34 vMultScalar_C . . . 208
4.34.1 Constants . . . 209
4.34.2 Parameters . . . 209
4.34.3 Functions . . . 212
4.35 vMultScalar_R . . . 212

4.35.1 Constants . . . 213
4.35.2 Parameters . . . 213
4.35.3 Functions . . . 215
4.36 vMultScalarConj . . . 215
4.36.1 Constants . . . 216
4.36.2 Parameters . . . 217
4.36.3 Functions . . . 219
4.37 vConj . . . 219
4.37.1 Parameters . . . 220
4.37.2 Functions . . . 222
4.38 vMag . . . 222
4.38.1 Constants . . . 223
4.38.2 Parameters . . . 223
4.38.3 Functions . . . 225
4.39 vSqMag . . . 225
4.39.1 Constants . . . 226
4.39.2 Parameters . . . 226
4.39.3 Functions . . . 228
4.40 InnerProduct_C . . . 228
4.40.1 Constants . . . 229
4.40.2 Parameters . . . 229
4.40.3 Functions . . . 231
4.41 InnerProduct_R . . . 232
4.41.1 Constants . . . 233
4.41.2 Parameters . . . 233
4.41.3 Functions . . . 235
4.42 vLinLog2 . . . 235
4.42.1 Constants . . . 236
4.42.2 Parameters . . . 237
4.42.3 Functions . . . 238
4.43 vLog2Lin . . . 238
4.43.1 Constants . . . 239
4.43.2 Parameters . . . 240
4.43.3 Functions . . . 241
4.44 vPolarRect . . . 241
4.44.1 Parameters . . . 242
4.44.2 Functions . . . 244
4.45 vRectPolar . . . 244
4.45.1 Parameters . . . 245
4.45.2 Functions . . . 246
4.46 vCopy . . . 247
4.46.1 Parameters . . . 247
4.46.2 Functions . . . 249
4.47 vCopy2 . . . 249

4.47.1 Parameters . . . 250
4.47.2 Functions . . . 251
4.48 vCopy4 . . . 251
4.48.1 Parameters . . . 252
4.48.2 Functions . . . 253
4.49 L1NormDiff . . . 254
4.49.1 Constants . . . 254
4.49.2 Parameters . . . 255
4.49.3 Functions . . . 257
4.50 vAbsolute . . . 257
4.50.1 Parameters . . . 258
4.50.2 Functions . . . 259
4.51 vBound . . . 260
4.51.1 Parameters . . . 260
4.51.2 Functions . . . 263
4.52 vShift . . . 263
4.52.1 Constants . . . 264
4.52.2 Parameters . . . 265
4.52.3 Functions . . . 267
4.53 vShiftScalar . . . 267
4.53.1 Constants . . . 268
4.53.2 Parameters . . . 269
4.53.3 Functions . . . 271
4.54 vMax . . . 271
4.54.1 Constants . . . 272
4.54.2 Parameters . . . 272
4.54.3 Functions . . . 274
4.55 vMaxScalar . . . 275
4.55.1 Parameters . . . 275
4.55.2 Functions . . . 277
4.56 vMin . . . 278
4.56.1 Constants . . . 278
4.56.2 Parameters . . . 279
4.56.3 Functions . . . 281
4.57 vMinScalar . . . 281
4.57.1 Parameters . . . 282
4.57.2 Functions . . . 284
4.58 vNLog2Abs . . . 284
4.58.1 Constants . . . 285
4.58.2 Parameters . . . 285
4.58.3 Functions . . . 287
4.59 AutoCorrelation_C . . . 287
4.59.1 Parameters . . . 288
4.59.2 Functions . . . 290

4.60 AutoCorrelation_R 290
4.60.1 Parameters 291
4.60.2 Functions 293
4.61 AvgMag 293
4.61.1 Constants 294
4.61.2 Parameters 294
4.61.3 Functions 296
4.62 AvgSqMag 296
4.62.1 Constants 297
4.62.2 Parameters 297
4.62.3 Functions 299
4.63 Correlation_C 299
4.63.1 Parameters 300
4.63.2 Functions 303
4.64 Correlation_R 303
4.64.1 Parameters 304
4.64.2 Functions 307
4.65 MaxElement 307
4.65.1 Parameters 308
4.65.2 Functions 309
4.66 MinElement 309
4.66.1 Parameters 310
4.66.2 Functions 311
4.67 Mean 311
4.67.1 Constants 312
4.67.2 Parameters 313
4.67.3 Functions 314
4.68 Sum 315
4.68.1 Constants 315
4.68.2 Parameters 316
4.68.3 Functions 317
4.69 Variance_C 317
4.69.1 Constants 318
4.69.2 Parameters 319
4.69.3 Functions 321
4.70 Variance_R 321
4.70.1 Constants 322
4.70.2 Parameters 323
4.70.3 Functions 324
5. HEAR Configuration Tool 325
5.1 Usage 325
5.2 HCT Language 327
5.2.1 Lexical Conventions 327
5.2.2 Language Syntax 328

5.2.2.1 Argument Lists 328
5.2.2.2 Expressions 329
5.2.2.3 Identifiers 329
5.2.2.4 Input File Structure 329
5.2.3 The Preprocessor 329
5.3 Built-in Macros 330
5.4 Built-in Constants 333
5.5 Microcode Functions and Constants 334
5.5.1 Parameter Checking 334
5.6 Library Files 334
5.7 CFX Header File Output 335
5.8 CFX Object File Output 336
5.9 Summary File Output 337
A. Optimized WOLA Windows 339
B. Glossary 345

CHAPTER 1
Introduction

1.1 Purpose

This manual describes the configuration and run-time usage of the HEAR Configurable Accelerator. It also describes how to configure the microcode which executes on the HEAR as part of your application.

The HEAR Configurable Accelerator is a microcode-configurable signal processing core which is part of the system for Ezairo® 7100. The microcode is composed of multiple configurable modules which you can configure and arrange in a manner suitable for your applications.

1.2 Intended Audience

This manual is intended for anyone responsible for developing microcode configurations for the HEAR Configurable Accelerator. It is also useful for those who are writing CFX applications that interface to the HEAR.

It is assumed that you have a general familiarity with the system architecture of this family of ON Semiconductor chips. It is also assumed that you are familiar with digital signal processing (DSP) in general, including the underlying theory and real-time DSP applications.

1.3 Conventions

This book uses the following conventions:

- Code snippets and text that would appear in code, such as register, module, function and parameter names, are shown in a `monospace font`.
- Angle brackets “<” and “>” indicate a placeholder for specific information. To replace a placeholder, specify the information within the brackets; do not include the brackets themselves.
- When new terms are introduced, or equation variables appear in sentences, they are shown in an *italic typeface*.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes “0x” and “0b” respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 Manual Organization

The *HEAR Configurable Accelerator Reference Manual* contains the following chapters and appendices:

- **Chapter 1: Introduction** provides a basic introduction to the manual including the purpose, intended audience, conventions and a list of further reading material.
- **Chapter 2: Usage Model** describes the general model which is used to configure and interface with the HEAR. First, the concept of a microcode module is introduced, including the attributes, parameters and conventions which define a module. This chapter also covers how you can interface to and control the HEAR from their CFX applications.
- **Chapter 3: Modules** provides a high-level description of all the microcode modules available in the new library (Version 2.0) for the HEAR. They are described in terms of their signal processing functionality and their inputs and outputs. Any general information needed to understand and configure each module is described in this chapter.
- **Chapter 4: Module Reference** is a reference chapter with a comprehensive list of all the parameters and functions for each microcode module in Version 2.0 of the library.
- **Chapter 5: HEAR Configuration Tool** describes how to use the HEAR Configuration Tool (HCT) to configure an application’s microcode modules. It also describes the command line interface to the HCT.

- **Appendix A: Optimized WOLA Windows** is a listing of all the WOLA filterbank configurations which have optimized windows available.
- **Appendix B: Glossary** is a list of acronyms and terms used in this manual.

1.5 Further Reading

For more information, refer to the following documents:

- *Hardware Reference Manual for Ezairo 7100*
- *CFX DSP Architecture Manual*
- *Firmware Reference Manual for Ezairo 7100*
- Any university-level introductory DSP textbook
- Crochiere, Ronald E. and Rabiner, Lawrence R., *Multirate Digital Signal Processing*. (Prentice-Hall Signal Processing Series, 1983)

CHAPTER 2
Usage Model

2.1 Introduction to Using the HEAR

This section describes how to use the HEAR Configurable Accelerator as part of an application. There are two aspects to making use of the HEAR:

- Creating an application-specific HEAR microcode configuration at compile-time.
- Configuring and using the system at run-time so that the microcode configuration operates correctly.

The HEAR microcode is designed around a particular model for using the HEAR Configurable Accelerator and the rest of the system. This model is meant to enable the development of efficient applications while still taking advantage of the system's flexibility. The remainder of this chapter describes the various aspects of this usage model.

An application's microcode configuration is defined using a macro-based language and the HEAR Configuration Tool (HCT), which is described in Chapter 5, "HEAR Configuration Tool" on page 325. It may be helpful to refer to this chapter about the HCT while reading about the HEAR usage model.

2.2 Model Overview

The HEAR Configurable Accelerator is configured as part of algorithm development on the DSP architecture. This configuration is done through a set of microcode modules using the HEAR Configuration Tool (HCT).You may select which modules to use, how the modules are configured and how they are executed at run-time. Each module exposes one or more functions that you can organize into function chains.

Important terms involved in configuring and using the HEAR microcode include:

Microcode Module A collection of microcode and data used to execute one or more functions on the HEAR.

Microcode Function The portion of a microcode module which performs processing on the HEAR.

Microcode Library The collection of all microcode modules available for use on the HEAR.

Function Chain A sequence of microcode functions that are executed in response to a command from the CFX or an event from the FIFO Controller.

At run-time, function chains are executed by the HEAR in response to commands from the CFX or in response to events from the FIFO controller. The HEAR can send interrupts to the CFX at any time during a function chain's execution in order to notify it of when functions have completed their processing. Run-time inputs and outputs to the microcode functions are all located in shared memory.

Figure 1 on page 16 illustrates the relationship between microcode modules, microcode functions and function chains. The microcode library that is available through the HCT is partitioned into different modules as depicted in the figure. Each module has one or more functions that can be selected and arranged into function chains in an application-specific way by the developer. These application-specific function chains represent what can be executed at run-time.

IMPORTANT: Understanding the relationship between microcode modules, their functions and how they are configured and mapped to function chains is key to making efficient use of the HEAR microcode in an application.

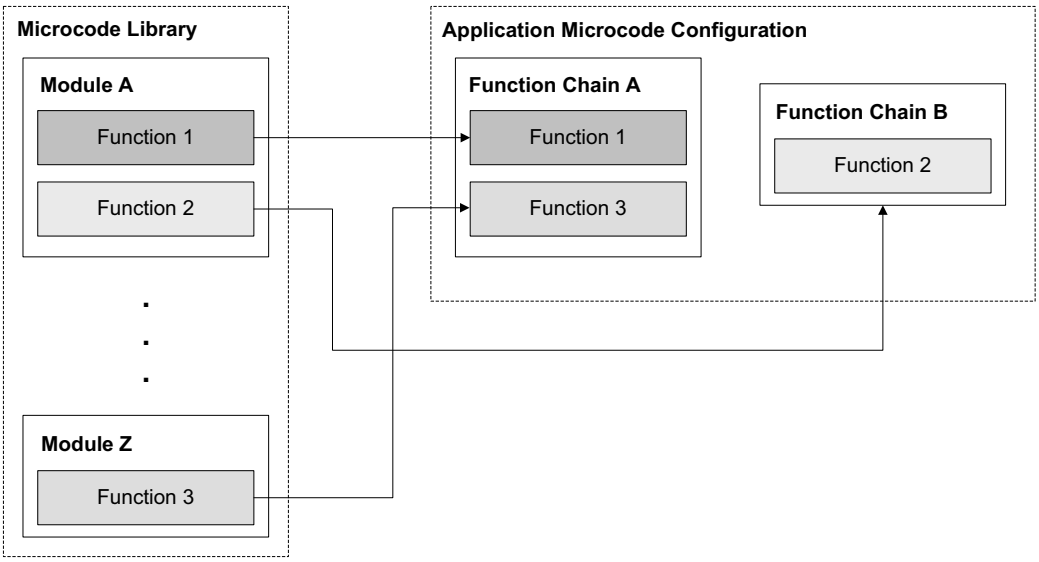


Figure 1. Relationship Between Microcode Modules, Functions and Function Chains

More information about the individual microcode modules that are available and how to use them is specified in Chapter 3, “Modules” on page 35. The rest of this chapter describes in greater detail the mechanics of using the HEAR, both during the microcode configuration and at run-time.

2.3 Microcode Module Configuration and Usage

A microcode module is described by the following:

Attributes — Each module has a number of standard attributes that document some high-level information about how the module can be used.

Constants — Each module may define constants that can be used to specify configuration parameter values. While constants are specified by individual modules, they are globally available to all modules for the purposes of specifying parameter restrictions and configuring the microcode using the HCT.

Configuration Parameters — Configuration parameters define the behavior of each instance of a microcode module.

Address Parameters — Address parameters specify the memory addresses for input, output and temporary variables used by the microcode module.

State Variables — State variables are named locations in memory which are exported by a microcode module instance. They are used to maintain any internal state of the microcode module and to control the processing of the module at run-time.

Multiple instances of the same microcode module can be used in a particular microcode configuration subject to the overall memory limitations of the DSP architecture (i.e., available microcode memory). For configuration purposes, you will need to supply a unique name for each instance.

The following sub-sections elaborate on how module attributes, constants, parameters and state variables are described. It is recommended that the reader consult Chapter 4, “Module Reference” on page 95 for examples of modules and parameters when reading these sections.

2.3.1 Module Attributes

The module attributes summarized in Table 1 are used to describe some top-level behaviors and features of a microcode module.

Table 1. Module Attributes

Module Attribute	Description
Single-function	Specifies whether the module has one function or several related functions. This has implications for how a module is configured via the HCT. Single-function modules are configured at the point where the function is called in an HCT input file. Modules with multiple functions provide a separate configuration function that must be used in the HCT input file before any of the actual processing functions can be used inside a function chain.
Reset	Specifies whether the module can be reset. Modules with this functionality provide a reset function which initializes any internal state. More details of the reset functionality are module-specific.
BFP Support	Specifies whether the module supports the use of block floating point (BFP) data. This parameter can be *None*, *Shared* or *Full*. The module-specific documentation provides more information about the exact usage of BFP for each module. Full support means that the module makes full use of BFP, including initializing it as necessary and rescaling data via the BFP unit when appropriate. Shared support means that the module makes use of the BFP unit state provided by another module with full BFP support. That is, it can process block floating point data from modules with full support and update the BFP unit state accordingly, thereby “sharing” the BFP state from the module with full support. The shared support modules assume that the BFP unit is configured appropriately (by a full support module) before execution. No support (i.e., None) means that the module does not use the BFP unit and does not support BFP data types at all. Use of multiple modules that support the BFP unit requires that the BFP unit’s state be saved and restored when switching from one module to another. The HCT and the modules themselves allow you to control this saving and restoring of the BFP state. For more information, consult Section 2.5.3, “Block Floating Point” on page 29.

2.3.2 Constants

Microcode module constants are simply used to define convenient aliases for standard configuration values. Each microcode constant has an integer value associated with it. Microcode module constants can be used like built-in constants when configuring an application’s microcode using the HCT.

Constants are defined as part of a particular microcode module, but there is no concept of scope when using these constants. All constants are available for all microcode modules to use when specifying their parameters.

2.3.3 Configuration Parameters

A module’s configuration parameters control the behavior of the module and are specified at configuration-time using the HCT. Examples include filter lengths and input/output block sizes. Configuration parameters are fixed for an application and cannot be changed at run-time.

Every module includes a configuration parameter which gives each instance of the module a unique identifier. This identifier is used wherever a unique reference to the module instance is required and all such identifiers must be unique across all module instances in an application's microcode configuration.

Table 2 summarizes the attributes which define a configuration parameter. In general, configuration parameters have restricted ranges and may restrict, or be restricted by, the values of other configuration parameters.

Table 2. Configuration Parameter Attributes

Parameter Attribute	Description
Type	Specifies the datatype for this parameter - either of *String* or *Integer*. The string type is only used for the unique identifiers which name the module instance. The integer type is used to specify all other parameters, including those which may be constrained to a set of defined microcode constants.
Range	A range or finite set of valid values for this configuration parameter. For string parameters, the range is implicitly restricted to the set of characters and string length which defines a valid symbol in the HCT. For integers, a valid range or fixed set of values will normally be given. The values in a set may include microcode module constants. The range given is still subject to any of the restrictions for this parameter.
Restrictions	A set of zero or more restrictions for the parameter as a function of itself or other configuration parameters. These are used to define interrelationships between different configuration parameters and restrictions which are not captured as part of the parameter's Range attribute.
Run-Time Parameter (optional)	If present, it states the symbol used to export this parameter's address in shared memory. The symbols for run-time parameters always follow the format <name>_<parameter> where <name> is the unique identifier for this module instance and <parameter> is the name of the parameter itself. All run-time parameters are stored in the CD1 memory address space of shared memory in single 24-bit word entries. Run-time parameters are stored and exported in a manner similar to that of state variables described in Section 2.3.6, "State Variables". For more information about using run-time parameters, see Section 2.5.5, "Run-Time Usage".
Default Value (optional)	If present, it states the default value taken by this parameter when it is omitted in the HCT source file.

2.3.4 Address Parameters

A module's address parameters specify the locations in shared memory of all input, output and scratch (i.e., temporary) areas of memory used by the module. Like configuration parameters, they are specified at configuration-time using the HCT and are fixed for an application.

The module itself does not allocate or control the memory specified by the address parameters since the memory may actually be shared across multiple modules. For example, the output of one module may be an input to another module. It is up to the application developer to properly manage the allocation of this memory based on how the modules are configured.

IMPORTANT: We recommend that the CFX application allocate memory associated with the microcode module inputs and outputs using non-relocatable data segments in shared memory. This ensures that the toolchain does not locate any relocatable data segments in these areas of memory.

Table 3 summarizes the attributes that define address parameters. Address parameters are generally restricted to certain physical or logical memory mappings and the actual address may be restricted as a function of configuration parameters or other address parameters.

Table 3. Address Parameter Attributes

Parameter Attribute	Description
Type	Specifies whether the address parameter is for an *input*, an *output*, an *input/output*, or for *scratch* (temporary) memory. The type of the address parameter provides some guarantees about how the microcode module will treat that memory. An input will only be read by the microcode module. Thus, data in the memory specified by an input address parameter is guaranteed to not be changed by the module's functions. An output will only be written by the microcode module and correspondingly the data in the memory specified by an output address parameter is not guaranteed to be unchanged by a module's functions. Scratch memory is used for intermediate processing by the microcode module and any data stored at the address may be overwritten as part of the execution of a module's functions. Scratch memory can often be shared with other input or output address parameters in order to reduce the overall module memory requirements. An input/output can be read and written to by the module's functions and therefore the data is not guaranteed to be unchanged. If an input and output parameter are shared (see Shared Parameters below), then the location is also technically an input/output location and may be both read from or overwritten depending on the function being performed.
Memory Spaces	A set of valid memory spaces for this parameter. In this context, memory space refers to the different logical areas in shared memory, e.g., A0MEM, CD0MEM and H0MEM.
Shared Parameters	A set of zero or more address parameters that can refer to the same memory location. That is, these parameters can actually share the same assigned memory. Sharing address parameters is useful for re-using scratch memory as an input or output parameter or for using the same memory for both input and output (e.g., in-place processing). Note that if a single memory location is shared across multiple address parameters then it must still conform to all the restrictions of every address parameter it is used for.
Configuration Parameter Restrictions	A set of zero or more restrictions for this parameter as a function of other configuration parameters. These restrictions typically specify modulo restrictions on the address parameter because of circular addressing used by the HEAR to implement the module.
Address Parameter Restrictions	A set of zero or more restrictions for this parameter as a function of itself or other address parameters. These often specify further restrictions on which memory mappings are allowed for an address parameter based on what mappings are used for other address parameters. For example, given two parameters `input_a` and `output_b` which can both be in any interleaved memory mapping, there may be a further restriction that `input_a` and `output_b` cannot be in the same interleaved memory space. This kind of restriction can also be used to restrict the address parameter to 48-bit or 96-bit word alignments.
Size	The size of the memory block (in 24-bit words) pointed to by this address parameter as a function of the module's configuration parameters. For example, if an address parameter points to *N* 48-bit values, then the size for the address parameter is actually specified as *2N*. You can use the size as a guideline when allocating the memory associated with each module's inputs and outputs.

Table 3. Address Parameter Attributes

Parameter Attribute	Description
Run-Time Parameter (optional)	This is an optional parameter attribute. If present, it states the symbol used to export this parameter's address in shared memory. The symbols for run-time parameters always follow the format <name>_<parameter> where <name> is the unique identifier for this module instance and <parameter> is the name of the parameter itself. All run-time parameters are stored in the CD1 memory address space of shared memory in single 24-bit word entries. Run-time parameters are stored and exported in a manner similar to that of state variables described in Section 2.3.6, "State Variables". For more information about using run-time parameters, see Section 2.5.5, "Run-Time Usage".
Default Value (optional)	This is an optional parameter attribute. If present, it states the default value taken by this parameter when it is omitted in the HCT source file.

2.3.5 Parameter Restrictions

As described above, both configuration and address parameters can specify restrictions that are a function of other parameters. Normally, each restriction is specified as a boolean expression that must be true for the module configuration to be valid. These expressions are written using a C programming language style expression syntax. Restrictions can also contain a conditional element, whereby the restriction is only applied if some other boolean condition is met.

Several built-in functions are also used in these expressions to simplify the description of common relationships between parameters. These functions are described in Table 4. For examples of parameter restrictions, consult Chapter 4, "Module Reference" on page 95.

Table 4. Parameter Restriction Functions

Function Syntax	Description
`memtype(address)`	Returns the shared memory mapping for an address. For example, an address at 0x4040 would return the mapping H01MEM, and an address at 0x8100 would return AB0MEM. Two memory mappings are considered to be equal if there is any overlap in the physical memory that is part of the mapping. For example, for the purposes of checking parameter restrictions, A0MEM and AB0MEM are equal because they both use A0 memory. This allows parameter restrictions to specify that certain module inputs and outputs cannot be in the same physical memory regardless of how that physical memory is accessed.
`npow2(n)`	Returns the next power of two greater than or equal to the integer value given. For example, given the value 46, it would return 64 since that is the next power of two greater than 46.
`abs(n)`	Returns the absolute value of the integer value given. For example, given the value +5 it would return +5, but given the value -5 it would return +5.

2.3.6 State Variables

State variables provide a way to modify the behavior of module at run-time. The module-specific documentation describes how each state variable is used and how it affects the module's behavior.

All state variables are stored in the interleaved CD1 memory space of shared memory as part of the overall configuration memory reserved by the microcode configuration (see Section 2.4.3, "Shared Memory" on page 23). Each state variable is a single 24-bit word in this memory space. The exact addresses of each module instance's state variables are exported by the HCT as an appropriate symbol in the object file and header file outputs. Each state

variable's symbol always starts with the module's unique name, followed by a suffix which is unique to that state variable.

2.3.7 Numeric Types and Data Formats

The following list describes the standard numeric types used by the microcode modules:

Signed Fractional

The standard numeric type for the microcode modules is a signed fractional 24-bit word (i.e., the native numeric type of the system). This is equivalent to the frac24 numeric type described in the Math library section of the *Firmware Reference Manual for Ezairo 7100*.

Complex Signed Fractional

Microcode modules frequently work with complex data. The standard complex numeric type is two signed fractional 24-bit words in adjacent memory locations. Even address words are always considered to be the real component of a complex number and odd address words are always considered to be the imaginary component of a complex number.

Double-Precision Signed Fractional

The double precision signed fractional numeric type is a 48-bit version of the normal 24-bit signed fractional numeric type. This is equivalent to the frac48 numeric type described in the Math library section of the *Firmware Reference Manual for Ezairo 7100*. Double precision data is always stored in adjacent memory locations. Even address words are always considered to be the most significant word (MSW) and odd address words are always considered to be the least significant word (LSW).

Single-Precision Mixed Format

In some cases, the standard fractional numeric type does not have sufficient range to represent a microcode module's inputs or outputs. Where possible, the standard mixed precision numeric types described in the Math library section of the *Firmware Reference Manual for Ezairo 7100* are used (e.g., m8p16). Consult the *Firmware Reference Manual for Ezairo 7100* for information about these formats. In cases where there is no Math library compatible type, a general notation of 'p.q' is used where 'p' represents the number of signed integer bits in a word and 'q' represents the number of unsigned fractional bits in a word. In this notation, the normal signed fractional datatype would be described as a 1.23 format.

Double Precision Mixed Format

Double precision mixed format data is analogous to single-precision mixed format data, however the data is 48 bits wide. Once again, the standard mixed precision numeric types described in the *Firmware Reference Manual for Ezairo 7100* are used whenever possible (e.g., m2p46). Double precision data is always stored in adjacent memory locations. Even address words are always considered to be the most significant word (MSW) and odd address words are always considered to be the least significant word (LSW).

Block Floating Point (BFP) Format

Individual words of a BFP numeric type use the standard signed fractional 24-bit format, however this datatype is augmented by an exponent which is applied to an entire block of these values. The BFP datatype is supported by the HEAR's block floating point unit. For more information about the possible BFP formats and using the block floating point unit, consult Section 2.5.3, "Block Floating Point".

In addition to the numeric type of individual data elements, each microcode module description in Chapter 3, “Modules” on page 35 describes the format of the memory used for each module input or output. These formats describe the layout of the data elements in memory. Where possible, common formats are used across modules to allow for easy re-use of inputs or outputs across multiple modules.

2.4 HEAR INTERFACING

The HEAR Configurable Accelerator interfaces to the rest of the system through four different mechanisms:

- Command signals from the CFX to the HEAR
- Event signals from the FIFO controller to the HEAR
- Interrupt signals from the HEAR to the CFX
- Shared memory for storage of microcode function inputs, outputs, coefficients and state variables

Each of these mechanisms is described in greater detail in the following sub-sections.

NOTE: There are no direct command or interrupt signals between the HEAR and the ARM® Cortex™-M3 subsystems. You can use the FIFO controller or the CFX to provide indirect communications between the HEAR and the ARM Cortex-M3 core.

Similarly all HEAR and FENG interfacing are coordinated using the FIFO controller as there are no direct command or interrupt signals between the HEAR and the FENG.

2.4.1 Signals to the HEAR

Command signals from the CFX and input/output block (ioblock) full events from the FIFO controller can both be used to trigger the execution of a function chain. Each signal or event is associated with a unique function chain. Function chains are enabled or disabled by using the function chain controller as described in the *Hardware Reference Manual for Ezairo 7100* and Section 2.5.2, “Function Chain Controller”.

There are 16 function chains associated with command signals from the CFX. In some situations, one of these is reserved for initializing or resetting the state of all microcode modules, although this is ultimately configurable by you.

There are eight function chains associated with events from the FIFO. Each one can be configured to be triggered by any of the 32 FIFOs. When the FIFO controller performs a block update for a FIFO which is configured as the source of a HEAR FIFO interrupt, it triggers the corresponding function chain for the HEAR. For example, when the FIFO A0_0 is configured as the source of HEAR FIFO interrupt 0 and the FIFO controller has collected a new block of data in A0_0, the FIFO 0 function chain on the HEAR is executed if it is enabled via the function chain controller.

Each function chain is linked to a command from the CFX or an interrupt event from the FIFO controller by an entry in the HEAR function chain vector table. Examples of using the function chain vector table to control which function chains are executed at run-time can be found in Section 2.5.2, “Function Chain Controller”.

If a new function chain is triggered during the execution of another function chain, the second chain is not executed until the first completes. If two function chains are waiting to execute simultaneously, the next one to execute is determined by the priority setting for the function chains. Function chain priorities are described in greater detail in the *Hardware Reference Manual for Ezairo 7100*.

2.4.2 Signals from the HEAR

The HEAR can send an interrupt to the CFX before a microcode function begins or after a microcode function is finished in a function chain. These signals are meant to interrupt the CFX at developer-selected points during the function chain execution, normally to indicate the completion of a specific function or set of functions.

There are eight interrupt signals from the HEAR to the CFX. As part of configuring the function chains, you can select which interrupt to use and what its meaning is in the context of the application. In some situations, one of these interrupt signals is reserved to indicate the completion of a reset/initialization function chain, although this is ultimately configurable by you.

2.4.3 Shared Memory

The shared memory is accessible by both the CFX and the HEAR, and is the only means for the two cores to share data. Any memory associated with a microcode module and its functions is located in the shared memory, including inputs, outputs, coefficients, state variables and configuration data. Shared access to the memory is controlled through a priority access scheme. If both the CFX and HEAR attempt to access the same physical part of shared memory (e.g., both access A memory), then the core with lower priority is held for that clock cycle. More detailed information about the physical memories, bus structure, and shared access to this memory is described in the *Hardware Reference Manual for Ezairo 7100*.

Each named memory bank in the HEAR (A, B, C and D) has two memories. For example, memory bank A has A0 and A1 memories.

IMPORTANT: When you pass values for address parameters in an Ezairo 7100 HEAR configuration that are restricted to A, B, C and D memory banks, the address you pass can fall into either the 0 or 1 part of the memory. For example, the `output_fifo_addr` parameter of the `WOLA_Synthesis` module is restricted to BMEM, so you can pass an address that is in either B0 or B1.

A number of conventions govern the usage of these memories in microcode modules. These are only conventions though; you have ultimate control over the microcode configuration, subject to the module-specific address parameter restrictions. The conventions for using shared memory are described below.

- A0, A1, B0 and B1 memories are used primarily for FIFO buffers, since they are the only memories accessible through the FIFO controller. By convention, A0 and A1 memories are used for FIFOs that are primarily system inputs, and B0 and B1 memories are used for FIFOs that are primarily system outputs.
- C0, C1, D0 and D1 memories are used primarily for algorithm coefficients. Coefficients in these memories can be organized to align with a corresponding FIFO in A0/A1 or B0/B1 memory. This allows the HEAR to efficiently access FIFO data and coefficient data through the interleaved FIFO and coefficient memory spaces.
- The H[0-5] memories and their interleaved mappings are used for general function inputs or outputs, and as temporary memory by some microcode modules. An important use for these memories is storage of the analysis results and gain values for WOLA filterbank processing.

2.4.3.1 Microcode Configuration Reserved Memory

Some portions of the shared memory are reserved for use by the microcode. There is one fixed-size reserved area located at the top of the interleaved CD1 memory space (i.e., a portion is reserved at the top of both C1 memory and D1 memory).

This area extends from 0xBFA0 to 0xBFFF under the interleaved CD1 memory mapping, or from 0xEFD0 to 0xEFFF in C1 memory and 0xFFD0 to 0xFFFF in D1 memory. This fixed reserved area contains global configuration

data for the microcode modules and function chains. Microcode configurations generated by the HCT automatically reserve this area of memory using the linker.

An additional amount of the interleaved memory is reserved for module-specific configuration data and variables. This configuration memory is automatically allocated and reserved by the HCT. You can determine the size and location of this configuration memory by examining the summary file which can be generated by the HCT (see Chapter 5, “HEAR Configuration Tool” on page 325). This information can also be determined by examining the HCT object file output with CFLIB, or by examining the *.map* file output for the application binary. For more information about CFLIB and the build process, consult the *CFX Toolchain Reference*.

A diagram depicting the positioning of the fixed-size reserved area and the variable-size reserved areas of CD1 memory is shown in Figure 2.

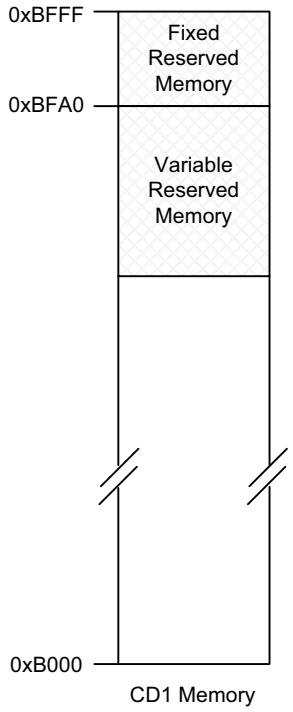


Figure 2. Memory Map of Microcode Reserved Shared Memory

IMPORTANT: The WOLA module reserves additional shared memory in C0, C1, D0 and D1 memories for analysis and synthesis windows. See Section 3.2.1.4, “Analysis FIFO Configuration” on page 43 and Section 3.2.1.5, “Synthesis FIFO Configuration” on page 44 for more information about the shared memory used for these coefficients.

2.4.3.2 FIFO Buffer Reserved Memory

The HCT will generate a macro for reserving the memory required by the FIFO buffers you have defined in the HCT input file. This FIFO memory allocation macro can be used by the CFX application code to reserve memory for

FIFO buffers. This memory is not explicitly allocated by the HCT, therefore you are free to allocate the FIFO buffer memory in some alternate fashion.

IMPORTANT: It is strongly recommended that the HCT generated FIFO buffer memory allocation macro be used to ensure that other relocatable memory segments are not placed in the same memory location as a FIFO buffer.

2.4.3.3 Sine/Cosine Tables

The shared memory also includes a ROM-based table of sine and cosine values. The table is mapped to `D_HEAR_ROM_BASE` in the X memory space. The table is organized in sine and cosine value pairs, so that accessing a 48-bit double word gives a single complex value. The table is arranged according to the following formula for a given address offset *x*, where *x* is assumed to be even:

$$y = e^{j\frac{2\pi}{1024}(x/2)}$$

The even address portion of the data at *x* corresponds to the real part of *y* and the odd address portion corresponds to the imaginary part of *y*. Each real or imaginary value in the table is a 24-bit signed fractional value.

2.5 System Configuration and Usage

Proper execution of the HEAR microcode depends on the proper configuration and usage of the HEAR's internal components such as the block floating point support and the function chain controller. The HEAR microcode is also designed to work closely with the FIFO controller. Configuring the FIFO controller appropriately can simplify the usage of many microcode functions.

This section describes the general configuration and usage guidelines for ensuring that the HEAR microcode works with the rest of the system.

IMPORTANT: The system must be correctly configured before starting the HEAR in order for the microcode to work as designed.

2.5.1 FIFO Controller

The FIFO controller's configuration is key to efficient usage of the HEAR's microcode modules. Microcode modules work with fixed input and output addresses. The use of a FIFO conveniently remaps a circular buffer such that the most recent samples are always located at the same fixed address in memory. You may forego the use of FIFOs and manually move data to the appropriate locations for the modules, however it is clearly more efficient to use a FIFO whenever one is available.

Each microcode module documents the specific format of input or output data and how it can be efficiently mapped to a FIFO where possible. This module-specific information is available in Chapter 3, "Modules" on page 35. The FIFOs being used must be configured properly from the CFX before starting the HEAR and executing any of the associated function chains.

The remainder of this section documents general principles for the FIFO configuration used by microcode modules. It also provides additional information about using a software controlled FIFO.

For specific information about the registers for configuring and using the FIFO controller, consult the *Hardware Reference Manual for Ezairo 7100*.

2.5.1.1 FIFO Configuration

From the perspective of a microcode module, any of its address parameters may point to memory which is part of a FIFO. When the address parameter is for an input or output which relies on past or future samples of data, then there is an expectation that the FIFO will be configured in a manner which is compatible with the expected ordering and format of the data.

This expected configuration can be described in terms of the relative location of the most recent input or output block to the address parameter value, and how much data is expected before or after this most recent block. A generalized view of this is depicted in Figure 3. In this view, the region of the FIFO being operated on by the microcode module is split into three regions - a processing area for *future samples*, the *most recent block of samples* (also called the current block), and the processing area for *past samples*.

In the case of an input, the most recent block of samples is normally considered to be part of the past samples, since it represents samples that have already arrived. In the case of an output, the most recent block of samples is normally considered to be part of the future samples, since it represents samples that have yet to be written by the microcode module.

NOTE: When using FIFOs with the IOC, the ordering of samples in time is such that older samples are at higher addresses. Within a block, the oldest sample is always at the top of the block and the newest sample is always at the bottom.

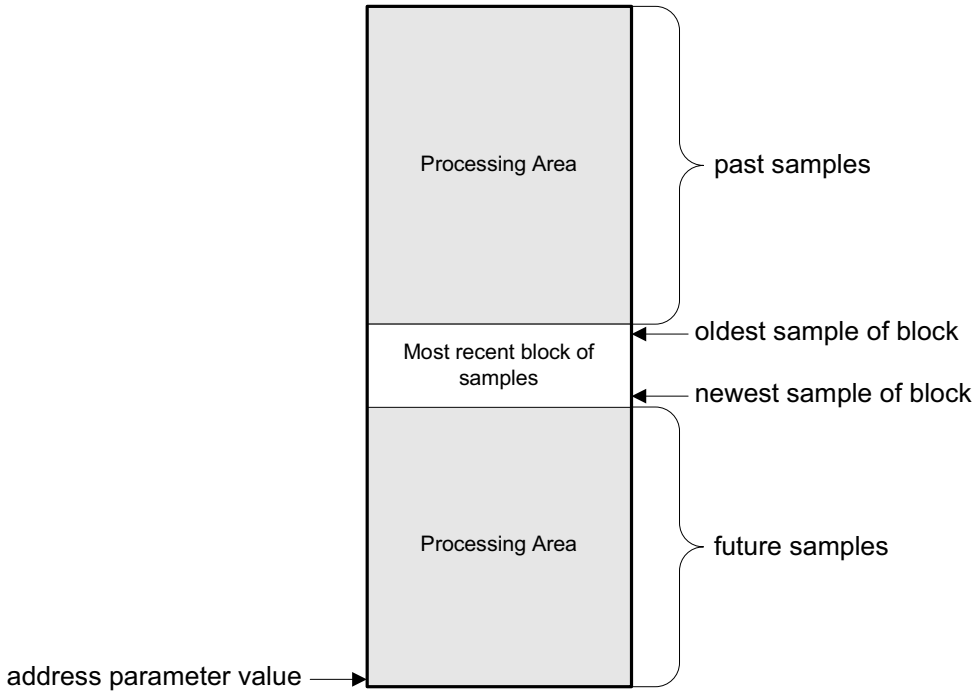


Figure 3. General View of FIFO Configuration Using a Microcode Module Address Parameter

The FIFO layout(s) for a particular microcode module are described in terms of suggested values for the following FIFO parameters, assuming that the address parameter in question points to the beginning (i.e. the base address) of the configured FIFO:

- Minimum FIFO size[1]
- Block size
- Base pointer
- IO block pointer

Using the suggested values for these FIFO parameters and defining the address parameter value to be the same as the FIFO's start address will result in a FIFO layout which is suitable for the microcode module, assuming that an IOC input or output is used to write or read samples from the FIFO. The FIFO configuration information for each microcode modules shows the location of the *write-in block* or *read-out block* for the IOC, assuming that IOC is used to move samples in or out of the FIFO. The write-in block is the block of samples currently being written by the IOC when the FIFO is used as an input. The read-out block is the block of samples currently being read by the IOC when the FIFO is used as an output. These blocks may overlap with the processing areas depending on the FIFO configuration. Where the module's input and/or output cannot be practically connected to the IOC, the location of these blocks and suggested values for the IO block pointer are omitted.

When the FIFO is software controlled, this same layout can be used and the samples can be written or read in one of two ways: either by accessing the FIFO through the appropriate `D_FIFO_xx_x_ACCESS` register, or by accessing the module's most recent block of samples through direct addressing into the FIFO.

NOTE: The most recent block of samples does not necessarily have to be the same for the hardware FIFO configuration and the microcode module's perspective. Similarly, the area of memory defined for the hardware FIFO does not necessarily have to correspond one-to-one with the area of memory used for the module input or output. The microcode modules only require that data be formatted in a particular manner for input or output, and the FIFOs simply provide a convenient and efficient way to achieve this format.

2.5.1.2 Software Controlled FIFOs

As described in the *Hardware Reference Manual for Ezairo 7100*, a FIFO can be connected directly to a system input or output via the IOC, or it can also be used directly by the application software. In this second case, the FIFO acts as a hardware-supported circular buffer. This FIFO usage is very useful for storing intermediate inputs or outputs between microcode modules. Software controlled FIFOs provide an efficient mechanism for connecting microcode modules together.

For example, suppose an analog input is routed directly into FIFO A0_0 via the IOC. Then, an FIR filter module instance is used to filter this input and the FIR filter places its output into FIFO A1_0. FIFO A1_0 is then used as input to a WOLA analysis operation. In this situation, FIFO A1_0 is not directly connected to hardware input or output, and so it must be a software controlled FIFO.

The point at which a software FIFO's pointers are updated will depend on the application structure. If samples are read or written via the `D_FIFO_xx_x_ACCESS` register, then the update happens automatically. In this case, a clear distinction should be made between which application code or microcode module is responsible for accessing data via this register.

1. The minimum FIFO size is specified in the microcode module description but the actual FIFO size is constrained to be the next higher multiple of 32 due to the hardware constraints as described in the *Hardware Reference Manual for Ezairo 7100*.

A more general method for updating the FIFO pointers is to write a value into the D_FIFO_xx_x_BLOCK_CNT register that is equal to the value stored in D_FIFO_xx_x_BLOCK_SIZE. This immediately triggers a FIFO update. The pointers will be updated, the block count will be reset to zero, a FIFO interrupt will be generated and the appropriate FIFO related function chain will execute if it is enabled. When using this method, the FIFO update is normally postponed until after every microcode module that accesses the FIFO is completed. The HCT provides a method to trigger this kind of FIFO update within a function chain, as described in Chapter 5, “HEAR Configuration Tool” on page 325.

2.5.1.3 Dual FIFOs

A *dual FIFO* is composed of one A0/A1 memory FIFO and one B0/B1 memory FIFO, both of which are configured with the same settings. When the memory for these FIFOs is accessed through the interleaved AB0/AB1 memory mappings, this has the effect of creating one larger dual-memory FIFO out of two smaller single-memory FIFOs, hence the name “dual FIFO”. The combination of an A0/A1 and B0/B1 memory FIFO into a dual (AB0/AB1) FIFO is depicted in Figure 4.

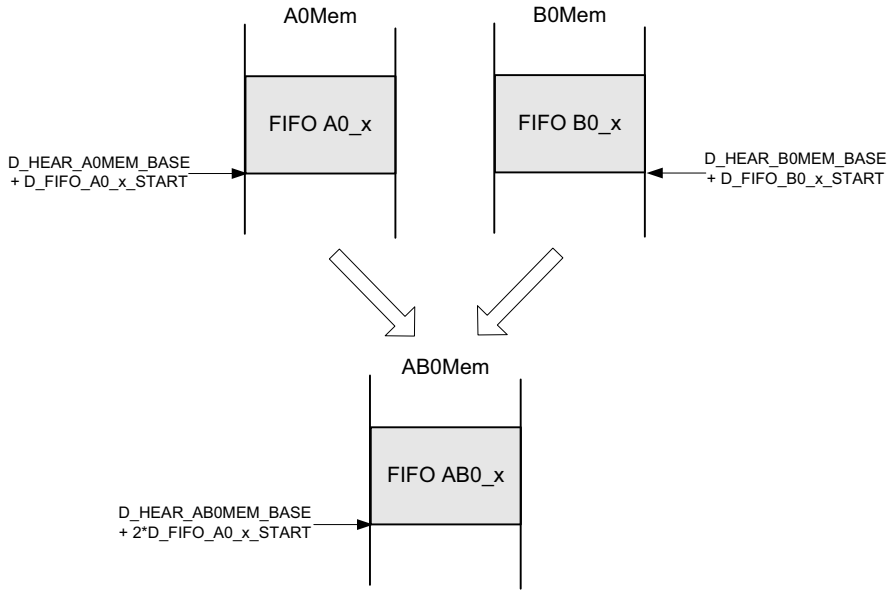


Figure 4. Dual FIFO Mapping

There is no special hardware support for dual FIFOs but the HCT does provide direct support for configuring and using dual FIFOs. Because there is no additional hardware support, it is not meaningful to connect either of the single FIFOs in a dual FIFO to the IOC. Dual FIFOs are normally used only as software controlled FIFOs. FIFO pointer updates for the A0/A1 and B0/B1 FIFOs which make up the dual FIFO must always be done together. This ensures that the A0/A1 and B0/B1 memory portions of the dual FIFO are in sync.

There are two primary reasons for using a dual FIFO:

- To create a FIFO buffer which is bigger than what is possible with a single FIFO
- To create a FIFO buffer with an interleaved memory mapping for microcode modules which require interleaved memory inputs or outputs

There are some limitations on the possible configurations when using dual FIFOs. Because the AB0/AB1 memories are interleaved as 48-bit words, the block size for the component A0/A1 and B0/B1 memory FIFOs must be even for an equal number of 48-bit words to exist in A0/A1 and B0/B1 memory. Since the dual FIFO block size is twice the underlying single FIFO block size, this means that the dual FIFO block size must always be a multiple of four. The normal single FIFOs also have a hardware limitation that the start address, end address and size parameters are integer multiples of 32. When considering a dual FIFO, the effective parameter sizes are doubled and thus the limitation is that the start address, end address and size of a dual FIFO must all be integer multiples of 64.

More examples of using dual FIFOs are provided as part of some module descriptions in Chapter 3, “Modules” on page 35.

2.5.2 Function Chain Controller

Code running on the CFX must enable all of the function chains that are used by the application’s microcode configuration. This can be done by setting the appropriate bits in the `D_HEAR_FC_ENABLE` register. This must be done before the HEAR is initially started.

If you are expecting concurrent events or commands to occur, you may need to configure the function chain priorities accordingly so that the function chains are executed in the proper order.

The HEAR function chain vector table located at `D_HEAR_FC_VECT_BASE` can be modified at run-time to change the command or FIFO event to which a specific function chain is linked. Each function chain has an entry point in microcode memory and the HCT generates a symbol for these entry points so that they can be referenced from the developer’s CFX application. The macro `Sys_Set_HEAR_FunctionChain_Vector` from the System Library can be used to assign a function chain to a command or FIFO event in the vector table. The example below demonstrates setting the FIFO 0 event function chain vector to a new function chain:

```
// Set FIFO 0 event to trigger the function chain called fc1
Sys_Set_HEAR_FunctionChain_Vector(HEAR_FC_FIFO_0_POS, fc1_entry)
```

For more information about configuring the function chain controller, including specific register settings, consult the *Hardware Reference Manual for Ezairo 7100*.

2.5.3 Block Floating Point

The HEAR Accelerator’s block floating point support is configured through the `D_HEAR_BFP` register. Any microcode modules with full BFP support will automatically initialize this register as appropriate and the module-specific documentation indicates what inputs or outputs use the BFP datatype.

2.5.3.1 BFP Data Types

The HEAR supports 3 separate BFP data types: 21-, 22- and 23-bit. The number of bits specified in the BFP datatype indicates the size of the signed fractional portion of the datatype. The additional bits left over to make up the 24-bit data word are essentially extension (or integer) bits. When used, the BFP unit will detect when the extension bits are in use, i.e., when the extension bits do not all equal the most significant fractional bit. When this happens, the BFP unit will perform a rescaling of the data such that the extension bits are no longer in use (see Section 2.5.3.4, “BFP Implementation”). The use of a BFP datatype means that the BFP unit can compensate for data growth equal to the number of extension bits in a single pass. For example, using the 22-bit BFP datatype the BFP unit can compensate for data growth of up to 2 bits, or a growth factor of 4.

2.5.3.2 BFP Usage

From a user's perspective, the most important information is the block exponent applied to the data thus far, which is stored in the HEAR_BFP_BLOCK_EXP field of the D_HEAR_BFP register. This value is also mirrored in the D_HEAR_BFP_BLOCK_EXP register. This allows for easier read and write access to this value without modifying the rest of the D_HEAR_BFP register. The block exponent is a 6-bit unsigned value. This value is the total number of right shifts applied to the data thus far and should always be taken into account when processing BFP data.

Several firmware macros, which are described in the *Firmware Reference Manual for Ezairo 7100*, are available to manipulate the BFP registers. Examples of reading and writing the block exponent using these macros are given below:

```
// Read the block exponent into x0
Sys_Get_HEAR_BFP_BLOCK_EXP(x0)

// Write a new block exponent from x1
Sys_SetReg_HEAR_BFP_BLOCK_EXP(x1)
```

2.5.3.3 BFP Unit Sharing

When multiple microcode modules make full use of the BFP unit, it is necessary to save and restore the BFP unit's state when each module's functions are run. Also, some modules will reuse or update the current BFP state but must rely on another module with full BFP support or CFX application code to first initialize the BFP unit.

Saving and restoring the BFP unit's state is normally done using dedicated BFP support functions in the BFP_State microcode module. These functions follow the format shown in Table 5, where the parameter name is the developer-supplied unique name. The BFP_Save function saves the D_HEAR_BFP register into a state variable name_bfp_state which you can access to extract the block exponent or other BFP state information. The BFP_Restore function restores the D_HEAR_BFP register with the value read from the state variable name_bfp_state. For more information, consult Chapter 4, "Module Reference" on page 95.

Table 5. Standard BFP Save and Restore Functions

Microcode Function	Description
BFP_Save (name)	Saves the state of the BFP unit to the state variable name_bfp_state.
BFP_Restore (name)	Restores the state of the BFP unit from the state variable name_bfp_state.

2.5.3.4 BFP Implementation

Figure 5 depicts a block diagram view of the BFP unit, including the internal state portions of the D_HEAR_BFP register. The functionality of the BFP unit can be explained by following the events which occur during a particular read/write cycle of BFP data:

1. The HEAR processing reads from shared memory and the data is rescaled by the current value in the HEAR_BFP_SCALE_FACTOR field of the D_HEAR_BFP register. E.g., if the current value of the scale factor is 2, then the data is shifted right by 2 (with sign extension).
2. The HEAR performs some processing on the data that was read.
3. The HEAR writes new data to shared memory, and the maximum magnitude of the block of data written is saved in the HEAR_BFP_BLOCK_MAX_MAG field of the D_HEAR_BFP register. If the HEAR_BFP_FMD_MODE bit is set, then the value saved to the HEAR_BFP_BLOCK_MAX_MAG field records the maximum magnitude of the data regardless of whether rescaling is required. If the HEAR_BFP_FMD_MODE is not set, then the value saved to

the HEAR_BFP_BLOCK_MAX_MAG field only records the magnitude when it exceeds the fractional portion of the BFP datatype, i.e., when rescaling is required. Note that normally this bit is not set, since this saves power by only examining the top guard bits of the data written.

4. The HEAR BFP unit updates the BFP state by performing the following steps:
   a. Add the current scale factor to the block exponent (HEAR_BFP_BLOCK_EXP field) in the D_HEAR_BFP register. This records the rescaling which was done on the most recent read.
   b. Determine the next scale factor by examining the HEAR_BFP_BLOCK_MAX_MAG field and writing a new value to the HEAR_BFP_SCALE_FACTOR bits. The next scale factor is the number of right shifts necessary to rescale the data back to the fractional range when the data is read from memory by the HEAR next time.

Once this BFP read/write cycle is complete, the data is ready for another pass of processing by the HEAR or for reading by the CFX application. At this point, the D_HEAR_BFP_BLOCK_EXP register contains the total number of right shifts applied to the data thus far, and should be used as the block exponent for this data.

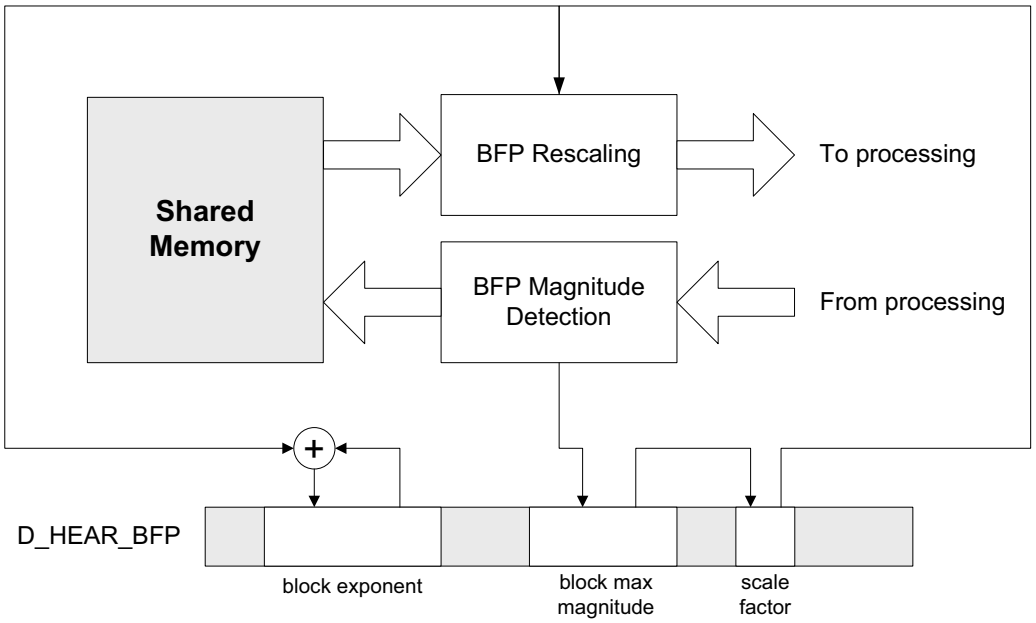


Figure 5. Block Floating Point Unit Block Diagram

2.5.4 Arithmetic Flags and Control

The HEAR Configurable Accelerator's internal datapath has control bits and flags for modifying its arithmetic behavior and reporting the results of arithmetic operations. These bits are not directly accessible from the CFX but the HCT and HEAR microcode library provide some built-in facilities to change or save them from within function chains. These facilities are described in the sub-sections which follow.

2.5.4.1 Saturation Detection

The HEAR Configurable Accelerator's internal datapath has a flag to detect when saturation occurs in an arithmetic operation. This flag is not directly accessible from the CFX; however, the HCT provides a mechanism to reset or save this flag to a location in shared memory. This is done using the built-in ResetSaturationFlag and SaveSaturationFlag macros described in Chapter 5, "HEAR Configuration Tool" on page 325.

This flag can be used to detect if saturation occurred during the computation of a particular microcode module. The normal method of using this is as follows:

1. Use the `ResetSaturationFlag` macro in a function chain immediately before the module or modules in which you would like to detect saturation.
2. Use the `SaveSaturationFlag` macro in a function chain immediately after the module or modules in which you would like to detect saturation.
3. Check the saturation flag in shared memory using the `HEAR_SATURATION_FLAG` symbol generated by the HCT. This can be done from the CFX application after the function chain is completed. A value of one in this memory location means that saturation has occurred and a value of zero means that it has not.

2.5.4.2 Accumulator Overflow Detection

The HEAR Configurable Accelerator's internal data path has a flag to detect when overflow occurs during an accumulation operation. This flag is not directly accessible from the CFX; however, the HCT provides a mechanism to reset or save this flag to a location in shared memory. The built-in macros, called `ResetOverflowFlag` and `SaveOverflowFlag`, are described in Section 5.3, "Built-in Macros" on page 330. This overflow detection differs from the saturation detection in that it signals when an accumulation has overflowed the internal accumulator registers of the HEAR including its extension bits.

This flag can be used to detect if accumulator overflow occurred during the computation of a particular microcode module. The normal method of using this flag is as follows:

1. Use the `ResetOverflowFlag` macro in a function chain immediately before the module or modules in which you would like to detect overflow.
2. Use the `SaveOverflowFlag` macro in a function chain immediately after the module or modules in which you would like to detect overflow.
3. Check the overflow flag in shared memory using the `HEAR_OVERFLOW_FLAG` symbol generated by the HCT. The CFX application can perform the check after the function chain is completed. A value of one in this memory location means that overflow has occurred and a value of zero means that it has not.

2.5.4.3 Overflow Protection Control

The HEAR Configurable Accelerator's internal data path has a control bit to enable or disable overflow protection in arithmetic operations. This control bit is not directly accessible from the CFX; however, the HCT provides a mechanism to set or reset this flag from a function chain. These built-in macros, called `EnableOverflowProtect` and `DisableOverflowProtect`, are described in Section 5.3, "Built-in Macros" on page 330. The default state of the HEAR at the beginning of each function chain is for overflow protection to be enabled. Disabling overflow protection only has defined behavior for the single-precision additive vector arithmetic modules listed below:

- `vAdd_R`
- `vAdd_C`
- `vSub_R`
- `vSub_C`
- `vAddScalar_R`
- `vAddScalar_C`
- `vSubScalar_R`
- `vSubScalar_C`

Other modules are either unaffected by the overflow protect control or will produce undefined results if it is disabled.

2.5.5 Run-Time Usage

At run-time, the HEAR and its microcode can be controlled from the CFX DSP. This section discusses several issues related to run-time control of the HEAR:

- Direct control of the HEAR's execution from the CFX, including stopping and starting the HEAR, and executing function chains
- Resetting the HEAR both in terms of the hardware and the application's microcode configuration
- Run-time modification of parameters in the HEAR microcode

2.5.5.1 Control from the CFX

The HEAR can be controlled from the CFX through the `D_SYSTEM_CTRL` register. The CFX can start, pause or reset the HEAR by writing the appropriate bit in this register. The HEAR reset signal is also tied to the FIFO controller reset, so that when the HEAR is reset, the FIFO controller registers are also cleared. Resetting the HEAR does not clear the microcode memory or shared memory. The appropriate firmware macros for controlling the HEAR through the `D_SYSTEM_CTRL` register are described in the *Firmware Reference Manual for Ezairo 7100*, and are repeated in the code example below:

```
// Reset the HEAR
Sys_HEAR_Reset

// Start the HEAR
Sys_HEAR_Start

// Pause the HEAR
Sys_HEAR_Pause
```

Initialization

Normally, reset the HEAR as part of the system initialization before any of the following are done:

- FIFO controller configuration
- HEAR function chain controller configuration
- Enabling of any system interrupts

Running and Pausing

Once the system initialization is complete, the HEAR can be started. This enables the execution of function chains on the HEAR. During normal system operation, the HEAR does not need to be manually paused since it automatically executes microcode only in response to a function chain command or FIFO signal. The pause feature can, however, be used to interrupt a function chain if the system must immediately cease any signal processing on the HEAR. The pause feature simply stops the clock to the HEAR and puts any shared memory accesses in a non-blocking state. Function chains still become pending while the HEAR is paused, and execute according to their priority once the HEAR is unpaused. For more information about starting and pausing the HEAR, consult the *Hardware Reference Manual for Ezairo 7100*.

Executing Function Chains

The HEAR executes a function chain in response to a command from the CFX or in response to a block full event from a FIFO. The code example below shows how to use the standard firmware macros to launch a function chain from the CFX:

```
// Launch function chain associated with command 5 on the HEAR
Sys_HEAR_Command(5)
```

2.5.5.2 Reset Behavior

The HEAR hardware is reset using the `Sys_HEAR_Reset` macro which sets the appropriate bit in the `D_SYSTEM_CTRL` register as described in Section 2.5.5.1, “Control from the CFX”. This hardware reset affects the HEAR accelerator, the function chain controller, the BFP controller and the FIFO controller. Detailed information about the hardware reset behavior of the HEAR is included in the *Hardware Reference Manual for Ezairo 7100*.

The hardware HEAR reset *does not* modify any of the state variables associated with the microcode modules used in the current microcode configuration. These state variables are stored in shared memory as described in Section 2.4.3.1, “Microcode Configuration Reserved Memory”. To properly reset these state variables, any reset functions associated with the microcode modules must be executed.

2.5.5.3 Modification of Run-Time Parameters

Microcode modules with run-time accessible parameters are identified by the run-time parameter attribute in the associated module parameters as described in Section 2.3.3, “Configuration Parameters” on page 17 and Section 2.3.4, “Address Parameters” on page 18. Only selected configuration and address parameters are run-time configurable. These parameters are exported as 24-bit words in the CD1 memory address space of shared memory using symbols generated by the HCT in the object code and header file outputs. The following rules apply for modifying run-time parameters:

1. The CFX can modify these parameters by writing to the memory location referenced by the exported symbol. Only modify these parameters when the HEAR is not executing the associated module. Modification of a run-time parameter during the module’s execution produces undefined behavior.
2. The valid values for these parameters are still restricted by the associated compile-time configuration parameter restrictions. Setting a run-time parameter to an invalid value produces undefined behavior.
3. The initial value of the run-time parameter is normally the same value that is given to the HCT at compile-time for the associated configuration parameter. In some cases, the run-time parameter value stored in memory is modified relative to the compile-time value (e.g., the value in memory might be negated). In these cases, the differences are documented with the configuration parameter in Chapter 4, “Module Reference” on page 95.
4. The value stored in CD1 memory for a run-time configurable address parameter is the address within the HEAR’s memory space, not the address within the CFX’s memory space where the HEAR shared memory is mapped into. When you specify an address parameter at compile-time (in the .hct file), you use the address within the CFX’s memory space, such as `D_HEAR_A0MEM_BASE`, and the HEAR Configuration Tool processes it and puts the corresponding HEAR address at the address parameter location. However, if an address parameter is run-time configurable, when you update it at run-time you must use the native HEAR address. We recommend that you use the firmware macro `Sys_HEAR_RunTime_Addr` to translate a shared memory address in the CFX memory space into an address in the HEAR memory space.

CHAPTER 3
Modules

This chapter describes the general usage of all the microcode modules available for the HEAR Configurable Accelerator. More reference information about the modules and their parameters is available in Chapter 4, “Module Reference” on page 95.

3.1 Overview

The microcode modules can be divided into five general groups:

WOLA Filterbank — This group includes the Weighted-Overlap Add (WOLA) Filterbank module as well as several support modules designed to work with the filterbank outputs, e.g., gain application, subband channel mixing.

Transforms — These modules implement common signal processing transforms, such as the Discrete Fourier Transform (DFT).

Filters — These modules include various time-domain and subband-domain filtering operations.

Vector Operations — These modules implement vector-based operations such as element-wise arithmetic operations.

Signal Statistics — These modules are used for computing statistical values of vectors such as average magnitude or cross-correlations.

3.1.1 Conventions

In addition to understanding the module groupings, you will find it helpful to be familiar with the module, function and parameter naming conventions when you are navigating the documentation. The module and function naming conventions are:

- Module names use a mix of “CamelCase” style naming and the underscore character as a general purpose separator.
- Multi-function modules are named for the general operation or signal processing structure they implement. Individual functions in these modules use the module name followed by an underscore, followed by the function’s specific operation.
- Single-function modules are always named for the operation they perform and the function name is the same as the module name.
- The character ‘v’ is used as a prefix for modules which take vector inputs, but where the operation itself does not indicate a vector operation. For example, the module `InnerProduct_R` clearly requires vector inputs, so the prefix ‘v’ is not used.
- The suffixes ‘_C’ and ‘_R’ are used to denote modules which perform the same operation but work with complex or real data, respectively.

The parameter naming conventions are given below:

- Parameters are normally specified in lower-case with underscore separators, with the exception of capital letters used for parameters which correspond to mathematical variables such as *L*, *N*, *K*, etc.
- The parameter *L* is used for specifying FIR filter lengths, including filterbank window sizes.
- The parameter *N* is used for specifying the transform size, or number of complex bands, in filterbank related modules.

- The parameter *R* is used for specifying the input/output block size for block-based processing.
- The parameter *M* is used for specifying IIR filter lengths.
- The parameter *K* is used for specifying the number of elements in a vector to be processed.
- Configuration parameters involving the above generic variables use the module name as a distinguishing prefix.

3.2 WOLA FILTERBANK

The Weighted-Overlap Add (WOLA) filterbank is the oversampled filterbank architecture patented by ON Semiconductor for efficiently dividing an input signal into a group of subband signals having uniform frequency band-width. The WOLA filterbank operates on time-domain input blocks and the subband outputs are a time-frequency representation of the input signal on a block-by-block basis. It implements a complex modulated filterbank and thus the subband samples are complex with both magnitude and phase components. The WOLA filterbank architecture is highly configurable, and through oversampling it provides excellent band separation and very low group delay.

The WOLA filterbank modules implement the filterbank itself as well as a number of support modules that perform common subband processing tasks. The modules are summarized in Table 6.

Table 6. WOLA Filterbank Modules

Module Name	Description	References
`WOLA_Analysis`	The `WOLA_Analysis` module implements the analysis (time to time-frequency) filterbank function.	Section 3.2.1, “Analysis and Synthesis” Section 3.2.1.1, “Analysis Implementation” Section 4.2, “WOLA_Analysis”
`WOLA_Synthesis`	The `WOLA_Synthesis` module implements the synthesis (time-frequency to time) filterbank function.	Section 3.2.1, “Analysis and Synthesis” Section 3.2.1.2, “Synthesis Implementation” Section 4.3, “WOLA_Synthesis”
`Gain_R`	Applies a real-valued set of gains to the subband signals	Section 3.2.2, “Gain Application” Section 4.4, “Gain_R”
`Gain_C`	Applies a complex-valued set of gains to the subband signals	Section 3.2.2, “Gain Application” Section 4.5, “Gain_C”
`Mix_Channels`	Down-mixes the output of a stereo WOLA filterbank analysis to a single channel for input to a mono WOLA filterbank synthesis	Section 3.2.3, “Subband Channel Mixing” Section 4.6, “Mix_Channels”
`Energy`	Computes the squared magnitude of the complex subband signals	Section 3.2.4, “Subband Energy Calculation” Section 4.7, “Energy”
`Normalize`	Normalizes the block floating point subband signals to a user-specified block floating point exponent	Section 3.2.5, “Subband Normalization” Section 4.8, “Normalize”

The WOLA filterbank architecture allows for a number of important parameters that are summarized in Table 7. Note that not all of the parameters are independent. For the WOLA microcode module, the developer specifies the parameters L_a, L_s, N, and R, as well as the band stacking and channel mode. The dependent parameters are thus OS and DF.

Table 7. WOLA Filterbank Parameters

Parameter	Description
Analysis Window Length (L_a)	The length of the filter used for analysis. In the WOLA filterbank architecture, this filter is applied as a window to the input data and thus is often referred to simply as the analysis window. The `WOLA_Analysis` module provides default and optimized analysis windows, or you can specify a custom analysis window. Can be a power of two from 16 to 1024.
Synthesis Window Length (L_s)	The length of the filter used for synthesis. Often referred to as the synthesis window, for the same reasons as described above for the analysis window. Due to oversampling, this window can have a wider bandwidth and hence be shorter in time which helps to reduce group delay. The `WOLA_Synthesis` module provides default and optimized synthesis windows, or the user can specify a custom synthesis window. Can be a power of two from 16 to L_a.
FFT Size (N)	The size of the Fast Fourier Transform (FFT) used to modulate the frequency bands. In even stacking, the transform produces (N/2 - 1) full-width frequency bands and 2 half-width frequency bands. In odd stacking, the transform produces N/2 full-width bands. Can be a power of two from 16 to 512 for mono filterbanks, or up to 256 for stereo filterbanks.
Input Block Size (R)	The number of input samples collected before performing a new analysis. Equivalent to the subband decimation factor, so that the subband sampling rate is F_s/R, where F_s is the system's sampling frequency. Can be a power of two from 2 to (L_a/2).
Oversampling Factor (OS)	The degree to which the subband signals are oversampled, defined as OS=N/R.
Window Decimation Factor (DF)	The ratio of the lengths of the analysis and synthesis windows, i.e., DF=L_a/L_s.
Band Stacking	The arrangement of the frequency bands, either *even* or *odd*. Even stacking produces (N/2 - 1) full-width, complex-valued frequency bands and two half-width, real-valued frequency bands for DC and Nyquist. In odd stacking, the bands are offset by 1/2 the nominal full bandwidth, and thus the filterbank produces N/2 full-width, complex-valued frequency bands.
Channel Mode	The processing mode that determines the number of time-domain channels being processed by the WOLA filterbank, either mono or stereo. In *mono mode*, the filterbank analyzes or synthesizes only one channel. In *stereo* mode, the filterbank analyzes or synthesizes two interleaved channels. NOTE: To implement a mono input, stereo output filterbank configuration, use a mono filterbank for analysis, and a stereo filterbank for synthesis. To implement a stereo input, mono output filterbank configuration, use a stereo filterbank for analysis, and a mono filterbank for synthesis.

3.2.1 Analysis and Synthesis

The WOLA filterbank can be divided into an analysis operation and a synthesis operation. The analysis operation splits a time-domain signal into a number of complex-valued subband signals, and the synthesis operation converts these subband signals back to a full-band, time-domain signal. Figure 6 on page 38 shows both operations based on a complex modulator model of the filterbank.

From a signal processing perspective, the analysis operation modulates the full-band signal such that the center frequency of each subband is moved to DC. This center frequency is denoted as w_k in the figure. The modulated signal

is then low-pass filtered by a filter denoted *h(n)*. Finally, the filtered signal is decimated by a factor of R. The synthesis operation performs the inverse operations: first interpolating by a factor of R, then low-pass filtering to remove any images, and finally modulating the subband back to its original frequency position. It is important to note that this view of the filterbank produces N subband signals; however, because the input signal is real, there is complex conjugate symmetry of the input frequency spectrum and approximately half of these signals are simply complex conjugates of other subbands. Thus the WOLA microcode module only generates the N/2+1 or N/2 (for even and odd stacking respectively) unique subband signals.

These operations are done on a block-by-block basis. When the signal is divided into subband signals, each subband signal can be interpreted as a complex-valued time signal representing a band-limited frequency region of the original input signal. Alternatively, the subband samples for one block can be thought of as representing the magnitude and phase of the input signal spectrum for that time block. Thus, the subband signals present a time-frequency view of the input signal.

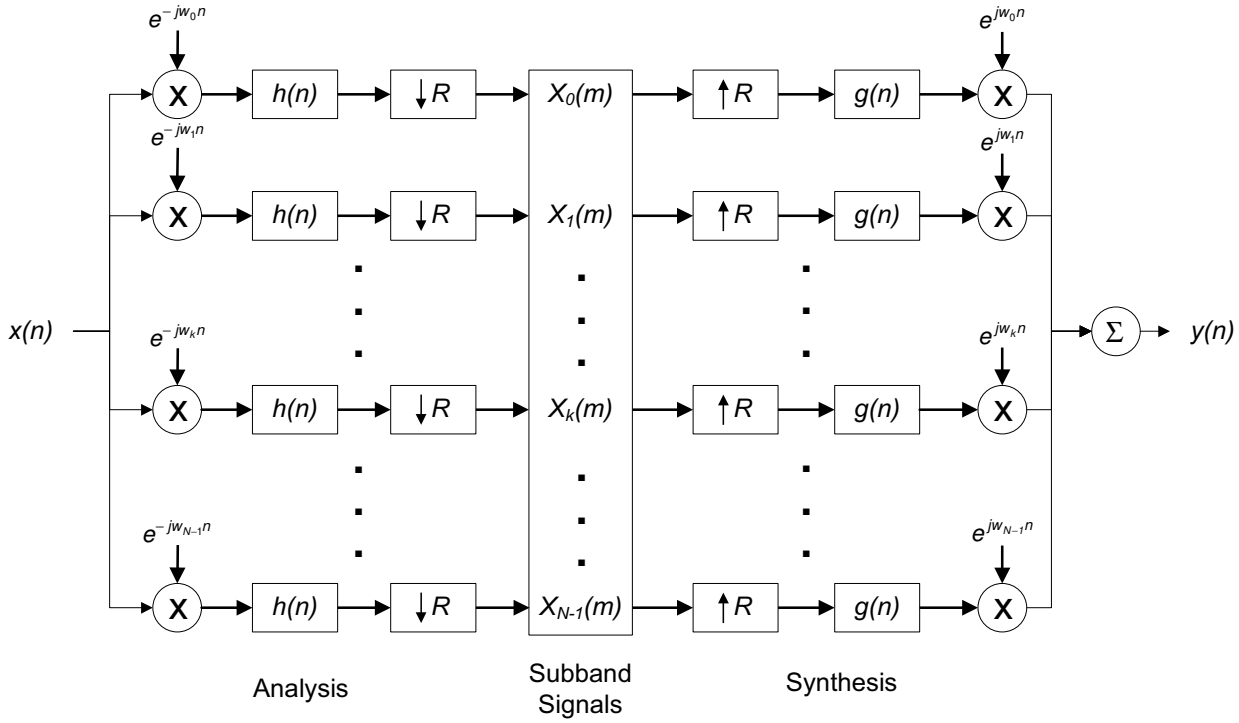


Figure 6. Complex-modulated Filterbank Analysis and Synthesis

The WOLA filterbank on the HEAR accelerator efficiently implements the system described in Figure 6 using a window-and-fold technique for the analysis filtering, an FFT for the modulation and demodulation, and an overlap-add technique for the synthesis filtering. The following sections describe this implementation in greater detail.

3.2.1.1 Analysis Implementation

Figure 7 shows a graphical representation of the operations that occur during the analysis operation. A new block of R samples is shifted into an input buffer of length L_a and then the buffer is windowed with the analysis filter (W). The windowed data is then folded and added to create the polyphase filtered results. A circular shift is applied to these results and then an N-point even or odd FFT is computed. The resulting output is the subband samples for the new block

of R samples. In odd stacking mode, a sign sequencer also modulates the input blocks to ensure that the complex exponential basis functions are continuous.

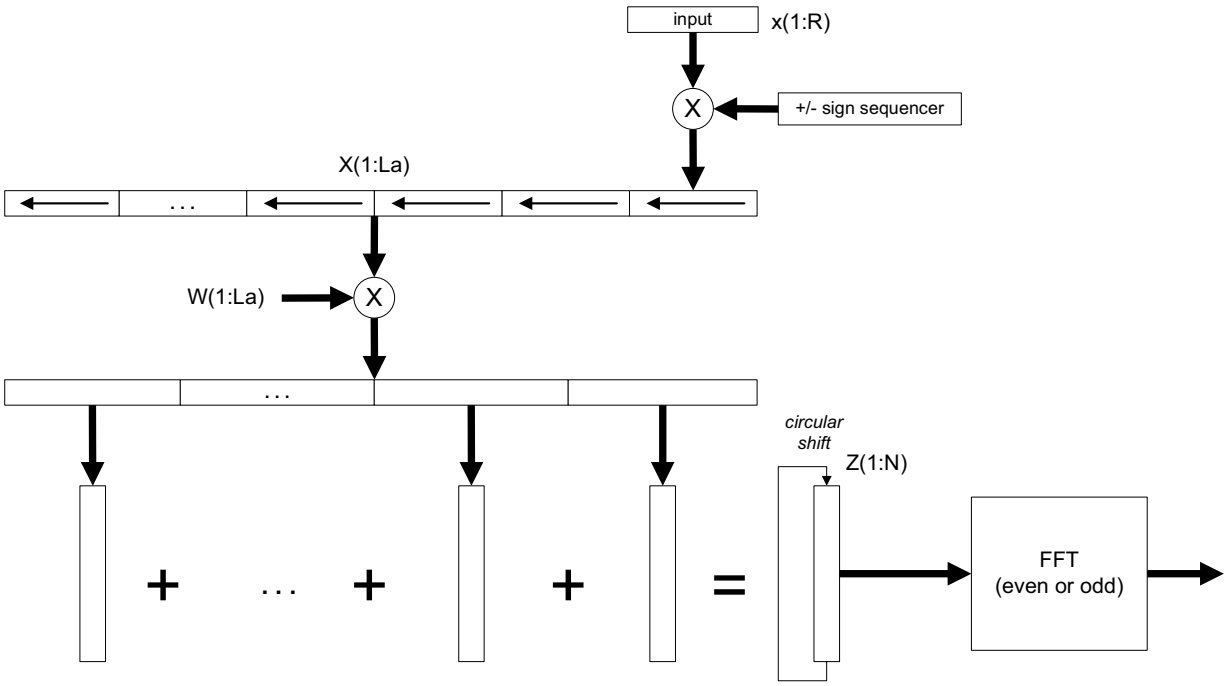


Figure 7. Analysis Implementation

Mono Analysis Results

The results of the analysis operation are stored in the shared memory locations according to the WOLA module configuration. In odd stacking, a complex-valued result is generated for N/2 subbands. In even stacking, N/2-1 complex-valued results are generated for N/2-1 subbands and two real-valued results are generated for the DC and Nyquist bands. These two half-bands are packed into the same complex-pair memory location with the DC portion in the real component and the Nyquist portion in the imaginary component. This format of the output results is illustrated in Figure 8.

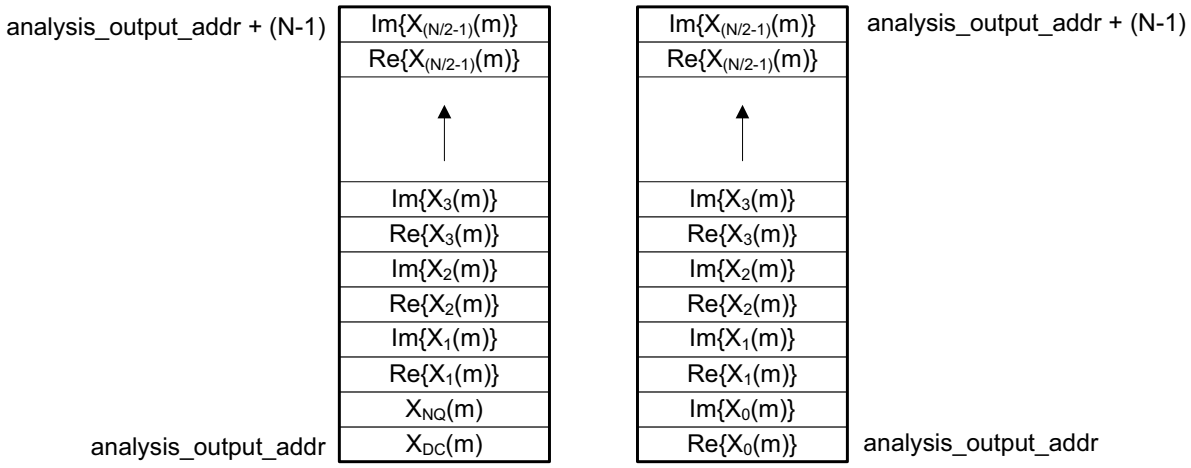


Figure 8. Mono analysis results format. The left array depicts the format for even stacking and the right array depicts the format for odd stacking.

Stereo Analysis Results

Stereo analysis provides a means to efficiently compute the analysis operation for two interleaved input channels. The implementation is similar to that shown in Figure 6, except that the input block now contains interleaved samples from two channels and the effective sizes of all blocks and buffers are doubled. A single complex FFT is done on the interleaved data and then the results for each channel are separated into their independent signals.

The format of the results for each channel is the same as that depicted in Figure 8 with the two channels stacked on top of each other. This data organization is depicted in Figure 9.

IMPORTANT: In Figure 9, channel 0 corresponds to the input signal located at odd addresses in the FIFO, and channel 1 corresponds to the input signal located at even addresses in the FIFO. The first sample in each new input block arriving from the IOC is considered to be channel 0, and this sample is written at the top of the new input block because the IOC fills the input block in a top-down manner. The actual mapping between the channels and the system inputs depends on your IOC and FIFO configuration.

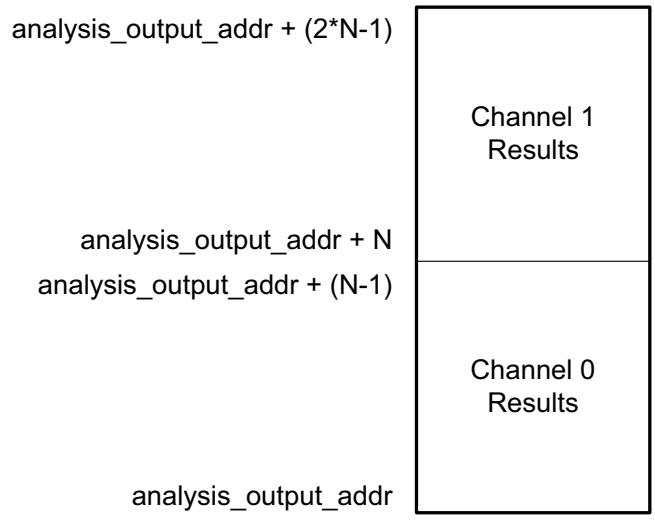


Figure 9. Stereo Analysis Results Format

3.2.1.2 Synthesis Implementation

Figure 10 shows a graphical representation of the operations that occur during synthesis. The subband samples are first processed by an even or odd inverse FFT (IFFT). The IFFT output is circularly shifted, periodically extended and multiplied by the synthesis filter. The output samples are then reconstructed using an overlap-add technique in the output buffer. In odd stacking mode, a block modulation is applied to the output block that is analogous to the one applied during analysis.

The organization of the input data to the synthesis operation is the same as the organization of the analysis results. This applies for both mono and stereo synthesis operations. For example, a mono synthesis operation expects the input data to be organized in the same way as the output of a mono analysis operation, regardless of whether a stereo or mono analysis operation had actually been executed prior to the synthesis operation.

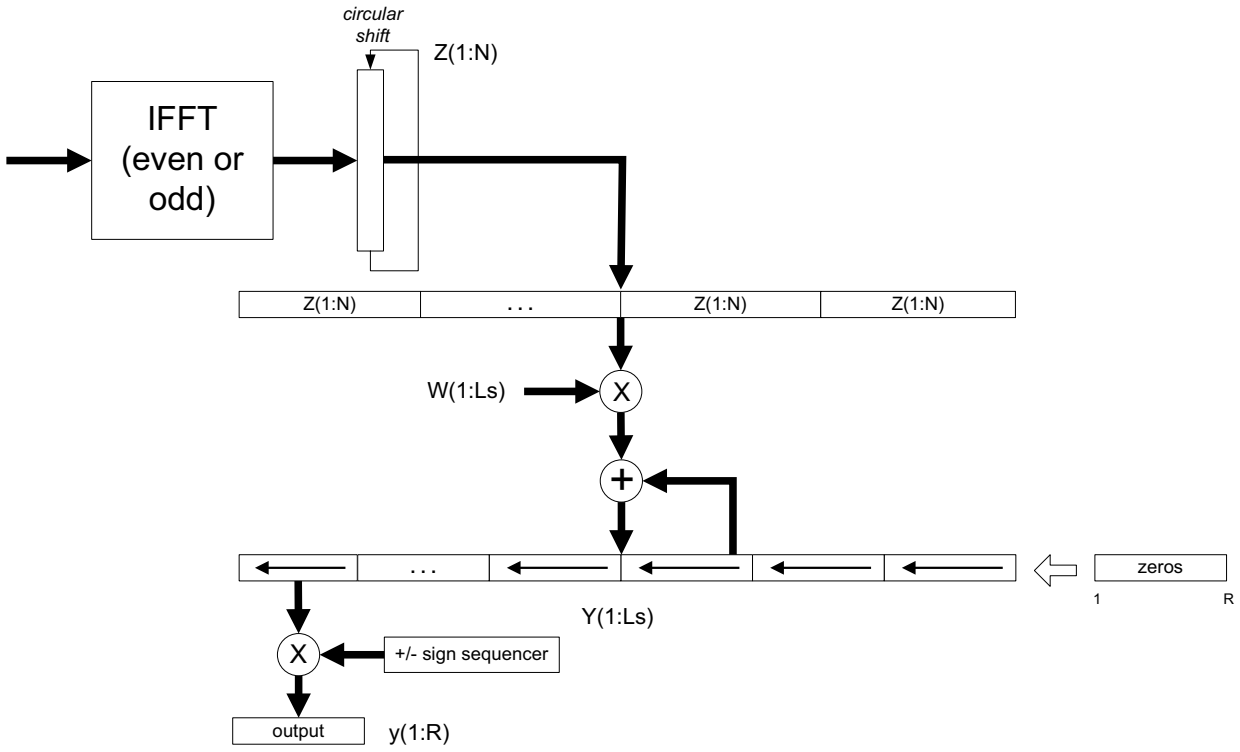


Figure 10. Synthesis Implementation

3.2.1.3 Input and Output Data Types

Input Samples

The input samples for the WOLA analysis operation can be 18-bit or 24-bit obtained either directly from the system's input stage or from some other pre-filterbank processing. If the input sample size is 18-bit, the samples must be stored in the lower 18 bits (i.e., right-aligned) in each 24-bit word. The `WOLA_Analysis` module automatically rescales this data format to the 22-bit block floating point format described below as the first step during analysis.

This rescaling is depicted in Figure 11. For an 18-bit input sample, it is shifted so that two bits are reserved at the top of the 24-bit word, and four bits are reserved at the bottom of the 24-bit word. After this rescaling, the block floating point exponent is initialized to a value of two, effectively meaning that the full-scale in the input is equivalent to full-scale in the block floating point format. However, using this rescaling ensures that there are two bits to accommodate data growth in the first pass of the WOLA analysis, and four bits for the accumulation of noise from rounding throughout the WOLA analysis.

A 24-bit input sample is shifted so that two bits are reserved at the top of the 24-bit word, and the two least significant bits in the raw data no longer exist. Using this rescaling ensures that there are two bits to accommodate data growth in the first pass of the WOLA analysis, while the resolution is reduced by 2 bits.

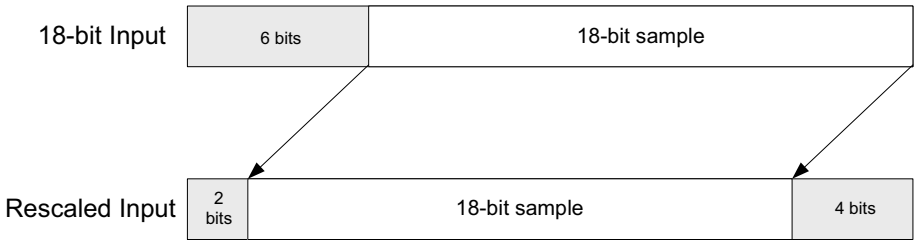


Figure 11. WOLA Analysis Input Rescaling

Block Floating Point Subband Samples

The subband samples which are the output from the analysis operation and input to the synthesis operation are stored in a block floating point datatype. The `WOLA_Analysis and WOLA_Synthesis` modules take advantage of the HEAR's block floating point support to automatically rescale the data during each step of processing to avoid saturation during analysis or synthesis. The `WOLA_Analysis and WOLA_Synthesis` modules support the 22-bit block floating point datatype, which treats the top two bits of each 24-bit word as extension bits. The BFP data is rescaled as necessary when the computation overflows into these extension bits.

Note that because of this full support for the block floating point datatype, the state of the HEAR's block floating point register (`D_HEAR_BFP`) must be properly saved and restored if multiple `WOLA_Analysis and WOLA_Synthesis` modules are used in an application's microcode configuration. For more information about handling this situation, see Section 2.5.3, "Block Floating Point".

Output Samples

The `WOLA_Synthesis` module automatically rescales the block floating point data used during synthesis to 18-bit or 24-bit samples before the overlap-add portion of the synthesis operation. If the output sample size is 18-bit, these samples are right-justified in each 24-bit word, and the output samples of either size can be used as input to the system's output stage via the IOC.

3.2.1.4 Analysis FIFO Configuration

The analysis operation is designed to work with a FIFO as its input (i.e. the `input_fifo_addr` parameter normally points to an appropriately configured input FIFO). The layout of a typical input FIFO for the analysis operation in a mono filterbank is shown in Figure 12. The first value in the input data must always be the most recent sample and past samples must be located at ascending addresses. Assuming a mono input signal, the first *R* samples in the buffer represent the current input block, and the remaining L_a - *R* samples represent the past input blocks. For a stereo input FIFO, the lengths involved are $2L_a$ and *2R*. A summary of the typical input FIFO configuration parameters for both mono and stereo filterbanks is given in Table 8.

IMPORTANT: The `WOLA_Analysis` module requires that the input FIFO be located in A0 or A1 memory.

Table 8. Typical WOLA Analysis FIFO Configuration Values

FIFO Parameter	Mono Filterbank	Stereo Filterbank
Minimum FIFO Size	L_a+*R*	$2L_a$+*2R*
Block Size	*R*	*2R*

Table 8. Typical WOLA Analysis FIFO Configuration Values

FIFO Parameter	Mono Filterbank	Stereo Filterbank
Base Pointer	0	0
IO Block Pointer	0	0

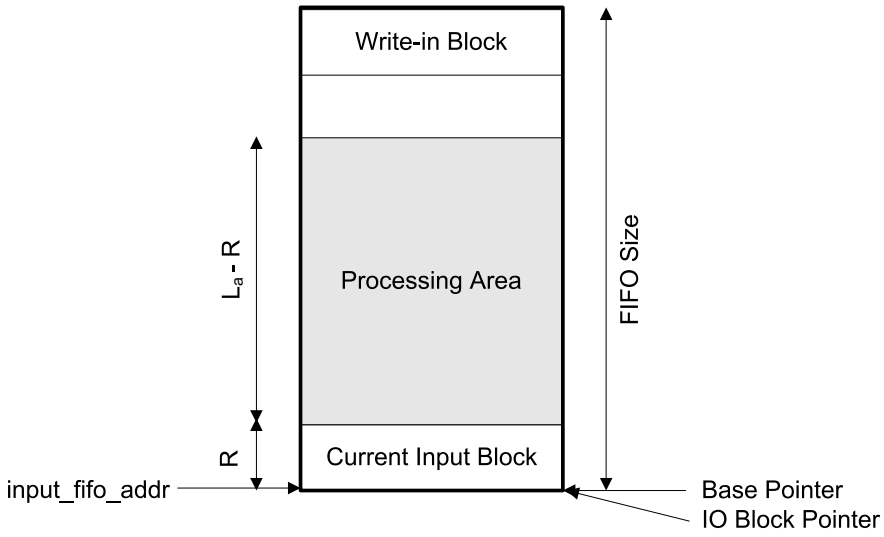


Figure 12. Analysis FIFO configuration for mono input

Analysis Window

The analysis window is closely tied to the input FIFO. For efficiency reasons, the analysis window is always located in C0 or C1 memory and must be located at the same offset into C0/C1 memory as the input FIFO's offset into A0/A1 memory. This configuration results in the input FIFO and analysis window being interleaved in 48-bit double words when accessed through AC0/AC1 memory by the HEAR. For more information about specifying the synthesis window, see Section 3.2.1.6, "Analysis and Synthesis Windows".

3.2.1.5 Synthesis FIFO Configuration

The synthesis operation is designed to work with a FIFO for its output (i.e. the `output_fifo_addr` parameter normally points to an appropriately configured output FIFO). Because of the overlap-add nature of the synthesis operation, the output buffer actually contains partially reconstructed samples, and these partial samples are also an input to the synthesis operation. The layout of a typical output FIFO for the synthesis operation in a mono filterbank is shown in Figure 13. For a mono output signal, the synthesis buffer is L_s samples in size, and after synthesis, the R samples at the highest addresses of the buffer contain the newest output block. For a stereo output FIFO, the sizes involved are $2L_s$ and $2R$. A summary of the typical output FIFO configuration parameters for both mono and stereo filterbanks is given in Table 9.

IMPORTANT: The `WOLA_Synthesis` module requires that the output FIFO be located in B0 or B1 memory.

Table 9. Typical WOLA Synthesis FIFO Configuration Values

FIFO Parameter	Mono Filterbank	Stereo Filterbank
Minimum FIFO Size	L_S+*R*	$2L_S$+*2R*
Block Size	*R*	*2R*
Base Pointer	0	0
IO Block Pointer	L_S+*R*	$2L_S$+*2R*

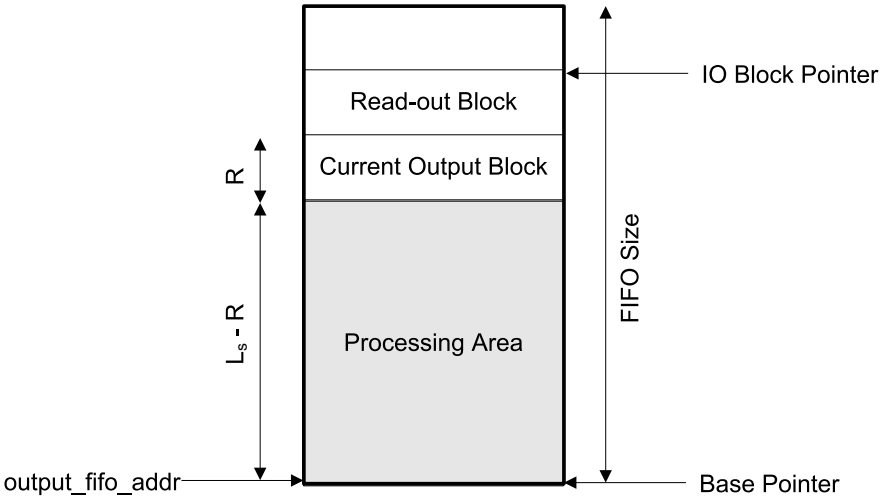


Figure 13. Synthesis FIFO configuration for mono output

Synthesis Window

The synthesis window is closely tied to the output FIFO. For efficiency reasons, the synthesis window is always located in D0 or D1 memory and must be located at the same offset into D0/D1 memory as the output FIFO's offset into B0/B1 memory. This configuration results in the output FIFO and synthesis window being interleaved in 48-bit double words when accessed through BD0/BD1 memory by the HEAR. For more information about specifying the synthesis window, see Section 3.2.1.6, "Analysis and Synthesis Windows".

3.2.1.6 Analysis and Synthesis Windows

The `WOLA_Analysis and WOLA_Synthesis` modules support three different methods of specifying the analysis and synthesis windows: *default*, *optimized* or *custom* windows. In the case of default or optimized windows, the `WOLA_Analysis or WOLA_Synthesis` module automatically puts the corresponding analysis or synthesis window into C and D memory respectively. The windows will be included as static data in the HCT object file output. More information about each type of window and how to specify custom windows is given below.

IMPORTANT: The `WOLA_Analysis and WOLA_Synthesis` modules allow you to independently specify the window mode for the analysis and synthesis windows. However, the default and optimized windows are designed as analysis-synthesis pairs. This means that you should not mix default windows with optimized windows. For example, do *not* specify an optimized analysis window and then use a default synthesis window to synthesize the filterbank outputs.

Default Windows

The default windows are created using a windowed sinc filter design method. This method provides reasonably good frequency selectivity, good stopband attenuation and small passband ripple for all configurations except DF=1. Default windows are provided for every valid WOLA filterbank configuration.

Optimized Windows

The optimized windows which are included in the `WOLA_Analysis and WOLA_Synthesis` modules provide improved stopband attenuation within each subband while guaranteeing no more than 3 dB of passband ripple in the overall filterbank frequency response. Most of these windows provide significantly better ripple than this, typically less than 0.1 dB. These windows are designed using a constrained optimization approach.

Optimized windows are provided for most WOLA filterbank configurations. Although these windows generally provide improved performance over the default windows, it is important to carefully assess the audio and algorithm performance before deciding which windows to use in a particular application.

A complete list of WOLA filterbank configurations with optimized windows is given in Appendix A, “Optimized WOLA Windows” on page 339.

Custom Windows

If you opt to use a custom window, you must specifically put the window at the correct location in C0/C1 or D0/D1 memory via the CFX software or some other method; the HCT will not place the window in memory for you. For example, if the input FIFO starts at address 0xC100, then its offset in A0 memory is 0x100. Thus, the analysis window must be placed at offset 0x100 into C0 memory, or address 0xE100. Similarly, if the output FIFO starts at address 0xDA00, then its offset in B1 memory is 0x200 and the synthesis window must be placed at offset 0x200 in D1 memory, or address 0xFA00. This arrangement of FIFOs and analysis and synthesis windows is depicted for the general case in Figure 14, where the FIFO offset and window offset must be equal for each FIFO/window pair.

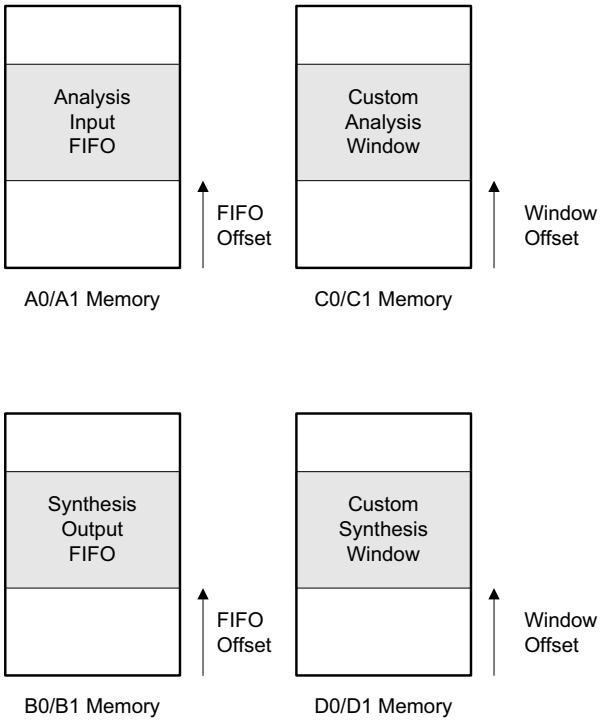


Figure 14. Custom Analysis and Synthesis Window Positioning

Both the analysis and synthesis windows are stored in an impulse response reversed order. That is, if the analysis filter is described by a filter impulse response $h(n)$, $n = 0...L_a-1$, then the analysis window must be stored in memory in the order $h(L_a-1), h(L_a-2), ..., h(0)$. If the synthesis window is described by a filter impulse response $g(n)$, $n=0...L_s-1$, then the synthesis window must be stored in memory in the order $g(L_s-1), g(L_s-2), ..., g(0)$.

For stereo operations, the windows for each channel must be interleaved in the same fashion as the input samples.

3.2.1.7 Initialization

The `WOLA_Analysis and WOLA_Synthesis` modules include reset/initialization functionality which can reset the internal state to the default values. When the microcode is configured and loaded from non-volatile memory, this state is already set to the default values, so normally this reset does not need to execute at system startup. It may need to be executed as part of any soft reset functionality for the application.

3.2.1.8 State Variables

The `WOLA_Analysis` module exposes one state variable, `<name>_i_shift`, for the end user. This variable keeps track of the circular shift during analysis.

The `WOLA_Synthesis` module also exposes one state variable, `<name>_o_shift`, for the end user. This variable keeps track of the circular shift during synthesis.

If an analysis or synthesis has to be done repeatedly without updating the state, you must save and restore the values between calls to the analysis or synthesis function. Arbitrarily changing the value of these variables can cause the WOLA filterbank to no longer function correctly.

IMPORTANT: Improper modification of the circular shift state variables can cause the WOLA filterbank to not work properly. Modification of these variables should only be considered by advanced users.

3.2.2 Gain Application

The gain application modules, `Gain_R` and `Gain_C`, are used to multiply the analysis outputs by a set of real or complex-valued gains. The gain application modules are compatible with any valid WOLA configuration, including even and odd stacking modes, as well as mono and stereo channel modes. For the mono channel mode, the gain application is mono. For the stereo channel modes, the gain application is stereo.

Both gain application modules export a state variable which acts as a gain exponent. This gain exponent is the number of left shifts that will be applied to the data after multiplication with the fractional gain. This sequence of applying both the individual gains and the bulk gain from the gain exponent is shown in Figure 15. Since it is exported as a state variable, the gain exponent is stored as a 24-bit signed value in shared memory. However, be careful when applying this exponent since it can cause the subband samples to saturate if both of the following are true:

- They are close to full scale.
- The number of left shifts exceeds the number of extension bits in the block floating point datatype.

Therefore, the suggested maximum value for this exponent is 2, which is the number of extension bits in the WOLA filterbank module's block floating point format. If required, you can apply additional digital gain to subband signals by directly manipulating the block exponent.

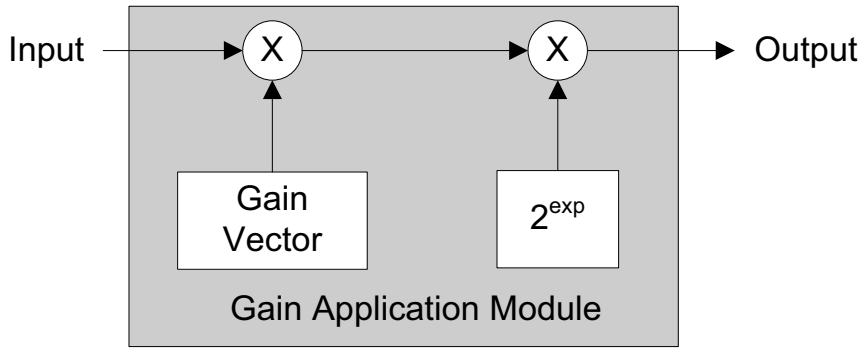


Figure 15. Gain Vector Application and Gain Exponent Scaling

The input and output formats of the gain application modules are identical to the analysis results formats described in Section 3.2.1.1, "Analysis Implementation". This includes making use of the block floating point datatype to rescale the output data when the gain application causes the data to exceed the fractional range of the BFP datatype.

The format of the gains and how they are applied is described in the sections that follow.

3.2.2.1 Real Gains

The real gain application multiplies each complex subband sample with a fractional real-valued gain factor. For even stacking, separate gain values for DC and Nyquist can be specified. Figure 16 illustrates the gain vector organization for both even and odd stacking for a single set of mono gains. For stereo gain applications, the gain vector is formed by stacking two mono gain vectors on top of each other in memory, similar to organization of the stereo analysis results as shown in Figure 9 on page 41.

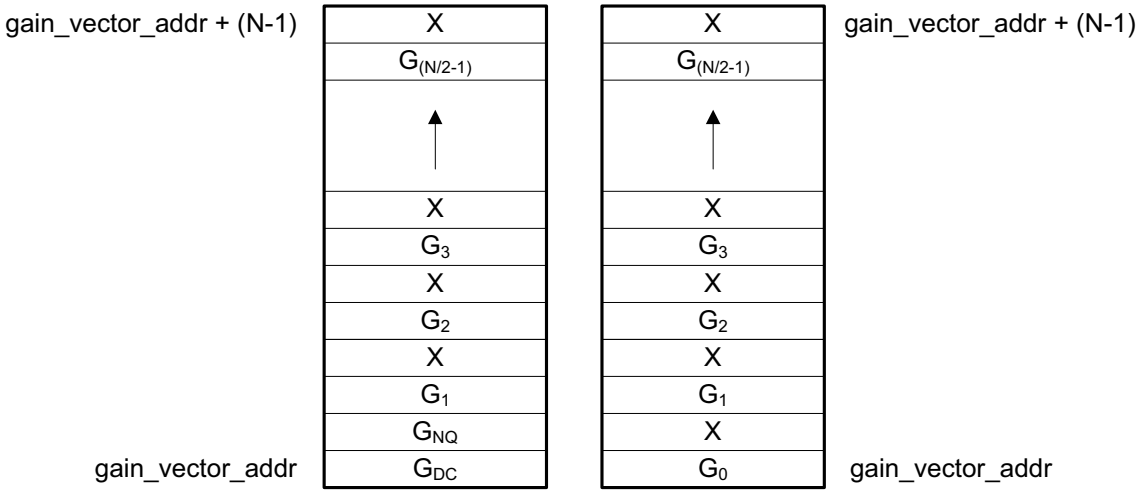


Figure 16. Real gain vector formats. The left array depicts the format for even stacking and the right array depicts the format for odd stacking.

3.2.2.2 Complex Gains

The complex gain application multiplies each complex subband sample with a fractional complex-valued gain factor. For even stacking, separate real-valued gain values for DC and Nyquist can be specified. Figure 17 illustrates the gain vector organization for both even and odd stacking for a single set of mono gains. For stereo gain applications, the gain vector is formed by stacking two mono gain vectors on top of each other in memory, similar to organization of the stereo analysis results as shown in Figure 9 on page 41.

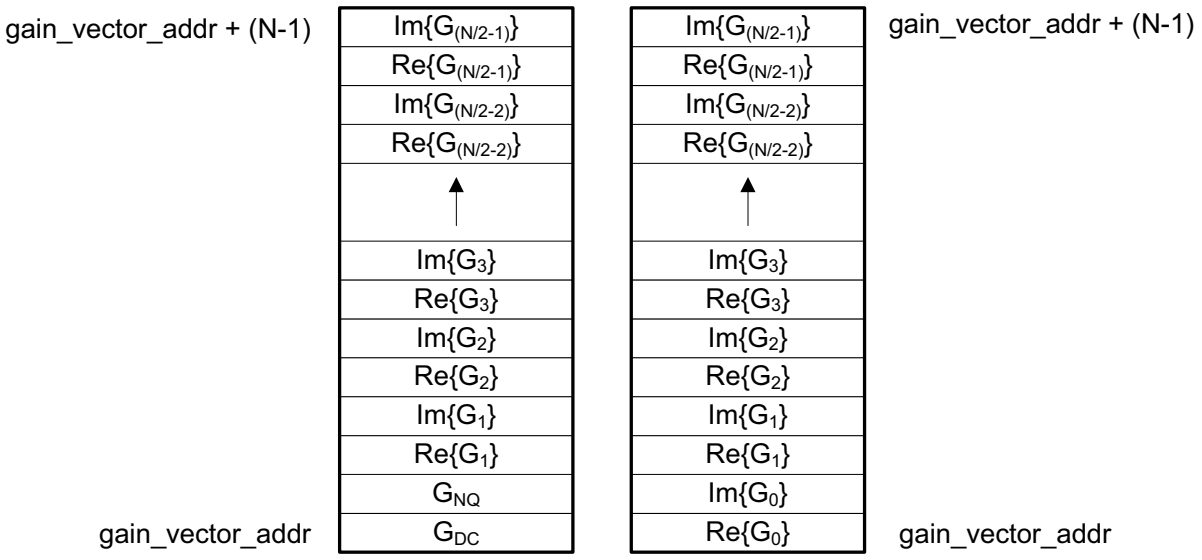


Figure 17. Complex gain vector formats. The left array depicts the format for even stacking and the right array depicts the format for odd stacking.

3.2.3 Subband Channel Mixing

The `Mix_Channels` module is used to down-mix a set of stereo subband samples to a set of mono subband samples suitable for input to a mono synthesis operation. It is designed to be used with a stereo WOLA analysis, and mono WOLA synthesis configuration. The down-mixing operation is conceptually shown in Figure 18 and is the same for both even and odd stacking. The input and output formats are identical to the standard analysis results formats described in Section 3.2.1.1, “Analysis Implementation”.

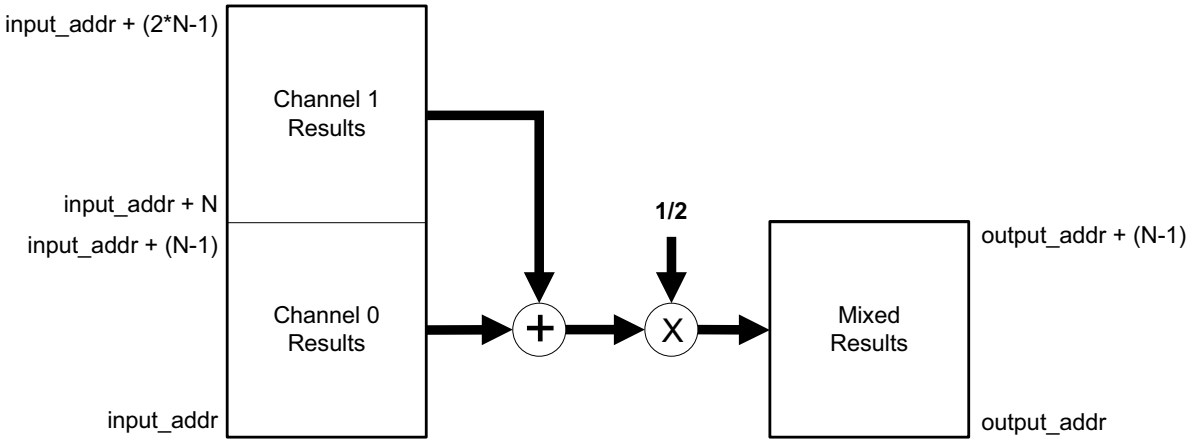


Figure 18. Channel Mixing Operation

3.2.4 Subband Energy Calculation

The `Energy` module is used to compute the instantaneous energy in each subband by computing the squared magnitude of each subband sample. The `Energy` module differs from the basic vector squared-magnitude module in

that it is designed to work on input data that is organized in the same way as the results of `WOLA_Analysis`. This is primarily an issue for even stacking results, where the DC and Nyquist bands must be handled separately from the other complex-valued bands. For a complex subband value $X_k(m)$, the energy, E_k, is defined as follows:

$$E_k = \left|X_k(m)\right|^2 = Re\{X_k(m)\}^2 + Im\{X_k(m)\}^2$$

The squaring operation of this calculation involves the multiplication of two 22-bit block floating point values. The result is stored in two consecutive words as a 48-bit result with 2 integer bits and 46 fractional bits. Because of the squaring operation, the effective block exponent of these double-precision results is double the block exponent of the original analysis results. The HEAR does not have support for double-precision block floating data, so this exponent is not actually stored in the block floating point register and the `Energy` module does not modify the BFP register in any way.

The organization of the 48-bit results for a set of mono analysis results is depicted in Figure 19. Note that for even stacking the results require two additional words in memory because of the need to store separate 48-bit results for both the DC and Nyquist bands. For stereo results, the data format is simply two sets of mono results stacked on top of each other in memory, similar to the organization of the stereo analysis results as shown in Figure 9 on page 41.

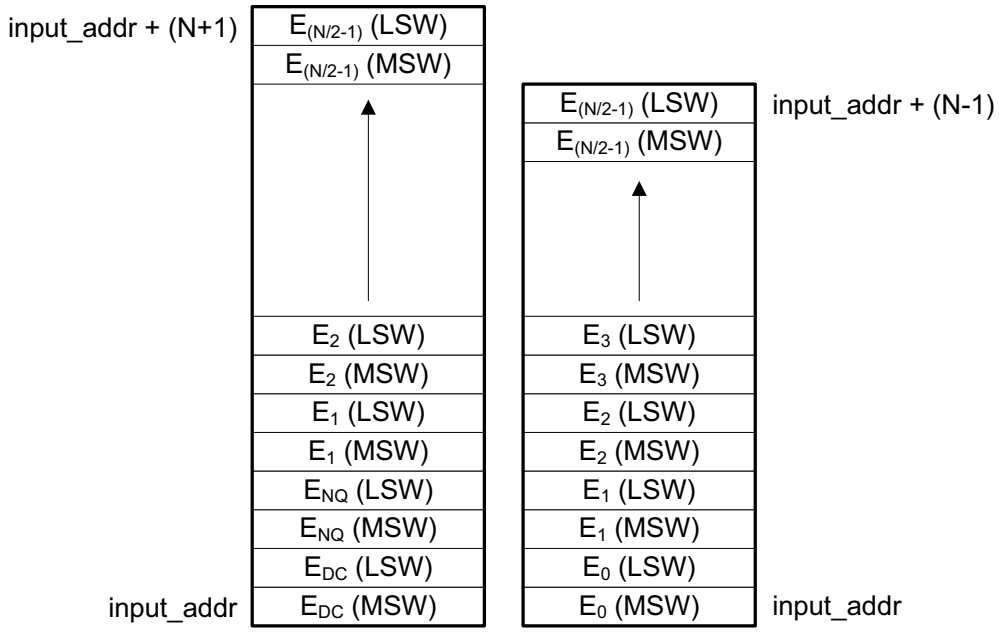


Figure 19. Energy calculation results format. The left array depicts the format for even stacking and the right array depicts the format for odd stacking. MSW refers to the most significant 24-bit word and LSW refers to the least significant 24-bit word for a 48-bit double-precision value.

3.2.5 Subband Normalization

The `Normalize` module is used to normalize a set of subband samples to a fixed block exponent, called the normalized block exponent. This can be useful when performing computations with subband samples across multiple time blocks, such as for subband filtering. The normalized block exponent should always be selected such that it is equal to or greater than the largest block exponent that is expected during normal operation of the signal processing

application. Otherwise, saturation of the subband samples may occur when they are normalized to the smaller fixed block exponent. The net number of right shifts that are performed on the data is determined according to the following formula:

```
num_shifts = new_block_exp - block_exp
```

It is clear from this formula that if `block_exp` > `new_block_exp`, then the number of right shifts is actually negative, indicating a left shift and leading to potential saturation if the samples are near full-scale.

The `Normalize` module operates on a number of 24-bit block floating point words specified by the `K` parameter. For normalizing a set of mono-mode analysis results, set this parameter to the value of `N` used in the associated `WOLA_Analysis` module configuration. For normalizing a set of stereo-mode analysis results, set this parameter to two times `N`.

The `Normalize` module can also optionally update the `D_HEAR_BFP` register to the new normalized block exponent, although the need to do this will depend on the particular application.

3.2.6 WOLA Filterbank Performance

The `WOLA_Analysis and WOLA_Synthesis` configuration parameters can affect a number of performance characteristics of the filterbank:

- The number of subbands and consequently the width of each subband (by varying N)
- The center frequency of each band (using even or odd stacking)
- The level of aliasing distortion in each subband (by varying L_a, L_s and OS)
- The block rate and the number of cycles available per block (by varying R)
- Group delay of the filterbank (by varying L_a, L_s and R)
- Relative power consumption

A number of these important performance characteristics are described in the sections that follow.

3.2.6.1 Block Rate

The block rate (sometimes called the *tick* rate) is the time it takes to completely fill up one input block of samples. It is a function of the input sampling frequency and the `WOLA_Analysis` module's block size. Specifically, the block rate for the WOLA filterbank processing is equal to R/F_s where F_s is the system sampling rate. The block rate will normally be tied to the rate of FIFO interrupts that are generated by the FIFO controller, assuming that one or more FIFOs have been configured with a block size equal to the WOLA module's input block size (*R* for mono FIFOs, or *2R* for stereo FIFOs).

The number of cycles available for processing between input blocks is equal to the block rate multiplied by the system clock frequency. The `WOLA_Synthesis` module operation and any other post-synthesis processing must normally be completed within the amount of time equal to the block rate and before the next input block is ready, so that the next block of output samples can be made available in time for the output stage.

3.2.6.2 Group Delay

The WOLA filterbank has the following sources of delay which contribute to the overall group delay of the filterbank:

- The block pipelining
- The analysis window

- The synthesis window

The analysis and synthesis windows are essentially finite-impulse response (FIR) filters and each filter introduces a delay equal to half of its respective filter length. Thus, the analysis window delay is $L_a/2$ samples and the synthesis window delay is $L_s/2$ samples. There is an additional delay of R samples due to the collection of the input block. This delay of R samples is actually incurred by the FIFO data collection process and not the WOLA filterbank processing itself. Assuming that this delay is accounted for, the total group delay through a WOLA filterbank system is found with the following formula:

$$\text{GroupDelay(samples)} = \frac{L_a}{2} + \frac{L_s}{2} + R$$

This result can be divided by the system sampling frequency to determine the group delay in seconds. Taking into account this formula, the group delay can be decreased by one or more of the following changes to the filterbank configuration:

- Reducing the analysis window length (L_a)
- Reducing the synthesis window length (L_s) or, analogously, increasing DF ($DF=L_a/L_s$)
- Decreasing the input block size (R) or, analogously, increasing the oversampling factor ($OS=N/R$)
- Increasing the system sampling frequency (F_s)

3.2.6.3 Frequency Response

Two types of frequency response are of interest in determining the performance of the WOLA filterbank. The first is the overall frequency response through the analysis and synthesis operations without any additional processing. This represents the overall transfer function of the filterbank. Nominally, this response would be flat with unity gain. The second frequency response of interest is the frequency response of the modulated filter used to isolate a single subband, sometimes called the band frequency response. The ideal band response is a perfect brick-wall filter around the bandwidth of interest.

The degree of flatness for the overall frequency response is generally a function of the analysis and synthesis window design. The default windows included with the WOLA module attempt to keep any passband ripple to less than 1 dB. However this is not guaranteed for all configurations. The passband gain is equal to the ratio OS/DF. This represents inherent filterbank gain due to the oversampling which occurs in each subband.

The band response determines how much leakage there is between bands and the overall level of aliasing or imaging distortion present in the filterbank. Generally, this response can be improved by any of the following changes to the filterbank configuration:

- Increasing the analysis filter length (L_a)
- Increasing the synthesis filter length (L_s)
- Increasing the oversampling factor ($OS=N/R$)

3.2.6.4 Power Consumption

The relative power consumption of the WOLA filterbank processing depends on how much computation must be done for each filterbank operation (primarily analysis and synthesis) and also how often these operations must be performed. The latter is a function of the block rate and hence both the input block size (R) and the system sampling

frequency can affect power consumption. Assuming everything else remains constant, then the relative power consumption can always be decreased by increasing R and hence reducing the oversampling factor.

The amount of computation required is generally proportional to L_a, L_s and N. Assuming N is kept constant, the relative power consumption can be decreased by decreasing the analysis and/or synthesis window lengths.

3.3 TRANSFORMS

The transform modules implement block signal processing transforms. This includes DFT operations for both real and complex valued inputs. The modules available are summarized in Table 10.

Table 10. Transform Modules

Module Name	Description	References
DFT_C_Forward	Implements a Discrete Fourier Transform for complex-valued inputs.	Section 3.3.1, “Discrete Fourier Transforms” Section 4.10, “DFT_C_Forward”
DFT_C_Inverse	Implements an Inverse Discrete Fourier Transform for complex-valued inputs.	Section 3.3.1, “Discrete Fourier Transforms” Section 4.11, “DFT_C_Inverse”
DFT_R_Forward	Implements a Discrete Fourier Transform for real-valued inputs.	Section 3.3.1, “Discrete Fourier Transforms” Section 4.12, “DFT_R_Forward”
DFT_R_Forward	Implements an Inverse Discrete Fourier Transform for real-valued inputs.	Section 3.3.1, “Discrete Fourier Transforms” Section 4.13, “DFT_R_Inverse”
Win_DFT_R_Forward	Implements a Discrete Fourier Transform for real-valued inputs with windowing of the forward DFT inputs.	Section 3.3.1, “Discrete Fourier Transforms” Section 4.14, “Win_DFT_R_Forward”
Win_DFT_R_Inverse	Implements an Inverse Discrete Fourier Transform for real-valued outputs with windowing of the inverse DFT outputs.	Section 3.3.1, “Discrete Fourier Transforms” Section 4.15, “Win_DFT_R_Inverse”

3.3.1 Discrete Fourier Transforms

The DFT_C_Forward and DFT_C_Inverse modules implement the forward and inverse Fourier transforms for complex valued inputs, and DFT_R_Forward and DFT_R_Inverse modules implement the forward and inverse Fourier transforms for real valued inputs. Both real and complex transforms are implemented using the FFT algorithm.

The *K*-point forward DFT of a sequence, *x(n)*, is defined as follows, for *k = 0, ..., K-1*:

$$X(k) = \sum_{n=0}^{K-1} x(n) e^{-j\frac{2\pi}{K}kn}$$

Similarly, the *K*-point inverse DFT of a sequence, *X(k)*, is defined as follows, for *n = 0, ..., K-1*:

$$x(n) = \frac{1}{K}\sum_{k=0}^{K-1} X(k)e^{j\frac{2\pi}{K}kn}$$

In some cases, it is desirable to window the forward DFT input and/or the inverse DFT output. We can define a windowed DFT as follows, where $w_1(n)$ is a K-point forward DFT window and $w_2(n)$ is a K-point inverse DFT window:

$$X(k) = \sum_{n=0}^{K-1} [w_1(n)x(n)]e^{-j\frac{2\pi}{K}kn}$$

$$x(n) = w_2(n)\left[\frac{1}{K}\sum_{k=0}^{K-1} X(k)e^{j\frac{2\pi}{K}kn}\right]$$

The individual forward/inverse DFT module pairs are classified based on whether the time domain sequence, $x(n)$, is complex or real-valued. Both real and complex forward transform modules take signed fractional inputs, and both real and complex inverse transform modules produce signed fractional outputs. The intermediate frequency-domain sequence, $X(k)$, uses the 22-bit block floating point datatype. The block floating point unit is used throughout the DFT modules to avoid overflow and saturation.

Note that because of this full support for the block floating point datatype, the state of the HEAR's block floating point register must be properly saved and restored if the DFT modules are combined with other modules that also make use of the BFP unit. For more information about handling this situation, see Section 2.5.3, "Block Floating Point".

3.3.1.1 Complex Input DFT

The complex input DFT takes a complex-valued input to the forward transform, i.e., $x(n)$ is a K-point complex sequence. This produces K unique complex-frequency samples. The input and output formats for the forward complex DFT module (`DFT_C_Forward`) are shown in Figure 20. The input and output formats for the inverse complex DFT module (`DFT_C_Inverse`) are simply reversed relative to the formats for the forward transform module. In other words, the input format for the inverse module is the same as the output format for the forward module, and the output format for the inverse module is the same as the input format for the forward module.

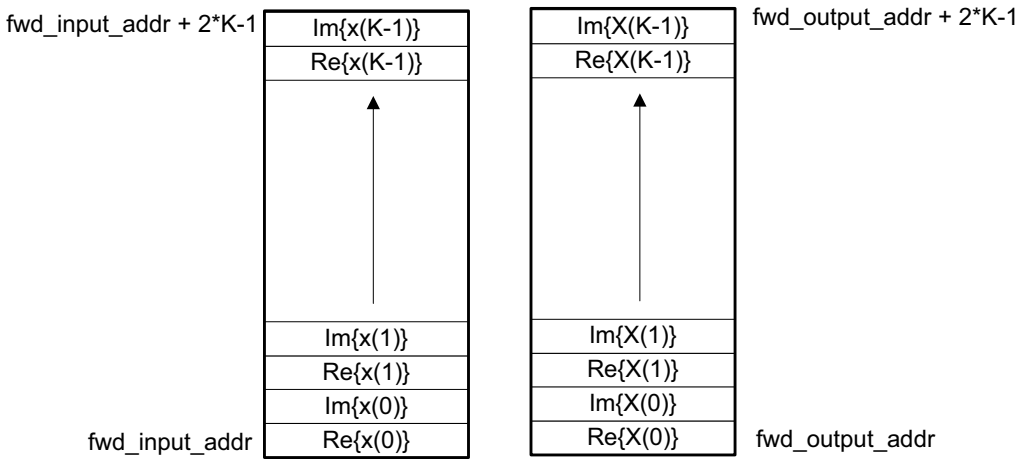


Figure 20. Complex-valued forward DFT input (left) and output (right) formats. The input and output formats are reversed for the inverse DFT module.

3.3.1.2 Real Input DFT

The real input DFT takes a real-valued input to the forward transform, i.e., *x(n)* is a *K*-point real sequence. This produces *K/2 + 1* unique complex frequency samples. The remaining complex frequency samples are simply complex conjugates of the other samples. The frequency domain samples *X(0)* and *X(K/2)* are always real-valued and are packed into the first two words of the frequency domain output. The input and output formats for the forward complex DFT module (`DFT_R_Forward`) are shown in Figure 21. Similar to the complex DFT modules, the input and output formats for the inverse real DFT module (`DFT_R_Inverse`) are simply reversed relative to the formats for the forward transform module.

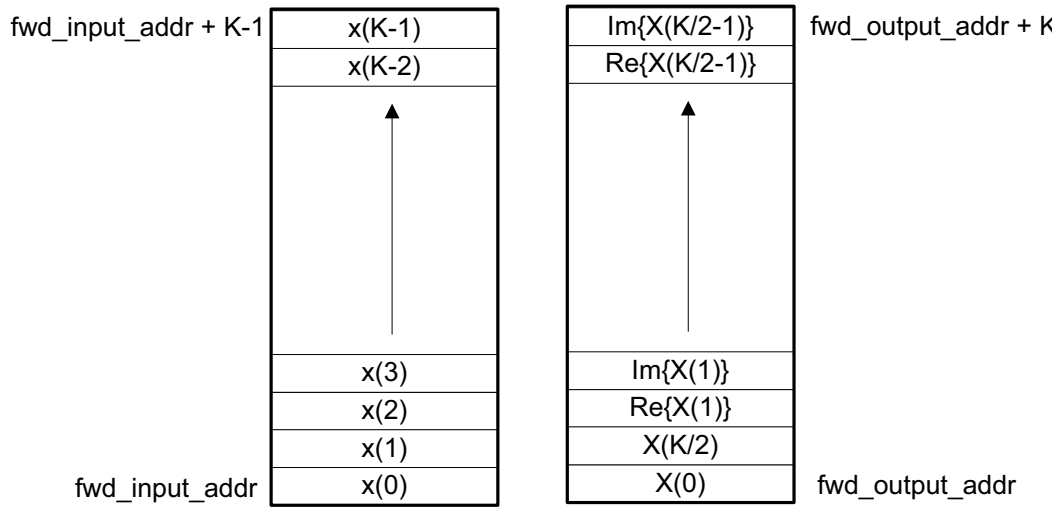


Figure 21. Real valued forward DFT input (left) and output (right) formats. The input and output formats are reversed for the inverse DFT module.

3.3.1.3 Windowed DFT

The `Win_DFT_R_Forward and Win_DFT_R_Inverse` modules implement real-valued forward and inverse DFT with windowing. The same or different windows can be used for each operation. All window coefficients are specified as 24-bit fractional values and are organized in memory in the same order as the input sequence.

3.4 Block Floating Point Support

The `BFP_State` module has two functions, `BFP_Save` and `BFP_Restore`, to facilitate saving and restoring the current block floating point state. See Section 2.5.3.3, “BFP Unit Sharing” on page 30 for more information.

3.5 Filters

The filter modules implement a set of time-domain, subband-domain and averaging filters useful in a variety of signal processing applications. The modules available are summarized in Table 11. The time-domain based filtering operations work on a block-by-block basis and the subband based filtering operations work on a sample-by-sample basis, but across multiple subbands or channels with each execution. In general, the filtering modules do not manage or maintain their own filtering buffers. Each module’s description outlines the formats you must provide for the input and output buffers.

Table 11. Filter Modules

Module Name	Description	References
`FIR_R`	Implements a real-valued finite impulse response (FIR) filter suitable for time-domain signal processing, using single-precision (24-bit) or double-precision (48-bit) coefficients	Section 3.5.1, “Time-Domain FIR Filter” Section 4.17, “FIR_R”
`IIR_R`	Implements a real-valued infinite impulse response (IIR) filter suitable for time-domain signal processing, using single-precision (24-bit) or double-precision (48-bit) output samples	Section 3.5.2, “Time-Domain IIR Filter” Section 4.18, “IIR_R”
`Biquad_IIR`	Implements a real-valued infinite impulse response (IIR) filter composed of cascaded biquad (second order section) filters	Section 3.5.3, “Biquad IIR Filter” Section 4.19, “Biquad_IIR”
`sFIR_C`	Implements a complex-valued FIR filter suitable for subband-domain signal processing	Section 3.5.4, “Subband FIR Filter” Section 4.20, “sFIR_C”
`sIIR_C`	Implements a complex-valued IIR filter suitable for subband-domain signal processing	Section 3.5.5, “Subband IIR Filter” Section 4.21, “sIIR_C”
`Attack_Release`	Implements a multi-channel first-order attack-release averaging filter useful in multi-channel dynamic range compression systems	Section 3.5.6, “Attack-Release Averaging” Section 4.16, “Attack_Release”
`Exp_Average`	Implements a multi-channel first-order IIR filter useful in exponential averaging of values such as signal energy calculations	Section 3.5.7, “Exponential Averaging” Section 4.22, “Exp_Average”

3.5.1 Time-Domain FIR Filter

The `FIR_R` module implements block-based FIR filtering of real-valued inputs with real-valued filter coefficients. The expression for FIR filtering is given below using the following notation:

- $x(n)$ is the input signal
- $y(n)$ is the output signal
- $h(n)$ is the FIR filter coefficients
- L is the length of the filter

- A_{sf} is a scaling factor designed to allow for rescaling of the filter coefficients

$$y(n) = 2^{A_{sf}}\left[\sum_{k=0}^{L-1} x(n-k)h(k)\right]$$

The module works on a block-by-block basis. A new block of *R* input samples is first collected and then a new block of *R* output samples is computed. The module supports interleaved input and output data, so for example, one channel of an interleaved stereo FIFO can be filtered or the output can be placed in one channel of an interleaved stereo output FIFO. This interleaved input and output is depicted in Figure 22.

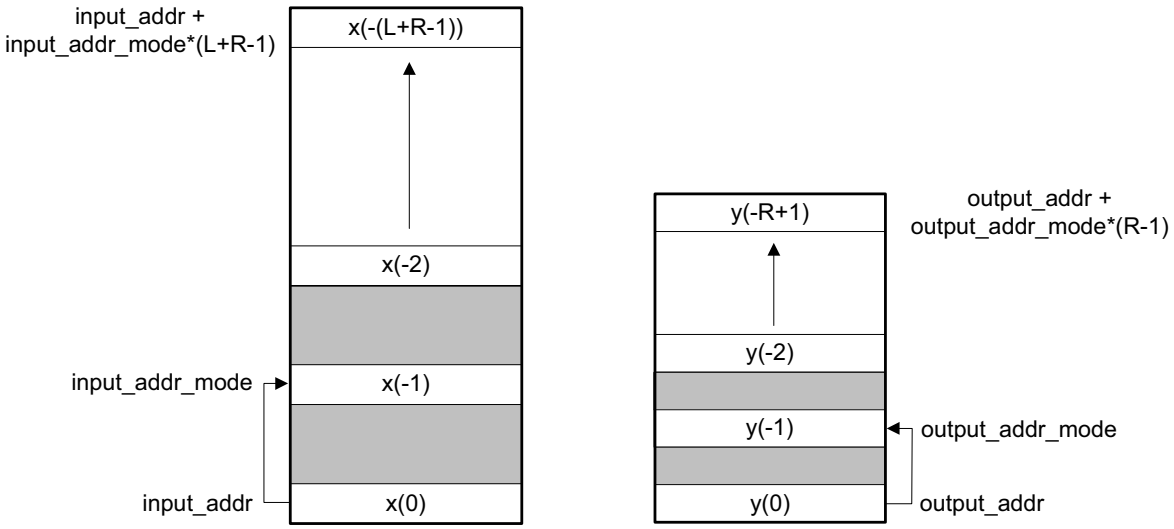


Figure 22. FIR filter input (left) and output (right) formats. Gray areas indicate skipped samples in memory.

The coefficients for the FIR filter are 24-bit or 48-bit signed fractional values, and the precision is specified by an optional configuration parameter. In addition, the coefficients are optionally scaled by a fixed scale factor, also specified as part of the `FIR_R` module configuration. Store the coefficients in the natural order of the filter's impulse response (i.e., *h(0), h(1), h(2),..., h(L-1)*). they can be ordered either contiguously in memory, or in an interleaved fashion specified by the optional coefficient addressing mode.

If the filter coefficients are double-precision, each 48-bit coefficient is stored as two adjacent 24-bit words with the most significant word first. The output, *y(n)*, is computed using a 48-bit by 24-bit multiply-accumulate between the filter coefficients and the inputs. The result is then scaled by A_{sf} and rounded to a 24-bit output sample.

3.5.1.1 FIR Filter FIFO Configuration

Figure 23 depicts a typical input FIFO configuration for storing the input samples to the FIR filter module. This sample configuration assumes contiguous input samples, i.e., `input_addr_mode` is set to one. A summary of the typical input FIFO configuration parameters for the `FIR_R` module is given in Table 12.

Table 12. Typical FIR_R Input FIFO Configuration Values

FIFO Parameter	Value
Minimum FIFO Size	*L*+R
Block Size	*R*
Base Pointer	0
IO Block Pointer	0

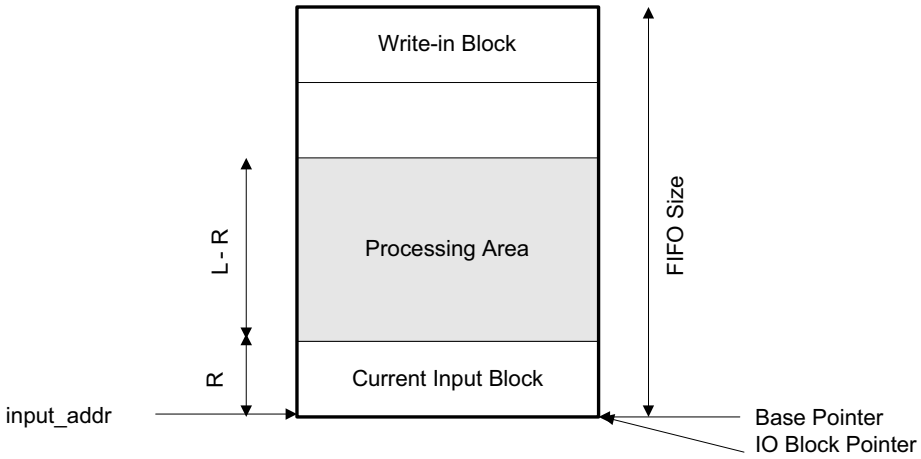


Figure 23. FIR filter input FIFO configuration

3.5.2 Time-Domain IIR Filter

The `IIR_R` module implements block-based IIR filtering of real-valued inputs with real-valued filter coefficients. The expression for the HEAR IIR filtering is given below using the following notation:

- *x(n)* is the input signal
- *y(n)* is the output signal
- *a(n)* is the set of IIR filter feedback coefficients
- *b(n)* is the set of IIR filter feedforward coefficients
- M_a is the number of feedback coefficients
- M_b is the number of feedforward coefficients
- A_{sf} is a scaling factor designed to allow for rescaling of the filter coefficients

Note that in this formulation, *a(0)* is always assumed to be the reciprocal of the scaling factor. For unscaled coefficients, this value is *a(0)=1*.

$$y(n) = 2^{A_{sf}}\left[\sum_{k=0}^{M_b-1} x(n-k)b(k) - \sum_{k=1}^{M_a-1} y(n-k)a(k)\right]$$

Like the FIR_R module, this module works on a block-by-block basis. A new block of *R* input samples is first collected and then a new block of *R* output samples is computed. The module also supports the same kinds of interleaved inputs and outputs as the FIR_R module.

In single-precision mode, the IIR_R module uses a single-precision Direct-Form I implementation of the IIR filter with 24-bit output samples and thus only 24-bit feedback. Single-precision mode is not recommended for higher order filters due to undesirable finite precision effects. The general input and output format is depicted in Figure 24.

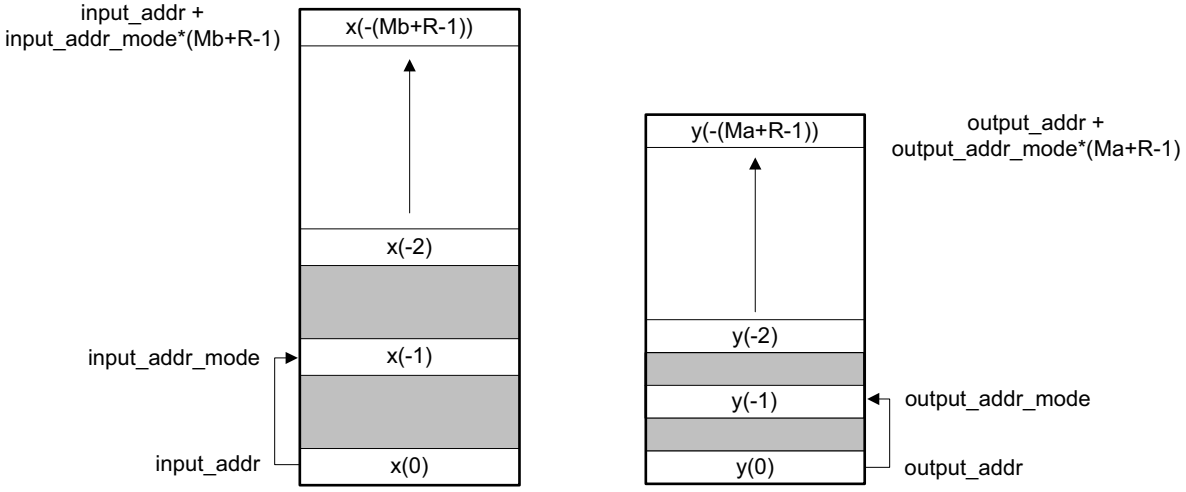


Figure 24. IIR_R filter input (left) and single-precision output (right) formats. Gray areas indicate skipped samples in memory.

In double-precision mode, the IIR_R module produces double-precision outputs, and they are used in the feedback portion of the IIR filtering. The feedback portion of the filter is computed using a 48-bit by 24-bit multiply-accumulate between the recursive filter coefficients, *a(k)*, and the previous outputs with only the 48 most significant bits of the multiply-accumulate being saved. The input and output sample formats and addressing are the same as depicted in Figure 24, except the elements depicted for *y(n)* are each 48-bits in size and the output_addr_mode step size is in terms of 48-bit elements. Each 48-bit output sample is stored as two adjacent 24-bit words with the most significant word first.

The coefficients for the IIR filter are all 24-bit signed fractional values, optionally scaled by a fixed scale factor specified as part the IIR_R module configuration. Store both sets of coefficients in the ascending order used for the transfer function view of the filter (i.e., *a(1), a(2),..., a(M_a-1)* and *b(0), b(1),..., b(M_b-1)*). Each of the two sets can be stored either contiguously in memory, or in an interleaved fashion specified by their respective coefficient addressing mode.

3.5.2.1 IIR Filter FIFO Configuration

Figure 25 depicts a typical set of input and output FIFO configurations for storing the input and output samples for the IIR_R filter module in single-precision mode. The use of a FIFO buffer provides convenient storage of the new input/output samples as well as past input/output samples. This sample configuration assumes contiguous samples, i.e., input_addr_mode and output_addr_mode are both set to one. A summary of the typical FIFO configuration parameters for the IIR_R module in single-precision mode is given in Table 13.

Table 13. Typical IIR_R FIFO Configuration Values in Single-Precision Mode

FIFO Parameter	Input	Output
Minimum FIFO Size	M_b+*R*	M_a+*R*
Block Size	*R*	*R*
Base Pointer	0	0
IO Block Pointer	0	*2R*

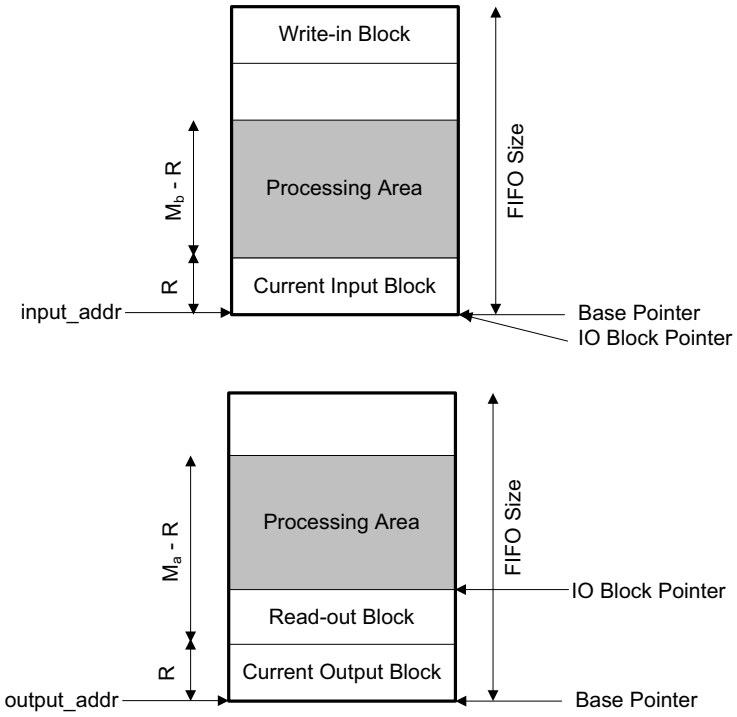


Figure 25. Single-precision IIR filter input (top) and output (bottom) FIFO configuration

In double-precision mode, a typical input FIFO configuration is the same as depicted in Figure 25. However, because of the double-precision outputs, the typical output FIFO configuration is different. Figure 26 depicts a typical output FIFO configuration for storing the output samples for the double-precision IIR filter. This sample configuration assumes contiguous samples (i.e., `output_addr_mode` is set to one). A summary of the typical FIFO configuration parameters for the `IIR_R` module in double-precision mode is given in Table 14.

NOTE: Because of the double-precision output samples, it is not normally possible to route a double-precision IIR output FIFO directly to any of the system outputs via the IOC.

Table 14. Typical IIR_R FIFO Configuration Values in Double-Precision Mode

FIFO Parameter	Input	Output
Minimum FIFO Size	M_b+*R*	$2M_a$+*2R*
Block Size	*R*	*2R*

Table 14. Typical IIR_R FIFO Configuration Values in Double-Precision Mode

FIFO Parameter	Input	Output
Base Pointer	0	0
IO Block Pointer	0	N/A

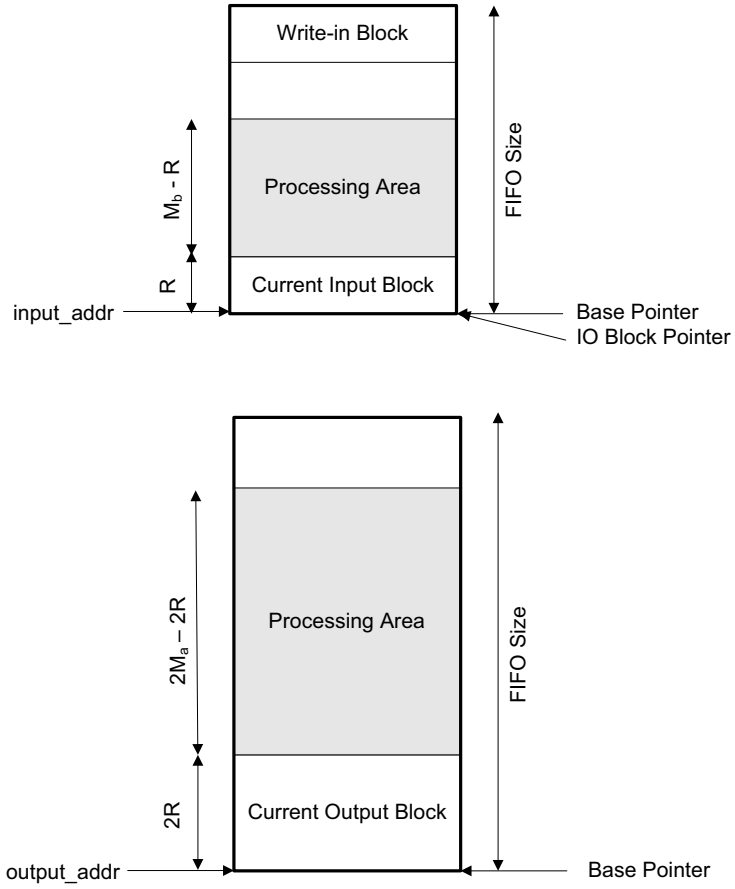


Figure 26. Double-precision IIR filter input (top) and output (bottom) FIFO configuration

3.5.3 Biquad IIR Filter

The `Biquad_IIR` module implements block-based IIR filtering of real-valued inputs with real-valued filter coefficients using a cascade of biquad (also known as second order section) filters. Using the following z-domain notation, the filtering can be described by the expression below:

- *H(z)* is the overall z-domain transfer function of the cascaded filters
- $a_k(n)$ is the IIR filter feedback coefficients for the *k*-th cascaded filter
- $b_k(n)$ is the IIR filter feedforward coefficients for the *k*-th cascaded filter
- *K* is the total number of cascaded filters

$$H(z) = \prod_{k=0}^{K-1} \frac{b_k(0) + b_k(1)z^{-1} + b_k(2)z^{-2}}{a_k(0) + a_k(1)z^{-1} + a_k(2)z^{-2}}$$

The *k*-th stage of filtering produces an output according to the IIR filtering equation given below using the following additional notation:

- $x_k(n)$ is the filter input
- $y_k(n)$ is the filter output
- A_k is a scaling factor designed to allow for rescaling of the filter coefficients

As with the `IIR_R` module, $a_k(0)$ is always assumed to be the reciprocal of the scaling factor. For unscaled coefficients, this value is $a_k(0)=1$. It is also important to note that the definition of the scale factor exponent is negated relative to the other filtering modules, due to the fact that the scale factor is read from memory as part of the coefficients. The organization of the coefficients within memory is described below.

$$y_k(n) = 2^{-A_k}[b_k(0)x_k(n) + b_k(1)x_k(n-1) + b_k(2)x_k(n-2) - a_k(1)y_k(n-1) - a_k(2)y_k(n-2)]$$

Like the other time-domain filtering modules, this module works on a block-by-block basis. A new block of *R* input samples is first collected and then a new block of *R* output samples is computed. The module also supports the same kinds of interleaved inputs and outputs as the other time-domain filtering modules. The coefficients are stored contiguously. The general input and output format is depicted in Figure 27. Note that this module only requires two past inputs and two past outputs after the new input/output blocks of *R* samples.

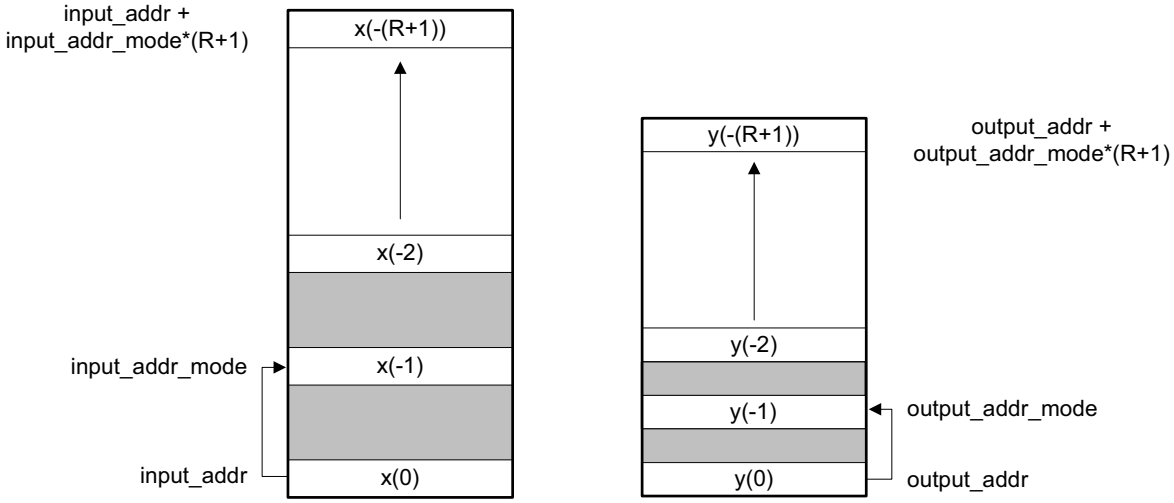


Figure 27. Biquad_IIR filter input (left) and output (right) formats. Gray areas indicate skipped samples in memory.

The `Biquad_IIR` module uses a single-precision Direct-Form I implementation of each IIR filter with 24-bit feedback. The filter coefficients of each IIR filter are all 24-bit signed fractional values, optionally scaled by a 24-bit

signed integer scale factor in a manner similar to the IIR_R module coefficients. The coefficients for each filter, including the scale factor, are specified as a set of 6 values in memory, with each filter's values located contiguously in memory as depicted in Figure 28. The scale factors A_k are the number of right shifts to perform on the filter output of each stage as described in the filtering equation above.

IMPORTANT: The filter coefficient scale factors for the Biquad_IIR module that are specified in memory have the opposite sign of the configuration parameter scale factors used in the other filtering modules. Ensure that the correct scale factors are stored in memory when using the Biquad_IIR module.

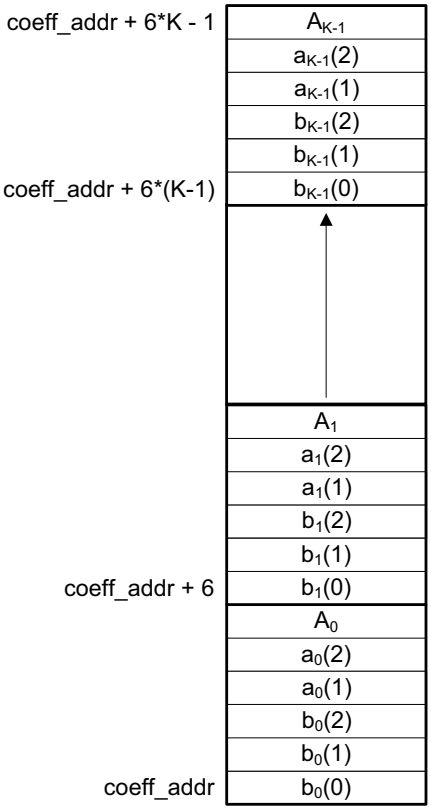


Figure 28. Biquad_IIR module filter coefficient memory format. Gray areas indicate skipped samples in memory.

The Biquad_IIR module also uses a block of temporary memory to store the intermediate inputs and outputs for each filter stage. This memory is only used if *K > 1*.

3.5.3.1 Biquad_IIR Filter FIFO Configuration

Figure 29 depicts a typical set of input and output FIFO configurations for storing the input and output samples for the biquad IIR filter module. The use of a FIFO buffer provides convenient storage of the new input/output samples as well as past input/output samples. This sample configuration assumes contiguous samples (i.e., input_addr_mode and output_addr_mode are both set to one). A summary of the typical FIFO configuration parameters for the Biquad_IIR module is given in Table 15.

Table 15. Typical Biquad_IIR FIFO Configuration Values

FIFO Parameter	Input	Output
Minimum FIFO Size	*2R+2*	*2R*
Block Size	*R*	*R*
Base Pointer	0	0
IO Block Pointer	0	*2R*

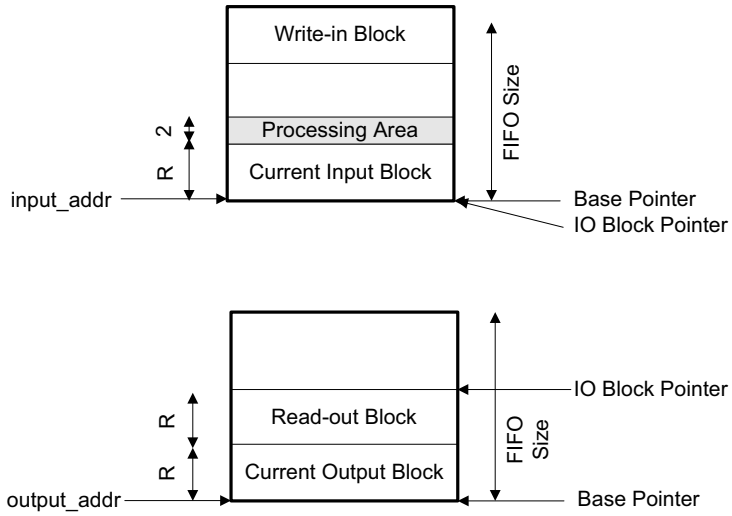


Figure 29. Biquad_IIR filter input (top) and output (bottom) FIFO configurations

3.5.4 Subband FIR Filter

The `sFIR_C` module computes FIR filtered outputs for a set of complex subbands using complex-valued coefficients. Unlike the time-domain filtering modules, the `sFIR_C` module computes new filtered outputs on a sample-by-sample basis. However, it computes new outputs for multiple subbands at a time. The expression below describes this filtering using the following notation:

- $X_k(m)$ represents the complex input signal for each subband
- $Y_k(m)$ is the filtered output for each subband
- $H_k(m)$ is the set of complex filter coefficients for each subband
- L is the length of the subband filters
- k is the subband index, which ranges from *0* to *K-1*
- A_{sf} is a scaling factor designed to allow for rescaling of the filter coefficients

$$Y_k(m) = 2^{A_{sf}}\left[\sum_{p=0}^{L-1} X_k(m-p)H_k(p)\right]$$

The module assumes a set of complex interleaved values representing the complex signal in each subband. Older samples are located at higher addresses in memory. The output is stored as a single set of interleaved complex subband samples. This input and output format is depicted in Figure 30, where each cell in the figure indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value.

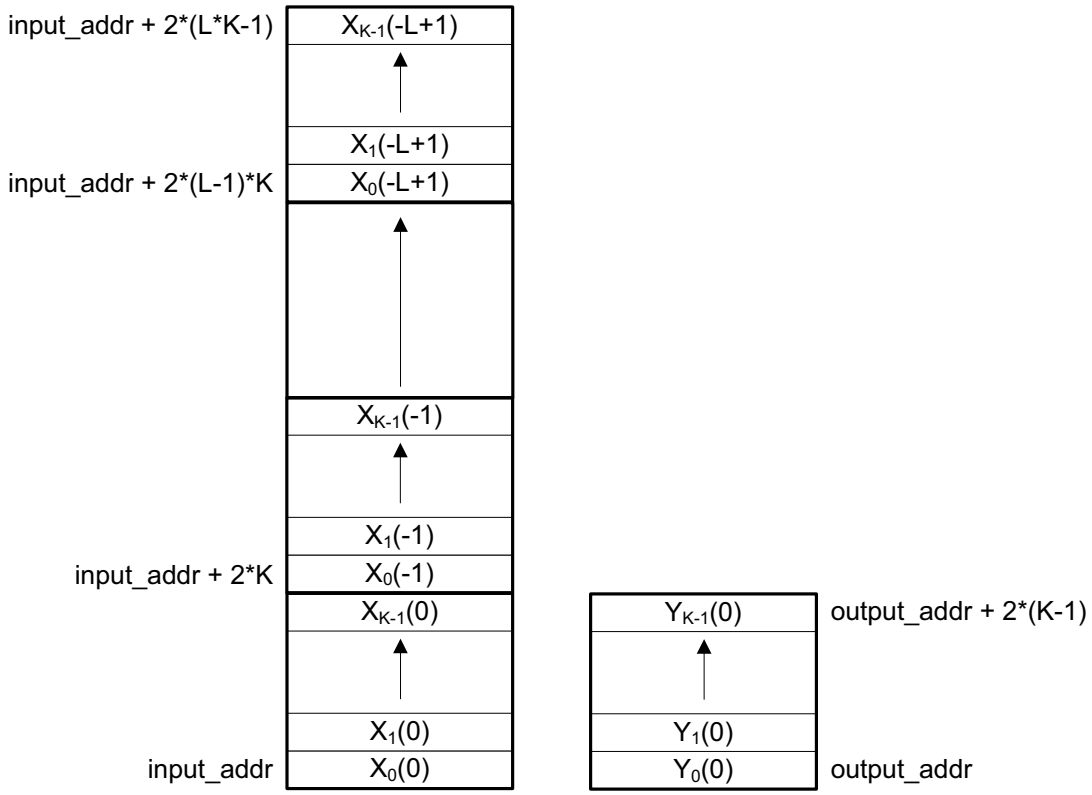


Figure 30. Subband FIR input (left) and output (right) formats. Each cell indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. Addresses in the figure refer to the first 24-bit word of each cell.

Unlike the subband samples, the FIR coefficients themselves are stored contiguously for each subband rather than in an interleaved fashion. This format is meant to simplify updating or modifying these coefficients on a band-by-band basis. Figure 31 depicts the organization of the coefficients. Each cell in the figure indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. Each coefficient is a complex value composed of two 24-bit fractional values. Note that the coefficient definition used in this module does not conjugate the coefficients as part of the filtering operation. As such, you may have to manually conjugate the coefficients before storing them in memory if they are defined for a conjugate filtering operation.

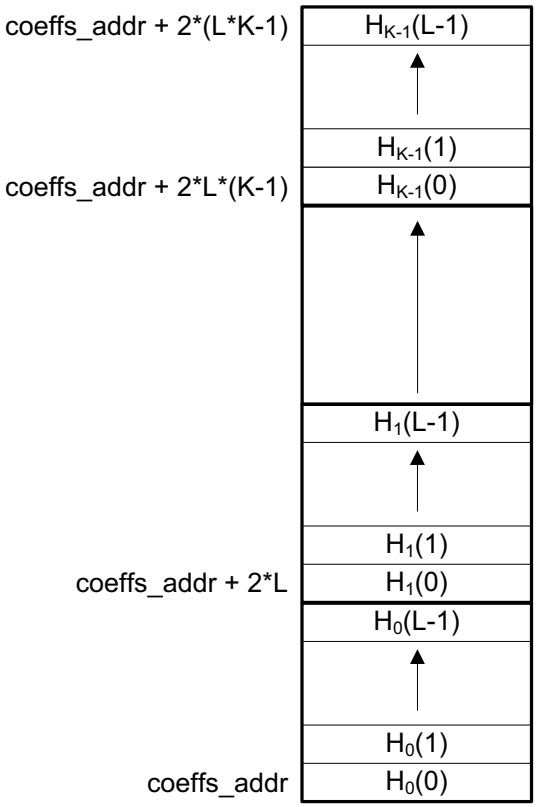


Figure 31. Subband FIR filter coefficient format. Each cell indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. Addresses in the figure refer to the first 24-bit word of each cell.

3.5.4.1 Subband FIR Filter FIFO Configuration

The subband FIR filter FIFO configuration requires storage of past sets of samples much like the time-domain FIR filter module. Since the inputs and outputs from these FIFOs are complex-valued subband signals, it does not make sense to connect these FIFOs to the IOC. Thus, these are generally used only as software controlled FIFOs and they should both be updated at the same time from either the HEAR or the CFX. The typical input FIFO layout for the `sFIR_C` module is depicted in Figure 32. A summary of the typical input FIFO configuration parameters for the `sFIR_C` module is given in Table 16.

Table 16. Typical sFIR_C Input FIFO Configuration Values

FIFO Parameter	Value
Minimum FIFO Size	*2LK*
Block Size	2*K*
Base Pointer	0
IO Block Pointer	N/A

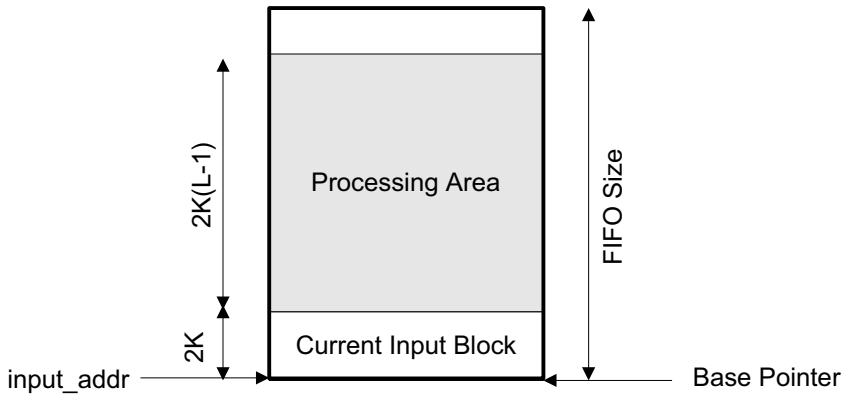


Figure 32. Subband FIR input FIFO configuration.

This FIFO is normally implemented using a dual FIFO due to the large number of data samples involved and for easier integration with other filterbank related modules (e.g. the `WOLA_Analysis and WOLA_Synthesis` modules). For odd values of *K*, it is not possible to use a dual FIFO structure because the block size for the individual A and B FIFOs would then be odd (see the limitations described in Section 2.5.1.3, “Dual FIFOs”).

3.5.5 Subband IIR Filter

The `sIIR_C` module computes IIR filtered outputs for a set of complex subbands using complex-valued coefficients. Unlike the time-domain filtering modules, the `sIIR_C` module computes new filtered outputs on a sample-by-sample basis. However, it computes new outputs for multiple subbands at a time. The expression below describes this filtering using the following notation:

- $X_k(m)$ represents the complex input signal in each subband
- $Y_k(m)$ is the filtered output in each subband
- $A_k(m)$ is the set of feedback complex filter coefficients for each subband
- $B_k(m)$ is the set of feedforward complex filter coefficients for each subband
- M_a is the length of the subband filter feedforward coefficients
- M_b is the length of the subband filter feedback coefficients
- k is the subband index, which ranges from *0* to *K-1*
- A_{sf} is a scaling factor designed to allow for rescaling of the filter coefficients

As with the `IIR_R` module, this formulation assumes that the first feedback coefficient ($A_k(0)$ in this case) is the reciprocal of $2^{A_{sf}}$. For unscaled coefficients, this value is $A_k(0)=1$.

$$Y_k(m) = 2^{A_{sf}}\left[\sum_{p=0}^{M_b-1} X_k(m-p)B_k(p) - \sum_{p=1}^{M_a-1} Y_k(m-p)A_k(p)\right]$$

The `sIIR_C` module uses a single-precision Direct Form I IIR filter implementation with 24-bit feedback. This is analogous to the structure used in the `IIR_R` module. As such, it is not recommended for high order filters due to finite precision effects.

The module assumes a set of complex interleaved values representing the complex signal in each subband. Older samples are located at higher addresses in memory for the input and output signals. The output is stored as a single set of interleaved complex subband samples. This input and output format is depicted in Figure 33, where each cell in the figure indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. For the output, the bottom *K* complex values represent where the newest output will be written. The older output samples located at higher addresses are used as inputs to the recursive portion of the IIR filter.

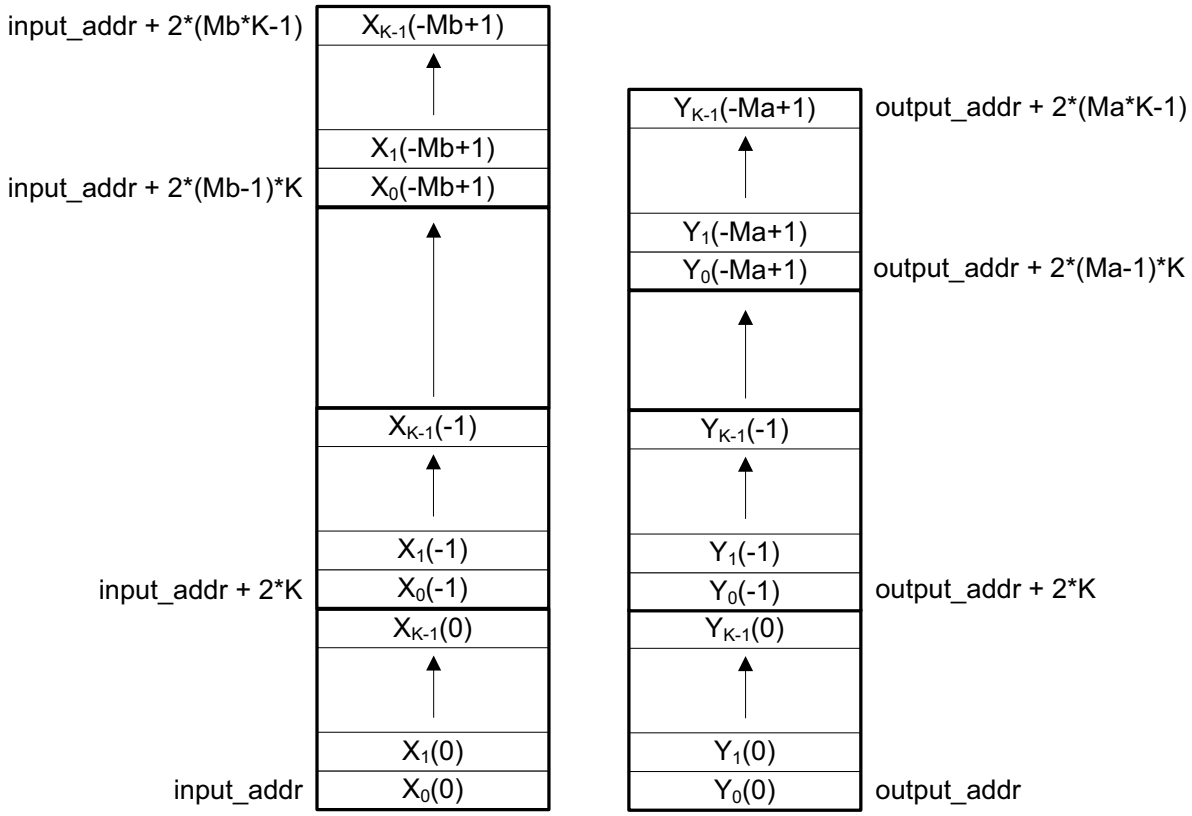


Figure 33. Subband IIR input (left) and output (right) formats. Each cell indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. Addresses in the figure refer to the first 24-bit word of each cell.

The IIR coefficients are stored contiguously for each subband, i.e., in the same manner as the filter coefficients for the `sFIR_C` module. The feedback and feedforward coefficients may be stored in different memory blocks. Figure 34 depicts the organization of the coefficients. Each cell in the figure indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. Each coefficient is a complex value composed of two 24-bit fractional values.

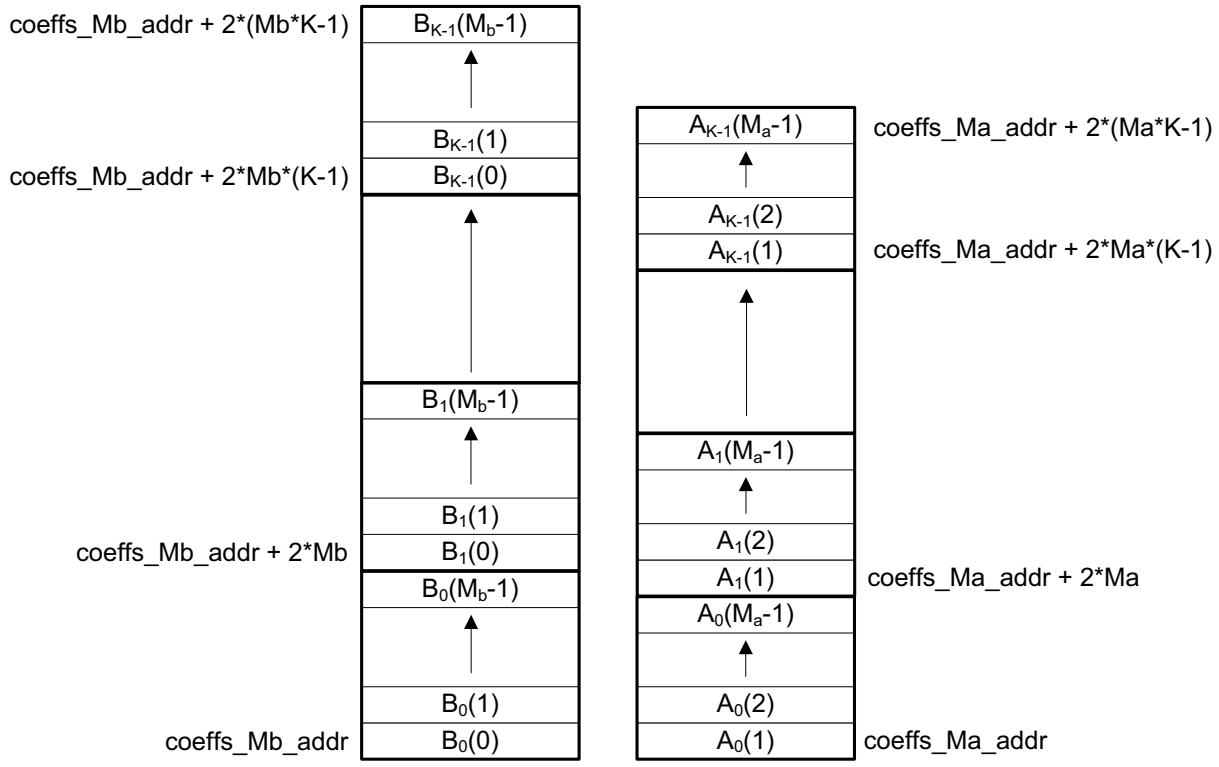


Figure 34. Subband IIR filter coefficient format. Feedforward coefficients are shown on the left, and feedback coefficients are shown on the right. Each cell indicates a 48-bit complex value where the even address is the real value and the odd address is the imaginary value. Addresses in the figure refer to the first 24-bit word of each cell.

3.5.5.1 Subband IIR Filter FIFO Configuration

The subband IIR filter FIFO configuration requires storage of past sets of input and output samples much like the time-domain IIR filter module. Since the inputs and outputs from these FIFOs are complex-valued subband signals, it does not make sense to connect these FIFOs to the IOC. Thus, these are generally used only as software controlled FIFOs and they should both be updated at the same time from either the HEAR or the CFX. The typical FIFO layout for the `sIIR_C` module is depicted in Figure 35. A summary of the typical FIFO configuration parameters for the `sIIR_C` module is given in Table 17.

Table 17. Typical sIIR_C FIFO Configuration Values

FIFO Parameter	Input	Output
Minimum FIFO Size	$2KM_b$	$2KM_a$
Block Size	$2K$	$2K$
Base Pointer	0	0
IO Block Pointer	N/A	N/A

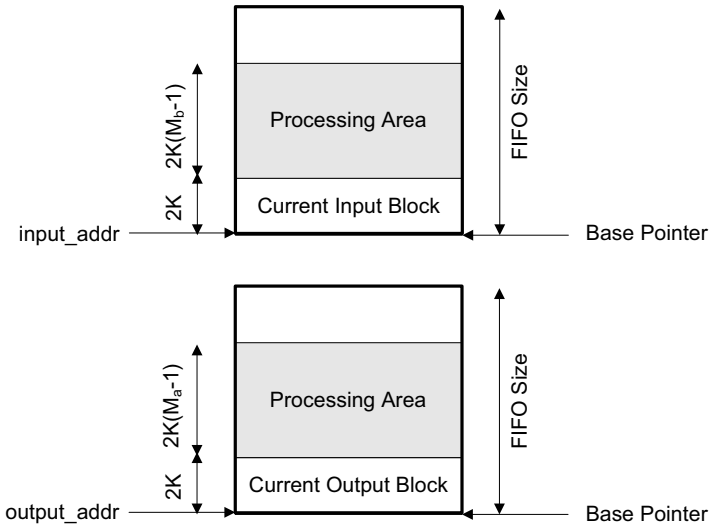


Figure 35. Subband IIR input (top) and output (bottom) FIFO configurations.

These FIFOs are normally implemented using dual FIFOs due to the large number of data samples involved and for easier integration with other filterbank related modules (e.g. the `WOLA` module). For odd values of *K*, it is not possible to use dual FIFOs because the block sizes for the individual A and B FIFOs would then be odd (see the limitations described in Section 2.5.1.3, “Dual FIFOs”). When using an odd number of subbands, it is necessary to use normal A or B memory FIFOs for input and output.

3.5.6 Attack-Release Averaging

The `Attack_Release` module implements a multi-channel, first-order, attack-release averaging filter. It can be used to average signal values such as energy calculations in a dynamic range compression system. With each execution, it computes a new filter output for each input channel. Independent attack and release filter coefficients can be specified for each channel. The formula for the attack-release filtering is shown below with *y(n)* as the output, *x(n)* as the new input, *a* as the attack filter coefficient, and *b* as the release filter coefficient.

$$\begin{aligned} y(n) &= c \cdot x(n) + (1 - c) \cdot y(n-1) \\ &= c \cdot [x(n) - y(n-1)] + y(n-1) \end{aligned}$$

$$\begin{aligned} \text{where} \quad c &= a \ \text{ if } \ x(n) \geq y(n-1) \\ c &= b \ \text{ if } \ x(n) < y(n-1) \end{aligned}$$

The module accepts single-precision, 24-bit inputs or double-precision, 48-bit inputs. The module stores the outputs in double-precision 48-bit values. For double-precision inputs, the full 48 bits are used to compute the difference $x(n) - y(n-1)$ and then only the most significant 24 bits of the difference are used in the subsequent multiplication. The past output (i.e., y(n-1)) must remain in the output buffer prior to launching the function, and the new filtered output (i.e., y(n)) will overwrite the past output at the same location. The application software is responsible for initializing the output buffer (e.g., to zero) to ensure that the results of the filter’s first execution are correct. It is important to note that the intermediate 48-bit result $x(n) - y(n-1)$ might saturate, causing erroneous outputs. However, for a sufficiently

slowly varying input signal this is normally not an issue. The inputs and outputs are either contiguous or interleaved in memory as depicted in Figure 36, where the subscripts denote channel indices and not time indices.

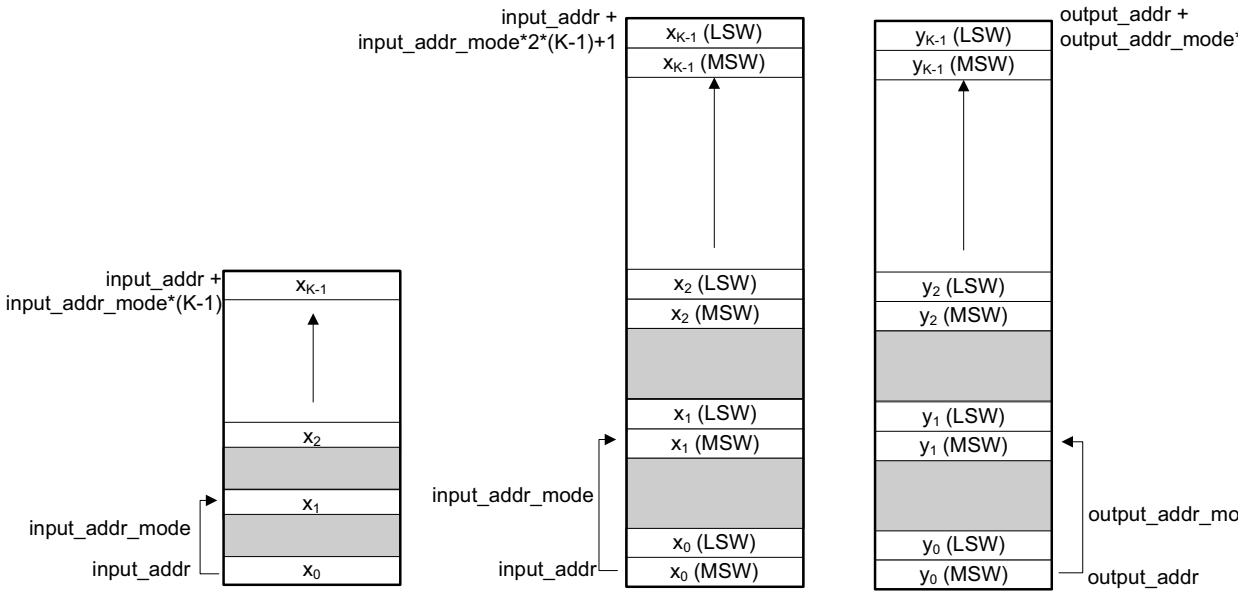


Figure 36. Attack-Release filtering single-precision input (left), double-precision input (center) and double-precision output (right) formats. MSW refers to the most significant 24-bit word and LSW refers to the least significant 24-bit word for a 48-bit double-precision value. Gray areas indicate skipped samples in memory.

By default, the coefficients for both attack and release are stored as 24-bit fractional values in separate vectors. The coefficients must be ordered in the same manner as the inputs (i.e., the first coefficient in each array is used for filtering the first input channel), and can be either contiguous or interleaved in memory. The coefficient organization for the vector format is shown in Figure 37. Alternatively, the coefficients for both attack and release can be scalars, and in this case the same coefficient(s) will be applied to all input channels. This can be realized by setting the corresponding coefficient addressing mode to zero.

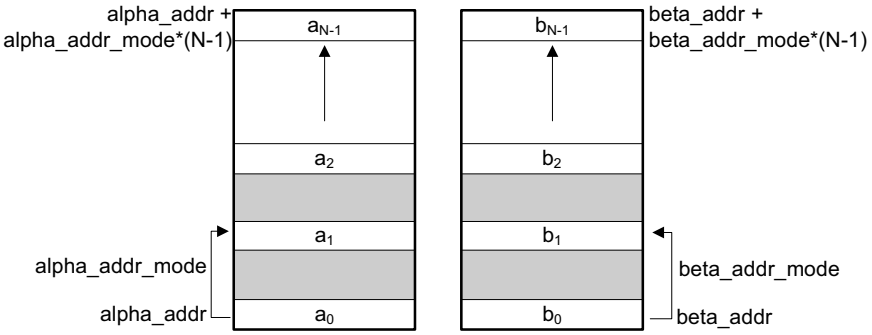


Figure 37. Attack-Release filtering attack (left) and release (right) coefficients. Gray areas indicate skipped samples in memory.

3.5.7 Exponential Averaging

The `Exp_Average` module implements a multi-channel, first-order, IIR filter. It can be used to average signal values such as energy calculations in a dynamic range compression system. With each execution, it computes a new filter output for each input channel. Filter coefficients are specified either as a scalar (a single coefficient applied to all channels) or as a vector (an independent coefficient for each channel). The formula for the attack-release filtering is shown below with *y(n)* as the output, *x(n)* as the new input, and *c* as the filter coefficient.

$$\begin{aligned} y(n) &= c \cdot x(n) + (1 - c) \cdot y(n - 1) \\ &= c \cdot [x(n) - y(n - 1)] + y(n - 1) \end{aligned}$$

The module accepts single-precision, 24-bit inputs or double-precision, 48-bit inputs. The module stores the outputs in double-precision 48-bit values. For double-precision inputs, the full 48 bits are used to compute the difference $x(n) - y(n - 1)$, then only the most significant 24 bits of the difference are used in the subsequent multiplication. The past output (i.e., y(n-1)) must remain in the output buffer prior to launching the function, and the new filtered output (i.e., y(n)) will overwrite the past output at the same location. The application software is responsible for initializing the output buffer (e.g., to zero) to ensure that the results of the filter's first execution are correct. It is important to note that the intermediate 48-bit result $x(n) - y(n - 1)$ might saturate, causing erroneous outputs. However, for a sufficiently slowly varying input signal, this is normally not an issue. The inputs and outputs are either contiguous or interleaved in memory as depicted in Figure 36 on page 72, where the subscripts denote channel indices and not time indices.

The coefficient(s) are stored as 24-bit fractional values. If a vector of coefficients is used, the coefficients must be ordered in the same manner as the inputs (i.e., the first coefficient in the array is used for filtering the first input channel). The coefficient organization for the vector format case is similar to the alpha (or beta) organization shown in Figure 37 on page 72. In the case of a scalar coefficient, a single 24-bit fractional value is stored, and by setting alpha_addr_mode to zero, it is used for all channels.

3.6 VECTOR OPERATIONS

The vector operation modules provide basic arithmetic operations for manipulating arbitrary vectors in memory. They can be used to efficiently add, multiply, subtract or analyze vectors as part of an application's signal processing. The addition, multiplication and subtraction modules have options for single-precision 24-bit or double-precision 48-bit operations. The vector operation modules that are available for the HEAR are summarized in Table 18.

Table 18. Vector Operations Modules

Module Name	Description	References
`vAdd_C`	Adds two complex-valued vectors of equal length in an element-wise fashion	Section 3.6.2, "Complex Vector Addition" Section 4.23, "vAdd_C"
`vAdd_R`	Adds two real-valued vectors of equal length in an element-wise fashion	Section 3.6.1, "Vector Addition" Section 4.24, "vAdd_R"
`vAddScalar_C`	Adds a complex-valued scalar to each element of a complex-valued vector	Section 3.6.4, "Complex Vector-Scalar Addition" Section 4.25, "vAddScalar_C"
`vAddScalar_R`	Adds a real-valued scalar to each element of a real-valued vector	Section 3.6.3, "Vector-Scalar Addition" Section 4.26, "vAddScalar_R"

Table 18. Vector Operations Modules (Continued)

Module Name	Description	References
vSub_C	Subtracts two complex-valued vectors of equal length in an element-wise fashion	Section 3.6.6, “Complex Vector Subtraction” Section 4.27, “vSub_C”
vSub_R	Subtracts two real-valued vectors of equal length in an element-wise fashion	Section 3.6.5, “Vector Subtraction” Section 4.28, “vSub_R”
vSubScalar_C	Subtracts a complex-valued scalar from each element of a complex-valued vector, or subtracts each element of a complex-valued vector from a complex-valued scalar	Section 3.6.8, “Complex Vector-Scalar Subtraction” Section 4.29, “vSubScalar_C”
vSubScalar_R	Subtracts a real-valued scalar from each element of a real-valued vector, or subtracts each element of a real-valued vector from a real-valued scalar	Section 3.6.7, “Vector-Scalar Subtraction” Section 4.30, “vSubScalar_R”
vMult_C	Multiplies two complex-valued vectors of equal length in an element-wise fashion	Section 3.6.10, “Complex Vector Multiplication” Section 4.31, “vMult_C”
vMult_R	Multiplies two real-valued vectors of equal length in an element-wise fashion	Section 3.6.12, “Vector-Scalar Multiplication” Section 4.32, “vMult_R”
vMultConj	Multiplies two complex-valued vectors of equal length in an element-wise fashion and conjugates the second multiplicand	Section 3.6.11, “Complex Conjugate Vector Multiplication” Section 4.33, “vMultConj”
vMultScalar_C	Multiplies each element of a complex-valued vector with a complex-valued scalar	Section 3.6.13, “Complex Vector-Scalar Multiplication” Section 4.34, “vMultScalar_C”
vMultScalar_R	Multiplies each element of a real-valued vector with a real-valued scalar	Section 3.6.12, “Vector-Scalar Multiplication” Section 4.35, “vMultScalar_R”
vMultScalarConj	Multiplies each element of a complex-valued vector with a complex-valued scalar and conjugates one of the multiplicands	Section 3.6.14, “Complex Conjugate Vector-Scalar Multiplication” Section 4.36, “vMultScalarConj”
vConj	Makes a conjugated copy of a complex-valued vector	Section 3.6.15, “Complex Conjugate Vector Copy” Section 4.37, “vConj”
vMag	Calculates the magnitude of each element in a vector of complex values	Section 3.6.16, “Vector Magnitude” Section 4.38, “vMag”
vSqMag	Calculates the squared magnitude of each element in a vector of complex values	Section 3.6.17, “Vector Squared Magnitude” Section 4.39, “vSqMag”
InnerProduct_C	Calculates the sum of the element-wise products of two complex-valued vectors of equal length	Section 3.6.19, “Complex Inner Product” Section 4.40, “InnerProduct_C”
InnerProduct_R	Calculates the sum of the element-wise products of two real-valued vectors of equal length	Section 3.6.18, “Inner Product” Section 4.41, “InnerProduct_R”
vLinLog2	Calculates the base-2 logarithm of each element in a vector of linear (fractional) values	Section 3.6.20, “Linear to Log2 Conversion” Section 4.42, “vLinLog2”

Table 18. Vector Operations Modules (Continued)

Module Name	Description	References
vLog2Lin	Calculates the exponent base-2 of each element in a vector of m8p16 values	Section 3.6.21, “Log2 to Linear Conversion” Section 4.43, “vLog2Lin”
vPolarRect	Converts a vector of complex values from polar format (magnitude-phase) to rectangular format (real-imaginary) using an iterative CORDIC approximation	Section 3.6.22.1, “Polar-to-Rectangular Conversion” Section 4.44, “vPolarRect”
vRectPolar	Converts a vector of complex values from rectangular format (real-imaginary) to polar format (magnitude-phase) using an iterative CORDIC approximation	Section 3.6.22.2, “Rectangular-to-Polar Conversion” Section 4.45, “vRectPolar”
vCopy	Makes a copy of a vector using 24-bit (single word) data moves	Section 3.6.23, “Single Word Vector Copy” Section 4.46, “vCopy”
vCopy2	Makes a copy of a vector using 48-bit (two word) data moves	Section 3.6.24, “Two Word Vector Copy” Section 4.47, “vCopy2”
vCopy4	Makes a copy of a vector using 96-bit (four word) data moves	Section 3.6.25, “Four Word Vector Copy” Section 4.48, “vCopy4”
vMax	Computes the element-wise maximum between two vectors	Section 3.6.26, “Vector Maximum” Section 4.54, “vMax”
vMaxScalar	Computes the maximum between a scalar and each element of a vector	Section 3.6.27, “Vector-Scalar Maximum” Section 4.55, “vMaxScalar”
vMin	Computes the element-wise minimum between two vectors	Section 3.6.28, “Vector Minimum” Section 4.56, “vMin”
vMinScalar	Computes the minimum between a scalar and each element of a vector	Section 3.6.29, “Vector-Scalar Minimum” Section 4.57, “vMinScalar”
vBound	Applies upper and lower bounds to an input vector	Section 3.6.30, “Vector Bound” Section 4.51, “vBound”
vShift	Applies an element-wise shift to an input vector with shift counts specified in another vector	Section 3.6.31, “Vector Shift” Section 4.52, “vShift”
vShiftScalar	Applies a shift to each element of an input vector using a scalar shift count	Section 3.6.32, “Vector-Scalar Shift” Section 4.53, “vShiftScalar”
vAbsolute	Computes the absolute value of each element of a real-valued input vector	Section 3.6.33, “Vector Absolute Value” Section 4.50, “vAbsolute”
L1NormDiff	Computes the L1 norm of the difference between two equal length, real-valued vectors	Section 3.6.34, “L1 Norm of Vector Difference” Section 4.49, “L1NormDiff”

3.6.1 Vector Addition

The vAdd_R module performs element-wise single- or double-precision addition of two equal length vectors. The two inputs and outputs can be addressed in an interleaved fashion as depicted in Figure 38. This can be used, for example, to address one or more interleaved audio streams in a stereo FIFO for use as a module input or output. The

figure also depicts overall vector sizes that are different based on different numbers of skipped samples in both the input and the output.

NOTE: In double-precision mode, interpret *x(k)*, *y(k)* and *z(k)* in the figure as double-precision elements composed of two 24-bit words in adjacent memory locations.

If the most significant word in a double-precision element is located at address *n*, then the least significant word for that element is located at address *n+1* as depicted in Figure 39.

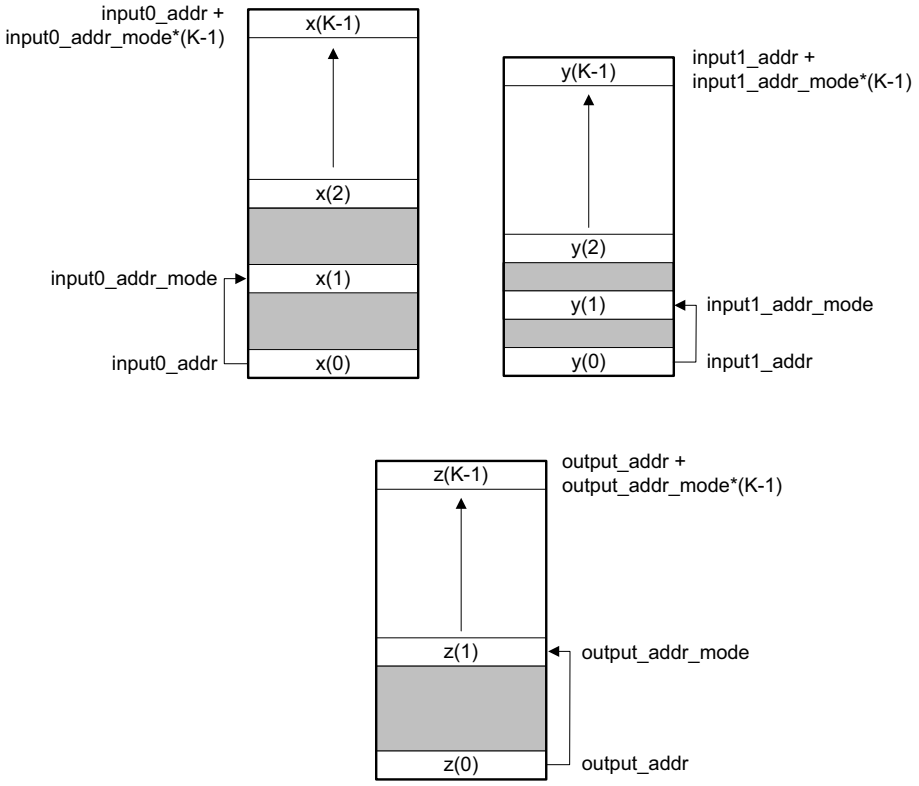


Figure 38. Vector addition input (top left and right) and output (bottom) formats. Gray areas indicate skipped samples in memory. Relative sizes of the buffers and address mode parameters are for illustration only.

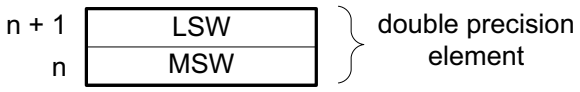


Figure 39. Real valued double-precision (48-bit) vector element at memory address n where MSW indicates the most significant 24-bit word and LSW indicates the least significant 24-bit word

In single-precision mode, the output saturates if the addition of two input elements exceeds the normal 24-bit range. In double-precision mode, the output wraps instead of saturating if the addition of two input elements exceeds the 48-bit range.

3.6.2 Complex Vector Addition

The `vAdd_C` module performs element-wise addition of two equal-length complex vectors with single- or double-precision elements.

In single-precision mode, each real and imaginary part of a complex element is represented by a 24-bit single-precision value. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 for the `vAdd_R` module. Note that you can now interpret *x(k)*, *y(k)* and *z(k)* in the figure as complex values. In other words, the real and imaginary components of each complex value must be adjacent, but the complex values themselves can be interleaved. The output saturates if the addition of two input elements exceeds the normal 24-bit range.

In double-precision mode, each real and imaginary part of a complex element is represented by its own 48-bit double-precision value. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 for the `vAdd_R` module. Note that *x(k)*, *y(k)* and *z(k)* in the figure should now be interpreted as double-precision complex elements composed of four 24-bit words in adjacent memory locations. If the most significant word of the real part is located at address *n*, then the least significant word for the real part is located at address *n+1*, the most significant word of the imaginary part is located at address *n+2* and the least significant word of the imaginary part is located at address *n+3* as depicted in Figure 40. Unlike the single-precision operation, the output wraps instead of saturating if the addition of two input elements exceeds the 48-bit range.

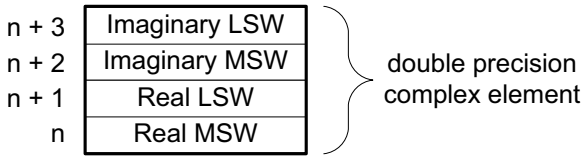


Figure 40. Complex valued double-precision (48-bit) vector element at memory address n where MSW indicates the most significant 24-bit word of each part and LSW indicates the least significant 24-bit word of each part

3.6.3 Vector-Scalar Addition

The `vAddScalar_R` module performs single- or double-precision addition between a vector with real elements and a real scalar. The vector input and output can be addressed in an interleaved fashion as depicted in Figure 38 for the `vAdd_R` module. In double-precision mode, the order of the double words in memory is depicted in Figure 39.

In single-precision mode, the output saturates if the addition exceeds the normal 24-bit range. In double-precision mode, the output wraps if the addition exceeds the 48-bit range.

IMPORTANT: The `vAddScalar_R` module is more efficient if the vector input address and the output address are in two different memory spaces. If the vector input address and the output address are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.4 Complex Vector-Scalar Addition

The `vAddScalar_C` module performs single- or double-precision addition between a vector with complex elements and a complex scalar.

The addressing mode and the arrangement of each element in memory are the same as for the vAdd_C module, except that one of the two vectors is now a scalar. In single-precision mode, the output saturates if the addition exceeds the normal 24-bit range. In double-precision mode, the output wraps if the addition exceeds the 48-bit range.

IMPORTANT: The vAddScalar_C module is more efficient if the vector input address and the output address are in two different memory spaces. If the vector input address and the output address are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.5 Vector Subtraction

The vSub_R module performs element-wise single- or double-precision subtraction of two equal length vectors. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 for the vAdd_R module. This can be used, for example, to address one or more interleaved audio streams in a stereo FIFO for use as a module input or output. In single-precision mode, the output saturates if the subtraction of two input elements exceeds the normal 24-bit range. In double-precision mode, the output wraps if the subtraction of two input elements exceeds the 48-bit range.

3.6.6 Complex Vector Subtraction

The vSub_C module performs element-wise single- or double-precision subtraction of two equal length complex vectors. The input and output address mode and data format are the same as for the vAdd_C module. In single-precision mode, the output saturates if the subtraction of two input elements exceeds the normal 24-bit range. In double-precision mode, the output wraps if the subtraction of two input elements exceeds the 48-bit range.

3.6.7 Vector-Scalar Subtraction

The vSubScalar_R module performs single- or double-precision subtraction between a vector with real elements and a real scalar. The order of the subtraction (i.e., scalar or vector element first) is selectable using a configuration parameter. The vector input and output can be addressed in an interleaved fashion as depicted in Figure 38 for the vAdd_R module. In double-precision mode, the order of the double words in memory is depicted in Figure 39.

In single-precision mode, the output saturates if the subtraction exceeds the normal 24-bit range. In double-precision mode, the output wraps if the subtraction exceeds the 48-bit range.

IMPORTANT: The vSubScalar_R module is more efficient if the vector input address and the output address are in two different memory spaces. If the vector input address and the output address are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.8 Complex Vector-Scalar Subtraction

The vSubScalar_C module performs single- or double-precision subtraction between a vector with complex elements and a complex scalar. The order of the subtraction (i.e., scalar or vector element first) is selectable using a configuration parameter.

The addressing mode and the arrangement of each element in memory are the same as for the `vSub_C` module, except that one of the two vectors is now a scalar. In single-precision mode, the output saturates if the subtraction exceeds the normal 24-bit range. In double-precision mode, the output wraps if the subtraction exceeds the 48-bit range.

IMPORTANT: The `vSubScalar_C` module is more efficient if the vector input address and the output address are in two different memory spaces. If the vector input address and the output address are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.9 Vector Multiplication

The `vMult_R` module performs element-wise single- or double-precision multiplication of two equal length vectors. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 for the `vAdd_R` module.

In single-precision mode, the module performs a 24-bit by 24-bit fractional multiply with each pair of input elements, rounds the 48-bit multiplication result to 24 bits and then saves this value to memory.

In double-precision mode, the first input vectors is composed of double-precision (48-bit) elements and the second input vector is composed of single-precision (24-bit) elements. The format of the double-precision vector elements is the same as for the other double-precision vector operations and is depicted in Figure 39. The module performs a 48-bit by 24-bit fractional multiply with each pair of input elements, and saves the 48 most significant bits of the multiplication result to memory.

3.6.10 Complex Vector Multiplication

The `vMult_C` module performs element-wise single- or double-precision multiplication of two equal length complex vectors. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 for the `vAdd_R` module. The input vectors must be located in different memories for efficiency.

In single-precision mode, for each pair of complex input elements, the module performs 24-bit by 24-bit fractional multiplies, accumulates the 48-bit multiplication results, rounds the 48-bit results to 24 bits and then saves the single-precision complex value to memory.

In double-precision mode, the first input vector is composed of double-precision (48-bit) real and imaginary parts while the second vector is composed of single-precision (24-bit) real and imaginary parts. The format of the double-precision complex vector elements is the same as for the other double-precision complex vector operations and is depicted in Figure 40. For each pair of complex input elements, the module performs 48-bit by 24-bit fractional multiply-accumulates for the real and imaginary parts, and then saves the 48 most significant bits of the complex results to memory.

3.6.11 Complex Conjugate Vector Multiplication

The `vMultConj` module performs element-wise single- or double-precision multiplication of two equal length complex vectors and conjugates one of the two operands for every complex multiplication. In single-precision mode, the second input vector is conjugated during multiplication. In double-precision mode, The vector to be conjugated in the multiplication is selectable through a configuration parameter. With the exception of this extra conjugation, the module is identical to the `vMult_C` module.

3.6.12 Vector-Scalar Multiplication

The vMultScalar_R module performs single- or double-precision multiplication between each element of a real valued vector and a real valued scalar. The input vector and the output can be addressed in an interleaved fashion as depicted in Figure 38 for the vAdd_R module. The precision mode is the same as for the vMult_R module. Note that in double-precision mode, the input vector and the output are double precision and the scalar input is single precision.

IMPORTANT: The vMultScalar_R module is more efficient if the vector input address and the output address are in two different memory spaces. If the vector input address and the output address are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.13 Complex Vector-Scalar Multiplication

The vMultScalar_C module performs single- or double-precision multiplication between each element of a complex valued vector and a complex valued scalar. The addressing mode, the arrangement of each element in memory, and the precision mode are the same as for the vMult_C module, except that the second input vector is now a scalar. Note that in double-precision mode, the input vector and the output are double precision and the scalar input is single precision.

3.6.14 Complex Conjugate Vector-Scalar Multiplication

The vMultScalarConj module performs single- or double-precision multiplication between each element of a complex valued vector and a complex valued scalar and conjugates one of the two operands for every complex multiplication. The input vector or the input scalar can be selected for conjugation using an optional configuration parameter. With the exception of the extra conjugation, this module is identical to the vMultScalar_C module.

3.6.15 Complex Conjugate Vector Copy

The vConj module creates a copy of a complex-valued vector and conjugates the elements in the new copy, i.e., the imaginary component of each complex element is negated. It performs the copy in a manner similar to the vCopy2 module. (See Section 3.6.24, “Two Word Vector Copy” on page 87.)

3.6.16 Vector Magnitude

The vMag module computes the magnitude of each element in a complex input vector. The module uses a look-up table in order to approximate the required square root operation. The lookup method is identical to the square root method used by the Math library as described in the *Firmware Reference Manual for Ezairo 7100*. The results are stored in memory either as 24-bit values with 2 integer bits and 22 fractional bits, or as 48-bit values with 2 integer bits and 46 fractional bits. Figure 41 depicts the arrangement of input and output vector elements when they are stored contiguously in memory and when the output is double precision. However, note that both input vector elements and the output vector elements can be interleaved in memory, in the same way a comparable vector in a vector arithmetic module is interleaved.

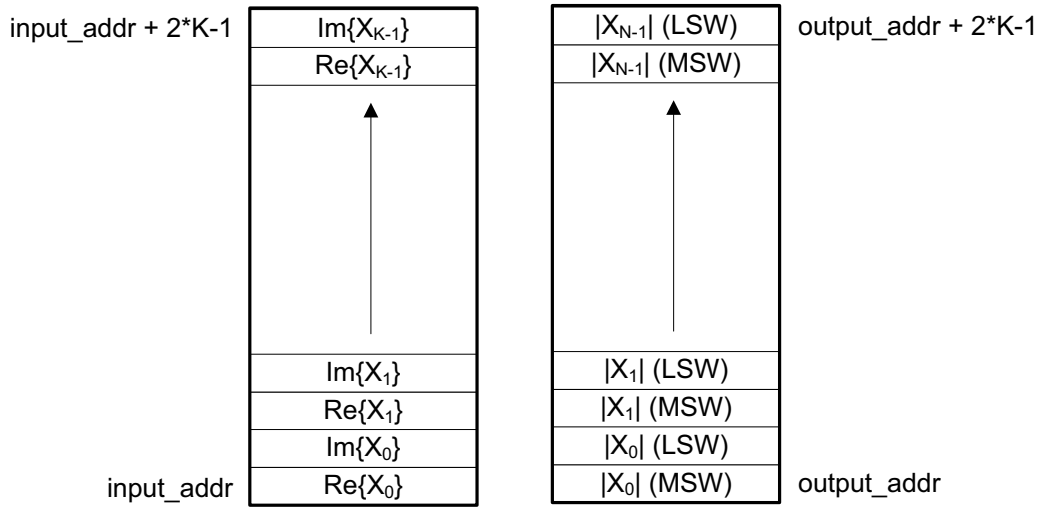


Figure 41. Vector magnitude input (left) and output (right) formats. MSW refers to the most significant 24-bit word and LSW refers to the least significant 24-bit word for a 48-bit double-precision value.

3.6.17 Vector Squared Magnitude

The `vSqMag` module computes the squared magnitude of each element in a complex input vector. The computation is similar to that of the `Energy` module, however no special formatting of the input or output vectors is used. The results are stored in memory either as 24-bit values with 2 integer bits and 22 fractional bits, or as 48-bit values with 2 integer bits and 46 fractional bits. The data format ensures that the summation portion of the squared magnitude operation does not result in the output being saturated. Figure 42 depicts the arrangement of input and output vector elements when they are stored contiguously in memory and when the output is double precision. However, note that both input vector elements and the output vector elements can be interleaved in memory, in the same way a comparable vector in a vector arithmetic module is interleaved.

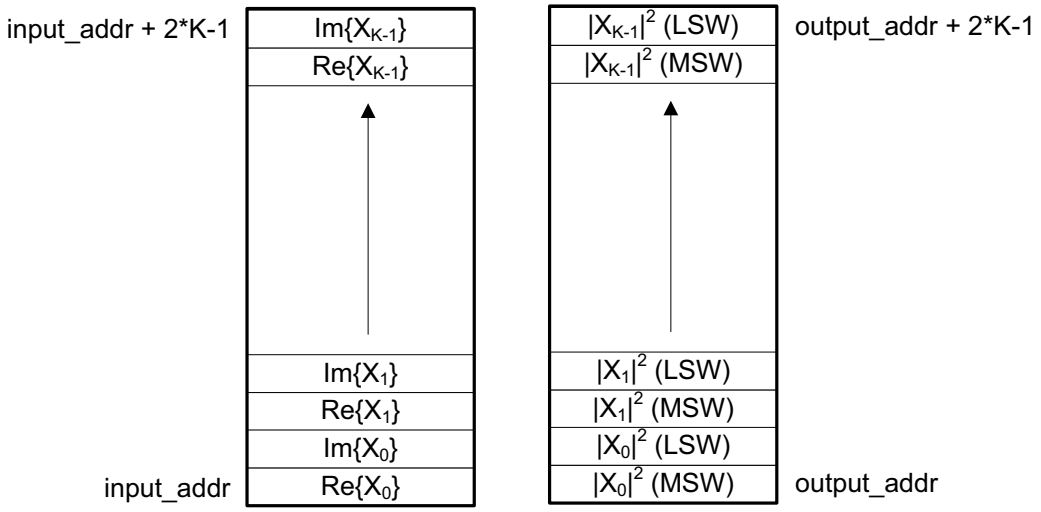


Figure 42. Vector squared magnitude input (left) and output (right) formats. MSW refers to the most significant 24-bit word and LSW refers to the least significant 24-bit word for a 48-bit double-precision value.

3.6.18 Inner Product

The `InnerProduct_R` module computes the inner product (i.e., the dot product) of two real-valued vectors of equal length. The inner product is defined as the summation of the element-wise multiplication of each vector element pair. The elements of each vector can be interleaved in memory. The module performs a 24-bit by 24-bit fractional multiply for each pair of input elements and then accumulates the 48-bit multiplication result in a 56-bit accumulator. After all products have been summed, the summation result is scaled by the user-specified scale factor and then saved to memory either as a 24-bit single-precision value, or as a 48-bit double-precision value. The scale factor can be set by the user in order to avoid saturation in the summation result. If the summation result exceeds the 56-bit accumulator range, then saturation will occur regardless of this scale factor.

3.6.19 Complex Inner Product

The `InnerProduct_C` module computes the inner product (i.e., the dot product) of two complex-valued vectors of equal length. The inner product is defined as the summation of the element-wise multiplication of each complex vector element pair. The elements of each vector can be interleaved in memory. For each pair of complex input elements, the module performs 24-bit by 24-bit fractional multiplies and accumulates the 48-bit multiplication results in separate 56-bit accumulators. The real and imaginary parts in the accumulators are each scaled by a user-specified scale factor and then saved to memory either as a pair of 24-bit single-precision values, or as a pair of 48-bit double-precision values. The scale factor can be set by the user in order to avoid saturation in the summation result. If the summation results exceed the 56-bit accumulator range, then saturation will occur regardless of this scale factor.

3.6.20 Linear to Log2 Conversion

The `vLinLog2` module calculates the base-2 logarithm of each element in a vector of linear (fractional) values. The module uses a table-lookup for the log2 calculations. The lookup method is identical to the log2 method used by the Math library as described in the *Firmware Reference Manual for Ezairo 7100*, with the exception that it returns a different undefined value for zero inputs. The input vector may consist of single (24-bit) or double (48-bit) precision values. Double-precision values are stored with MSW first and LSW at the next location in memory. The output vector elements are in mixed format with 8 integer bits and 16 fractional bits (see the m8p16 numeric format described in the *Firmware Reference Manual for Ezairo 7100*). The vector elements can be interleaved in memory.

3.6.21 Log2 to Linear Conversion

The `vLog2Lin` module calculates the exponent base-2 of each element in a vector of m8p16 values. The module uses a table-lookup for the exp2 calculations. The lookup method is identical to the exp2 method used by the Math library as described in the *Firmware Reference Manual for Ezairo 7100*. The input vector elements are in mixed format with 8 integer bits and 16 fractional bits (see the m8p16 numeric format described in the *Firmware Reference Manual for Ezairo 7100*). Thus, the output of the `vLinLog2` module is the same format as the input to the `vLog2Lin` module. The output vector may consist of single (24-bit) or double (48-bit) precision values. Double-precision values are stored with MSW first and LSW at the next location in memory. The vector elements can be interleaved in memory.

3.6.22 Complex Number Coordinate System Conversions

The HEAR microcode library includes modules for converting between rectangular (i.e., cartesian) and polar (i.e., exponential) coordinate systems. In both cases, the conversion is done using an iterative COordinate Rotational DIgital Computer (CORDIC) approximation. The implementation details for CORDIC-based conversions are described in Section 3.6.22.3, “CORDIC Implementation”.

3.6.22.1 Polar-to-Rectangular Conversion

The `vPolarRect` module converts a vector of complex values from polar format (magnitude-phase) to rectangular format (real-imaginary). The conversion is done using an iterative CORDIC approximation described in Section 3.6.22.3, “CORDIC Implementation”.

The input to the `vPolarRect` module is a vector made up of 24-bit magnitude values with 2 integer bits and 22 fractional bits and 24-bit fractional values giving the normalized radian frequency between -π and π. That is, the minimum fractional value corresponds to -π and the maximum fractional corresponds to π. Each element of the input vector is one magnitude/phase pair with the magnitude stored in the even address and the phase stored in the odd address. The output of the `vPolarRect` module is two 24-bit fractional values, one for the real component and one for the imaginary component of the complex value. The vector input and output format is depicted in Figure 43. Although not explicitly shown in Figure 43, the vector input and output elements can be interleaved in memory, in the same way a comparable vector in a vector arithmetic module is interleaved.

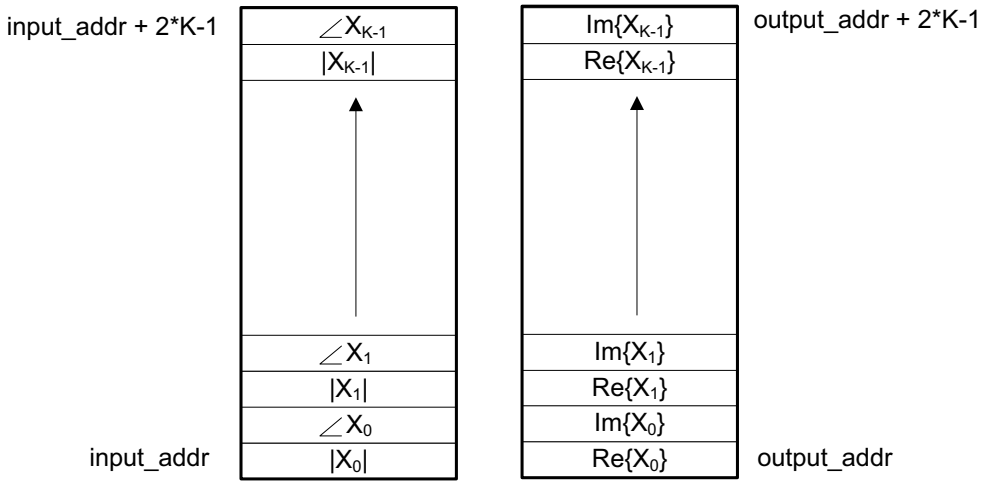


Figure 43. Polar-to-rectangular conversion input (left) and output (right) format. Magnitude is stored in 24-bit format with 2 integer bits and 22 fractional bits. Phase is stored as a normalized radian value in 24-bit fractional format.

3.6.22.2 Rectangular-to-Polar Conversion

The `vRectPolar` module converts a vector of complex values from rectangular coordinates (real-imaginary) to polar format (magnitude-phase). The conversion is done using an iterative CORDIC approximation, which is described in Section 3.6.22.3, “CORDIC Implementation”.

The input format to the `vRectPolar` function is a vector of 24-bit complex values representing the real and imaginary components of the rectangular representation. The output is a vector of magnitude/phase pairs having the same format as the input to the `vPolarRect` module. The vector input and output format is depicted in Figure 44. Although not explicitly shown in Figure 44, the vector input and output elements can be interleaved in memory, in the same way a comparable vector in a vector arithmetic module is interleaved.

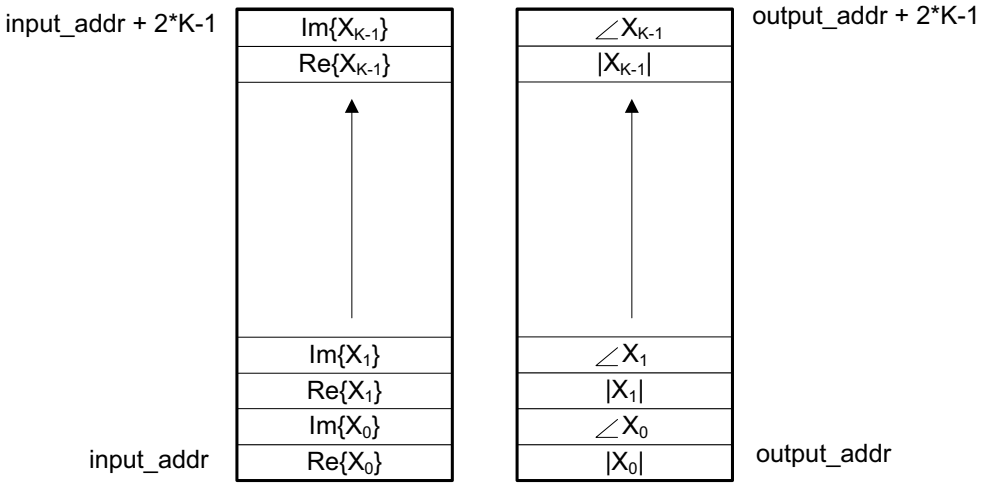


Figure 44. Rectangular-to-polar conversion input (left) and output (right) format. Magnitude is stored in 24-bit format with 2 integer bits and 22 fractional bits. Phase is stored as a normalized radian value in 24-bit fractional format.

3.6.22.3 CORDIC Implementation

The HEAR includes specialized functionality for implementing COordinate Rotation Digital Computer (CORDIC) based rectangular-to-polar and polar-to-rectangular conversions. CORDIC is a well-documented technique for iteratively calculating trigonometric functions without using multiplication. The CORDIC algorithm can be executed for additional iterations to improve the accuracy of the output, at the expense of more computational cycles. CORDIC works by rotating the phase of a complex number with successive multiplications of power-of-two values implemented as simple shift operations.

Rectangular-to-Polar Conversion

The basic idea for rectangular-to-polar conversion is to take a rectangular complex number input and rotate it until the imaginary component is “close” to zero as shown in Figure 45, where *x* is the real component of the original number, *y* is the imaginary component of the original number, and *theta* (θ) is the angle of rotation. The phase of this rotation is the phase of the original number, and the magnitude of the real component after rotation is the magnitude of the original number.

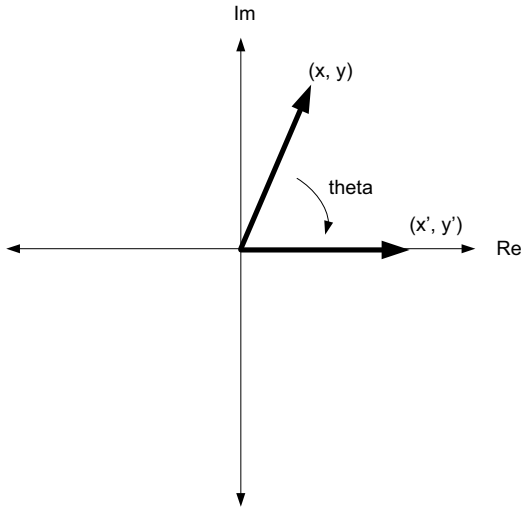


Figure 45. Rotation for rectangular-to-polar conversion

The rotation can be expressed in matrix form by the following equation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

As CORDIC is an iterative process, the input vector is actually put through a series of rotations which bring the imaginary component successively closer to zero. At iteration *n*, the rotation is given by the following equation:

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} \cos\theta_{n-1} & -\sin\theta_{n-1} \\ \sin\theta_{n-1} & \cos\theta_{n-1} \end{bmatrix} \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix} = \cos\theta_{n-1} \begin{bmatrix} 1 & -\tan\theta_{n-1} \\ \tan\theta_{n-1} & 1 \end{bmatrix} \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix}$$

To simplify the computation, the rotation angles (θ_n) are chosen such that $\theta_n = \arctan(2^{-n})$. Thus, the tangent of θ_n evaluates to a power of 2 and can be implemented using a shift operation.The values of θ_n are predetermined and are stored as an internal look-up table in the HEAR.

The sum of θ_n for all iterations is the net rotation angle, i.e. $\theta = \sum \theta_n S_n$ where $S_n = \{\pm 1\}$. Using this relation, we can express the tangent of θ_n as $\tan\theta_n = S_n 2^{-n}$. The value of S_n depends on the current value of the imaginary component at each iteration. If the imaginary component is positive, then a clockwise (or negative angle) rotation is required, and S_n is negative. If the imaginary component is negative, then a counter-clockwise (or positive angle) rotation is required, and S_n is positive.

Using these definitions of θ_n and S_n, we can re-write the iterative algorithm as follows:

$$\begin{bmatrix} x_n \\ y_n \end{bmatrix} = \cos\theta_{n-1} \begin{bmatrix} 1 & -S_{n-1}2^{-(n-1)} \\ S_{n-1}2^{-(n-1)} & 1 \end{bmatrix} \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix}$$

The correction factor(s) $\cos\theta_n$ can be wrapped into a constant, *K*, computed as $K = \prod \cos(\arctan(2^{-n}))$. This constant is automatically computed and used by the microcode functions.

When the complex input is not in the first quadrant of the Cartesian plane, then some pre-processing is required before applying this algorithm. This pre-processing involves rotating the vector through 180°, 90° or -90° to move it into the first quadrant. This pre-rotation is then simply added to the summation which computes θ to get the final phase value.

Polar-to-Rectangular Conversion

Conversion from polar to rectangular coordinates is performed in a similar manner as the rectangular-to-polar conversion described above. The basic idea involves rotating the magnitude of the polar number until the total angle of rotation equals the polar number's original phase. This process is depicted in Figure 46.

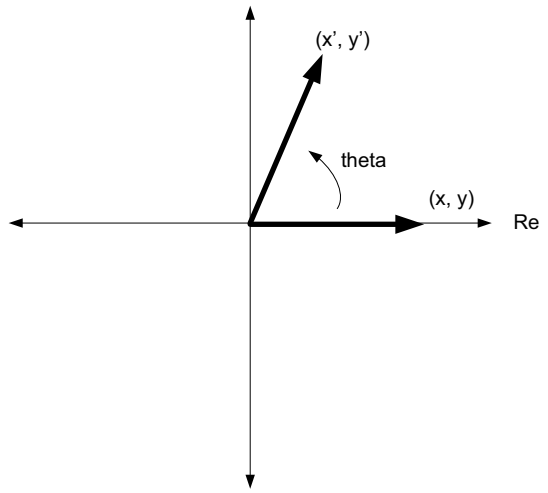


Figure 46. Rotation for polar-to-rectangular conversion

The same rotation and values for θ_n apply as in the rectangular-to-polar conversion. The sign of the rotation (S_n) is now determined by the difference between the desired angle and the current iteration's angle. If this difference is positive, a counter-clockwise rotation is required and S_n must be positive. If this difference is negative, a clockwise rotation is required and S_n must be negative.

The same look-up table of angles and correction factor for $\cos\theta_n$ is used in the polar-to-rectangular conversion algorithm.

If the initial phase angle is outside of the -90° to +90° range, then the initial complex vector is adjusted to have an angle of -90° or +90° and the initial angle of rotation must be adjusted accordingly.

3.6.23 Single Word Vector Copy

The `vCopy` module copies a vector of 24-bit words from one location in shared memory to another, one word at a time. This is less efficient than the two and four word vector copy modules (`vCopy2` and `vCopy4`), but allows flexible addressing of the source and destination words. The source and destination vectors can be either contiguous or interleaved in memory.

IMPORTANT: The `vCopy` module is more efficient if the source and destination vectors are in two different memory spaces. If you need to copy a vector to somewhere within the same memory space, this must be specified at compile-time. In other words, if the source and destination vectors are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.24 Two Word Vector Copy

The `vCopy2` module copies a vector of 24-bit words from one location in shared memory to another using the HEAR's 48-bit data movement operations. This means that two words (48 bits) are moved in one cycle during this operation, allowing significantly more efficient data movement than can be achieved using the CFX's access to the shared memory bus. Since all shared memory is 48 bits wide, the only restriction for the source and destination addresses are that they must be in different physical memories.

IMPORTANT: Note that the `vCopy2` module is more efficient if the source and destination vectors are in two different memory spaces. If you need to copy a vector to somewhere within the same memory space, this must be specified at compile-time. In other words, if the source and destination vectors are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.25 Four Word Vector Copy

The `vCopy4` module copies a vector of 24-bit words from one location in shared memory to another using the HEAR's 96-bit data movement operations. This means that four words (96 bits) are moved in one cycle during this operation, allowing significantly more efficient data movement than can be achieved using the CFX's access to the shared memory bus. The 96-bit moves require that the source and destination addresses be located in interleaved shared memory. This allows the HEAR to access 96 bits at a time by reading 48 bits from one of the interleaved physical memories, and 48 bits from the other interleaved physical memory.

IMPORTANT: The `vCopy4` module is more efficient if the source and destination vectors are in two different memory spaces. If you need to copy a vector to somewhere within the same memory space, this must be specified at compile-time. In other words, if the source and destination vectors are in two different memory spaces at compile-time, they must not be changed to the same memory space at run-time.

3.6.26 Vector Maximum

The `vMax` module computes the element-wise maximum of two equal length vectors. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the `vAdd_R` module.

3.6.27 Vector-Scalar Maximum

The vMaxScalar module compares a scalar value to each element of a vector, computing the maximum in each case. The input and the output vectors can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module.

3.6.28 Vector Minimum

The vMin module computes the element-wise minimum of two equal length vectors. The two inputs and the output can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module.

3.6.29 Vector-Scalar Minimum

The vMinScalar module compares a scalar value to each element of a vector, computing the minimum in each case. The input and the output vectors can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module.

3.6.30 Vector Bound

The vBound module applies upper and lower bounds to an input vector. The module computes the element-wise maximum of the input vector and a lower-bound vector, and then computes the element-wise minimum between the result and an upper-bound vector. The input, lower-bound, upper-bound and output vectors can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module.

3.6.31 Vector Shift

The vShift module applies an element-wise shift to data in an input vector using shift counts specified in another vector. A positive shift count value results in an arithmetic right shift, with the sign bit being shifted into the most significant bit position. A negative shift count value results in a left shift with zeros shifted into the least significant bit position. The input, shift count and output vectors can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module. The output saturates if a shifted element exceeds the normal 24-bit range.

3.6.32 Vector-Scalar Shift

The vShiftScalar module applies a shift to each element of an input vector using a scalar shift count. A positive shift count value results in an arithmetic right shift, with the sign bit being shifted into the most significant bit position. A negative shift count value results in a left shift with zeros shifted into the least significant bit position. The input and output vectors can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module. The output saturates if a shifted element exceeds the normal 24-bit range.

3.6.33 Vector Absolute Value

The vAbsolute module computes the absolute value of each element of a real-valued input vector. The input and output vectors can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the vAdd_R module.

3.6.34 L_1 Norm of Vector Difference

The L1NormDiff module computes the element-wise difference of two equal length, real-valued input vectors, calculates the absolute value of the differences, sums the absolute values and scales the result by applying a right shift to the total. Given two input vectors with elements x_k and y_k, both with length K, the output z is defined as shown in the equation below:

$$z = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K-1} |x_k - y_k|$$

NOTE: The scale factor can be used to reduce the chance of overflow when the output is single precision.

The input vector can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the `vAdd_R` module. The result is either single precision (1.23 format when the scale factor is zero), or double precision (24.24 format when the scale factor is zero). The output will be inaccurate if the result of the subtraction exceeds the normal 24-bit range.

3.7 SIGNAL STATISTICS

The signal statistics modules support the computation of statistical values for signal vectors. The signal statistics modules available for the HEAR are summarized in Table 19.

Table 19. Signal Statistics Modules

Module Name	Description	References
`AutoCorrelation_C`	Calculates the autocorrelation of a vector of complex values	Section 3.7.1, “Complex Autocorrelation” Section 4.59, “AutoCorrelation_C”
`AutoCorrelation_R`	Calculates the autocorrelation of a vector of real values	Section 3.7.2, “Autocorrelation” Section 4.60, “AutoCorrelation_R”
`AvgMag`	Calculates the average magnitude of a vector of complex values	Section 3.7.3, “Average Magnitude” Section 4.61, “AvgMag”
`AvgSqMag`	Calculates the average squared magnitude of a vector of complex values	Section 3.7.4, “Average Squared Magnitude” Section 4.62, “AvgSqMag”
`Correlation_C`	Calculates the correlation of two equal length vectors of complex values	Section 3.7.5, “Complex Correlation” Section 4.63, “Correlation_C”
`Correlation_R`	Calculates the correlation of two equal length vectors of real values	Section 3.7.6, “Correlation” Section 4.64, “Correlation_R”
`Mean`	Calculates the mean of a vector of real values	Section 3.7.7, “Mean” Section 4.67, “Mean”
`Sum`	Calculates the sum of a vector of real values	Section 3.7.8, “Sum” Section 4.68, “Sum”
`Variance_C`	Calculates the variance of a vector of complex values with zero mean	Section 3.7.10, “Complex Variance” Section 4.69, “Variance_C”
`Variance_R`	Calculates the variance of a vector of real values with zero mean	Section 3.7.9, “Variance” Section 4.70, “Variance_R”
`MaxElement`	Determines the element of maximum value within a real-valued vector	Section 3.7.11, “Maximum Element” Section 4.65, “MaxElement”
`MinElement`	Determines the element of minimum value within a real-valued vector	Section 3.7.12, “Minimum Element” Section 4.66, “MinElement”

3.7.1 Complex Autocorrelation

The `AutoCorrelation_C` module calculates the autocorrelation function estimate for a complex-valued input vector. This module only computes positive lags, since the autocorrelation function is conjugate symmetric around the zero-th lag. The user must specify the starting lag and the final lag. The calculation assumes that the mean of the vector is zero. Given an input vector with complex elements X_k and length K, the output Y for the jth lag is defined as shown in the equation below.

$$Y(j) = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K-1} (X_k \times X^*_{k-j})$$

NOTE: The scale factor can be used to reduce the chance of overflow. The * operator represents conjugation.

The input vector is a set of K 24-bit complex values, either contiguous or interleaved in memory. The output is a single-precision, 24-bit complex value for each lag. These output values are stored in memory either contiguously or in an interleaved fashion, starting from the first lag.

3.7.2 Autocorrelation

The `AutoCorrelation_R` module calculates the autocorrelation function estimate for a real-valued input vector. This module only computes positive lags, since the autocorrelation function is symmetric around the zero-th lag. The user must specify the starting lag and the final lag. The calculation assumes that the mean of the vector is zero. Given an input vector with real elements X_k and length K, the output Y for the jth lag is defined as shown in the equation below.

$$Y(j) = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K-1} (X_k \times X_{k-j})$$

NOTE: The scale factor can be used to reduce the chance of overflow.

The input vector is a set of K 24-bit real values, either contiguous or interleaved in memory. The output is a single-precision, 24-bit complex value for each lag. These output values are stored in memory either contiguously or in an interleaved fashion, starting from the first lag.

3.7.3 Average Magnitude

The `AvgMag` module computes the sum of the magnitudes of a complex-valued input vector. The magnitudes are calculated using a square root operation that is based on a table look-up. The look-up method used for the square root operation is identical to that used by the `vMag` module and the Math library as described in the *Firmware Reference Manual for Ezairo 7100*. The result of all magnitude calculations are summed and then rescaled according to a fixed power-of-two scaling factor specified during the module configuration. Given an input vector with complex elements X_k and length K, the output Y is defined as shown in the equation below.

$$Y = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K-1} |X_k|$$

NOTE: The scale factor should generally be greater than or equal to $\lceil \log_2 K \rceil$ to ensure that saturation cannot occur.

The input vector is a set of *K* 24-bit complex values, either contiguous or interleaved in memory. The output is either a double-precision 48-bit value with 2 integer bits and 46 fractional bits, stored most-significant word first, or a single-precision 24-bit value with 2 integer bits and 22 fractional bits.

3.7.4 Average Squared Magnitude

The `AvgSqMag` module computes the sum of the squared magnitudes of a complex-valued input vector. It then rescales this sum according to a fixed power-of-two scaling factor specified during the module configuration. Given an input vector with complex elements X_i and length *K*, the output *Y* is defined as shown in the equation below.

$$Y = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K-1} |X_k|^2$$

NOTE: The scale factor should generally be greater than or equal to $\lceil \log_2 K \rceil$ to ensure that saturation cannot occur.

The input vector is a set of *K* 24-bit complex values, either contiguous or interleaved in memory. The output is either a double-precision 48-bit value with 2 integer bits and 46 fractional bits, stored most-significant word first, or a single-precision 24-bit value with 2 integer bits and 22 fractional bits.

3.7.5 Complex Correlation

The `Correlation_C` module calculates the correlation function estimate for two complex-valued input vectors. You must specify the starting lag and the final lag. The calculation assumes that the mean of each vector is zero. Given one input vector with complex elements X_k and length *K0* and a second input vector with complex elements Z_k and length K1, the output *Y* for the *j*th lag is defined as shown in the equation below.

$$Y(j) = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K0-1} (X_k \times Z^*_{k-j})$$

NOTE: The scale factor can be used to reduce the chance of overflow. The lag, *j*, can be positive or negative.

Each input vector is a set of *K0* or K1 24-bit complex values, either contiguous or interleaved in memory. The output is a single-precision, 24-bit value for each lag. These output values are stored in memory either contiguously or in an interleaved fashion, starting from the first lag.

3.7.6 Correlation

The `Correlation_R` module calculates the correlation function estimate for two real-valued input vectors. You must specify the starting lag and the final lag. The calculation assumes that the mean of each vector is zero. Given one input vector with real elements X_k and length *K0* and a second input vector with real elements Z_k and length K1, the output *Y* for the *j*th lag is defined as shown in the equation below.

$$Y(j) = \frac{1}{2^{\text{scalefactor}}} \sum_{k=0}^{K0-1} (X_k \times Z_{k-j})$$

NOTE: The scale factor can be used to reduce the chance of overflow. The lag, *j*, can be positive or negative.

Each input vector is a set of *K0* or K1 24-bit real values, either contiguous or interleaved in memory. The output is a single-precision, 24-bit value for each lag. These output values are stored in memory either contiguously or in an interleaved fashion, starting from the first lag.

3.7.7 Mean

The `Mean` module calculates the mean of a vector of real values. Given an input vector with real elements X_k and length *K*, the output *Y* is defined as shown in the equation below. The scale factor *1/K* is approximated as a 24-bit fractional value.

$$Y = \frac{1}{K} \sum_{k=0}^{K-1} X_k$$

The input vector is a set of *K* 24-bit or *K* 48-bit real values, either contiguous or interleaved in memory. When 24-bit inputs are used, the input values are sign-extended to 48 bits before the summation, but only the part containing the most significant 24 non-zero bits in the summation result is multiplied by the 24-bit *1/K* scale factor. When 48-bit inputs are used, the 48-bit summation result is multiplied by the 24-bit *1/K* scale factor using a mixed-precision (48-bit by 24-bit) multiplication. Saturation in the summation can occur depending on the scale and size of the inputs.

The output can be either single- or double-precision. If the output is double-precision, it is stored most-significant word first.

3.7.8 Sum

The `Sum` module calculates the sum of a vector of real values. Given an input vector with real elements X_k and length *K*, the output *Y* is defined as shown in the equation below.

$$Y = \sum_{k=0}^{K-1} X_k$$

The input vector is a set of *K* 24-bit or *K* 48-bit real values, either contiguous or interleaved in memory. When 24-bit inputs are used, the input values are sign-extended to 48 bits before the summation. The output is a double-precision, 48-bit value, stored most-significant word first. No scaling of the data occurs, so the output value might be saturated depending on the precision and scale of the inputs.

3.7.9 Variance

The `Variance_R` module calculates the variance estimate for a vector of real values, with the assumption that the mean of the vector is zero. Given an input vector with real elements X_k and length *K*, the output *Y* is defined as shown in the equation below. The user may select whether the calculation is a biased estimate where the multiplication factor *1/K* is used, or unbiased estimate where the factor *1/(K-1)* is used. These factors are approximated using a 24-bit fractional multiplier. The equation below illustrates a biased estimate. The summation occurs using a 56-bit accumulator. This summation result is rescaled by `scalefactor` and the rescaled value is saturated to 48-bits. Finally, the 24-bit biased/unbiased factional multiplier and the 48-bit rescaled summation are multiplied to produce a 48-bit output.

$$Y = \frac{1}{K} \times \frac{1}{2^{\text{scalefactor}}} \times \sum_{k=0}^{K-1} X_k^2$$

The input vector is a set of *K* 24-bit real values, either contiguous or interleaved in memory. The output is either a single-precision, 24-bit value or double-precision, 48-bit value. If the output is double-precision, it is stored most-significant word first.

3.7.10 Complex Variance

The `Variance_C` module calculates the variance estimate of a vector of complex values, with the assumption that the mean of the vector is zero. Given an input vector with complex elements X_k and length *K*, the output *Y* is defined as shown in the equation below. The user may select whether the calculation is a biased estimate where the multiplication factor *1/K* is used, or unbiased estimate where the factor *1/(K-1)* is used. These factors are approximated using a 24-bit fractional multiplier. The equation below illustrates a biased estimate. The summation occurs using a 56-bit accumulator. This summation result is rescaled by `scalefactor` and the rescaled value is saturated to 48-bits. Finally, the 24-bit biased/unbiased factional multiplier and the 48-bit rescaled summation are multiplied to produce a 48-bit output.

$$Y = \frac{1}{K} \times \frac{1}{2^{\text{scalefactor}}} \times \sum_{k=0}^{K-1} (X_k \times X_k^*)$$

The input vector is a set of *K* 24-bit complex values, either contiguous or interleaved in memory. The output is either a single-precision, 24-bit value or double-precision, 48-bit value. If the output is double-precision, it is stored most-significant word first.

3.7.11 Maximum Element

The `MaxElement` module determines the maximum value within a real-valued input vector. The input vector can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the `vAdd_R` module.

3.7.12 Minimum Element

The `MinElement` module determines the minimum value within a real-valued input vector. The input vector can be addressed in an interleaved fashion as depicted in Figure 38 on page 76 for the `vAdd_R` module.

CHAPTER 4
Module Reference

4.1 Overview

This module reference documents the features of each HEAR microcode module, including a summary of all module attributes, constants, parameters and functions. Detailed information about each parameter is also given, including all of the restrictions associated with a particular parameter.

4.2 WOLA_Analysis

This module implements the analysis operation of a highly configurable WOLA Filterbank.

Module Attributes

Single Function No

Reset Yes

BFP Support Full

Table 20. WOLA_Analysis Constants

Constants	Description
WOLA_STACKING_EVEN	Specify even stacking of the WOLA subbands
WOLA_STACKING_ODD	Specify odd stacking of the WOLA subbands
WOLA_MODE_MONO	Specify mono input channel mode
WOLA_MODE_STEREO	Specify stereo input channel mode
WOLA_WIN_MODE_DEFAULT	Use the default window for analysis or synthesis
WOLA_WIN_MODE_OPTIMIZED	Use an optimized window for analysis or synthesis
WOLA_WIN_MODE_CUSTOM	Use a custom (user-specified) window for analysis or synthesis
WOLA_SAMPLE_18BITS	Specify an 18-bit sample size
WOLA_SAMPLE_24BITS	Specify a 24-bit sample size

Table 21. WOLA_Analysis Parameters

Parameters	Description
name	A unique name for each instance of the module
La	The length of the analysis window
Ls	The length of the synthesis window
N	The FFT size (per channel) of the filterbank
R	The block size (per channel) of the filterbank
stacking	Stacking of the filterbank bands (even or odd)
channel_mode	Specifies the WOLA filterbank channel mode
ana_win_mode	Specifies whether to use the default, optimized or a custom analysis window
input_fifo_addr	Address for the beginning of the input FIFO

Table 21. WOLA_Analysis Parameters (Continued)

Parameters	Description
analysis_output_addr	Address for the analysis outputs
temp01_addr	Address for intermediate FFT/IFFT steps
temp23_addr	Address for intermediate FFT/IFFT steps
sample_size	The input sample size

Table 22. WOLA_Analysis State Variables

State Variables	Description
<name>_i_shift	The input circular shift value

Table 23. WOLA_Analysis Functions

Functions	Description
WOLA_Analysis	Performs the WOLA analysis
WOLA_Analysis_Reset	Resets the WOLA filterbank analysis state

4.2.1 Constants

WOLA_STACKING_EVEN

Specify even stacking of the WOLA subbands

Value	0

WOLA_STACKING_ODD

Specify odd stacking of the WOLA subbands

Value	1

WOLA_MODE_MONO

Specify mono input channel mode

Value	0

WOLA_MODE_STEREO

Specify stereo input channel mode

Value	2

WOLA_WIN_MODE_DEFAULT

Use the default window for analysis or synthesis

Value	0

WOLA_WIN_MODE_OPTIMIZED

Use an optimized window for analysis or synthesis

Value	1

WOLA_WIN_MODE_CUSTOM

Use a custom (user-specified) window for analysis or synthesis

Value	2

WOLA_SAMPLE_18BITS

Specify an 18-bit sample size

Value	0

WOLA_SAMPLE_24BITS

Specify a 24-bit sample size

Value	1

4.2.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

La

The length of the analysis window

Type	Integer
Range	16, 32, 64, 96, 128, 256, 512, 1024
Restrictions	(N/R) >= (La/Ls) La >= Ls La >= N if (N % 3 == 0), La == 96 La/Ls == npow2(La/Ls) La/N == npow2(La/N) if(channel_mode == WOLA_MODE_MONO), La <= 1024 if(channel_mode == WOLA_MODE_MONO), La >= 32 if(channel_mode != WOLA_MODE_MONO), La <= 512 if(channel_mode != WOLA_MODE_MONO), La >= 16

Ls

The length of the synthesis window

Type	Integer
Range	16, 32, 64, 96, 128, 256, 512, 1024
Restrictions	(N/R) >= (La/Ls) La >= Ls Ls >= N if (N % 3 == 0), Ls == 96 La/Ls == npow2(La/Ls) if(channel_mode != WOLA_MODE_STEREO), Ls <= 1024 if(channel_mode != WOLA_MODE_STEREO), Ls >= 32 if(channel_mode == WOLA_MODE_STEREO), Ls <= 512 if(channel_mode == WOLA_MODE_STEREO), Ls >= 16

N

The FFT size (per channel) of the filterbank

Type	Integer
Range	16, 24, 32, 48, 64, 96, 128, 256, 512
Restrictions	(N/R) >= (La/Ls) N >= 2*R La >= N Ls >= N La/N == npow2(La/N) N/R == npow2(N/R) if((channel_mode == WOLA_MODE_MONO) && (stacking == WOLA_STACKING_EVEN)), N <= 512 if((channel_mode == WOLA_MODE_MONO) && (stacking == WOLA_STACKING_ODD)), N <= 256 if((channel_mode != WOLA_MODE_MONO) && (stacking == WOLA_STACKING_EVEN)), N <= 256 if((channel_mode != WOLA_MODE_MONO) && (stacking == WOLA_STACKING_ODD)), N <= 128

R

The block size (per channel) of the filterbank

Type	Integer
Range	2, 4, 6, 8, 12, 16, 24, 32, 48, 64, 128, 256
Restrictions	(N/R) >= (La/Ls) (N/2) >= R N/R == npow2(N/R)

stacking

Stacking of the filterbank bands (even or odd)

Type	Integer
Range	WOLA_STACKING_EVEN, WOLA_STACKING_ODD
Restrictions	if (N % 3 == 0), stacking == WOLA_STACKING_EVEN

channel_mode

Specifies the WOLA filterbank channel mode

Type	Integer
Range	WOLA_MODE_MONO, WOLA_MODE_STEREO
Restrictions	if (N % 3 == 0), channel_mode == WOLA_MODE_MONO

ana_win_mode

Specifies whether to use the default, optimized or a custom analysis window

Type	Integer
Range	WOLA_WIN_MODE_DEFAULT, WOLA_WIN_MODE_OPTIMIZED, WOLA_WIN_MODE_CUSTOM
Restrictions	None

input_fifo_addr

Address for the beginning of the input FIFO. Should point to the most recent sample of the most recent input block.

Type	Input
Memory Spaces	AMEM
Shared Parameters	None
Configuration Parameter Restrictions	if ((channel_mode == WOLA_MODE_MONO) && (npow2(La) == La)), (input_fifo_addr % npow2(La)) == 0 if (channel_mode != WOLA_MODE_MONO), (input_fifo_addr % npow2(2*La)) == 0
Address Parameter Restrictions	(input_fifo_addr % 4) == 0
Size	if(channel_mode == WOLA_MODE_MONO), La if(channel_mode != WOLA_MODE_MONO), 2*La

analysis_output_addr

Address for the analysis outputs

Type	Output
Memory Spaces	H01MEM, H45MEM, ABMEM, ACMEM, BDMEM, CDMEM
Shared Parameters	synthesis_input_addr, temp01_addr
Configuration Parameter Restrictions	if ((channel_mode == WOLA_MODE_MONO) && (npow2(N) == N)), (analysis_output_addr % npow2(N)) == 0 if (channel_mode != WOLA_MODE_MONO), (analysis_output_addr % npow2(2*N)) == 0
Address Parameter Restrictions	(analysis_output_addr % 4) == 0
Size	if(channel_mode == WOLA_MODE_MONO), N if(channel_mode != WOLA_MODE_MONO), 2*N

temp01_addr

Address for intermediate FFT/IFFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	analysis_output_addr, synthesis_input_addr
Configuration Parameter Restrictions	if ((channel_mode == WOLA_MODE_MONO) && (npow2(N) == N)), (temp01_addr % npow2(N)) == 0 if ((N % 3) == 0), (temp01_addr % (2*N/3)) == 0 if (channel_mode != WOLA_MODE_MONO), (temp01_addr % npow2(2*N)) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	if(channel_mode == WOLA_MODE_MONO), N if(channel_mode != WOLA_MODE_MONO), 2*N

temp23_addr

Address for intermediate FFT/IFFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	if ((channel_mode == WOLA_MODE_MONO) && (npow2(N) == N)), (temp23_addr % npow2(N)) == 0 if ((N % 3) == 0), (temp23_addr % (2*N/3)) == 0 if (channel_mode != WOLA_MODE_MONO), (temp23_addr % npow2(2*N)) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	if(channel_mode == WOLA_MODE_MONO), N if(channel_mode != WOLA_MODE_MONO), 2*N

sample_size

The input sample size.

Type	Integer
Range	WOLA_SAMPLE_18BITS, WOLA_SAMPLE_24BITS
Restrictions	None
Default Value	WOLA_SAMPLE_18BITS

4.2.3 State Variables

<name>_i_shift

The input circular shift value. The number of complex elements to circularly shift by before the analysis FFT.

Size	1
Modified By	WOLA_Analysis

4.2.4 Functions

WOLA_Analysis

```
WOLA_Analysis( name,
 La,
 Ls,
 N,
 R,
 stacking,
 channel_mode,
 ana_win_mode,
 input_fifo_addr,
 analysis_output_addr,
 temp01_addr,
 temp23_addr,
 sample_size )
```

Performs the WOLA analysis

WOLA_Analysis_Reset

```
WOLA_Analysis_Reset( name )
```

Resets the WOLA filterbank analysis state

4.3 WOLA_SYNTHESIS

This module implements the synthesis operation of a highly configurable WOLA Filterbank.

Module Attributes

Single Function No

Reset Yes

BFP Support Full

Table 24. WOLA_Synthesis Constants

Constants	Description
WOLA_STACKING_EVEN	Specify even stacking of the WOLA subbands
WOLA_STACKING_ODD	Specify odd stacking of the WOLA subbands
WOLA_MODE_MONO	Specify mono output channel mode
WOLA_MODE_STEREO	Specify stereo output channel mode
WOLA_WIN_MODE_DEFAULT	Use the default window for analysis or synthesis
WOLA_WIN_MODE_OPTIMIZED	Use an optimized window for analysis or synthesis
WOLA_WIN_MODE_CUSTOM	Use a custom (user-specified) window for analysis or synthesis
WOLA_SAMPLE_18BITS	Specify an 18-bit sample size
WOLA_SAMPLE_24BITS	Specify a 24-bit sample size

Table 25. WOLA_Synthesis Parameters

Parameters	Description
name	A unique name for each instance of the module
La	The length of the analysis window
Ls	The length of the synthesis window
N	The FFT size (per channel) of the filterbank
R	The block size (per channel) of the filterbank
stacking	Stacking of the filterbank bands (even or odd)
channel_mode	Specifies the WOLA filterbank channel mode
syn_win_mode	Specifies whether to use the default, optimized or a custom synthesis window
output_fifo_addr	Address for the beginning of the output FIFO
synthesis_input_addr	Address for the synthesis inputs
temp01_addr	Address for intermediate FFT/IFFT steps
temp23_addr	Address for intermediate FFT/IFFT steps
sample_size	The output sample size

Table 26. WOLA_Synthesis State Variables

State Variables	Description
<name>_o_shift	The output circular shift value

Table 27. WOLA_Synthesis Functions

Functions	Description
WOLA_Synthesis	Performs the WOLA Synthesis
WOLA_Synthesis_Reset	Resets the WOLA filterbank synthesis state

4.3.1 Constants

WOLA_STACKING_EVEN

Specify even stacking of the WOLA subbands

Value	0

WOLA_STACKING_ODD

Specify odd stacking of the WOLA subbands

Value	1

WOLA_MODE_MONO

Specify mono output channel mode

Value	0

WOLA_MODE_STEREO

Specify stereo output channel mode

Value	2

WOLA_WIN_MODE_DEFAULT

Use the default window for analysis or synthesis

Value	0

WOLA_WIN_MODE_OPTIMIZED

Use an optimized window for analysis or synthesis

Value	1

WOLA_WIN_MODE_CUSTOM

Use a custom (user-specified) window for analysis or synthesis

Value	2

WOLA_SAMPLE_18BITS

Specify an 18-bit sample size

Value	0

WOLA_SAMPLE_24BITS

Specify a 24-bit sample size

Value	1

4.3.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

La

The length of the analysis window

Type	Integer
Range	16, 32, 64, 96, 128, 256, 512, 1024
Restrictions	(N/R) >= (La/Ls) La >= Ls La >= N if (N % 3 == 0), La == 96 La/N == npow2(La/N) La/Ls == npow2(La/Ls) if(channel_mode == WOLA_MODE_MONO), La <= 1024 if(channel_mode == WOLA_MODE_MONO), La >= 32 if(channel_mode != WOLA_MODE_MONO), La <= 512 if(channel_mode != WOLA_MODE_MONO), La >= 16

Ls

The length of the synthesis window

Type	Integer
Range	16, 32, 64, 96, 128, 256, 512, 1024
Restrictions	(N/R) >= (La/Ls) La >= Ls Ls >= N if (N % 3 == 0), Ls == 96 La/Ls == npow2(La/Ls) if(channel_mode != WOLA_MODE_STEREO), Ls <= 1024 if(channel_mode != WOLA_MODE_STEREO), Ls >= 32 if(channel_mode == WOLA_MODE_STEREO), Ls <= 512 if(channel_mode == WOLA_MODE_STEREO), Ls >= 16

N

The FFT size (per channel) of the filterbank

Type	Integer
Range	16, 24, 32, 48, 64, 96, 128, 256, 512
Restrictions	(N/R) >= (La/Ls) N >= 2*R La >= N Ls >= N La/N == npow2(La/N) N/R == npow2(N/R) if((channel_mode == WOLA_MODE_MONO) && (stacking == WOLA_STACKING_EVEN)), N <= 512 if((channel_mode == WOLA_MODE_MONO) && (stacking == WOLA_STACKING_ODD)), N <= 256 if((channel_mode != WOLA_MODE_MONO) && (stacking == WOLA_STACKING_EVEN)), N <= 256 if((channel_mode != WOLA_MODE_MONO) && (stacking == WOLA_STACKING_ODD)), N <= 128

R

The block size (per channel) of the filterbank

Type	Integer
Range	2, 4, 6, 8, 12, 16, 24, 32, 48, 64, 128, 256
Restrictions	(N/R) >= (La/Ls) (N/2) >= R N/R == npow2(N/R)

stacking

Stacking of the filterbank bands (even or odd)

Type	Integer
Range	WOLA_STACKING_EVEN, WOLA_STACKING_ODD
Restrictions	if (N % 3 == 0), stacking == WOLA_STACKING_EVEN

channel_mode

Specifies the WOLA filterbank channel mode

Type	Integer
Range	WOLA_MODE_MONO, WOLA_MODE_STEREO
Restrictions	if (N % 3 == 0), channel_mode == WOLA_MODE_MONO

syn_win_mode

Specifies whether to use the default, optimized or a custom synthesis window

Type	Integer
Range	WOLA_WIN_MODE_DEFAULT, WOLA_WIN_MODE_OPTIMIZED, WOLA_WIN_MODE_CUSTOM
Restrictions	None

output_fifo_addr

Address for the beginning of the output FIFO. This address must point to the sample that is Ls samples ahead of the most recent output block.

Type	Input/Output
Memory Spaces	BMEM
Shared Parameters	None
Configuration Parameter Restrictions	if ((channel_mode != WOLA_MODE_STEREO) && (npow2(Ls) == Ls)), (output_fifo_addr % npow2(Ls)) == 0 if (channel_mode == WOLA_MODE_STEREO), (output_fifo_addr % npow2(2*Ls)) == 0
Address Parameter Restrictions	(output_fifo_addr % 4) == 0
Size	if(channel_mode != WOLA_MODE_STEREO), Ls if(channel_mode == WOLA_MODE_STEREO), 2*Ls

synthesis_input_addr

Address for the synthesis inputs

Type	Input
Memory Spaces	H01MEM, H45MEM, ABMEM, ACMEM, BDMEM, CDMEM
Shared Parameters	analysis_output_addr, temp01_addr
Configuration Parameter Restrictions	if ((channel_mode != WOLA_MODE_STEREO) && (npow2(N) == N)), (synthesis_input_addr % npow2(N)) == 0 if (channel_mode == WOLA_MODE_STEREO), (synthesis_input_addr % npow2(2*N)) == 0
Address Parameter Restrictions	(synthesis_input_addr % 4) == 0
Size	if(channel_mode != WOLA_MODE_STEREO), N if(channel_mode == WOLA_MODE_STEREO), 2*N

temp01_addr

Address for intermediate FFT/IFFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	analysis_output_addr, synthesis_input_addr
Configuration Parameter Restrictions	if ((channel_mode == WOLA_MODE_MONO) && (npow2(N) == N)), (temp01_addr % npow2(N)) == 0 if ((N % 3) == 0), (temp01_addr % (2*N/3)) == 0 if (channel_mode != WOLA_MODE_MONO), (temp01_addr % npow2(2*N)) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	if(channel_mode == WOLA_MODE_MONO), N if(channel_mode != WOLA_MODE_MONO), 2*N

temp23_addr

Address for intermediate FFT/IFFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	if ((channel_mode == WOLA_MODE_MONO) && (npow2(N) == N)), (temp23_addr % npow2(N)) == 0 if ((N % 3) == 0), (temp23_addr % (2*N/3)) == 0 if (channel_mode != WOLA_MODE_MONO), (temp23_addr % npow2(2*N)) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	if(channel_mode == WOLA_MODE_MONO), N if(channel_mode != WOLA_MODE_MONO), 2*N

sample_size

The output sample size.

Type	Integer
Range	WOLA_SAMPLE_18BITS, WOLA_SAMPLE_24BITS
Restrictions	None
Default Value	WOLA_SAMPLE_18BITS

4.3.3 State Variables

<name>_o_shift

The output circular shift value. The number of complex elements to circularly shift by after the synthesis inverse FFT.

Size	1
Modified By	WOLA_Synthesis

4.3.4 Functions

WOLA_Synthesis

```
WOLA_Synthesis( name,
 La,
 Ls,
 N,
 R,
 stacking,
 channel_mode,
 syn_win_mode,
 output_fifo_addr,
 synthesis_input_addr,
 temp01_addr,
 temp23_addr,
 sample_size )
```

Performs the WOLA Synthesis

WOLA_Synthesis_Reset

```
WOLA_Synthesis_Reset( name )
```

Resets the WOLA filterbank synthesis state

4.4 GAIN_R

This module implements a real-valued gain application for WOLA analysis results.

Module Attributes

Single Function	Yes
Reset	No
BFP Support	Shared

Table 28. Gain_R Parameters

Parameters	Description
name	A unique name for each instance of the module
N	The size of the FFT used for the analysis results
stacking	The stacking format (even/odd) of the analysis results
channel_mode	The channel mode format (mono/stereo) of the analysis results
input_addr	Address of the analysis results
gain_vector_addr	Address of the gain vector
output_addr	Address of the output vector

Table 29. Gain_R State Variables

State Variables	Description
<name>_exp	The gain exponent for this gain application

Table 30. Gain_R Functions

Functions	Description
Gain_R	Performs a real gain application on the analysis results

4.4.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

N

The size of the FFT used for the analysis results

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

stacking

The stacking format (even/odd) of the analysis results

Type	Integer
Range	WOLA_STACKING_ODD, WOLA_STACKING_EVEN
Restrictions	None

channel_mode

The channel mode format (mono/stereo) of the analysis results

Type	Integer
Range	WOLA_MODE_MONO, WOLA_MODE_STEREO
Restrictions	None

input_addr

Address of the analysis results

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input_addr) != memtype(gain_vector_addr) (input_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N

gain_vector_addr

Address of the gain vector. This parameter is run-time configurable. The gain vector address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input_addr) != memtype(gain_vector_addr) (gain_vector_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N
Run-time Parameter	<name>_gain_vector_addr

output_addr

Address of the output vector

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(output_addr) != memtype(gain_vector_addr) (output_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N

4.4.2 State Variables

<name>_exp

The gain exponent for this gain application

Size	1
Modified By	None

4.4.3 Functions

Gain_R

```
Gain_R( name,
 N,
 stacking,
 channel_mode,
 input_addr,
 gain_vector_addr,
 output_addr )
```

Performs a real gain application on the analysis results.

4.5 GAIN_C

This module implements a complex-valued gain application for WOLA analysis results.

Module Attributes

Single Function Yes

Reset No

BFP Support Shared

Table 31. Gain_C Parameters

Parameters	Description
name	A unique name for each instance of the module
N	The size of the FFT used for the analysis results
stacking	The stacking format (even/odd) of the analysis results
channel_mode	The channel mode format (mono/stereo) of the analysis results
input_addr	Address of the analysis results
gain_vector_addr	Address of the gain vector
output_addr	Address of the output vector

Table 32. Gain_C State Variables

State Variables	Description
<name>_exp	The gain exponent for this gain application

Table 33. Gain_C Functions

Functions	Description
Gain_C	Performs a complex gain application on the analysis results

4.5.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

N

The size of the FFT used for the analysis results

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

stacking

The stacking format (even/odd) of the analysis results

Type	Integer
Range	WOLA_STACKING_ODD, WOLA_STACKING_EVEN
Restrictions	None

channel_mode

The channel mode format (mono/stereo) of the analysis results

Type	Integer
Range	WOLA_MODE_MONO, WOLA_MODE_STEREO
Restrictions	None

input_addr

Address of the analysis results

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input_addr) != memtype(gain_vector_addr) (input_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N

gain_vector_addr

Address of the gain vector. This parameter is run-time configurable. The gain vector address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input_addr) != memtype(gain_vector_addr) (gain_vector_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N
Run-time Parameter	<name>_gain_vector_addr

output_addr

Address of the output vector

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(output_addr) != memtype(gain_vector_addr) (output_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N

4.5.2 State Variables

<name>_exp

The gain exponent for this gain application

Size	1
Modified By	None

4.5.3 Functions

Gain_C

```
Gain_C( name,
 N,
 stacking,
 channel_mode,
 input_addr,
 gain_vector_addr,
 output_addr )
```

Performs a complex gain application on the analysis results

4.6 MIX_CHANNELS

This module mixes stereo analysis results into a single set of mono analysis results.

Module Attributes

Single Function Yes

Reset No

BFP Support Shared

Table 34. Mix_Channels Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`N`	The size of the FFT used for the analysis results
`input_addr`	Address of the analysis results
`output_addr`	Address of the output vector

Table 35. Mix_Channels Functions

Functions	Description
`Mix_Channels`	Mixes stereo analysis results

4.6.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

N

The size of the FFT used for the analysis results

Type	Integer
Range	16, 32, 64, 128, 256, 512, 1024
Restrictions	None

input_addr

Address of the analysis results

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 4) == 0
Size	2*N

output_addr

Address of the output vector

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 4) == 0
Size	N

4.6.2 Functions

Mix_Channels

```
Mix_Channels( name,
 N,
 input_addr,
 output_addr )
```

Mixes stereo analysis results

4.7 ENERGY

This module computes the squared magnitude of the WOLA analysis results.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 36. Energy Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`N`	The size of the FFT used for the analysis results
`stacking`	The stacking format (even/odd) of the analysis results
`channel_mode`	The channel mode format (mono/stereo) of the analysis results
`input_addr`	Address of the analysis results
`output_addr`	Address of the output vector

Table 37. Energy Functions

Functions	Description
`Energy`	Performs the energy calculation on the analysis results

4.7.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

N

The size of the FFT used for the analysis results

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

stacking

The stacking format (even/odd) of the analysis results

Type	Integer
Range	WOLA_STACKING_ODD, WOLA_STACKING_EVEN
Restrictions	None

channel_mode

The channel mode format (mono/stereo) of the analysis results

Type	Integer
Range	WOLA_MODE_MONO, WOLA_MODE_STEREO
Restrictions	None

input_addr

Address of the analysis results. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input_addr) != memtype(output_addr) (input_addr % 2) == 0
Size	if (channel_mode == WOLA_MODE_MONO), N if (channel_mode != WOLA_MODE_MONO), 2*N
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. Outputs are in double-precision (2.46) format. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(output_addr) != memtype(input_addr) (output_addr % 2) == 0
Size	if ((stacking == WOLA_STACKING_EVEN) && (channel_mode == WOLA_MODE_MONO)), (N + 2) if ((stacking == WOLA_STACKING_EVEN) && (channel_mode != WOLA_MODE_MONO)), 2*(N + 2) if ((stacking == WOLA_STACKING_ODD) && (channel_mode == WOLA_MODE_MONO)), N if ((stacking == WOLA_STACKING_ODD) && (channel_mode != WOLA_MODE_MONO)), 2*N
Run-time Parameter	<name>_output_addr

4.7.2 Functions

Energy

```
Energy( name,
 N,
 stacking,
 channel_mode,
 input_addr,
 output_addr )
```

Performs the energy calculation on the analysis results

4.8 NORMALIZE

This module normalizes a set block floating values, e.g. the outputs from a WOLA filterbank analysis operation. It takes the input data and rescales them to a normalized block exponent. The outputs will be saturated if this rescaling causes overflow.

Module Attributes

Single Function Yes

Reset No

BFP Support Shared

Table 38. Normalize Constants

Constants	Description
NORMALIZE_NO_BFP_UPDATE	Indicates not to update the BFP block exponent with the normalized block exponent value
NORMALIZE_BFP_UPDATE	Indicates to update the BFP block exponent with the normalized block exponent value

Table 39. Normalize Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of 24-bit block floating point values to normalize
new_block_exp	The block exponent to normalize to
update_block_exp	A flag indicating whether or not to update the block floating point register with the new normalized value
input_addr	Address of the input vector
output_addr	Address of the normalized output vector

Table 40. Normalize Functions

Functions	Description
Normalize	Normalizes a set of block floating point values

4.8.1 Constants

NORMALIZE_NO_BFP_UPDATE

Indicates not to update the BFP block exponent with the normalized block exponent value

Value	0

NORMALIZE_BFP_UPDATE

Indicates to update the BFP block exponent with the normalized block exponent value

Value	1

4.8.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of 24-bit block floating point values to normalize.

Type	Integer
Range	[2-512]
Restrictions	None

new_block_exp

The block exponent to normalize to. This parameter is run-time configurable. The normalized block exponent can be changed by writing a new integer value to the run-time parameter's memory location.

Type	Integer
Range	[0-24]
Restrictions	None
Run-time Parameter	<name>_new_block_exp

update_block_exp

A flag indicating whether or not to update the block floating point register with the new normalized value.

Type	Integer
Range	NORMALIZE_NO_BFP_UPDATE, NORMALIZE_BFP_UPDATE
Restrictions	None

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, ACMEM, BDMEM, CDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	K
Run-time Parameter	<name>_input_addr

output_addr

Address of the normalized output vector. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location.

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, ACMEM, BDMEM, CDMEM
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	K
Run-time Parameter	<name>_output_addr

4.8.3 Functions

Normalize

```
Normalize( name,
 K,
 new_block_exp,
 update_block_exp,
 input_addr,
 output_addr )
```

Normalizes a set of block floating point values

4.9 BFP_STATE

This module contains functions to save and restore the current block floating point state.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 41. BFP_State Parameters

Parameters	Description
name	A unique name for each instance of the module

Table 42. BFP_State State Variables

State Variables	Description
<name>_bfp_state	The block floating point unit's state

Table 43. BFP_State Functions

Functions	Description
BFP_Save	Saves the current block floating point state
BFP_Restore	Restores the current block floating point state

4.9.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

4.9.2 State Variables

<name>_bfp_state

The block floating point unit's state.

Size	1
Modified By	BFP_Save

4.9.3 Functions

BFP_Save

```
BFP_Save( name )
```

Saves the current block floating point state

BFP_Restore

```
BFP_Restore( name )
```

Restores the current block floating point state

4.10 DFT_C_FORWARD

This module computes a complex valued Discrete Fourier Transform (DFT). It takes complex inputs and produces complex outputs. It performs the DFT using the Fast Fourier Transform (FFT) algorithm. Note that the non-power-of-2 DFT sizes use a built-in cos/sin table of 512 24-bit words located at the base of H45MEM, so if your project uses this module to compute any non-power-of-2 sized DFT, make sure you do not overwrite this part of H45MEM.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 44. DFT_C_Forward Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The size of the transform (the number of complex inputs)
fwd_input_addr	Address of the complex inputs for the forward DFT
fwd_output_addr	Address of the complex outputs for the forward DFT
temp01_addr	Address for intermediate FFT steps
temp23_addr	Address for intermediate FFT steps
fwd_input_addr_mode	The addressing mode for the input signal

Table 45. DFT_C_Forward Functions

Functions	Description
DFT_C_Forward	Computes the forward DFT of the input

4.10.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The size of the transform (the number of complex inputs).

Type	Integer
Range	4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384
Restrictions	None

fwd_input_addr

Address of the complex inputs for the forward DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	fwd_output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_input_addr % 2) == 0
Size	2*fwd_input_addr_mode*K

fwd_output_addr

Address of the complex outputs for the forward DFT

Type	Output
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	fwd_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_output_addr % 2) == 0
Size	2*K

temp01_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	None
Configuration Parameter Restrictions	if (npow2(K) == K), (temp01_addr % (2*K)) == 0 if ((K % 3) == 0), (temp01_addr % (2*K/3)) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	2*K

temp23_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	if (npow2(K) == K), (temp23_addr % (2*K)) == 0 if ((K % 3) == 0), (temp23_addr % (2*K/3)) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	2*K

fwd_input_addr_mode

The addressing mode for the input signal. Specifies the pointer increment to use when reading the next input data sample.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

4.10.2 Functions

DFT_C_Forward

```
DFT_C_Forward( name,
 K,
 fwd_input_addr,
 fwd_output_addr,
 temp01_addr,
 temp23_addr,
 fwd_input_addr_mode )
```

Computes the forward DFT of the input

4.11 DFT_C_INVERSE

This module computes a complex valued Inverse Discrete Fourier Transform (IDFT). It takes complex inputs and produces complex outputs. It performs the IDFT using the Inverse Fast Fourier Transform (IFFT) algorithm. Note that the non-power-of-2 IDFT sizes use a built-in cos/sin table of 512 24-bit words located at the base of H45MEM, so if your project uses this module to compute any non-power-of-2 sized IDFT, make sure you do not overwrite this part of H45MEM.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 46. DFT_C_Inverse Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The size of the transform (the number of complex inputs)
inv_input_addr	Address of the complex inputs for the inverse DFT
inv_output_addr	Address of the complex outputs for the inverse DFT
temp01_addr	Address for intermediate FFT steps
temp23_addr	Address for intermediate FFT steps
inv_output_addr_mode	The addressing mode for the output signal

Table 47. DFT_C_Inverse Functions

Functions	Description
DFT_C_Inverse	Computes the inverse DFT of the input

4.11.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The size of the transform (the number of complex inputs).

Type	Integer
Range	4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384
Restrictions	None

inv_input_addr

Address of the complex inputs for the inverse DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	inv_output_addr
Configuration Parameter Restrictions	if (npow2(K) == K), (inv_input_addr % (2*K)) == 0 if ((K == 12) || (K == 24)), (inv_input_addr % (8*K/3)) == 0
Address Parameter Restrictions	(inv_input_addr % 2) == 0
Size	2*K

inv_output_addr

Address of the complex outputs for the inverse DFT

Type	Output
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	inv_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(inv_output_addr % 2) == 0
Size	2*inv_output_addr_mode*K

temp01_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	None
Configuration Parameter Restrictions	if (npow2(K) == K), (temp01_addr % (2*K)) == 0 if ((K % 3) == 0), (temp01_addr % (2*K/3)) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	2*K

temp23_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	if (npow2(K) == K), (temp23_addr % (2*K)) == 0 if ((K % 3) == 0), (temp23_addr % (2*K/3)) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	2*K

inv_output_addr_mode

The addressing mode for the output signal. Specifies the pointer increment to use when writing the next output data sample.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

4.11.2 Functions

DFT_C_Inverse

```
DFT_C_Inverse( name,
 K,
 inv_input_addr,
 inv_output_addr,
 temp01_addr,
 temp23_addr,
 inv_output_addr_mode )
```

Computes the inverse DFT of the input

4.12 DFT_R_FORWARD

This module computes a real valued Discrete Fourier Transform (DFT). It takes real valued inputs and produces complex outputs. Because a real valued sequence has a conjugate symmetric DFT, only the first half of the data points are generated. This module performs the DFT using the Fast Fourier Transform (FFT) algorithm.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 48. DFT_R_Forward Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The size of the transform (the number of real-valued inputs)
`fwd_input_addr`	Address of the real inputs for the forward DFT
`fwd_output_addr`	Address of the complex outputs for the forward DFT
`temp01_addr`	Address for intermediate FFT steps
`temp23_addr`	Address for intermediate FFT steps
`fwd_input_addr_mode`	The addressing mode for the input signal

Table 49. DFT_R_Forward Functions

Functions	Description
`DFT_R_Forward`	Computes the forward DFT of the input

4.12.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The size of the transform (the number of real-valued inputs)

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

fwd_input_addr

Address of the real inputs for the forward DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	fwd_output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_input_addr % 2) == 0
Size	fwd_input_addr_mode*K

fwd_output_addr

Address of the complex outputs for the forward DFT

Type	Output
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	fwd_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_output_addr % 2) == 0
Size	K

temp01_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp01_addr % K) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	K

temp23_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp23_addr % K) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	K

fwd_input_addr_mode

The addressing mode for the input signal. Specifies the pointer increment to use when reading the next input data sample.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

4.12.2 Functions

DFT_R_Forward

```
DFT_R_Forward( name,
 K,
 fwd_input_addr,
 fwd_output_addr,
 temp01_addr,
 temp23_addr,
 fwd_input_addr_mode )
```

Computes the forward DFT of the input

4.13 DFT_R_INVERSE

This module computes a real valued Inverse Discrete Fourier Transform (IDFT). It takes the first half of the DFT data points (complex valued) as input and produces real valued outputs. This module performs the IDFT using the Inverse Fast Fourier Transform (IFFT) algorithm.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 50. DFT_R_Inverse Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The size of the transform (the number of real-valued outputs)
`inv_input_addr`	Address of the complex inputs for the inverse DFT
`inv_output_addr`	Address of the real outputs for the inverse DFT
`temp01_addr`	Address for intermediate FFT steps
`temp23_addr`	Address for intermediate FFT steps
`inv_output_addr_mode`	The addressing mode for the output signal

Table 51. DFT_R_Inverse Functions

Functions	Description
DFT_R_Inverse	Computes the inverse DFT of the input

4.13.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The size of the transform (the number of real-valued outputs)

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

inv_input_addr

Address of the complex inputs for the inverse DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	inv_output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(inv_input_addr % 2) == 0
Size	K

inv_output_addr

Address of the real outputs for the inverse DFT

Type	Output
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	inv_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(inv_output_addr % 2) == 0
Size	inv_output_addr_mode*K

temp01_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp01_addr % K) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	K

temp23_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp23_addr % K) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	K

inv_output_addr_mode

The addressing mode for the output signal. Specifies the pointer increment to use when writing the next output data sample.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

4.13.2 Functions

DFT_R_Inverse

```
DFT_R_Inverse( name,
 K,
 inv_input_addr,
 inv_output_addr,
 temp01_addr,
 temp23_addr,
 inv_output_addr_mode )
```

Computes the inverse DFT of the input

4.14 WIN_DFT_R_FORWARD

This module computes a real valued Discrete Fourier Transform (DFT) with windowing of the forward DFT input. It takes real valued inputs and produces complex outputs. Because a real valued sequence has a conjugate symmetric DFT, only the first half of the data points are generated. This module performs the DFT using the Fast Fourier Transform (FFT) algorithm.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 52. Win_DFT_R_Forward Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The size of the transform (the number of real-valued inputs)
fwd_input_addr	Address of the real inputs for the forward DFT
fwd_output_addr	Address of the complex outputs for the forward DFT
temp01_addr	Address for intermediate FFT steps
temp23_addr	Address for intermediate FFT steps
fwd_window_addr	Address of the window to use during the forward DFT
fwd_input_addr_mode	The addressing mode for the input signal
fwd_window_addr_mode	The addressing mode for the window to use during forward DFT

Table 53. Win_DFT_R_Forward Functions

Functions	Description
Win_DFT_R_Forward	Computes the windowed forward DFT of the input

4.14.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The size of the transform (the number of real-valued inputs)

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

fwd_input_addr

Address of the real inputs for the forward DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	fwd_output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_input_addr % 2) == 0 memtype(fwd_window_addr) != memtype(fwd_input_addr)
Size	fwd_input_addr_mode*K

fwd_output_addr

Address of the complex outputs for the forward DFT

Type	Output
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	fwd_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_output_addr % 2) == 0
Size	K

temp01_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp01_addr % K) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	K

temp23_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp23_addr % K) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	K

fwd_window_addr

Address of the window to use during the forward DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(fwd_window_addr % 2) == 0 memtype(fwd_window_addr) != memtype(fwd_input_addr)
Size	fwd_window_addr_mode*K

fwd_input_addr_mode

The addressing mode for the input signal. Specifies the pointer increment to use when reading the next input data sample.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

fwd_window_addr_mode

The addressing mode for the window to use during forward DFT. Specifies the pointer increment to use when reading the next element of the window.

Type	Integer
Range	[1-32]
Restrictions	None
Default Value	1

4.14.2 Functions

Win_DFT_R_Forward

```
Win_DFT_R_Forward( name,
 K,
 fwd_input_addr,
 fwd_output_addr,
 temp01_addr,
 temp23_addr,
 fwd_window_addr,
 fwd_input_addr_mode,
 fwd_window_addr_mode )
```

Computes the windowed forward DFT of the input

4.15 WIN_DFT_R_INVERSE

This module computes a real valued Inverse Discrete Fourier Transform (IDFT) with windowing of the inverse DFT output. It takes the DFT data points (complex valued) as input and produces real valued outputs. This module performs the inverse DFT using the Inverse Fast Fourier Transform (IFFT) algorithm.

Module Attributes

Single Function No

Reset No

BFP Support Full

Table 54. Win_DFT_R_Inverse Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The size of the transform (the number of real-valued outputs)
inv_input_addr	Address of the complex inputs for the inverse DFT
inv_output_addr	Address of the real outputs for the inverse DFT
temp01_addr	Address for intermediate FFT steps
temp23_addr	Address for intermediate FFT steps
inv_window_addr	Address of the window to use during the inverse DFT
inv_output_addr_mode	The addressing mode for the output signal
inv_window_addr_mode	The addressing mode for the window to use during inverse DFT

Table 55. Win_DFT_R_Inverse Functions

Functions	Description
Win_DFT_R_Inverse	Computes the windowed inverse DFT of the input

4.15.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The size of the transform (the number of real-valued outputs)

Type	Integer
Range	16, 32, 64, 128, 256, 512
Restrictions	None

inv_input_addr

Address of the complex inputs for the inverse DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	inv_output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(inv_input_addr % 2) == 0
Size	K

inv_output_addr

Address of the real outputs for the inverse DFT

Type	Output
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	inv_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(inv_output_addr % 2) == 0 if ((inv_output_addr_mode != 1) || (inv_window_addr_mode != 1)), memtype(inv_window_addr) != memtype(inv_output_addr)
Size	inv_output_addr_mode*K

temp01_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H01MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp01_addr % K) == 0
Address Parameter Restrictions	(temp01_addr % 4) == 0
Size	K

temp23_addr

Address for intermediate FFT steps

Type	Scratch
Memory Spaces	H23MEM
Shared Parameters	None
Configuration Parameter Restrictions	(temp23_addr % K) == 0
Address Parameter Restrictions	(temp23_addr % 4) == 0
Size	K

inv_window_addr

Address of the window to use during the inverse DFT

Type	Input
Memory Spaces	AMEM, BMEM, CMEM, DMEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(inv_window_addr % 2) == 0 if ((inv_output_addr_mode != 1) || (inv_window_addr_mode != 1)), memtype(inv_window_addr) != memtype(inv_output_addr)
Size	inv_window_addr_mode*K

inv_output_addr_mode

The addressing mode for the output signal. Specifies the pointer increment to use when writing the next output data sample.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

inv_window_addr_mode

The addressing mode for the window to use during inverse DFT. Specifies the pointer increment to use when reading the next element of the window.

Type	Integer
Range	[1-32]
Restrictions	None
Default Value	1

4.15.2 Functions

Win_DFT_R_Inverse

```
Win_DFT_R_Inverse( name,
 K,
 inv_input_addr,
 inv_output_addr,
 temp01_addr,
 temp23_addr,
 inv_window_addr,
 inv_output_addr_mode,
 inv_window_addr_mode )
```

Computes the windowed inverse DFT of the input

4.16 ATTACK_RELEASE

This module implements first-order attack-release filtering for averaging signal values such as energy calculations. It is designed to perform multi-channel filtering. Filter coefficients for each channel can be specified separately. The inputs can be single or double precision. The outputs are always stored in double precision.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 56. Attack_Release Constants

Constants	Description
ATTACK_RELEASE_SINGLE_PRECISION	Specify single precision inputs
ATTACK_RELEASE_DOUBLE_PRECISION	Specify double precision inputs

Table 57. Attack_Release Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of input channels to be filtered

Table 57. Attack_Release Parameters (Continued)

Parameters	Description
input_addr	Address of the filter inputs
output_addr	Address of the filter outputs
alpha_addr	Address of the attack coefficients
beta_addr	Address of the release coefficients
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
alpha_addr_mode	Specifies the pointer increment to use when reading the next element of alpha
beta_addr_mode	Specifies the pointer increment to use when reading the next element of beta
input_precision	The size of each input element, either single precision (one word) or double precision (two words)

Table 58. Attack_Release Functions

Functions	Description
Attack_Release	Computes attack-release filtered outputs

4.16.1 Constants

ATTACK_RELEASE_SINGLE_PRECISION

Specify single precision inputs

Value	1

ATTACK_RELEASE_DOUBLE_PRECISION

Specify double precision inputs

Value	2

4.16.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of input channels to be filtered. This parameter is run-time configurable. The value of K can be changed by writing a new value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the filter inputs. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_precision == ATTACK_RELEASE_DOUBLE_PRECISION), (input_addr % 2) == 0
Size	abs(input_addr_mode)*input_precision*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the filter outputs. The past output (i.e., y(n-1)) must remain in the output buffer prior to launching the function, and the new filtered output (i.e., y(n)) will overwrite the past output at the same location. The application software is responsible for initializing the output buffer (e.g., to zero). This parameter is run-time configurable.

Type	Input/Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*2*K
Run-time Parameter	<name>_output_addr

alpha_addr

Address of the attack coefficients. This parameter is run-time configurable. The coefficient address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	if (alpha_addr_mode == 0), 1 if (alpha_addr_mode != 0), abs(alpha_addr_mode)*K
Run-time Parameter	<name>_alpha_addr

beta_addr

Address of the release coefficients. This parameter is run-time configurable. The coefficient address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	if (beta_addr_mode == 0), 1 if (beta_addr_mode != 0), abs(beta_addr_mode)*K
Run-time Parameter	<name>_beta_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512] when (input_precision == ATTACK_RELEASE_SINGLE_PRECISION), [-256-256] when (input_precision == ATTACK_RELEASE_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

alpha_addr_mode

Specifies the pointer increment to use when reading the next element of alpha. When this parameter is not zero, it is the addressing mode for the alpha vector. When this parameter is zero, it indicates a scalar format for alpha (one coefficient to be applied to all input data elements).

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

beta_addr_mode

Specifies the pointer increment to use when reading the next element of beta. When this parameter is not zero, it is the addressing mode for the beta vector. When this parameter is zero, it indicates a scalar format for beta (one coefficient to be applied to all input data elements).

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input_precision

The size of each input element, either single precision (one word) or double precision (two words).

Type	Integer
Range	ATTACK_RELEASE_SINGLE_PRECISION, ATTACK_RELEASE_DOUBLE_PRECISION
Restrictions	None
Default Value	ATTACK_RELEASE_SINGLE_PRECISION

4.16.3 Functions

Attack_Release

```
Attack_Release( name,
 K,
 input_addr,
 output_addr,
 alpha_addr,
 beta_addr,
```

```
input_addr_mode,
output_addr_mode,
alpha_addr_mode,
beta_addr_mode,
input_precision )
```

Computes attack-release filtered outputs

4.17 FIR_R

This module computes the FIR filtered outputs for a block of input samples (i.e., it implements a block-based FIR routine). The module can access interleaved data streams for inputs and outputs (e.g. for filtering one channel of a stereo FIFO). This module uses either single-precision (24-bit) or double-precision (48-bit) filter coefficients. The input and output samples are single precision (24-bit) values.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 59. FIR_R Constants

Constants	Description
FIR_R_SINGLE_PRECISION	Specify single-precision coefficients
FIR_R_DOUBLE_PRECISION	Specify double-precision coefficients

Table 60. FIR_R Parameters

Parameters	Description
name	A unique name for each instance of the module
R	The block size for the FIR filter processing
L	The length of the FIR filter (i.e., the number of coefficients)
scalefactor	The number of left shifts to apply to the FIR filter outputs
input_addr	Address for the most recent sample
output_addr	Address for the most recent FIR filter output
coeff_addr	Address of the FIR filter coefficients
input_addr_mode	Channel mode for the input signal
output_addr_mode	Channel mode for the output signal
coeff_addr_mode	The addressing mode for the filter coefficients vector
coeff_precision	The size of each filter coefficient, either single precision (one word) or double precision (two words)

Table 61. FIR_R Functions

Functions	Description
FIR_R	Computes the FIR filtered outputs for a new input block

4.17.1 Constants

FIR_R_SINGLE_PRECISION

Specify single-precision coefficients

Value	1

FIR_R_DOUBLE_PRECISION

Specify double-precision coefficients

Value	2

4.17.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

R

The block size for the FIR filter processing

Type	Integer
Range	[2-512]
Restrictions	(R % 2) == 0

L

The length of the FIR filter (i.e., the number of coefficients)

Type	Integer
Range	[3-512]
Restrictions	None

scalefactor

The number of left shifts to apply to the FIR filter outputs. Useful if the magnitude of the FIR filter coefficients exceeds one, since coefficients like these cannot be represented in a purely fractional format. If the scale factor is

negative, then the module actually applies a right shift. This parameter is run-time configurable. The parameter value stored in shared memory is negated relative to the parameter value supplied at compile-time.

Type	Integer
Range	[-24 - 24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input_addr

Address for the most recent sample. Past samples in the same block are expected to be at higher addresses from this one at intervals specified by input_addr_mode.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((coeff_precision == FIR_R_SINGLE_PRECISION) && (input_addr_mode == 1)), (input_addr % 2) == 0
Size	input_addr_mode*(R + L)

output_addr

Address for the most recent FIR filter output. Past outputs in the same block are placed at higher addresses from this one at intervals specified by output_addr_mode. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((coeff_precision == FIR_R_SINGLE_PRECISION) && (output_addr_mode == 1)), (output_addr % 2) == 0
Size	output_addr_mode*R
Run-time Parameter	<name>_output_addr

coeff_addr

Address of the FIR filter coefficients. This parameter is run-time configurable. The coefficient address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	if (coeff_addr_mode > 0), (coeff_addr % npow2(abs(coeff_addr_mode)*coeff_precision*L)) == 0 if (coeff_addr_mode < 0), ((coeff_addr - abs(coeff_addr_mode)*coeff_precision*(L-1)) % npow2(abs(coeff_addr_mode)*coeff_precision*L)) == 0
Address Parameter Restrictions	memtype(coeff_addr) != memtype(input_addr)
Size	abs(coeff_addr_mode)*coeff_precision*L
Run-time Parameter	<name>_coeff_addr

input_addr_mode

Channel mode for the input signal. Specifies the pointer increment to use when reading the next input data sample; e.g., for a single mono input stream set this to 1, and for a stereo input stream set this to 2.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

output_addr_mode

Channel mode for the output signal. Specifies the pointer increment to use when writing the next output data sample; e.g., for a single mono output stream set this to 1, and for a stereo output stream set this to 2.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

coeff_addr_mode

The addressing mode for the filter coefficients vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256] when (coeff_precision == FIR_R_SINGLE_PRECISION), [-128-128] when (coeff_precision == FIR_R_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

coeff_precision

The size of each filter coefficient, either single precision (one word) or double precision (two words).

Type	Integer
Range	FIR_R_SINGLE_PRECISION, FIR_R_DOUBLE_PRECISION
Restrictions	None
Default Value	FIR_R_SINGLE_PRECISION

4.17.3 Functions

FIR_R

```
FIR_R( name,
 R,
 L,
 scalefactor,
 input_addr,
 output_addr,
 coeff_addr,
 input_addr_mode,
 output_addr_mode,
 coeff_addr_mode,
 coeff_precision )
```

Computes the FIR filtered outputs for a new input block

4.18 IIR_R

This module implements a time-domain (real-valued) infinite impulse response (IIR) filter. It uses a Direct-Form I implementation and supports both single- and double-precision output formats. The filter coefficients and input samples are single-precision. The coefficients are stored in the same order and format used for the transfer function view of the filter.

The implementation can support input or output signals that are interleaved with other data in memory (e.g., one channel from a stereo FIFO as input and one channel from a stereo FIFO as output).

The recursive and non-recursive coefficients are located in individual contiguous or interleaved portions of memory.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 62. IIR_R Constants

Constants	Description
IIR_R_SINGLE_PRECISION	Specify single-precision coefficients and outputs
IIR_R_DOUBLE_PRECISION	Specify double-precision coefficients and outputs

Table 63. IIR_R Parameters

Parameters	Description
name	A unique name for each instance of the module
R	The block size for the IIR filter processing
Mb	The number of non-recursive (input-filtered) filter coefficients
Ma	The number of recursive (output-filtered) filter coefficients
scalefactor	The number of left shifts to apply to the filter outputs
input_addr	Address for the most recent input sample
output_addr	Address where the last new filter output should go
coeff_Mb_addr	Address for the non-recursive IIR filter coefficients
coeff_Ma_addr	Address for the recursive IIR filter coefficients
input_addr_mode	Channel mode for the input signal
output_addr_mode	Channel mode for the output signal
coeff_Mb_addr_mode	The addressing mode for the non-recursive filter coefficients vector
coeff_Ma_addr_mode	The addressing mode for the recursive filter coefficients vector
output_precision	The size of each filter coefficient and output element, either single precision (one word) or double precision (two words)

Table 64. IIR_R Functions

Functions	Description
IIR_R	Computes the IIR filtered outputs for the most recent input block

4.18.1 Constants

IIR_R_SINGLE_PRECISION

Specify single-precision coefficients and outputs

Value	1

IIR_R_DOUBLE_PRECISION

Specify double-precision coefficients and outputs

Value	2

4.18.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	any
Restrictions	None

R

The block size for the IIR filter processing

Type	Integer
Range	[2-512] when (output_precision == IIR_R_SINGLE_PRECISION), [2-256] when (output_precision == IIR_R_DOUBLE_PRECISION)
Restrictions	(R % 2) == 0

Mb

The number of non-recursive (input-filtered) filter coefficients

Type	Integer
Range	[1-128]
Restrictions	None

Ma

The number of recursive (output-filtered) filter coefficients. The first coefficient is always assumed to be 1, so only (Ma-1) coefficients must be stored in memory.

Type	Integer
Range	[2-128]
Restrictions	None

scalefactor

The number of left shifts to apply to the filter outputs. Useful if the magnitude of the filter coefficients exceeds one, since coefficients like these cannot be represented in a purely fractional format. If the scale factor is negative, then the module actually applies a right shift. This parameter is run-time configurable. The parameter value stored in shared memory is negated relative to the parameter value supplied at compile-time.

Type	Integer
Range	[-24 - 24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input_addr

Address for the most recent input sample. Past samples in the same block are expected to be at higher addresses from this one at intervals according to input_addr_mode.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	input_addr_mode*(R + Mb)

output_addr

Address where the last new filter output should go. That is, the base of the new output block. Past outputs in the same block are placed at higher addresses from this one in intervals according to ouput_addr_mode. The IIR_R function will fill in this block starting at output_addr + output_addr_mode*(R - 1).

Type	Input/Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	output_addr_mode*output_precision*(R + Ma - 1)

coeff_Mb_addr

Address for the non-recursive IIR filter coefficients. Ordered from b(0), b(1), ... b(Mb-1). This parameter is run-time configurable. The coefficient address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	if (coeff_Mb_addr_mode > 0), (coeff_Mb_addr % npow2(abs(coeff_Mb_addr_mode)*Mb)) == 0 if (coeff_Mb_addr_mode < 0), (coeff_Mb_addr - abs(coeff_Mb_addr_mode)*(Mb-1)) % npow2(abs(coeff_Mb_addr_mode)*Mb) == 0
Address Parameter Restrictions	memtype(coeff_Mb_addr) != memtype(input_addr)
Size	abs(coeff_Mb_addr_mode)*Mb
Run-time Parameter	<name>_coeff_Mb_addr

coeff_Ma_addr

Address for the recursive IIR filter coefficients. Ordered from a(1), a(2), ... a(Ma-1). The lead coefficient, a(0), is assumed to be one. This parameter is run-time configurable. The coefficient address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	if (coeff_Ma_addr_mode > 0), (coeff_Ma_addr % npow2(abs(coeff_Ma_addr_mode)*(Ma-1))) == 0 if (coeff_Ma_addr_mode < 0), ((coeff_Ma_addr - abs(coeff_Ma_addr_mode)*(Ma-2)) % npow2(abs(coeff_Ma_addr_mode)*(Ma-1))) == 0
Address Parameter Restrictions	memtype(coeff_Ma_addr) != memtype(output_addr)
Size	abs(coeff_Ma_addr_mode)*(Ma - 1)
Run-time Parameter	<name>_coeff_Ma_addr

input_addr_mode

Channel mode for the input signal. Specifies the pointer increment to use when reading the next input data sample; e.g., for a single mono input stream set this to 1, and for a stereo input stream set this to 2.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

output_addr_mode

Channel mode for the output signal. Specifies the pointer increment to use when writing the next output data sample; e.g., for a single mono output stream set this to 1, and for a stereo output stream set this to 2.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

coeff_Mb_addr_mode

The addressing mode for the non-recursive filter coefficients vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

coeff_Ma_addr_mode

The addressing mode for the recursive filter coefficients vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_precision

The size of each filter coefficient and output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	IIR_R_SINGLE_PRECISION, IIR_R_DOUBLE_PRECISION
Restrictions	None
Default Value	IIR_R_SINGLE_PRECISION

4.18.3 Functions

IIR_R

```
IIR_R( name,
 R,
 Mb,
 Ma,
 scalefactor,
 input_addr,
 output_addr,
 coeff_Mb_addr,
 coeff_Ma_addr,
 input_addr_mode,
 output_addr_mode,
 coeff_Mb_addr_mode,
 coeff_Ma_addr_mode,
 output_precision )
```

Computes the IIR filtered outputs for the most recent input block

4.19 BIQUAD_IIR

This module implements a time-domain (real-valued) infinite impulse response (IIR) filter composed of one or more second-order IIR filters, sometimes called biquad filters or second-order sections. Each filter is implemented with a Direct-Form I implementation.

The implementation can support input or output signals that are interleaved with other data in memory (e.g., one channel from a stereo FIFO as input and one channel from a stereo FIFO as output).

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 65. Biquad_IIR Parameters

Parameters	Description
name	A unique name for each instance of the module
R	The block size for the filter processing
K	The number of biquad filter sections
input_addr	Address for the most recent sample
output_addr	Address for the most recent IIR filter output
temp_addr	Address for storing the intermediate outputs from each biquad filter
coeff_addr	Address for biquad filter coefficients
input_addr_mode	Channel mode for the input signal
output_addr_mode	Channel mode for the output signal

Table 66. Biquad_IIR Functions

Functions	Description
Biquad_IIR	Computes the cascaded biquad filtered outputs for the most recent input block

4.19.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	any
Restrictions	None

R

The block size for the filter processing.

Type	Integer
Range	[2-512]
Restrictions	(R % 2) == 0

K

The number of biquad filter sections

Type	Integer
Range	[1-16]
Restrictions	None

input_addr

Address for the most recent sample. Past samples in the same block are expected to be at higher addresses from this one at intervals according to input_addr_mode. The past data must include the two most recent samples from the previous block of input samples.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input_addr) != memtype(coeff_addr)
Size	input_addr_mode*(R + 2)

output_addr

Address for the most recent IIR filter output. Past outputs in the same block are placed at higher addresses from this one in intervals according to output_addr_mode. The past data must include the two most recent samples from the previous block of output samples.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(output_addr) != memtype(coeff_addr)
Size	output_addr_mode*(R + 2)

temp_addr

Address for storing the intermediate outputs from each biquad filter. This area must normally be cleared to zero before the filter's first execution as it represents the past internal state of each filter section.

Type	Input/Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(temp_addr % 2) == 0 memtype(temp_addr) != memtype(coeff_addr)
Size	(K - 1)*(R + 2)

coeff_addr

Address for biquad filter coefficients. Ordered as b(0), b(1), b(2), a(1), a(2), scale_factor where b(n) represents the feedforward (non-recursive) coefficients, a(n) represents the feedback (recursive) coefficients and scale_factor represents a right shift used to scale the filter output. A negative scale factor implies a left shift to scale the output. Each biquad's coefficients are located contiguously in this format from the first filter to the last filter. This parameter is

run-time configurable. The coefficient address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(coeff_addr % 2) == 0 memtype(coeff_addr) != memtype(input_addr) memtype(coeff_addr) != memtype(output_addr) memtype(coeff_addr) != memtype(temp_addr)
Size	6*K
Run-time Parameter	<name>_coeff_addr

input_addr_mode

Channel mode for the input signal. Specifies the pointer increment to use when reading the next input data sample. For example, for a single mono input stream, set this to 1; for a stereo input stream, set this to 2.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

output_addr_mode

Channel mode for the output signal. Specifies the pointer increment to use when writing the next output data sample. For example, for a single mono output stream, set this to 1; for a stereo output stream, set this to 2.

Type	Integer
Range	[1-4]
Restrictions	None
Default Value	1

4.19.2 Functions

Biquad_IIR

```
Biquad_IIR( name,
 R,
 K,
 input_addr,
 output_addr,
 temp_addr,
 coeff_addr,
 input_addr_mode,
 output_addr_mode )
```

Computes the cascaded biquad filtered outputs for the most recent input block

4.20 sFIR_C

This module computes FIR filtered outputs for a set of complex subbands. The FIR coefficients used are also complex-valued. One set of FIR outputs is produced per execution (i.e., one complex output per subband). The filtering process assumes a set of complex interleaved values representing the complex signal in each subband. Older samples are located at higher addresses in memory.

The FIR coefficients themselves are not interleaved, but are rather stored in contiguous "chunks". That is, all the coefficients for the first band are stored contiguously, and then all the coefficients for the next band and so on.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 67. sFIR_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of subbands to filter
L	The length of each complex filter
scalefactor	The number of left shifts to apply to the filter outputs
input_addr	Address for the input signals
output_addr	Address for the FIR filter output
coeff_addr	Address for the FIR filter coefficients

Table 68. sFIR_C Functions

Functions	Description
sFIR_C	Performs a multi-band complex FIR filtering of normalized subband signals

4.20.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of subbands to filter

Type	Integer
Range	[2-256]
Restrictions	None

L

The length of each complex filter

Type	Integer
Range	[2-128]
Restrictions	None

scalefactor

The number of left shifts to apply to the filter outputs. Useful if the magnitude of the filter coefficients exceeds one, since coefficients like these cannot be represented in a purely fractional format. If the scale factor is negative, then the module actually applies a right shift. This parameter is run-time configurable. The parameter value stored in shared memory is negated relative to the parameter value supplied at compile-time.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input_addr

Address for the input signals. Older samples are assumed to be at higher addresses from this base and the bands are assumed to be interleaved.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(coeff_addr) != memtype(input_addr) (input_addr % 2) == 0
Size	2*K*L

output_addr

Address for the FIR filter output. The outputs are stored in interleaved fashion. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	2*K
Run-time Parameter	<name>_output_addr

coeff_addr

Address for the FIR filter coefficients. This parameter is run-time configurable. The coefficient address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(coeff_addr) != memtype(input_addr) (coeff_addr % 2) == 0
Size	2*L*K
Run-time Parameter	<name>_coeff_addr

4.20.2 Functions

sFIR_C

```
sFIR_C( name,
 K,
 L,
 scalefactor,
 input_addr,
 output_addr,
 coeff_addr )
```

Performs a multi-band complex FIR filtering of normalized subband signals

4.21 sIIR_C

This module computes IIR filtered outputs for a set of complex subbands. The IIR coefficients used are also complex-valued. One set of IIR outputs is produced per execution (i.e., one complex output per subband). The filtering process assumes a set of complex interleaved values representing the complex signal in each subband. Older samples are located at higher addresses in memory.

This module uses a single precision Direct-Form I implementation and as such is not recommended for higher-order filters due to finite precision effects.

The coefficients are stored in the same order and format used in the transfer function view of the filter. The IIR coefficients are not interleaved like the bands, but are rather stored in contiguous "chunks". That is, all the coefficients for the first band are stored contiguously, and then all the coefficients for the next band and so on. The feedforward and feedback coefficients are stored separately.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 69. sIIR_C Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of subbands to filter
`Mb`	The number of non-recursive (input filtered) filter coefficients
`Ma`	The number of recursive (output filtered) filter coefficients
`scalefactor`	The number of left shifts to apply to the filter outputs
`input_addr`	Address of the input signals
`output_addr`	Address of the output signals
`coeffs_Mb_addr`	Address for the non-recursive IIR filter coefficients
`coeffs_Ma_addr`	Address for the recursive IIR filter coefficients

Table 70. sIIR_C Functions

Functions	Description
`sIIR_C`	Performs multi-band complex IIR filtering of normalized subband signals

4.21.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of subbands to filter

Type	Integer
Range	[2-256]
Restrictions	None

Mb

The number of non-recursive (input filtered) filter coefficients

Type	Integer
Range	[1-128]
Restrictions	None

Ma

The number of recursive (output filtered) filter coefficients. The first coefficient is always assumed to be 1, so only (Ma-1) coefficients must be stored in memory.

Type	Integer
Range	[2-128]
Restrictions	None

scalefactor

The number of left shifts to apply to the filter outputs. Useful if the magnitude of the filter coefficients exceeds one, since coefficients like these cannot be represented in a purely fractional format. If the scale factor is negative, then the module actually applies a right shift. This parameter is run-time configurable. The parameter value stored in shared memory is negated relative to the parameter value supplied at compile-time.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input_addr

Address of the input signals. Older samples are assumed to be at higher addresses from this base and the bands are assumed to be interleaved.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(coeffs_Mb_addr) != memtype(input_addr) (input_addr % 2) == 0
Size	2*K*Mb

output_addr

Address of the output signals. Older samples are assumed to be at higher addresses from this base and the bands are assumed to be interleaved.

Type	Input/Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(coeffs_Ma_addr) != memtype(output_addr) (output_addr % 2) == 0
Size	2*K*Ma

coeffs_Mb_addr

Address for the non-recursive IIR filter coefficients. Ordered from b(0), b(1), ... b(Mb-1) and stored for each band in sequential order. This parameter is run-time configurable. The address for non-recursive coefficients can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(coeffs_Mb_addr) != memtype(input_addr) (coeffs_Mb_addr % 2) == 0
Size	2*K*Mb
Run-time Parameter	<name>_coeffs_Mb_addr

coeffs_Ma_addr

Address for the recursive IIR filter coefficients. Ordered from a(1), a(2), ... a(Ma-1) and stored for each band in sequential order. The lead coefficient, a(0), is assumed to one for all bands. This parameter is run-time configurable. The address for recursive coefficients can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(coeffs_Ma_addr) != memtype(output_addr) (coeffs_Ma_addr % 2) == 0
Size	2*K*(Ma - 1)
Run-time Parameter	<name>_coeffs_Ma_addr

4.21.2 Functions

sIIR_C

```
sIIR_C( name,
 K,
 Mb,
 Ma,
 scalefactor,
 input_addr,
 output_addr,
 coeffs_Mb_addr,
 coeffs_Ma_addr )
```

Performs multi-band complex IIR filtering of normalized subband signals

4.22 EXP_AVERAGE

This module implements a first-order IIR filter for exponential averaging signal values such as energy calculations. It is designed to perform multi-channel filtering. Filter coefficients can be specified as a scalar or a vector. The inputs can be single or double precision. The outputs are always stored in double precision.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 71. Exp_Average Constants

Constants	Description
EXP_AVERAGE_SINGLE_PRECISION	Specify single-precision inputs
EXP_AVERAGE_DOUBLE_PRECISION	Specify double-precision inputs

Table 72. Exp_Average Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of input channels to be filtered
input_addr	Address of the filter inputs
output_addr	Address of the filter outputs
alpha_addr	Address of the coefficients
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
alpha_addr_mode	Specifies the pointer increment to use when reading the next element of alpha
input_precision	The size of each input element, either single precision (one word) or double precision (two words)

Table 73. Exp_Average Functions

Functions	Description
`Exp_Average`	Computes exponential averaged outputs

4.22.1 Constants

EXP_AVERAGE_SINGLE_PRECISION

Specify single-precision inputs

Value	1

EXP_AVERAGE_DOUBLE_PRECISION

Specify double-precision inputs

Value	2

4.22.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of input channels to be filtered. This parameter is run-time configurable. The value of K can be changed by writing a new value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the filter inputs. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_precision == EXP_AVERAGE_DOUBLE_PRECISION), (input_addr % 2) == 0
Size	abs(input_addr_mode)*input_precision*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the filter outputs. The past output (i.e., y(n-1)) must remain in the output buffer prior to launching the function, and the new filtered output (i.e, y(n)) will overwrite the past output at the same location. The application software is responsible for initializing the output buffer (e.g., to zero). This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input/Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*2*K
Run-time Parameter	<name>_output_addr

alpha_addr

Address of the coefficients. This parameter is run-time configurable. The coefficient address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	if (alpha_addr_mode == 0), 1 if (alpha_addr_mode != 0), abs(alpha_addr_mode)*K
Run-time Parameter	<name>_alpha_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512] when (input_precision == EXP_AVERAGE_SINGLE_PRECISION), [-256-256] when (input_precision == EXP_AVERAGE_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

alpha_addr_mode

Specifies the pointer increment to use when reading the next element of alpha. When this parameter is not zero, it is the addressing mode for the alpha vector. When this parameter is zero, it indicates a scalar format for alpha (one coefficient to be applied to all input data elements).

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input_precision

The size of each input element, either single precision (one word) or double precision (two words).

Type	Integer
Range	EXP_AVERAGE_SINGLE_PRECISION, EXP_AVERAGE_DOUBLE_PRECISION
Restrictions	None
Default Value	EXP_AVERAGE_SINGLE_PRECISION

4.22.3 Functions

Exp_Average

```
Exp_Average( name,
 K,
 input_addr,
 output_addr,
 alpha_addr,
 input_addr_mode,
```

```
output_addr_mode,
alpha_addr_mode,
input_precision )
```

Computes exponential averaged outputs

4.23 vADD_C

This module adds two complex-valued vectors of equal length together in an element-wise fashion. The two input vectors and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 74. vAdd_C Constants

Constants	Description
VADD_C_SINGLE_PRECISION	Specify single-precision input/output
VADD_C_DOUBLE_PRECISION	Specify double-precision input/output

Table 75. vAdd_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of the real or imaginary part of each input/output element

Table 76. vAdd_C Functions

Functions	Description
vAdd_C	Performs the complex-valued vector addition

4.23.1 Constants

VADD_C_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VADD_C_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.23.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input0_addr % 2) == 0
Size	2*abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	2*abs(input1_addr_mode)*precision*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	2*abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

precision

The size of the real or imaginary part of each input/output element. It can be single precision (one word) or double precision (two words).

Type	Integer
Range	VADD_C_SINGLE_PRECISION, VADD_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VADD_C_SINGLE_PRECISION

4.23.3 Functions

vAdd_C

```
vAdd_C( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs the complex-valued vector addition

4.24 vADD_R

This module adds two real-valued vectors of equal length together in an element-wise fashion. The two input vectors and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used (e.g., blocks of stereo audio in a FIFO).

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 77. vAdd_R Constants

Constants	Description
VADD_R_SINGLE_PRECISION	Specify single-precision input/output
VADD_R_DOUBLE_PRECISION	Specify double-precision input/output

Table 78. vAdd_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each input/output element, either single precision (one word) or double precision (two words)

Table 79. vAdd_R Functions

Functions	Description
vAdd_R	Performs the real-valued vector addition

4.24.1 Constants

VADD_R_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VADD_R_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.24.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VADD_R_DOUBLE_PRECISION)), (input0_addr % 2) == 0 if (precision == VADD_R_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VADD_R_DOUBLE_PRECISION)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr) if (precision == VADD_R_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input1_addr_mode)*precision*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VADD_R_DOUBLE_PRECISION)), (output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each input/output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VADD_R_SINGLE_PRECISION, VADD_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VADD_R_SINGLE_PRECISION

4.24.3 Functions

vAdd_R

```
vAdd_R( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs the real-valued vector addition

4.25 vAddScalar_C

This module adds a complex-valued scalar to each element of a complex-valued vector. The input vector, input scalar and the output vector can be all single precision, or all double precision. It supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 80. vAddScalar_C Constants

Constants	Description
VADDSCALAR_C_SINGLE_PRECISION	Specify single-precision input/output
VADDSCALAR_C_DOUBLE_PRECISION	Specify double-precision input/output

Table 81. vAddScalar_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the vector input
vector_input_addr	Address of the input vector

Table 81. vAddScalar_C Parameters (Continued)

Parameters	Description
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
precision	The size of the real or imaginary part of each input/output element

Table 82. vAddScalar_C Functions

Functions	Description
vAddScalar_C	Performs the complex-valued scalar/vector addition

4.25.1 Constants

VADDSCALAR_C_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VADDSCALAR_C_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.25.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*2*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(scalar_input_addr % 2) == 0
Size	precision*2
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*2*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

precision

The size of the real or imaginary part of each input/output element. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VADDSCALAR_C_SINGLE_PRECISION, VADDSCALAR_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VADDSCALAR_C_SINGLE_PRECISION

4.25.3 Functions

vAddScalar_C

```
vAddScalar_C( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 precision )
```

Performs the complex-valued scalar/vector addition

4.26 vAddScalar_R

This module adds a real-valued scalar to each element of a real-valued vector. The input vector, input scalar and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 83. vAddScalar_R Constants

Constants	Description
VADDSCALAR_R_SINGLE_PRECISION	Specify single-precision input/output
VADDSCALAR_R_DOUBLE_PRECISION	Specify double-precision input/output

Table 84. vAddScalar_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector input
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each input/output element, either single precision (one word) or double precision (two words)

Table 85. vAddScalar_R Functions

Functions	Description
vAddScalar_R	Performs the real-valued scalar/vector addition

4.26.1 Constants

VADDSCALAR_R_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VADDSCALAR_R_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.26.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VADDSCALAR_R_DOUBLE_PRECISION)), (vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (precision == VADDSCALAR_R_DOUBLE_PRECISION), (scalar_input_addr % 2) == 0
Size	precision*1
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VADDSCALAR_R_DOUBLE_PRECISION)), (output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each input/output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VADDSCALAR_R_SINGLE_PRECISION, VADDSCALAR_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VADDSCALAR_R_SINGLE_PRECISION

4.26.3 Functions

vAddScalar_R

```
vAddScalar_R( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 precision )
```

Performs the real-valued scalar/vector addition

4.27 vSUB_C

This module subtracts one complex-valued vector from another complex-valued vector of equal length in an element-wise fashion. The two input vectors and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 86. vSub_C Constants

Constants	Description
VSUB_C_SINGLE_PRECISION	Specify single-precision input/output
VSUB_C_DOUBLE_PRECISION	Specify double-precision input/output

Table 87. vSub_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of the real or imaginary part of each input/output element

Table 88. vSub_C Functions

Functions	Description
vSub_C	Performs the complex-valued vector subtraction

4.27.1 Constants

VSUB_C_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VSUB_C_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.27.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input0_addr % 2) == 0
Size	2*abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	2*abs(input1_addr_mode)*precision*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	2*abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

precision

The size of the real or imaginary part of each input/output element. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VSUB_C_SINGLE_PRECISION, VSUB_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VSUB_C_SINGLE_PRECISION

4.27.3 Functions

vSub_C

```
vSub_C( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
```

```
output_addr_mode,
precision )
```

Performs the complex-valued vector subtraction

4.28 vSub_R

This module subtracts one real-valued vector from another real-valued vector of equal length in an element-wise fashion. The two input vectors and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used (e.g., blocks of stereo audio in a FIFO).

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 89. vSub_R Constants

Constants	Description
VSUB_R_SINGLE_PRECISION	Specify single-precision input/output
VSUB_R_DOUBLE_PRECISION	Specify double-precision input/output

Table 90. vSub_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each input/output element, either single precision (one word) or double precision (two words)

Table 91. vSub_R Functions

Functions	Description
vSub_R	Performs the real-valued vector subtraction

4.28.1 Constants

VSUB_R_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VSUB_R_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.28.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSUB_R_DOUBLE_PRECISION)), (input0_addr % 2) == 0 if (precision == VSUB_R_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSUB_R_DOUBLE_PRECISION)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr) if (precision == VSUB_R_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input1_addr_mode)*precision*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSUB_R_DOUBLE_PRECISION)), (output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each input/output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VSUB_R_SINGLE_PRECISION, VSUB_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VSUB_R_SINGLE_PRECISION

4.28.3 Functions

vSub_R

```
vSub_R( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs the real-valued vector subtraction

4.29 vSubScalar_C

This module subtracts a complex-valued scalar from each element of a complex-valued vector, or subtracts each element of the vector input from the scalar input. The input vector, input scalar and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 92. vSubScalar_C Constants

Constants	Description
VSUBSCALAR_C_NORMAL_ORDER	Subtract the input scalar from the input vector elements
VSUBSCALAR_C_REVERSE_ORDER	Subtract the input vector elements from the input scalar
VSUBSCALAR_C_SINGLE_PRECISION	Specify single-precision input/output
VSUBSCALAR_C_DOUBLE_PRECISION	Specify double-precision input/output

Table 93. vSubScalar_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the vector input
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
order	Specify the order of the operands for the subtract operation
precision	The size of the real or imaginary part of each input/output element

Table 94. vSubScalar_C Functions

Functions	Description
vSubScalar_C	Performs the complex-valued scalar/vector subtraction

4.29.1 Constants

VSUBSCALAR_C_NORMAL_ORDER

Subtract the input scalar from the input vector elements

Value	0

VSUBSCALAR_C_REVERSE_ORDER

Subtract the input vector elements from the input scalar

Value	1

VSUBSCALAR_C_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VSUBSCALAR_C_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.29.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*2*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(scalar_input_addr % 2) == 0
Size	precision*2
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*2*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

order

Specify the order of the operands for the subtract operation.

Type	Integer
Range	VSUBSCALAR_C_NORMAL_ORDER, VSUBSCALAR_C_REVERSE_ORDER
Restrictions	None
Default Value	VSUBSCALAR_C_NORMAL_ORDER

precision

The size of the real or imaginary part of each input/output element. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VSUBSCALAR_C_SINGLE_PRECISION, VSUBSCALAR_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VSUBSCALAR_C_SINGLE_PRECISION

4.29.3 Functions

vSubScalar_C

```
vSubScalar_C( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 order,
 precision )
```

Performs the complex-valued scalar/vector subtraction

4.30 vSubScalar_R

This module subtracts a real-valued scalar from each element of a real-valued vector, or subtracts each element of the vector input from the scalar input. The input vector, input scalar and the output vector can be all single precision, or all double precision. The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 95. vSubScalar_R Constants

Constants	Description
VSUBSCALAR_R_NORMAL_ORDER	Subtract the input scalar from the input vector elements
VSUBSCALAR_R_REVERSE_ORDER	Subtract the input vector elements from the input scalar
VSUBSCALAR_R_SINGLE_PRECISION	Specify single-precision input/output
VSUBSCALAR_R_DOUBLE_PRECISION	Specify double-precision input/output

Table 96. vSubScalar_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector input
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
order	Specify the order of the operands for the subtract operation
precision	The size of each input/output element, either single precision (one word) or double precision (two words)

Table 97. vSubScalar_R Functions

Functions	Description
vSubScalar_R	Performs the real-valued scalar/vector subtraction

4.30.1 Constants

VSUBSCALAR_R_NORMAL_ORDER

Subtract the input scalar from the input vector elements

Value	0

VSUBSCALAR_R_REVERSE_ORDER

Subtract the input vector elements from the input scalar

Value	1

VSUBSCALAR_R_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VSUBSCALAR_R_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

4.30.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSUBSCALAR_R_DOUBLE_PRECISION)), (vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (precision == VSUBSCALAR_R_DOUBLE_PRECISION), (scalar_input_addr % 2) == 0
Size	precision*1
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSUBSCALAR_R_DOUBLE_PRECISION)), (output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

order

Specify the order of the operands for the subtract operation.

Type	Integer
Range	VSUBSCALAR_R_NORMAL_ORDER, VSUBSCALAR_R_REVERSE_ORDER
Restrictions	None
Default Value	VSUBSCALAR_R_NORMAL_ORDER

precision

The size of each input/output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VSUBSCALAR_R_SINGLE_PRECISION, VSUBSCALAR_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VSUBSCALAR_R_SINGLE_PRECISION

4.30.3 Functions

vSubScalar_R

```
vSubScalar_R( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 order,
 precision )
```

Performs the real-valued scalar/vector subtraction

4.31 vMULT_C

This module multiplies two complex-valued vectors of equal length together in an element-wise fashion. The module supports both single-precision and double-precision multiplication. In single-precision mode, the two input vectors and the output vector are all single precision (one-word real part and one-word imaginary part); in double-precision mode, the first input vector and the output vector are double precision (two-word real part and two-word imaginary part) and the second input vector is single precision (one-word real part and one-word imaginary part). The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used (e.g., blocks of stereo audio in a FIFO).

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 98. vMult_C Constants

Constants	Description
VMULT_C_SINGLE_PRECISION	Specify single precision input/output
VMULT_C_DOUBLE_PRECISION	Specify double precision input/output

Table 99. vMult_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of the real or imaginary part of each element in the output vector and the first input vector

Table 100. vMult_C Functions

Functions	Description
vMult_C	Performs the complex-valued vector multiplication

4.31.1 Constants

VMULT_C_SINGLE_PRECISION

Specify single precision input/output

Value	1

VMULT_C_DOUBLE_PRECISION

Specify double precision input/output

Value	2

4.31.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input0_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	2*abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input1_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	2*abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr, input1_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	2*abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

precision

The size of the real or imaginary part of each element in the output vector and the first input vector. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VMULT_C_SINGLE_PRECISION, VMULT_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VMULT_C_SINGLE_PRECISION

4.31.3 Functions

vMult_C

```
vMult_C( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs the complex-valued vector multiplication

4.32 vMULT_R

This module multiplies two real-valued vectors of equal length together in an element-wise fashion. The module supports both single-precision and double-precision multiplication. In single-precision mode, the two input vectors and the output vector are all single precision (one word); in double- precision mode, the first input vector and the output vector are double precision (two words) and the second input vector is single precision (one word). The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used (e.g., blocks of stereo audio in a FIFO).

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 101. vMult_R Constants

Constants	Description
VMULT_R_SINGLE_PRECISION	Specify single precision input/output
VMULT_R_DOUBLE_PRECISION	Specify double precision input/output

Table 102. vMult_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each element in the output vector and the first input vector

Table 103. vMult_R Functions

Functions	Description
vMult_R	Performs the real-valued vector multiplication

4.32.1 Constants

VMULT_R_SINGLE_PRECISION

Specify single precision input/output

Value	1

VMULT_R_DOUBLE_PRECISION

Specify double precision input/output

Value	2

4.32.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMULT_R_DOUBLE_PRECISION)), (input0_addr % 2) == 0 if (precision == VMULT_R_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1) && (precision == VMULT_R_SINGLE_PRECISION)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr) if (precision == VMULT_R_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMULT_R_DOUBLE_PRECISION)), (output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each element in the output vector and the first input vector. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VMULT_R_SINGLE_PRECISION, VMULT_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VMULT_R_SINGLE_PRECISION

4.32.3 Functions

vMult_R

```
vMult_R( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs the real-valued vector multiplication

4.33 vMultConj

This module multiplies two complex-valued vectors of equal length together in an element-wise fashion and conjugates one of the input vectors during multiplication. The module supports both single-precision and double-precision conjugate-multiplication. In single-precision mode, the two input vectors and the output vector are all single precision, and the second input vector is conjugated in multiplication; in double-precision mode, the first input vector and the output vector are double precision and the second input vector is single precision, and either the first or the second input vector is conjugated in multiplication. The module supports addressing the input and output vectors with a configurable step size.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 104. vMultConj Constants

Constants	Description
VMULTCONJ_SINGLE_PRECISION	Specify single-precision input/output
VMULTCONJ_DOUBLE_PRECISION	Specify double-precision input/output
VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_SINGLE	Conjugate the single-precision input during multiplication
VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_DOUBLE	Conjugate the double-precision input during multiplication

Table 105. vMultConj Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of the real or imaginary part of each input0 and output element
select	Select whether to conjugate the single-precision or double-precision input during the multiplication

Table 106. vMultConj Functions

Functions	Description
vMultConj	Performs the complex-valued vector multiplication

4.33.1 Constants

VMULTCONJ_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VMULTCONJ_DOUBLE_PRECISION

Specify double-precision input/output

Value	2

VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_SINGLE

Conjugate the single-precision input during multiplication

Value	0

VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_DOUBLE

Conjugate the double-precision input during multiplication

Value	1

4.33.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input0_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	2*abs(input0_addr_mode)*precision*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input1_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	2*abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr, input1_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	2*abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

precision

The size of the real or imaginary part of each input0 and output element. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VMULTCONJ_SINGLE_PRECISION, VMULTCONJ_DOUBLE_PRECISION
Restrictions	None
Default Value	VMULTCONJ_SINGLE_PRECISION

select

Select whether to conjugate the single-precision or double-precision input during the multiplication.

Type	Integer
Range	VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_SINGLE, VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_DOUBLE
Restrictions	None
Default Value	VMULTCONJ_DOUBLE_PRECISION_CONJUGATE_SINGLE

4.33.3 Functions

vMultConj

```
vMultConj( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision,
 select )
```

Performs the complex-valued vector multiplication

4.34 vMultScalar_C

This module multiplies each element of a complex-valued vector with a complex-valued scalar. The module supports both single-precision and double-precision multiplication. In single-precision mode, the input vector, input scalar and the output vector are all single precision (one word for the real part and one word for the imaginary part); in double-precision mode, the input vector and the output vector are double precision (two words for the real part and two words for the imaginary part), and the input scalar is single precision (one word for the real part and one word for the imaginary part). The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 107. vMultScalar_C Constants

Constants	Description
VMULTSCALAR_C_SINGLE_PRECISION	Specify single-precision input/output
VMULTSCALAR_C_DOUBLE_PRECISION	Specify double-precision vector input and output

Table 108. vMultScalar_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the input vector
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
precision	The size of the real part or imaginary part of each vector input or output element

Table 109. vMultScalar_C Functions

Functions	Description
vMultScalar_C	Performs the complex-valued scalar/vector multiplication

4.34.1 Constants

VMULTSCALAR_C_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VMULTSCALAR_C_DOUBLE_PRECISION

Specify double-precision vector input and output

Value	2

4.34.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*2*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(scalar_input_addr % 2) == 0
Size	2
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*2*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

precision

The size of the real part or imaginary part of each vector input or output element. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VMULTSCALAR_C_SINGLE_PRECISION, VMULTSCALAR_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VMULTSCALAR_C_SINGLE_PRECISION

4.34.3 Functions

vMultScalar_C

```
vMultScalar_C( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 precision )
```

Performs the complex-valued scalar/vector multiplication

4.35 vMultScalar_R

This module multiplies each element of a real-valued vector with a real-valued scalar. The module supports both single-precision and double-precision multiplication. In single-precision mode, the input vector, input scalar and the output vector are all single precision (one word); in double-precision mode, the input vector and the output vector are double precision (two words), and the input scalar is single precision (one word). The module supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 110. vMultScalar_R Constants

Constants	Description
VMULTSCALAR_R_SINGLE_PRECISION	Specify single-precision input/output
VMULTSCALAR_R_DOUBLE_PRECISION	Specify double-precision vector input and output

Table 111. vMultScalar_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each element in the output vector and the input vector

Table 112. vMultScalar_R Functions

Functions	Description
vMultScalar_R	Performs the real-valued scalar/vector multiplication

4.35.1 Constants

VMULTSCALAR_R_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VMULTSCALAR_R_DOUBLE_PRECISION

Specify double-precision vector input and output

Value	2

4.35.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the vector input address and the output

address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMULTSCALAR_R_DOUBLE_PRECISION)), (vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	1
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location. Note that if the vector input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMULTSCALAR_R_DOUBLE_PRECISION)), (output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each element in the output vector and the input vector. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VMULTSCALAR_R_SINGLE_PRECISION, VMULTSCALAR_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VMULTSCALAR_R_SINGLE_PRECISION

4.35.3 Functions

vMultScalar_R

```
vMultScalar_R( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 precision )
```

Performs the real-valued scalar/vector multiplication

4.36 vMultScalarConj

This module multiplies each element of a complex-valued vector with the conjugate of a complex-valued scalar, or alternatively multiplies the conjugate elements of a complex-valued vector with a complex-valued scalar. The module supports both single-precision and double-precision conjugate-multiplication. In single-precision mode, the vector input, the scalar input and the output vector are all single precision; in double-precision mode, the input vector and the

output vector are double precision, and the input scalar is single precision. The module supports addressing the input and output vectors with a configurable step size.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 113. vMultScalarConj Constants

Constants	Description
VMULTSCALARCONJ_SINGLE_PRECISION	Specify single-precision input/output
VMULTSCALARCONJ_DOUBLE_PRECISION	Specify double-precision vector input and output
VMULTSCALARCONJ_CONJUGATE_SCALAR	Conjugate the scalar during multiplication
VMULTSCALARCONJ_CONJUGATE_VECTOR	Conjugate the vector elements during multiplication

Table 114. vMultScalarConj Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the input vector
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
select	Select whether to conjugate the scalar or the vector elements during the multiplication
precision	The size of the real or imaginary part of each vector input or output element

Table 115. vMultScalarConj Functions

Functions	Description
vMultScalarConj	Performs the complex-valued scalar/vector multiplication

4.36.1 Constants

VMULTSCALARCONJ_SINGLE_PRECISION

Specify single-precision input/output

Value	1

VMULTSCALARCONJ_DOUBLE_PRECISION

Specify double-precision vector input and output

Value	2

VMULTSCALARCONJ_CONJUGATE_SCALAR

Conjugate the scalar during multiplication

Value	0

VMULTSCALARCONJ_CONJUGATE_VECTOR

Conjugate the vector elements during multiplication

Value	1

4.36.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The vector input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*precision*2*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The scalar input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(scalar_input_addr % 2) == 0
Size	2
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*precision*2*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

select

Select whether to conjugate the scalar or the vector elements during the multiplication.

Type	Integer
Range	VMULTSCALARCONJ_CONJUGATE_SCALAR, VMULTSCALARCONJ_CONJUGATE_VECTOR
Restrictions	None
Default Value	VMULTSCALARCONJ_CONJUGATE_SCALAR

precision

The size of the real or imaginary part of each vector input or output element. It can be either single precision (one word) or double precision (two words).

Type	Integer
Range	VMULTSCALARCONJ_SINGLE_PRECISION, VMULTSCALARCONJ_DOUBLE_PRECISION
Restrictions	None
Default Value	VMULTSCALARCONJ_SINGLE_PRECISION

4.36.3 Functions

vMultScalarConj

```
vMultScalarConj( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 select,
 precision )
```

Performs the complex-valued scalar/vector multiplication

4.37 vCONJ

This module makes a conjugated copy of a complex valued vector in memory. It supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 116. vConj Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the vector
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 117. vConj Functions

Functions	Description
vConj	Performs the conjugate vector copy

4.37.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Run-time Parameter	<name>_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0 memtype(input_addr) != memtype(output_addr)
Size	2*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Run-time Parameter	<name>_output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0 memtype(input_addr) != memtype(output_addr)
Size	2*abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

4.37.2 Functions

vConj

```
vConj( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the conjugate vector copy

4.38 vMag

This module calculates the magnitude of each element in a complex vector. This calculation uses a table look-up based approximation for the square-root operation required. This function can output 24-bit (2.22) or 48-bit (2.46) values. It supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 118. vMag Constants

Constants	Description
VMAG_SINGLE_PRECISION	Specify single-precision outputs
VMAG_DOUBLE_PRECISION	Specify double-precision outputs

Table 119. vMag Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the input vector
input_addr	Address of the input vector

Table 119. vMag Parameters (Continued)

Parameters	Description
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
output_precision	The size of each output element

Table 120. vMag Functions

Functions	Description
vMag	Calculates the magnitude of each element in a complex vector

4.38.1 Constants

VMAG_SINGLE_PRECISION

Specify single-precision outputs

Value	1

VMAG_DOUBLE_PRECISION

Specify double-precision outputs

Value	2

4.38.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	2*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. The output is in 2.22 or 2.46 format. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == VMAG_DOUBLE_PRECISION), (output_addr % 2) == 0
Size	output_precision*abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512] when (output_precision == VMAG_SINGLE_PRECISION), [-256-256] when (output_precision == VMAG_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

output_precision

The size of each output element. It can be single precision (one word) or double precision (two words).

Type	Integer
Range	VMAG_SINGLE_PRECISION, VMAG_DOUBLE_PRECISION
Restrictions	None
Default Value	VMAG_DOUBLE_PRECISION

4.38.3 Functions

vMag

```
vMag( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 output_precision )
```

Calculates the magnitude of each element in a complex vector

4.39 vSqMag

This module calculates the squared magnitude of each element in a complex vector. It supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 121. vSqMag Constants

Constants	Description
VSQMAG_SINGLE_PRECISION	Specify single-precision outputs
VSQMAG_DOUBLE_PRECISION	Specify double-precision outputs

Table 122. vSqMag Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the input vector
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector

Table 122. vSqMag Parameters (Continued)

Parameters	Description
output_addr_mode	The addressing mode for the output vector
output_precision	The size of each output element, either single precision (one word) or double precision (two words)

Table 123. vSqMag Functions

Functions	Description
vSqMag	Calculates the squared magnitude of each element in a complex vector

4.39.1 Constants

VSQMAG_SINGLE_PRECISION

Specify single-precision outputs

Value	1

VSQMAG_DOUBLE_PRECISION

Specify double-precision outputs

Value	2

4.39.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	2*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. The output vector is either a single-precision value in 2.22 format, or a double-precision value in 2.46 format. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (output_precision == VSQMAG_DOUBLE_PRECISION)), (output_addr % 2) == 0
Size	output_precision*abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512] when (output_precision == VSQMAG_SINGLE_PRECISION), [-256-256] when (output_precision == VSQMAG_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

output_precision

The size of each output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VSQMAG_SINGLE_PRECISION, VSQMAG_DOUBLE_PRECISION
Restrictions	None
Default Value	VSQMAG_SINGLE_PRECISION

4.39.3 Functions

vSqMag

```
vSqMag( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 output_precision )
```

Calculates the squared magnitude of each element in a complex vector

4.40 INNERPRODUCT_C

This module computes the inner product (dot product) of two complex-valued vectors. This product is defined as the summation of an element-wise multiplication of each vector element pair. The result is stored as either a single-precision or double-precision complex value. Both the real and imaginary components can be rescaled to avoid overflow.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 124. InnerProduct_C Constants

Constants	Description
INNERPRODUCT_C_SINGLE_PRECISION	Specify single-precision output
INNERPRODUCT_C_DOUBLE_PRECISION	Specify double-precision output

Table 125. InnerProduct_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in each vector
scalefactor	The number of right shifts to apply to the summation result in order to avoid saturation

Table 125. InnerProduct_C Parameters (Continued)

Parameters	Description
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output value
input0_addr_mode	The addressing mode for the first input vector
input1_addr_mode	The addressing mode for the second input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 126. InnerProduct_C Functions

Functions	Description
InnerProduct_C	Computes the inner product

4.40.1 Constants

INNERPRODUCT_C_SINGLE_PRECISION

Specify single-precision output

Value	1

INNERPRODUCT_C_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.40.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a new value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts to apply to the summation result in order to avoid saturation. This parameter is run-time configurable. The scale factor can be changed by writing a new value to the run-time parameter's memory location.

Type	Integer
Range	[0-56]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input0_addr %2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*2*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input1_addr %2) == 0 memtype(input1_addr) != memtype(input0_addr)
Size	abs(input1_addr_mode)*2*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output value. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr, input1_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr %2) == 0
Size	2*output_precision
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the first input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the second input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	INNERPRODUCT_C_SINGLE_PRECISION, INNERPRODUCT_C_DOUBLE_PRECISION
Restrictions	None
Default Value	INNERPRODUCT_C_DOUBLE_PRECISION

4.40.3 Functions

InnerProduct_C

```
InnerProduct_C( name,
 K,
```

```
scalefactor,
input0_addr,
input1_addr,
output_addr,
input0_addr_mode,
input1_addr_mode,
output_precision )
```

Computes the inner product

4.41 INNERPRODUCT_R

This module computes the inner product (dot product) of two real-valued vectors. This product is defined as the summation of an element-wise multiplication of each vector element pair. The result is stored as either a single-precision or double-precision value, and can be rescaled to avoid overflow.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 127. InnerProduct_R Constants

Constants	Description
INNERPRODUCT_R_SINGLE_PRECISION	Specify single-precision output
INNERPRODUCT_R_DOUBLE_PRECISION	Specify double-precision output

Table 128. InnerProduct_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
scalefactor	The number of right shifts to apply to the summation result in order to avoid saturation
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output value
input0_addr_mode	The addressing mode for the first input vector
input1_addr_mode	The addressing mode for the second input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 129. InnerProduct_R Functions

Functions	Description
InnerProduct_R	Computes the inner product

4.41.1 Constants

INNERPRODUCT_R_SINGLE_PRECISION

Specify single-precision output

Value	1

INNERPRODUCT_R_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.41.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a new value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts to apply to the summation result in order to avoid saturation. This parameter is run-time configurable. The scale factor can be changed by writing a new value to the run-time parameter's memory location.

Type	Integer
Range	[0-56]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input0_addr_mode==1) && (input1_addr_mode==1)), (input0_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input0_addr_mode==1) && (input1_addr_mode==1)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(input0_addr)
Size	abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output value. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr, input1_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == INNERPRODUCT_R_DOUBLE_PRECISION), (output_addr % 2) == 0
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the first input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the second input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	INNERPRODUCT_R_SINGLE_PRECISION, INNERPRODUCT_R_DOUBLE_PRECISION
Restrictions	None
Default Value	INNERPRODUCT_R_DOUBLE_PRECISION

4.41.3 Functions

InnerProduct_R

```
InnerProduct_R( name,
 K,
 scalefactor,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_precision )
```

Computes the inner product

4.42 vLinLog2

This module converts a vector of linear (fractional) values into their equivalent logarithm base-2 values. This calculation uses a table-lookup based approximation for the log2() operation. The module accepts single-precision (24-bit) or double-precision (48-bit) inputs. It supports addressing the input and output vectors with configurable step sizes so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 130. vLinLog2 Constants

Constants	Description
VLINLOG2_SINGLE_PRECISION	Specify single-precision inputs
VLINLOG2_DOUBLE_PRECISION	Specify double-precision inputs

Table 131. vLinLog2 Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
input_precision	The size of each input element, either single precision (one word) or double precision (two words)

Table 132. vLinLog2 Functions

Functions	Description
vLinLog2	Calculates log2() of each element in the input vector

4.42.1 Constants

VLINLOG2_SINGLE_PRECISION

Specify single-precision inputs

Value	1

VLINLOG2_DOUBLE_PRECISION

Specify double-precision inputs

Value	2

4.42.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_precision == VLINLOG2_DOUBLE_PRECISION), (input_addr % 2) == 0
Size	input_precision*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. The output is in 8.16 format. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512] when (input_precision == VLINLOG2_SINGLE_PRECISION), [-256-256] when (input_precision == VLINLOG2_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input_precision

The size of each input element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VLINLOG2_SINGLE_PRECISION, VLINLOG2_DOUBLE_PRECISION
Restrictions	None
Default Value	VLINLOG2_SINGLE_PRECISION

4.42.3 Functions

vLinLog2

```
vLinLog2( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 input_precision )
```

Calculates log2() of each element in the input vector

4.43 vLog2Lin

This module calculates the exponent base 2 of a vector of 24-bit mixed (m8p16, or 8.16) values. This calculation uses a table-lookup based approximation for the exp2() operation. This function can output 24-bit or 48-bit values. It supports addressing the input and output vectors with configurable step sizes so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 133. vLog2Lin Constants

Constants	Description
VLOG2LIN_SINGLE_PRECISION	Specify single precision outputs
VLOG2LIN_DOUBLE_PRECISION	Specify double precision outputs

Table 134. vLog2Lin Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
output_precision	The size of each output element, either single precision (one word) or double precision (two words)

Table 135. vLog2Lin Functions

Functions	Description
vLog2Lin	Converts from a log2 value to a linear value by computing exp2() of each element in the input vector

4.43.1 Constants

VLOG2LIN_SINGLE_PRECISION

Specify single precision outputs

Value	1

VLOG2LIN_DOUBLE_PRECISION

Specify double precision outputs

Value	2

4.43.2 Parameters

name

A unique name for each instance of the module.

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. Inputs must be in 8.16 format. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. The output is in fractional 24-bit or 48-bit format. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == VLOG2LIN_DOUBLE_PRECISION), (output_addr % 2) == 0
Size	output_precision*abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512] when (output_precision == VLOG2LIN_SINGLE_PRECISION), [-256-256] when (output_precision == VLOG2LIN_DOUBLE_PRECISION)
Restrictions	None
Default Value	1

output_precision

The size of each output element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VLOG2LIN_SINGLE_PRECISION, VLOG2LIN_DOUBLE_PRECISION
Restrictions	None
Default Value	VLOG2LIN_SINGLE_PRECISION

4.43.3 Functions

vLog2Lin

```
vLog2Lin( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 output_precision )
```

Converts from a log2 value to a linear value by computing exp2() of each element in the input vector

4.44 vPolarRect

This modules converts a vector of complex values from a polar (magnitude-phase) data format to a rectangular (real-imaginary) format. This is done using an iterative CORDIC approximation. This module supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 136. vPolarRect Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
num_iterations	The number of iterations of the CORDIC algorithm to use
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 137. vPolarRect Functions

Functions	Description
vPolarRect	Performs the complex rectangular to complex polar conversion

4.44.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

num_iterations

The number of iterations of the CORDIC algorithm to use. Each iteration corresponds roughly to one additional bit of precision in the output.

Type	Integer
Range	[1-24]
Restrictions	None

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	2*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	2*abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

4.44.2 Functions

vPolarRect

```
vPolarRect( name,
 K,
 num_iterations,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the complex rectangular to complex polar conversion

4.45 vRectPolar

This modules converts a vector of complex values from a rectangular (real-imaginary) data format to a polar (magnitude-phase) format. This is done using an iterative CORDIC approximation. This module supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 138. vRectPolar Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
num_iterations	The number of iterations of the CORDIC algorithm to use
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 139. vRectPolar Functions

Functions	Description
vRectPolar	Performs the complex rectangular to complex polar conversion

4.45.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

num_iterations

The number of iterations of the CORDIC algorithm to use. Each iteration corresponds roughly to one additional bit of precision in the output.

Type	Integer
Range	[1-24]
Restrictions	None

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	2*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	(output_addr % 2) == 0
Address Parameter Restrictions	None
Size	2*abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

4.45.2 Functions

vRectPolar

```
vRectPolar( name,
 K,
 num_iterations,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the complex rectangular to complex polar conversion

4.46 vCopy

This module can be used to copy a vector of elements to/from any location in shared memory. This module performs the copy only one 24-bit word at a time and thus has no restrictions on the vector elements being contiguous.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 140. vCopy Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of 24-bit words in the vector
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 141. vCopy Functions

Functions	Description
vCopy	Performs the vector copy

4.46.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of 24-bit words in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location. Note that if the input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.46.2 Functions

vCopy

```
vCopy( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the vector copy

4.47 vCopy2

This module can be used to copy a vector of elements to/from any location in shared memory. In particular, this module must be used if either the input or output is in a non-interleaved mapping of shared memory.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 142. vCopy2 Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of 24-bit words in the vector
`input_addr`	Address of the input vector
`output_addr`	Address of the output vector
`input_addr_mode`	The addressing mode for the input vector
`output_addr_mode`	The addressing mode for the output vector

Table 143. vCopy2 Functions

Functions	Description
`vCopy2`	Performs the vector copy

4.47.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of 24-bit words in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-2048]
Restrictions	(K % 2) == 0
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location. Note that if the input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.47.2 Functions

vCopy2

```
vCopy2( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the vector copy

4.48 vCopy4

This module can be used to copy a vector of elements to/from interleaved memory mappings in shared memory. It does this by moving four 24-bit words with each memory read/write.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 144. vCopy4 Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of 24-bit words in the vector
`input_addr`	Address of the input vector

Table 144. vCopy4 Parameters (Continued)

Parameters	Description
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 145. vCopy4 Functions

Functions	Description
vCopy4	Performs the vector copy

4.48.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of 24-bit words in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[4-2048]
Restrictions	(K % 4) == 0
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location. Note that if the input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a new value to the run-time parameter's memory location. Note that if the input address and the output address are in different memory spaces at compile time, they must not be changed to the same memory space at run time.

Type	Output
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM, ACMEM, BDMEM
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

4.48.2 Functions

vCopy4

```
vCopy4( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the vector copy

4.49 L1NORMDIFF

This module calculates the L1 norm of the difference between two equal-length, real-valued input vectors. First, element-wise subtraction is performed to obtain the differences. Next, the absolute value of the differences is calculated. Finally, the sum of these absolute values is calculated and the total is shifted right by the scalefactor to obtain an output value. This module supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used. The result is stored in either single-precision (1.23) or double-precision (24.24) format.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 146. L1NormDiff Constants

Constants	Description
L1NORMDIFF_SINGLE_PRECISION	Specify single-precision output
L1NORMDIFF_DOUBLE_PRECISION	Specify double-precision output

Table 147. L1NormDiff Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
scalefactor	The number of right shifts by which to scale the output values
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output value
input0_addr_mode	The addressing mode for the first input vector
input1_addr_mode	The addressing mode for the second input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 148. L1NormDiff Functions

Functions	Description
L1NormDiff	Calculates the L1 norm of the difference of the given vectors

4.49.1 Constants

L1NORMDIFF_SINGLE_PRECISION

Specify single-precision output

Value	1

L1NORMDIFF_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.49.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts by which to scale the output values. Used to reduce the chance of overflows. This parameter is run-time configurable.

Type	Integer
Range	[0-56]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input0_addr_mode==1) && (input1_addr_mode==1)), (input0_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input0_addr_mode==1) && (input1_addr_mode==1)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(input0_addr)
Size	abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output value. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr, input1_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the first input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the second input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	L1NORMDIFF_SINGLE_PRECISION, L1NORMDIFF_DOUBLE_PRECISION
Restrictions	None
Default Value	L1NORMDIFF_SINGLE_PRECISION

4.49.3 Functions

L1NormDiff

```
L1NormDiff( name,
 K,
 scalefactor,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_precision )
```

Calculates the L1 norm of the difference of the given vectors.

4.50 vABSOLUTE

This module calculates the absolute value of each element of a real-valued vector. It supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 149. vAbsolute Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of elements in the vector input
`input_addr`	Address of the input vector
`output_addr`	Address of the output vector
`input_addr_mode`	The addressing mode for the input vector
`output_addr_mode`	The addressing mode for the output vector

Table 150. vAbsolute Functions

Functions	Description
vAbsolute	Calculates the absolute value of elements of a real-valued vector

4.50.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (output_addr_mode == 1)), (input_addr % 2) == 0 memtype(input_addr) != memtype(output_addr)
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (output_addr_mode == 1)), (output_addr % 2) == 0 memtype(input_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.50.2 Functions

vAbsolute

```
vAbsolute( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Calculates the absolute value of elements of a real-valued vector

4.51 vBound

This module reads three input vectors: a data vector, a lower bound vector and an upper bound vector. The i-th upper and lower bounds are applied to the i-th element in the data vector, and the resulting vector is output to the specified address. This module supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 151. vBound Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input_addr	Address of the data input vector, containing data to be bounded
lower_bound_addr	Address of the lower-bound input vector
upper_bound_addr	Address of the upper-bound input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
lower_bound_addr_mode	The addressing mode for the lower bound vector
upper_bound_addr_mode	The addressing mode for the upper bound vector
output_addr_mode	The addressing mode for the output vector

Table 152. vBound Functions

Functions	Description
vBound	Applies upper and lower bounds to an input vector

4.51.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the data input vector, containing data to be bounded. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (lower_bound_addr_mode == 1) && (upper_bound_addr_mode == 1) && (output_addr_mode == 1)), (input_addr % 2) == 0 memtype(input_addr) != memtype(upper_bound_addr)
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

lower_bound_addr

Address of the lower-bound input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (lower_bound_addr_mode == 1) && (upper_bound_addr_mode == 1) && (output_addr_mode == 1)), (lower_bound_addr % 2) == 0
Size	abs(lower_bound_addr_mode)*K
Run-time Parameter	<name>_lower_bound_addr

upper_bound_addr

Address of the upper-bound input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (lower_bound_addr_mode == 1) && (upper_bound_addr_mode == 1) && (output_addr_mode == 1)), (upper_bound_addr % 2) == 0 memtype(upper_bound_addr) != memtype(input_addr)
Size	abs(upper_bound_addr_mode)*K
Run-time Parameter	<name>_upper_bound_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr, lower_bound_addr, upper_bound_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (lower_bound_addr_mode == 1) && (upper_bound_addr_mode == 1) && (output_addr_mode == 1)), (output_addr % 2) == 0
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

lower_bound_addr_mode

The addressing mode for the lower bound vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

upper_bound_addr_mode

The addressing mode for the upper bound vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.51.2 Functions

vBound

```
vBound( name,
 K,
 input_addr,
 lower_bound_addr,
 upper_bound_addr,
 output_addr,
 input_addr_mode,
 lower_bound_addr_mode,
 upper_bound_addr_mode,
 output_addr_mode )
```

Applies upper and lower bounds to an input vector

4.52 vSHIFT

This module shifts elements of an input vector by the shift values specified in another vector. A positive value in the shift vector results in an arithmetic right shift with the sign bit being shifted into the most significant bit position. A negative value results in a left shift with zeros shifted into the least significant bit position. The module accepts

single-precision (24-bit) or double-precision (48-bit) inputs. In single- precision mode, the two input vectors and the output vector are all single precision (one word); in double-precision mode, the input0 vector and the output vector are double precision (two words), and the input1 vector (the shift amount vector) is single precision (one word). The module supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used (e.g., blocks of stereo audio in a FIFO).

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 153. vShift Constants

Constants	Description
VSHIFT_SINGLE_PRECISION	Specify single-precision inputs and outputs
VSHIFT_DOUBLE_PRECISION	Specify double-precision inputs and outputs

Table 154. vShift Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input0_addr	Address of the first input vector, containing data to be shifted
input1_addr	Address of the second input vector, containing the shift counts
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each element, either single precision (one word) or double precision (two words)

Table 155. vShift Functions

Functions	Description
vShift	Performs vector shifting

4.52.1 Constants

VSHIFT_SINGLE_PRECISION

Specify single-precision inputs and outputs

Value	1

VSHIFT_DOUBLE_PRECISION

Specify double-precision inputs and outputs

Value	2

4.52.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector, containing data to be shifted. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSHIFT_DOUBLE_PRECISION)), (input0_addr % 2) == 0 if (precision == VSHIFT_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector, containing the shift counts. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	if (precision == VSHIFT_DOUBLE_PRECISION), output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1) && (precision == VSHIFT_SINGLE_PRECISION)), (input1_addr % 2) == 0 if (precision == VSHIFT_SINGLE_PRECISION), memtype(input1_addr) != memtype(output_addr) if (precision == VSHIFT_DOUBLE_PRECISION), memtype(input0_addr) != memtype(input1_addr)
Size	abs(input1_addr_mode)*K

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr if (precision == VSHIFT_DOUBLE_PRECISION), input1_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSHIFT_DOUBLE_PRECISION)), (output_addr % 2) == 0 if (precision == VSHIFT_SINGLE_PRECISION), memtype(input1_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VSHIFT_SINGLE_PRECISION, VSHIFT_DOUBLE_PRECISION
Restrictions	None
Default Value	VSHIFT_SINGLE_PRECISION

4.52.3 Functions

vShift

```
vShift( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs vector shifting

4.53 vShiftScalar

This module performs an element-wise shift of vector data by a scalar shift count. A positive shift value results in an arithmetic right shift with the sign bit being shifted into the most significant bit position. A negative shift value results in a left shift with zeros shifted into the least significant bit position. The module accepts single-precision (24-bit) or double-precision (48-bit) inputs. In single-precision mode, the input vector, input scalar and the output vector are all single precision (one word); in double-precision mode, the input vector and the output vector are double precision

(two words), and the input scalar is single precision (one word). It supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 156. vShiftScalar Constants

Constants	Description
VSHIFTSCALAR_SINGLE_PRECISION	Specify single-precision inputs and outputs
VSHIFTSCALAR_DOUBLE_PRECISION	Specify double-precision inputs and outputs

Table 157. vShiftScalar Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector input
vector_input_addr	Address of the input vector
scalar_input_addr	Address of the input scalar
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each element, either single precision (one word) or double precision (two words)

Table 158. vShiftScalar Functions

Functions	Description
vShiftScalar	Performs an element-wise shift of a vector by a scalar shift count

4.53.1 Constants

VSHIFTSCALAR_SINGLE_PRECISION

Specify single-precision inputs and outputs

Value	1

VSHIFTSCALAR_DOUBLE_PRECISION

Specify double-precision inputs and outputs

Value	2

4.53.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSHIFTSCALAR_DOUBLE_PRECISION)), (vector_input_addr % 2) == 0 memtype(vector_input_addr) != memtype(output_addr)
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	1
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VSHIFTSCALAR_DOUBLE_PRECISION)), (output_addr % 2) == 0 memtype(vector_input_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VSHIFTSCALAR_SINGLE_PRECISION, VSHIFTSCALAR_DOUBLE_PRECISION
Restrictions	None
Default Value	VSHIFTSCALAR_SINGLE_PRECISION

4.53.3 Functions

vShiftScalar

```
vShiftScalar( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 precision )
```

Performs an element-wise shift of a vector by a scalar shift count

4.54 vMax

This module computes the element-wise maximum between two equal length vectors. The module accepts single-precision (24-bit) or double-precision (48-bit) inputs. It supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 159. vMax Constants

Constants	Description
VMAX_SINGLE_PRECISION	Specify single-precision inputs and outputs
VMAX_DOUBLE_PRECISION	Specify double-precision inputs and outputs

Table 160. vMax Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each element, either single precision (one word) or double precision (two words)

Table 161. vMax Functions

Functions	Description
vMax	Performs element-wise maximum operation between two vectors

4.54.1 Constants

VMAX_SINGLE_PRECISION

Specify single-precision inputs and outputs

Value	1

VMAX_DOUBLE_PRECISION

Specify double-precision inputs and outputs

Value	2

4.54.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMAX_DOUBLE_PRECISION)), (input0_addr % 2) == 0
Size	abs(input0_addr_mode)*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMAX_DOUBLE_PRECISION)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMAX_DOUBLE_PRECISION)), (output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VMAX_SINGLE_PRECISION, VMAX_DOUBLE_PRECISION
Restrictions	None
Default Value	VMAX_SINGLE_PRECISION

4.54.3 Functions

vMax

```
vMax( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs element-wise maximum operation between two vectors

4.55 vMaxScalar

This module compares the value of a real-valued scalar to each element of a real-valued vector and determines the maximum value from each comparison. It supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 162. vMaxScalar Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of elements in the vector input
`vector_input_addr`	Address of the input vector
`scalar_input_addr`	Address of the input scalar
`output_addr`	Address of the output vector
`input_addr_mode`	The addressing mode for the input vector
`output_addr_mode`	The addressing mode for the output vector

Table 163. vMaxScalar Functions

Functions	Description
`vMaxScalar`	Performs the maximum operation between a vector and a scalar

4.55.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (output_addr_mode == 1)), (vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	1
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (output_addr_mode == 1)), (output_addr % 2) == 0
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.55.2 Functions

vMaxScalar

```
vMaxScalar( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the maximum operation between a vector and a scalar

4.56 vMin

This module computes the element-wise minimum between two equal length vectors. The module accepts single-precision (24-bit) or double-precision (48-bit) inputs. It supports addressing the input and output vectors with a configurable step size, so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 164. vMin Constants

Constants	Description
VMIN_SINGLE_PRECISION	Specify single-precision inputs and outputs
VMIN_DOUBLE_PRECISION	Specify double-precision inputs and outputs

Table 165. vMin Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in each vector
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the input0 vector
input1_addr_mode	The addressing mode for the input1 vector
output_addr_mode	The addressing mode for the output vector
precision	The size of each element, either single precision (one word) or double precision (two words)

Table 166. vMin Functions

Functions	Description
vMin	Performs element-wise minimum operation between two vectors

4.56.1 Constants

VMIN_SINGLE_PRECISION

Specify single-precision inputs and outputs

Value	1

VMIN_DOUBLE_PRECISION

Specify double-precision inputs and outputs

Value	2

4.56.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in each vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input0_addr

Address of the first input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMIN_DOUBLE_PRECISION)), (input0_addr % 2) == 0
Size	abs(input0_addr_mode)*K
Run-time Parameter	<name>_input0_addr

input1_addr

Address of the second input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMIN_DOUBLE_PRECISION)), (input1_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(input1_addr_mode)*K
Run-time Parameter	<name>_input1_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input0_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (((input0_addr_mode == 1) && (input1_addr_mode == 1) && (output_addr_mode == 1)) || (precision == VMIN_DOUBLE_PRECISION)), (output_addr % 2) == 0 memtype(input1_addr) != memtype(output_addr)
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the input0 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the input1 vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

precision

The size of each element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VMIN_SINGLE_PRECISION, VMIN_DOUBLE_PRECISION
Restrictions	None
Default Value	VMIN_SINGLE_PRECISION

4.56.3 Functions

vMin

```
vMin( name,
 K,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode,
 precision )
```

Performs element-wise minimum operation between two vectors

4.57 vMinScalar

This module compares the value of a real-valued scalar to each element of a real-valued vector and determines the minimum value from each comparison. It supports addressing the input and output vectors with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 167. vMinScalar Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of elements in the vector input
`vector_input_addr`	Address of the input vector
`scalar_input_addr`	Address of the input scalar
`output_addr`	Address of the output vector
`input_addr_mode`	The addressing mode for the input vector
`output_addr_mode`	The addressing mode for the output vector

Table 168. vMinScalar Functions

Functions	Description
`vMinScalar`	Performs the minimum operation between a vector and a scalar

4.57.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

vector_input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (output_addr_mode == 1)), (vector_input_addr % 2) == 0
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_vector_input_addr

scalar_input_addr

Address of the input scalar. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	1
Run-time Parameter	<name>_scalar_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	vector_input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if ((input_addr_mode == 1) && (output_addr_mode == 1)), (output_addr % 2) == 0
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.57.2 Functions

vMinScalar

```
vMinScalar( name,
 K,
 vector_input_addr,
 scalar_input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Performs the minimum operation between a vector and a scalar

4.58 vNLog2Abs

This module performs a negated leading bit detection on an input vector. The resulting value is the amount of left shifts required to normalize the input value. The module accepts single-precision (24-bit) or double-precision (48-bit) inputs. When the input value is 0, the output value is 23 (single-precision) or 47 (double-precision). It supports addressing the input and output vectors with configurable step sizes so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 169. vNLog2Abs Constants

Constants	Description
VNLOG2ABS_SINGLE_PRECISION	Specify single-precision inputs
VNLOG2ABS_DOUBLE_PRECISION	Specify double-precision inputs

Table 170. vNLog2Abs Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector
input_precision	The size of each input element, either single precision (one word) or double precision (two words)

Table 171. vNLog2Abs Functions

Functions	Description
vNLog2Abs	Calculates negated leading bit of each element in the input vector

4.58.1 Constants

VNLOG2ABS_SINGLE_PRECISION

Specify single-precision inputs

Value	1

VNLOG2ABS_DOUBLE_PRECISION

Specify double-precision inputs

Value	2

4.58.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_precision == VNLOG2ABS_DOUBLE_PRECISION), (input_addr % 2) == 0
Size	input_precision*abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(output_addr_mode)*K
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when writing the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input_precision

The size of each input element, either single precision (one word) or double precision (two words).

Type	Integer
Range	VNLOG2ABS_SINGLE_PRECISION, VNLOG2ABS_DOUBLE_PRECISION
Restrictions	None
Default Value	VNLOG2ABS_SINGLE_PRECISION

4.58.3 Functions

vNLog2Abs

```
vNLog2Abs( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode,
 input_precision )
```

Calculates negated leading bit of each element in the input vector

4.59 AUTOCORRELATION_C

This module calculates the autocorrelation function estimate for an input vector with complex-valued elements. Since the autocorrelation function is conjugate-symmetric around the zero-th lag, this module only computes positive lags. The exact lags computed start at the first lag specified and continue until the final lag value is reached.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 172. AutoCorrelation_C Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K`	The number of elements in the input vector

Table 172. AutoCorrelation_C Parameters (Continued)

Parameters	Description
scalefactor	The number of right shifts to scale the output values by
start_lag	The starting lag to be computed
final_lag	The final lag to be computed
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 173. AutoCorrelation_C Functions

Functions	Description
AutoCorrelation_C	Calculates the estimated autocorrelation function for the input vector

4.59.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts to scale the output values by. Used to reduce the chance of overflows. This parameter is run-time configurable.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

start_lag

The starting lag to be computed

Type	Integer
Range	[0-511]
Restrictions	start_lag < final_lag start_lag <= (K - 1)

final_lag

The final lag to be computed

Type	Integer
Range	[0-511]
Restrictions	start_lag < final_lag final_lag <= (K - 1)

input_addr

Address of the input vector

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	abs(input_addr_mode)*2*K

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*2*(final_lag - start_lag + 1)

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

4.59.2 Functions

AutoCorrelation_C

```
AutoCorrelation_C( name,
 K,
 scalefactor,
 start_lag,
 final_lag,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Calculates the estimated autocorrelation function for the input vector

4.60 AUTOCORRELATION_R

This module calculates the autocorrelation function estimate for an input vector with real-valued elements. Since the autocorrelation function is symmetric around the zero-th lag, this module only computes positive lags. The exact lags computed start at the first lag specified and continue until the final lag value is reached.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 174. AutoCorrelation_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
scalefactor	The number of right shifts to scale the output values by
start_lag	The starting lag to be computed
final_lag	The final lag to be computed
input_addr	Address of the input vector
output_addr	Address of the output vector
input_addr_mode	The addressing mode for the input vector
output_addr_mode	The addressing mode for the output vector

Table 175. AutoCorrelation_R Functions

Functions	Description
AutoCorrelation_R	Calculates the estimated autocorrelation function for the input vector

4.60.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts to scale the output values by. Used to reduce the chance of overflows. This parameter is run-time configurable.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

start_lag

The starting lag to be computed

Type	Integer
Range	[0-1023]
Restrictions	start_lag < final_lag start_lag <= (K - 1)

final_lag

The final lag to be computed

Type	Integer
Range	[0-1023]
Restrictions	start_lag < final_lag final_lag <= (K - 1)

input_addr

Address of the input vector

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(input_addr_mode)*K

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(output_addr_mode)*(final_lag - start_lag + 1)
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.60.2 Functions

AutoCorrelation_R

```
AutoCorrelation_R( name,
 K,
 scalefactor,
 start_lag,
 final_lag,
 input_addr,
 output_addr,
 input_addr_mode,
 output_addr_mode )
```

Calculates the estimated autocorrelation function for the input vector

4.61 AVGMAG

This module calculates the average magnitude of a complex vector. It converts each complex value to its magnitude using a table lookup based square-root approximation, and sums the result. The final sum is then rescaled to approximate an averaging.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 176. AvgMag Constants

Constants	Description
AVGMAG_SINGLE_PRECISION	Specify single-precision output
AVGMAG_DOUBLE_PRECISION	Specify double-precision output

Table 177. AvgMag Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the vector
scalefactor	The number of right shifts to apply to the summation result to implement the averaging operation and to avoid overflow
input_addr	Address of the input vector
output_addr	Address of the output
input_addr_mode	The addressing mode for the input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 178. AvgMag Functions

Functions	Description
AvgMag	Calculates the average magnitude of a complex vector

4.61.1 Constants

AVGMAG_SINGLE_PRECISION

Specify single-precision output

Value	1

AVGMAG_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.61.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts to apply to the summation result to implement the averaging operation and to avoid overflow. Ideally should be at least ceil(log2(K)) to avoid overflow. This parameter is run-time configurable.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	abs(input_addr_mode)*2*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output. The output is a single-precision value in 2.22 format, or a double-precision value in 2.46 format. This parameter is run-time configurable.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == AVGMAG_DOUBLE_PRECISION), (output_addr % 2) == 0
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	AVGMAG_SINGLE_PRECISION, AVGMAG_DOUBLE_PRECISION
Restrictions	None
Default Value	AVGMAG_DOUBLE_PRECISION

4.61.3 Functions

AvgMag

```
AvgMag( name,
 K,
 scalefactor,
 input_addr,
 output_addr,
 input_addr_mode,
 output_precision )
```

Calculates the average magnitude of a complex vector

4.62 AVGSQMAG

This module calculates the average squared magnitude of a complex vector.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 179. AvgSqMag Constants

Constants	Description
AVGSQMAG_SINGLE_PRECISION	Specify single-precision output
AVGSQMAG_DOUBLE_PRECISION	Specify double-precision output

Table 180. AvgSqMag Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of complex elements in the vector
scalefactor	The number of right shifts to apply to the summation result to implement the averaging operation and to avoid overflow
input_addr	Address of the input vector
output_addr	Address of the output
input_addr_mode	The addressing mode for the input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 181. AvgSqMag Functions

Functions	Description
AvgSqMag	Calculates the average squared magnitude of a complex vector

4.62.1 Constants

AVGSQMAG_SINGLE_PRECISION

Specify single-precision output

Value	1

AVGSQMAG_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.62.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of complex elements in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K

scalefactor

The number of right shifts to apply to the summation result to implement the averaging operation and to avoid overflow. Ideally should be at least ceil(log2(K)) to avoid overflow. This parameter is run-time configurable.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	H01MEM, H23MEM, H45MEM, ABMEM, CDMEM, ACMEM, CDMEM
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	abs(input_addr_mode)*2*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output. The output is a single-precision value in 2.22 format, or a double-precision value in 2.46 format. This parameter is run-time configurable.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == AVGSQMAG_DOUBLE_PRECISION), (output_addr % 2) == 0
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	AVGSQMAG_SINGLE_PRECISION, AVGSQMAG_DOUBLE_PRECISION
Restrictions	None
Default Value	AVGSQMAG_DOUBLE_PRECISION

4.62.3 Functions

AvgSqMag

```
AvgSqMag( name,
 K,
 scalefactor,
 input_addr,
 output_addr,
 input_addr_mode,
 output_precision )
```

Calculates the average squared magnitude of a complex vector

4.63 CORRELATION_C

This module calculates the correlation function estimate between two complex valued vectors of either the same or different lengths. The correlation is computed starting at the lag value specified and continuing until the final lag value is reached.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 182. Correlation_C Parameters

Parameters	Description
`name`	A unique name for each instance of the module
`K0`	The number of elements in the first input vector

Table 182. Correlation_C Parameters (Continued)

Parameters	Description
K1	The number of elements in the second input vector
scalefactor	The number of right shifts to scale the output values by; used to reduce the chance of overflows
start_lag	The starting lag to be computed
final_lag	The final lag to be computed
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the first input vector
input1_addr_mode	The addressing mode for the second input vector
output_addr_mode	The addressing mode for the output vector

Table 183. Correlation_C Functions

Functions	Description
Correlation_C	Calculates the estimated correlation function for two input vectors

4.63.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K0

The number of elements in the first input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-256]
Restrictions	None
Run-time Parameter	<name>_K0

K1

The number of elements in the second input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-256]
Restrictions	None
Run-time Parameter	<name>_K1

scalefactor

The number of right shifts to scale the output values by; used to reduce the chance of overflows. This parameter is run-time configurable.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

start_lag

The starting lag to be computed

Type	Integer
Range	[-255 - 255]
Restrictions	start_lag < final_lag start_lag <= (K0 - 1) start_lag >= (-(K1-1))

final_lag

The final lag to be computed

Type	Integer
Range	[-255 - 255]
Restrictions	start_lag < final_lag final_lag <= (K0 - 1) final_lag >= (-(K1-1))

input0_addr

Address of the first input vector

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input0_addr % 2) == 0 memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*2*K

input1_addr

Address of the second input vector

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input1_addr % 2) == 0 memtype(input1_addr) != memtype(input0_addr)
Size	abs(input1_addr_mode)*2*K

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	abs(output_addr_mode)*2*(final_lag - start_lag + 1)
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the first input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-128-128]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the second input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-128-128]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-128-128]
Restrictions	None
Default Value	1

4.63.2 Functions

Correlation_C

```
Correlation_C( name,
 K0,
 K1,
 scalefactor,
 start_lag,
 final_lag,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode )
```

Calculates the estimated correlation function for two input vectors

4.64 CORRELATION_R

This module calculates the correlation function estimate between two real valued vectors of either the same or different lengths. The correlation is computed starting at the lag value specified and continuing until the final lag value is reached.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 184. Correlation_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K0	The number of elements in the first input vector
K1	The number of elements in the second input vector
scalefactor	The number of right shifts to scale the output values by
start_lag	The starting lag to be computed
final_lag	The final lag to be computed
input0_addr	Address of the first input vector
input1_addr	Address of the second input vector
output_addr	Address of the output vector
input0_addr_mode	The addressing mode for the first input vector
input1_addr_mode	The addressing mode for the second input vector
output_addr_mode	The addressing mode for the output vector

Table 185. Correlation_R Functions

Functions	Description
Correlation_R	Calculates the estimated correlation function for two input vectors

4.64.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K0

The number of elements in the first input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K0

K1

The number of elements in the second input vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-512]
Restrictions	None
Run-time Parameter	<name>_K1

scalefactor

The number of right shifts to scale the output values by. Used to reduce the chance of overflows. This parameter is run-time configurable.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

start_lag

The starting lag to be computed

Type	Integer
Range	[-511 - 511]
Restrictions	start_lag < final_lag start_lag <= (K0 - 1) start_lag >= (-(K1-1))

final_lag

The final lag to be computed

Type	Integer
Range	[-511 - 511]
Restrictions	start_lag < final_lag final_lag <= (K0 - 1) final_lag >= (-(K1 - 1))

input0_addr

Address of the first input vector

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input0_addr) != memtype(input1_addr)
Size	abs(input0_addr_mode)*K

input1_addr

Address of the second input vector

Type	Input
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	memtype(input1_addr) != memtype(input0_addr)
Size	abs(input1_addr_mode)*K

output_addr

Address of the output vector. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	None
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(output_addr_mode)*(final_lag - start_lag + 1)
Run-time Parameter	<name>_output_addr

input0_addr_mode

The addressing mode for the first input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

input1_addr_mode

The addressing mode for the second input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_addr_mode

The addressing mode for the output vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

4.64.2 Functions

Correlation_R

```
Correlation_R( name,
 K0,
 K1,
 scalefactor,
 start_lag,
 final_lag,
 input0_addr,
 input1_addr,
 output_addr,
 input0_addr_mode,
 input1_addr_mode,
 output_addr_mode )
```

Calculates the estimated correlation function for two input vectors

4.65 MAXELEMENT

This module determines the maximum value within a real-valued vector. It supports addressing the input vector with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 186. MaxElement Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector input
input_addr	Address of the input vector
output_addr	Address of the output value
input_addr_mode	The addressing mode for the input vector

Table 187. MaxElement Functions

Functions	Description
MaxElement	Determines the maximum element within a real-valued vector

4.65.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_addr_mode == 1) , (input_addr % 2) == 0
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output value. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.65.2 Functions

MaxElement

```
MaxElement( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode )
```

Determines the maximum element within a real-valued vector

4.66 MINELEMENT

This module determines the minimum value within a real-valued vector. It supports addressing the input vector with a configurable step size so that interleaved vectors can be used.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 188. MinElement Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector input
input_addr	Address of the input vector
output_addr	Address of the output value
input_addr_mode	The addressing mode for the input vector

Table 189. MinElement Functions

Functions	Description
MinElement	Determines the minimum element within a real-valued vector

4.66.1 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector input. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_addr_mode == 1) , (input_addr % 2) == 0
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output value. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

4.66.2 Functions

MinElement

```
MinElement( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode )
```

Determines the minimum element within a real-valued vector

4.67 MEAN

This module calculates the sample mean of a set of vector elements. It supports single- or double-precision inputs and produces single- or double-precision outputs. The 1/K factor required for the mean is approximated using a 24-bit fractional multiplier.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 190. Mean Constants

Constants	Description
MEAN_SINGLE_INPUT_PRECISION	Specify single-precision inputs
MEAN_DOUBLE_INPUT_PRECISION	Specify double-precision inputs
MEAN_SINGLE_OUTPUT_PRECISION	Specify single-precision output
MEAN_DOUBLE_OUTPUT_PRECISION	Specify double-precision output

Table 191. Mean Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector
input_addr	Address of the input vector
output_addr	Address of the output
input_addr_mode	The addressing mode for the input vector
input_precision	The size of each input element, either single precision (one word) or double precision (two words)
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 192. Mean Functions

Functions	Description
Mean	Calculates the mean of a vector of elements

4.67.1 Constants

MEAN_SINGLE_INPUT_PRECISION

Specify single-precision inputs

Value	1

MEAN_DOUBLE_INPUT_PRECISION

Specify double-precision inputs

Value	2

MEAN_SINGLE_OUTPUT_PRECISION

Specify single-precision output

Value	1

MEAN_DOUBLE_OUTPUT_PRECISION

Specify double-precision output

Value	2

4.67.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector

Type	Integer
Range	[2-1024]
Restrictions	None

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_precision == MEAN_DOUBLE_INPUT_PRECISION), (input_addr % 2) == 0
Size	abs(input_addr_mode)*input_precision*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == MEAN_DOUBLE_OUTPUT_PRECISION), (output_addr % 2) == 0
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input_precision

The size of each input element, either single precision (one word) or double precision (two words).

Type	Integer
Range	MEAN_SINGLE_INPUT_PRECISION, MEAN_DOUBLE_INPUT_PRECISION
Restrictions	None
Default Value	MEAN_SINGLE_INPUT_PRECISION

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	MEAN_SINGLE_OUTPUT_PRECISION, MEAN_DOUBLE_OUTPUT_PRECISION
Restrictions	None
Default Value	MEAN_DOUBLE_OUTPUT_PRECISION

4.67.3 Functions

Mean

```
Mean( name,
 K,
 input_addr,
```

```
output_addr,
input_addr_mode,
input_precision,
output_precision )
```

Calculates the mean of a vector of elements

4.68 SUM

This module calculates the sum of a set of vector elements. It supports single- or double-precision inputs and produces a double-precision output. If the inputs are single precision, they are sign-extended to double precision prior to summing. No rescaling of the data occurs, so there is potential for saturation.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 193. Sum Constants

Constants	Description
SUM_SINGLE_INPUT_PRECISION	Specify single-precision inputs
SUM_DOUBLE_INPUT_PRECISION	Specify double-precision inputs

Table 194. Sum Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the vector
input_addr	Address of the input vector
output_addr	Address of the output
input_addr_mode	The addressing mode for the input vector
input_precision	The size of each input element, either single precision (one word) or double precision (two words)

Table 195. Sum Functions

Functions	Description
Sum	Calculates the sum of a vector of elements

4.68.1 Constants

SUM_SINGLE_INPUT_PRECISION

Specify single-precision inputs

Value	1

SUM_DOUBLE_INPUT_PRECISION

Specify double-precision inputs

Value	2

4.68.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the vector. This parameter is run-time configurable. The number of elements can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[2-1024]
Restrictions	None
Run-time Parameter	<name>_K

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (input_precision == SUM_DOUBLE_INPUT_PRECISION), (input_addr % 2) == 0
Size	abs(input_addr_mode)*input_precision*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	(output_addr % 2) == 0
Address Parameter Restrictions	None
Size	2
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

input_precision

The size of each input element, either single precision (one word) or double precision (two words).

Type	Integer
Range	SUM_SINGLE_INPUT_PRECISION, SUM_DOUBLE_INPUT_PRECISION
Restrictions	None
Default Value	SUM_SINGLE_INPUT_PRECISION

4.68.3 Functions

Sum

```
Sum( name,
 K,
 input_addr,
 output_addr,
 input_addr_mode,
 input_precision )
```

Calculates the sum of a vector of elements

4.69 VARIANCE_C

This module calculates the sample variance of a set of complex vector elements, with the assumption that the elements have a sample mean of zero. It supports single-precision inputs and produces a single- or double-precision

output. The 1/K (biased estimate) or 1/(K-1) (unbiased estimate) factor required for the variance estimation is approximated using a 24-bit fractional multiplier.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 196. Variance_C Constants

Constants	Description
VARIANCE_C_UNBIASED	Specify that an unbiased estimator should be used for the variance
VARIANCE_C_BIASED	Specify that a biased estimator should be used for the variance
VARIANCE_C_SINGLE_PRECISION	Specify single-precision output
VARIANCE_C_DOUBLE_PRECISION	Specify double-precision output

Table 197. Variance_C Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
scalefactor	The number of right shifts to apply to the sum of squares before multiplying by 1/K or 1/(K-1)
biased_flag	Determines whether the variance calculation is biased or unbiased
input_addr	Address of the input vector
output_addr	Address of the output
input_addr_mode	The addressing mode for the input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 198. Variance_C Functions

Functions	Description
Variance_C	Calculates the sample variance of a vector of complex valued elements

4.69.1 Constants

VARIANCE_C_UNBIASED

Specify that an unbiased estimator should be used for the variance

Value	0

VARIANCE_C_BIASED

Specify that a biased estimator should be used for the variance

Value	1

VARIANCE_C_SINGLE_PRECISION

Specify single-precision output

Value	1

VARIANCE_C_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.69.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector

Type	Integer
Range	[2-256]
Restrictions	None

scalefactor

The number of right shifts to apply to the sum of squares before multiplying by 1/K or 1/(K-1). Can be used to avoid overflow and store the result in a mixed precision format. This parameter is run-time configurable. The scale factor can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

biased_flag

Determines whether the variance calculation is biased or unbiased

Type	Integer
Range	VARIANCE_C_UNBIASED, VARIANCE_C_BIASED
Restrictions	None

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(input_addr % 2) == 0
Size	abs(input_addr_mode)*2*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output. The output is either a single-precision or a double-precision value. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	(output_addr % 2) == 0
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-256-256]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	VARIANCE_C_SINGLE_PRECISION, VARIANCE_C_DOUBLE_PRECISION
Restrictions	None
Default Value	VARIANCE_C_DOUBLE_PRECISION

4.69.3 Functions

Variance_C

```
Variance_C( name,
 K,
 scalefactor,
 biased_flag,
 input_addr,
 output_addr,
 input_addr_mode,
 output_precision )
```

Calculates the sample variance of a vector of complex valued elements

4.70 VARIANCE_R

This module calculates the sample variance of a set of real vector elements, with the assumption that the elements have a sample mean of zero. It supports single-precision inputs and produces a single- or double-precision output. The 1/K (biased estimate) or 1/(K-1) (unbiased estimate) factor required for the variance estimation is approximated using a 24-bit fractional multiplier.

Module Attributes

Single Function Yes

Reset No

BFP Support None

Table 199. Variance_R Constants

Constants	Description
VARIANCE_R_UNBIASED	Specify that an unbiased estimator should be used for the variance
VARIANCE_R_BIASED	Specify that a biased estimator should be used for the variance
VARIANCE_R_SINGLE_PRECISION	Specify single-precision output
VARIANCE_R_DOUBLE_PRECISION	Specify double-precision output

Table 200. Variance_R Parameters

Parameters	Description
name	A unique name for each instance of the module
K	The number of elements in the input vector
scalefactor	The number of right shifts to apply to the sum of squares before multiplying by 1/K or 1/(K-1)
biased_flag	Determines whether the variance calculation is biased or unbiased
input_addr	Address of the input vector
output_addr	Address of the output
input_addr_mode	The addressing mode for the input vector
output_precision	The size of the output, either single precision (one word) or double precision (two words)

Table 201. Variance_R Functions

Functions	Description
Variance_R	Calculates the sample variance of a vector of real valued elements

4.70.1 Constants

VARIANCE_R_UNBIASED

Specify that an unbiased estimator should be used for the variance

Value	0

VARIANCE_R_BIASED

Specify that a biased estimator should be used for the variance

Value	1

VARIANCE_R_SINGLE_PRECISION

Specify single-precision output

Value	1

VARIANCE_R_DOUBLE_PRECISION

Specify double-precision output

Value	2

4.70.2 Parameters

name

A unique name for each instance of the module

Type	String
Range	N/A
Restrictions	None

K

The number of elements in the input vector

Type	Integer
Range	[2-512]
Restrictions	None

scalefactor

The number of right shifts to apply to the sum of squares before multiplying by 1/K or 1/(K-1). Can be used to avoid overflow and store the result in a mixed precision format. This parameter is run-time configurable. The scale factor can be changed by writing a value to the run-time parameter's memory location.

Type	Integer
Range	[-24-24]
Restrictions	None
Run-time Parameter	<name>_scalefactor

biased_flag

Determines whether the variance calculation is biased or unbiased

Type	Integer
Range	VARIANCE_R_UNBIASED, VARIANCE_R_BIASED
Restrictions	None

input_addr

Address of the input vector. This parameter is run-time configurable. The input address can be changed by writing a new value to the run-time parameter's memory location.

Type	Input
Memory Spaces	All
Shared Parameters	output_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	None
Size	abs(input_addr_mode)*K
Run-time Parameter	<name>_input_addr

output_addr

Address of the output. The output is either a single-precision or a double-precision value. This parameter is run-time configurable. The output address can be changed by writing a value to the run-time parameter's memory location.

Type	Output
Memory Spaces	All
Shared Parameters	input_addr
Configuration Parameter Restrictions	None
Address Parameter Restrictions	if (output_precision == VARIANCE_R_DOUBLE_PRECISION), (output_addr % 2) == 0
Size	output_precision*1
Run-time Parameter	<name>_output_addr

input_addr_mode

The addressing mode for the input vector. Specifies the pointer increment to use when reading the next element of this vector.

Type	Integer
Range	[-512-512]
Restrictions	None
Default Value	1

output_precision

The size of the output, either single precision (one word) or double precision (two words).

Type	Integer
Range	VARIANCE_R_SINGLE_PRECISION, VARIANCE_R_DOUBLE_PRECISION
Restrictions	None
Default Value	VARIANCE_R_DOUBLE_PRECISION

4.70.3 Functions

Variance_R

```
Variance_R( name,
 K,
 scalefactor,
 biased_flag,
 input_addr,
 output_addr,
 input_addr_mode,
 output_precision )
```

Calculates the sample variance of a vector of real valued elements

CHAPTER 5
HEAR Configuration Tool

This chapter describes the HEAR Configuration Tool (HCT) and how it can be used to generate application-specific microcode configurations. The HCT is a command-line tool that takes an input file which specifies the microcode configuration using a simple macro-based language. It outputs a CFX object file for linking with the application, a CFX header file which contains all of the symbol definitions and system configuration that was specified as part of the HCT input file, and an optional summary file with additional information about the generated microcode configuration. The dataflow of files and tools involving the HCT and the CFX toolchain is depicted in Figure 47.

The HCT is used by the build process in the IDE. The IDE builder automatically processes the HCT input file, links in the resulting object file and stores the generated include file in the main source directory of the project. For most users and projects, the default IDE behavior for processing HCT input files is sufficient.

IMPORTANT: If no HCT source file is included in an IDE project, the application will be built without any microcode and the HEAR microcode memory will not be initialized properly. Starting the HEAR when microcode memory is not initialized causes undefined behavior. If the HCT was not used to generate a valid microcode configuration, do not start the HEAR Configurable Accelerator from the application.

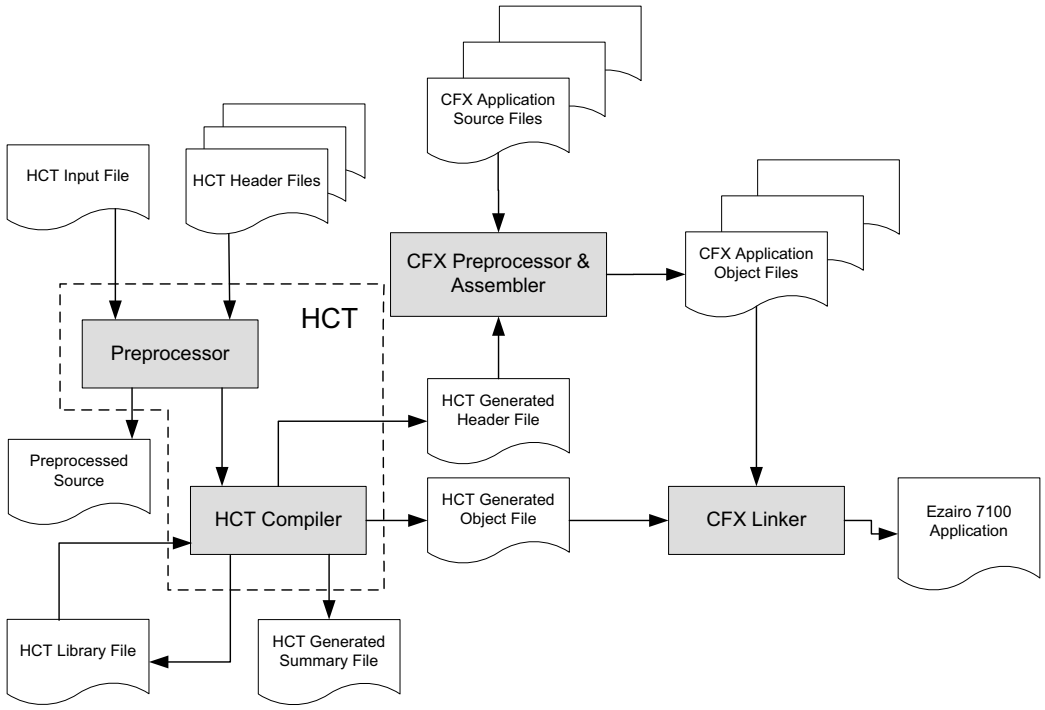


Figure 47. HCT Microcode Configuration Usage and Integration with CFX Compilation Toolchain

5.1 USAGE

The HCT uses the following command-line syntax:

```
hct <options> <input_file>
```

The input file can be any valid HCT source file, normally denoted with the file extension *.hct*. By default, without any options the HCT will output a *.cfi* header file and *.o* object code file with the same filename as the input (minus the *.hct* extension). Table 202 defines the options available for the HCT.

Table 202. HCT Command Line Options

Option	Description
`-h`	Display the usage (help).
`--version`	Display version information.
`-v`	Turn on verbose output. Verbose output provides informational messages to the console about the different stages of the HCT's processing.
`-b`<machine>	The argument <machine> specifies the target machine for compilation. It recognizes names of the form <arch>`c`<cid>, where <arch> is the instruction set architecture, and <cid> is the chip ID. For Ezairo 7100, you must specify `sk4c102` for this option.
`-s`	Generate a summary file. The summary file will have the same name and location as the object file output, but with a `.smy` file extension.
`-g`	Compile the object file output with debug information. Equivalent to specifying `-g` when compiling source code using *cfcc* (see the *CFX Toolchain Reference*).
`-E`	Stop after the preprocessing stage. Do not compile or link. If the `-o` option is specified, the preprocessed output is written to the specified file or directory. Otherwise, the output is printed to standard output.
`-C`	Preserve comments in preprocessed output.
`-t`	Suppress control lines which include the source line and file name. This option has no effect unless accompanied by the `-E` option.
`-S`<name>	Defines the identifier <name> as if you had included `#set` <name> `1` at the beginning of the input file.
`-D`<name>	Defines the identifier <name> as if you had included `#define` <name> `1` at the beginning of the input file.
`-D`<name>`=`<str>	Defines the identifier <name> as if you had included `#define` <name> <str> at the beginning of the input file.
`-U`<name>	Undefines the identifier `name` as if you had included `#undef` <name> at the beginning of the input file.
`-I`<dir>	Searches directory <dir> for included files if necessary.
`-Wundef`	Warns about undefined symbols used in `#if`,`#elif`,`#set`, and `#rept` directives. By default, the preprocessor substitutes a value of 0 for symbols that are not defined when it is evaluating the expressions for these directives.
`-l` <libraryfile_input>	The `-l` option lets you specify a filename for a library (*.hcl*) file input. You can use this option multiple times to specify several library files. The HCT imports their contents in the order that you give them on the command line. If you specify a relative name for the library file, the HCT follows the search path that you specify with the `-L` option to locate it. NOTE: You must include the standard library on the command line to gain access to the standard microcode modules described in this manual.

Table 202. HCT Command Line Options (Continued)

Option	Description
-L <library_folder>	The -L option lets you add a folder to the HCT's library search path. The HCT searches the folders in its search path whenever you specify a library file with a relative path using the -l option. You can use this option multiple times to build a search path to suit your purposes. The HCT begins its search in the current directory, then searches in the order that you give the -L options on the command line, stopping at the first matching library file it finds.
-i <includefile_output>	The -i setting lets you specify a directory or filename for the include (*.cfi*) file output. If you specify an existing directory, the HCT outputs an include file to that directory, giving it the same filename as the input but with the *.hct* extension changed to *.cfi*. If you specify a filename, the HCT uses the filename directly for the include file output. Any parent directories that you specify as part of the filename must already exist. If you specify a non-existent directory, the HCT will interpret it as a filename.
--defn-prefix <prefix>	Prefix the preprocessor definitions in the include file (*.cfi*) output with <prefix>. If you do not specify this option, the HCT prefixes definitions with HCT.
-o <main_output>	The -o setting lets you specify a directory or filename for the main output, which is either the object file or the preprocessed source (*.hcr*) file depending on whether the -E option is given. The object file extension is *.o* if the --toolchain option is set to gnu, or *.obj* if the --toolchain option is the default, archelon. If you specify an existing directory, the HCT writes its output to that directory, giving it the same filename as the input but with the *.hct* extension changed to either *.o* or *.hcr*. If you specify a filename, the HCT uses the filename directly for the output file name. Any parent directories that you specify as part of the filename must already exist. If you specify a non-existent directory, the HCT will interpret it as a filename.
-a <libraryfile_output>	The -a setting lets you specify a directory or filename for the library (*.hcl*) file output. If you specify an existing directory, the HCT outputs an object file to that directory, giving it the same filename as the input but with the *.hct* extension changed to *.hcl*. If you specify a filename, the HCT uses the filename directly for the include file output. Any parent directories that you specify as part of the filename must already exist. If you specify a non-existent directory, the HCT will interpret it as a filename.
--toolchain=<toolchain>	Selects the back end toolchain. Valid values for <toolchain> are archelon (default) and gnu.

5.2 HCT LANGUAGE

5.2.1 Lexical Conventions

The HCT language has only a few simple lexical conventions. In general, the HCT allows for flexible use of white space (spaces and tabs) to separate all tokens. White space includes newlines and carriage returns, allowing a single HCT input file statement to stretch across multiple lines.

The HCT language accepts C/C++ style comments. This includes line-based comments beginning with // and block-based comments delimited by /* and */.

The remainder of an HCT input source file is composed of identifiers, literals, delimiters and expression operators. The allowable syntax for these tokens is described in Section 5.2.2, “Language Syntax”.

An identifier must start with an alphabetic character and the remaining characters may be alphanumeric characters, the underscore character (‘_’), the dot character (‘.’) or the dollar sign character (‘$’). Identifiers are used to refer to all functions and constants, including user-defined constants.

Literals must be integer quantities specified in decimal, hexadecimal, octal or binary notation. Normal C language style notation is used for formatting literal values.

Delimiters include the parentheses characters, ‘(’ and ‘)’, as well as the comma character, ‘,’ for separating parameters in argument lists. More information about how delimiters are used is given in Section 5.2.2, “Language Syntax”.

Expression operators include the following: +, -, *, /, <<, and >>. Formatting and evaluation of expressions is described in Section 5.2.2, “Language Syntax”.

5.2.2 Language Syntax

The HCT language uses a macro-based (or function-call like) syntax for all configuration statements. Each HCT statement is a macro call with the following form:

```
<macro_name>(<argument_list>)
```

The first part of each statement, `<macro_name>`, must be an identifier referring to a valid built-in or microcode function. The `<argument_list>` is a comma-separated list of arguments to the macro or function. An empty argument list is allowed.

The HCT maps the arguments that you supply in `<argument_list>` to the parameters of a called function. A function can provide default values for some of its parameters. If you do not pass an argument for a parameter that has a default value, the default value is passed to the function automatically. You must supply arguments for all parameters that do not supply default values.

5.2.2.1 Argument Lists

Two styles of arguments are allowed in an argument list: simple and keyword. Simple arguments are only allowed to appear from the beginning of the list to the first keyword argument, after which all remaining arguments must be of the keyword style.

Simple arguments consist of a single identifier or expression. They are mapped to parameters in the order that they appear in the argument list. The first simple argument, for example, maps to the first parameter of the called function.

Keyword arguments have the following form:

```
<parameter_name>=<value>
```

The `<value>` is passed in for the parameter with the name that matches `<parameter_name>`. Like a simple argument, the `<value>` can be a single identifier or expression. Giving a `<parameter_name>` that does not exactly match the name of one of the parameters of the called function is invalid. Because keyword arguments are mapped to parameters by name, you can supply them in any order.

The allowed value of an argument depends on the specific macro or function being called. See Section 5.3, “Built-in Macros” on page 330 and Chapter 4, “Module Reference” on page 95 for details on a particular macro or function.

5.2.2.2 Expressions

Expressions are groups of tokens composed of the following:

- Literal values
- Expression operators
- Parentheses for grouping of expressions
- Identifiers which refer to valid built-in, microcode or user-defined constants

Expression parsing and evaluation occurs using normal C language style rules for formatting and operator precedence. The result of every expression is an integer value.

5.2.2.3 Identifiers

Identifiers are used to reference any of the following:

- Built-in macros and constants
- Microcode modules, functions and constants
- User-defined constants
- User-defined microcode module names

A single namespace is used for all identifiers, so no two identifiers of any type can be the same. Specifically, a user-defined identifier cannot be the same as any built-in or microcode identifier.

Additionally, the entire set of identifiers beginning with the string ‘HCT_’ are reserved for preprocessor definitions generated by the HCT in the `.cfi` include file output. This is because user-defined identifiers are also mapped to preprocessor definitions in this file as described in Section 5.7, “CFX Header File Output”, and a preprocessor definition naming conflict in this output file could generate compilation errors.

5.2.2.4 Input File Structure

An HCT input file is split into sections which are part of a function chain, and sections which are not part of a function chain. The sections of a file which are part of a function chain are delineated by the `StartFunctionChain` and `EndFunctionChain` built-in macros described in Section 5.3, “Built-in Macros” on page 330.

Microcode functions for the configuration of multi-function modules, i.e., `<module>_Configure` functions, are only valid outside of a function chain. All other microcode functions are only valid inside of a function chain.

Other built-in macros can only be used either inside or outside a function chain, as specified in Section 5.3, “Built-in Macros”.

5.2.3 The Preprocessor

Prior to compiling your source code, the HCT runs it through a C-style preprocessor, the same preprocessor used by CFCC to preprocess CFX assembly code. It supports all of the standard C preprocessor directives, such as `#define`, `#if`, and `#include`, along with several non-standard extensions, such as `#macro` and `#set`.

With one exception, the preprocessor runs exactly as it does in CFCC. The difference is that when the preprocessor is run from the HCT, it does not predefine the `_TARGET_<machine>` or `_CFXASM_VER` macros.

For a full description of the preprocessor and all of the directives that it supports, see the *CFX Toolchain Reference*.

5.3 BUILT-IN MACROS

The majority of macros in an HCT input file are calls to the microcode module functions described in Chapter 4, “Module Reference” on page 95. In addition to these microcode functions, a number of built-in macros are used to help configure the microcode or perform specialized tasks inside of a function chain. These macros are described below.

Define()

The `Define` macro is used to generate a user-defined constant. A constant defined using the `Define` macro is available for use in a parameter, either directly or as part of an expression. User-defined constants are only valid after the point in the input file at which they are defined. The `Define` macro can only be used outside of a function chain. The `Define` macro and its parameters are summarized in the table below.

Macro Syntax	`Define(id, value)`	
Parameters	`id`	- a valid user-defined identifier that is not the same as any other previously defined identifier
	`value`	- a literal numeric value or expression that will be set as the value of `id`

FIFOConfigure()

The `FIFOConfigure` macro is used to quickly and efficiently define a number of symbols associated with configuring a FIFO. It causes the HCT to generate preprocessor macros in the CFX include file which help to configure the FIFO and allocate the memory associated with the FIFO. The macro syntax and parameters are summarized in the table below. The `FIFOConfigure` macro can only be used outside of a function chain.

The `FIFOConfigure` macro allows for the configuration of single and dual FIFOs. A dual FIFO is a set of two identically configured FIFOs - one in A0/A1 memory (an A FIFO) and one in B0/B1 memory (a B FIFO). Only combinations where the FIFO indices are equal are allowed, e.g. a combination of FIFOs A0_0 and B0_0 is a valid dual FIFO for use with the `FIFOConfigure` macro, but a combination of FIFOs A0_0 and B1_0 is not.

Macro Syntax	`FIFOConfigure(fifo_name, start, length, block_size, base_ptr, ioblock_ptr)`	
Parameters	`fifo_name`	- the name of the FIFO being configured; must be one of the following built-in constants:`FIFO_A0_[0-7], FIFO_A1_[0-7], FIFO_B0_[0-7], FIFO B1_[0-7], FIFO_AB0_[0-7], FIFO_AB1_[0-7]`
	`start`	- the starting offset of the FIFO into A memory, B memory or AB memory, i.e., the value to be stored in the `D_FIFO_xx_x_START` register for single FIFOs, or twice the value for dual FIFOs
	`length`	- the length of the FIFO, i.e., the value to be stored in the `D_FIFO_xx_x_LENGTH` register for single FIFOs, or twice the value for dual FIFOs
	`block_size`	- the block size for the FIFO, i.e., the value to be stored in the `D_FIFO_xx_x_BLOCK_SIZE` register for single FIFOs, or twice the value for dual FIFOs
	`base_ptr`	- the base pointer for the FIFO, i.e., the value to be stored in the `D_FIFO_xx_x_BASE_PTR` register for single FIFOs, or twice the value for dual FIFOs
	`ioblock_ptr`	- the IO block pointer for the FIFO, i.e., the value to be stored in the `D_FIFO_xx_x_IOBLOCK_PTR` register for single FIFOs, or twice the value for dual FIFOs

StartFunctionChain() and EndFunctionChain()

The StartFunctionChain and EndFunctionChain macros are used to delineate the beginning and end of a function chain. Between these two macros, you specify the behavior of a function chain by invoking certain built-in macros and non-configuration microcode functions. Empty function chains do nothing. The StartFunctionChain macro and its parameters are summarized in the table below. The EndFunctionChain macro has no parameters and merely ends the current function chain.

Macro Syntax	StartFunctionChain(function_chain, id)	
Parameters	function_chain	- the command or FIFO event to which to initially bind this function chain; must be a built-in constant: one of CMD[0-15] for a CFX command function chain, FIFO_[0-7] for a FIFO-related function chains, or UNBOUND. If UNBOUND, the function chain is initially not bound to any command or FIFO event. You can change the binding of any function chain at run time using the system firmware macro Sys_Set_HEAR_FunctionChain_Vector.
	id	- a valid user-defined identifier string that will be used to uniquely identify this function chain. A label named <id>_entry is generated at the entry point of the function chain for use with the Sys_Set_HEAR_FunctionChain_Vector system macro.

If(), Else(), and EndIf()

The If, Else and EndIf macros are used to conditionally execute blocks of built-in macros and non-configuration microcode functions. As illustrated below, the syntax of an If-Else-EndIf sequence consists of:

1. An If macro followed by a block of macros or functions
2. An optional Else macro followed by a second block of macros or functions
3. A terminating EndIf macro.

The first block of macros and functions is hereafter called the consequent block. The second is called the alternative block.

```
If(condition, initial_value)
    // consequent block
Else()
    // alternative block
EndIf()
```

The If macro tests the value of a global variable at run time. If the variable is non-zero, execution continues with the macros and functions in the consequent block until it reaches a matching Else or EndIf macro. When an Else macro is reached, execution jumps past the alternative block to the matching EndIf macro. Alternatively, when the variable is zero, execution jumps past the consequent block to the matching Else macro, if one is given, or failing that, the matching EndIf macro.

If macros can be nested, allowing for complex conditions involving several global variables.

The If macro and its parameters are summarized in the table below. The Else and EndIf have no parameters.

Macro Syntax	`if(condition, initial_value)`	
Parameters	`condition`	- the label of the global variable to test at run time. The HCT allocates storage for the variable, exposing its address as an external symbol in the generated include file
	`initial_value`	- the initial value for the global variable. If the same `condition` is used in multiple `If` macros, the variable is initialized with the `initial_value` that occurs first in the source file.

Event()

The `Event` macro is used to send an interrupt signal from the HEAR to the CFX. It can be used at any point inside of a function chain. The macro and its parameters are summarized in the table below.

Macro Syntax	`Event(int_num)`	
Parameters	`int_num`	- the HEAR interrupt number to send to the CFX

FIFOUpdate()

The `FIFOUpdate` macro is used to update a software controlled FIFO. It writes the FIFO block size to the appropriate `D_FIFO_xx_x_BLOCK_CNT` register(s) in order to update the FIFO's pointers and generate a FIFO interrupt. To use this macro, the associated FIFO must have been already configured with the `FIFOConfigure` macro. This macro can only be used inside a function chain. The `FIFOUpdate` macro supports the same single memory FIFOs, as well as the same dual FIFOs which are supported by the `FIFOConfigure` macro. The `FIFOUpdate` macro and its parameters are summarized in the table below.

Macro Syntax	`FIFOUpdate(fifo_name)`	
Parameters	`fifo_name`	- the name of the FIFO to be updated; must be one of the following built-in constants: `FIFO_A0_[0-7]`, `FIFO_A1_[0-7]`, `FIFO_B0_[0-7]`, `FIFO_B1_[0-7]`, `FIFO_AB0_[0-7]`, `FIFO_AB1_[0-7]`

ResetSaturationFlag() and SaveSaturationFlag()

The `ResetSaturationFlag` and `SaveSaturationFlag` macros are used to reset the HEAR's internal saturation flag and save it to a variable in shared memory. They have no parameters. Using either or both of these macros causes the HCT to reserve a location in shared memory with the symbol `HEAR_SATURATION_FLAG`. The address of this location is included in the HCT's CFX header file output. These macros can only be used inside of a function chain. For more information about detecting saturation in the HEAR microcode, see Section 2.5.4.1, "Saturation Detection" on page 31.

ResetOverflowFlag() and SaveOverflowFlag()

The `ResetOverflowFlag` and `SaveOverflowFlag` macros are used to reset the HEAR's internal accumulator overflow flag and save it to a variable in shared memory. They have no parameters. Using either or both of these macros causes the HCT to reserve a location in shared memory with the symbol `HEAR_OVERFLOW_FLAG`. The address of this location is included in the HCT's CFX header file output. These macros can only be used inside of a function chain. For more information about detecting accumulator overflow in the HEAR microcode, see Section 2.5.4.1, "Saturation Detection" on page 31.

EnableOverflowProtect() and DisableOverflowProtect()

The `EnableOverflowProtect` and `DisableOverflowProtect` macros are used to enable and disable the HEAR's arithmetic overflow protection. They have no parameters. These macros can only be used inside of a function chain. For more information about overflow protection control in the HEAR microcode, see Section 2.5.4.1, "Saturation Detection" on page 31.

5.4 Built-in Constants

The HCT includes a number of built-in constants for specifying macro parameters. These built-in constants fall into several categories: address constants, FIFO names and function chain names.

The address constants listed below are provided to assist in specifying address parameters to microcode modules. Each has its value specified in *sk4_map.cfi* (see the *Firmware Reference Manual for Ezairo 7100*).

- `D_HEAR_A0MEM_BASE`
- `D_HEAR_A1MEM_BASE`
- `D_HEAR_B0MEM_BASE`
- `D_HEAR_B1MEM_BASE`
- `D_HEAR_C0MEM_BASE`
- `D_HEAR_C1MEM_BASE`
- `D_HEAR_D0MEM_BASE`
- `D_HEAR_D1MEM_BASE`
- `D_HEAR_AB0MEM_BASE`
- `D_HEAR_AB1MEM_BASE`
- `D_HEAR_CD0MEM_BASE`
- `D_HEAR_CD1MEM_BASE`
- `D_HEAR_AC0MEM_Y_BASE`
- `D_HEAR_AC1MEM_Y_BASE`
- `D_HEAR_BD0MEM_Y_BASE`
- `D_HEAR_BD1MEM_Y_BASE`
- `D_HEAR_H0MEM_BASE`
- `D_HEAR_H1MEM_BASE`
- `D_HEAR_H2MEM_BASE`
- `D_HEAR_H3MEM_BASE`
- `D_HEAR_H4MEM_BASE`
- `D_HEAR_H5MEM_BASE`
- `D_HEAR_H01MEM_BASE`
- `D_HEAR_H23MEM_BASE`
- `D_HEAR_H45MEM_BASE`

The FIFO name and function chain name constants are used as parameters for the built-in `FIFOConfigure`, `StartFunctionChain` and `FIFOUpdate` macros. These constants do not have a defined numeric value and are treated in a special manner by the HCT such that their use outside of the specific parameters described in Section 5.3, "Built-in Macros" on page 330 results in an error. These constants include the following:

- `FIFO_A0_[0-7]`
- `FIFO_A1_[0-7]`
- `FIFO_B0_[0-7]`
- `FIFO_B1_[0-7]`
- `FIFO_AB0_[0-7]`

- FIFO_AB1_[0-7]
- CMD[0-15]
- UNBOUND

5.5 Microcode Functions and Constants

The microcode module functions and constants as documented in Chapter 4, “Module Reference” on page 95 can be used directly in an HCT input file. Every call to a microcode module function requires the use of a unique module instance name which must conform to the requirements of a valid user-defined identifier (see Section 5.2.2.3, “Identifiers”).

Configuration functions for multi-function modules must be used outside of any function chain and must be used before the corresponding module’s non-configuration functions are called inside of a function chain. All non-configuration functions can only be specified inside of a function chain.

5.5.1 Parameter Checking

When a microcode function is used in an HCT input file, the configuration and address parameters for that microcode module are checked against the restrictions for the parameter. The parameter’s expression is first evaluated, and the result of the expression is used in the checks.

Parameters are checked in the order that they appear in the microcode function’s argument list. A parameter with a violated range, memory type or restriction causes the HCT to stop processing immediately and to output an error message identifying the microcode function, the parameter, the value of the parameter and the condition which was not met.

5.6 Library Files

The HCT can export and import library files(*.hcl* extension), allowing compiled code to be reused and exchanged without revealing the source code from which it was built. Libraries are imported using the `-l` option. They are generated using the `-a` option.

An HCT library file can contain any of the following elements:

- User-defined constants
- Microcode module configuration function calls
- FIFO configuration definitions
- Function chain definitions
- Microcode extension modules

When importing with the `-l` option, the HCT defines all the elements in the library before it reads from the HCT source file. Defined constants and microcode module configuration functions from a library act as though they were placed at the start of the source file. They can be referenced by any code in the source code file.

FIFO configurations and function chains that the HCT imports from a library file act as though they are defined before any other configurations or function chains are defined in the source code file. They cannot be overridden or extended.

Microcode extension modules extend the built-in set of microcode modules with new ones. Extension modules are available only from ON Semiconductor and cannot be generated with the HCT. Contact ON Semiconductor customer support for more details.

If you are importing an extension module from a library file, and that module has the same name as a module that is already defined, the one from the library file replaces the existing one.

If several library files are specified with the `-l` option, their contents are defined in the order in which the files are given on the command line.

To protect whatever intellectual property a library might contain, the HCT suppresses descriptions of imported libraries from appearing in the summary files it generates. It is the responsibility of the library provider to supply a description of the library contents to its users.

When generating a library with the `-a` option, the HCT draws the contents of the library from all of its inputs. This includes the source file and every library imported using the `-l` option. The generated library file contains all of the constants, configuration function calls, FIFO definitions, and function chains from its inputs. Microcode extension modules, however, are not transferred to the generated library. You must transmit them separately in their original library file if you want to share them with others.

5.7 CFX Header File Output

The CFX header file (*.cfi* extension) which is generated by the HCT contains preprocessor definitions and macros for interfacing with the generated microcode configuration. The items generated in the header file are described below. The generated header file relies heavily on the system firmware described in the *Firmware Reference Manual for Ezairo 7100*. The system firmware include file, *sk4_sys.cfi*, must be included before the HCT generated header file for the generated macros and constants to work correctly.

User-Defined Constants

All user-defined constants created with the built-in `Define` macro are mapped to equivalent pre-processor `#define` statements. The expression used to specify the constant's value is reproduced as a pre-processor expression wrapped in parentheses. This means that the expression will actually be re-evaluated by the preprocessor or assembler when these `#define` values are used in the CFX application source code. All of these tools (HCT, preprocessor and assembler) use the same C language style expression evaluation rules so the resulting value of a user-defined constant should be the same in all cases.

FIFO Configuration Constants

Preprocessor `#define` statements are generated for the start, length, base pointer, ioblock pointer and block size of each FIFO specified with the built-in `FIFOConfigure` macro. These constants all begin with the 'HCT_' prefix to distinguish them from user-defined constants. They are also used by the HCT-generated FIFO configuration and memory allocation macros as described below. Dual FIFOs result in three sets of HCT generated defines - one set for the A memory FIFO, one set for the B memory FIFO and one set for the combined AB memory FIFO.

FIFO Configuration Macro

A single preprocessor macro is generated for configuring all of the FIFOs specified using the built-in `FIFOConfigure` macro.This macro can be used in the CFX system initialization code to configure the FIFO controller with the FIFO configuration specified in the HCT input file. The macro is named `HCT_FIFO_Configure` and uses the system firmware macro `Sys_FIFO_Configure` to configure the FIFO controller registers.

FIFO Memory Allocation Macro

A single preprocessor macro is generated for allocating memory for all of the FIFOs specified using the built-in `FIFOConfigure` macro.This macro can be used in the CFX memory

allocation code to reserve the shared memory used by the FIFO configuration specified in the HCT input file. The macro is named `HCT_FIFO_Reserve_Mem` and uses the system firmware macro `Sys_Mem_Start_Absolute_Segment` to create new absolute data segments for each FIFO.

Initial Function Chain Controller Configuration

Preprocessor `#define` statements are generated for configuring and using the function chain controller. The constant `HCT_HEAR_FC_ENABLE_VAL` is generated by OR-ing together the `HEAR_FC_EBL_X` constants from the system firmware for every function chain which is bound to a command or FIFO event in the HCT input file. It is therefore highly recommended that you bind a function chain to every one of the commands and FIFO events that you intend to use in your application, even if you intend to change the bindings at run time with `Sys_Set_HEAR_FunctionChain_Vector`. The constant `HCT_HEAR_FC_PRIORITY_VAL` is also generated with the default value of 0, i.e. default function chain priorities.

In addition to these configuration values, `#define` statements for each user-defined function chain identifier are generated and mapped to the system firmware `HEAR_FC_X_POS` constants. This allows you to use your user-defined function chain identifiers when calling the system firmware macro `Sys_HEAR_Command`. These constants are also generated for FIFO-related function chains, although these function chains cannot be started using the `Sys_HEAR_Command` macro.

Shared Memory Addresses

The HCT generates `.extern` statements for all microcode state variables and certain special memory locations such as function chain entry points and the `HEAR_SATURATION_FLAG` allocated by the built-in `ResetSaturationFlag` and `SaveSaturationFlag` macros. Symbols for these memory locations are part of the object file output.

5.8 CFX Object File Output

The CFX object file (*.o* extension) which is generated by the HCT contains data segments which are allocated in the microcode memory areas of the P memory space and the shared memory area of the X memory space. These segments contain the microcode, configuration data and, in some cases, coefficients for the HCT generated microcode configuration. The object file also contains global symbols for all shared memory addresses which are exported by the microcode configuration. This object file must be linked into the CFX application for the microcode to be included in the final executable.

In the CFX object file output, microcode is allocated as ROM in memory type 0 (P memory space) using addresses 0xE000 to 0xEFFF. The microcode memory is allocated as ROM to keep it separate from the normal CFX program memory and to indicate that it does not contain CFX executable code. Any data or reserved areas in shared memory are allocated as RAM in memory type 1 (X memory space) using addresses 0x4000 to 0x67FF and 0x8000 to 0xFFFF, or as RAM in memory type 2 (Y memory space) using addresses 0xC000 to 0xFFFF. To link the microcode object file into a CFX application, you must therefore use the following linker flags:

```
--rom0=0xE000/0xEFFF
--mem1=0x4000/0x67FF
--mem1=0x8000/0xFFFF
--mem2=0xC000/0xFFFF
```

For more information about the linking process and typical memory allocation settings for applications, see the *CFX Toolchain Reference*.

Normally, there is no need to examine or change the contents of the HCT generated object file. If necessary, the locations and sizes of the data segments allocated in this object file can be examined by looking in the application's *.map* file or by using *cflib* to examine the contents of the object file itself. This may be done to determine, for example, how much memory is actually used by the microcode configuration. A more convenient alternative is to determine this information by examining the optional summary file which is described in Section 5.9, "Summary File Output".

5.9 Summary File Output

The HCT generated summary file (*.smy* extension) provides additional information about the generated microcode in a human-readable, text-based format. The information in this file is a combination of information in the header file output, the object file output and information generated by the HCT during analysis of the microcode. This file is normally located in the build outputs directory of an IDE project. This file contains the following information:

User-Defined Constants

Each user-defined constant generated using the `Define` macro is listed in the summary file with its user-defined identifier, the expression given for the constant's value and the resulting evaluated value of the expression in hexadecimal notation. This can be useful for debugging the value of constants and ensuring that expressions are being evaluated as intended by the HCT user.

FIFO Configurations

Each FIFO configuration defined using the `FIFOConfigure` macro is listed in the summary file with the final value of each FIFO configuration parameter. This can be useful for debugging the values of FIFO parameters and ensuring that expressions are being evaluated as intended by the HCT user.

Microcode Module Configurations

Each microcode module instance defined using the microcode functions is listed in the summary file with the final value of each configuration and address parameter. Configuration parameters are always listed in decimal notation and address parameters are always listed in hexadecimal notation. This can be useful for debugging the values of module parameters and ensuring that expressions are being evaluated as intended by the HCT user.

Function Chain Configurations

Each function chain is listed again with its user-defined identifier and the sequence of functions being executed in the function chain. Each function in the function chain is also repeated with its final parameter values as given in the microcode module configuration values section of the summary file.

Microcode Memory Usage

The amount of memory used by the microcode configuration is listed in the summary file. Specifically, the following items are given:

- The number of 32-bit words used in the microcode memory, both overall and for each function chain. This memory is always contiguous starting at the first address in the microcode memory (`D_HEAR_MICROCODE_MEM_BASE`).
- The starting address of the microcode configuration data which is reserved in CD1 memory.
- The number of 24-bit words used for microcode configuration data in CD1 memory.

Microcode Cycle Estimates

Cycle estimates for each function chain and each individual microcode function are generated and included in the summary file. The total for a function chain is the sum of each individual microcode function, plus any additional cycles for using built-in macros, such as the `Event` macro. A minimum execution time and a maximum execution time are specified, allowing for microcode modules which have variable execution times due to conditional operations. The majority of microcode modules do not have run-time variable execution times, so in most cases, the minimum and maximum cycle estimates are the same.

The cycle estimates do not take into account the normal delay between when a command or FIFO event is issued and when the HEAR begins processing the function chain. This fixed overhead is 5 HEAR clock cycles after the function chain is pending, if the HEAR is not currently processing a function chain. If another function chain is pending when the current function chain finishes, there is only a 2 clock cycle delay before the pending function chain begins execution. The actual number of cycles a function chain takes can also vary from the estimates in the summary file due to the way the HEAR fetches the microcode at run-time. Depending on the specific functions used, this fetching might introduce additional wait cycles that cannot be predicted during microcode generation, and hence are not taken into account in the summary file. Real-time measurements might be required to determine the exact number of cycles a function chain takes in a particular microcode configuration.

APPENDIX A
Optimized WOLA Windows

The table in this appendix summarizes all of the WOLA Filterbank configurations which have optimized analysis and synthesis windows.

La	Ls	N	R
32	16	16	2
32	16	16	4
32	16	16	8
32	32	16	2
32	32	16	4
32	32	16	8
64	16	16	2
64	16	16	4
64	32	16	2
64	32	16	4
64	32	16	8
64	64	16	2
64	64	16	4
64	64	16	8
128	16	16	2
128	32	16	2
128	32	16	4
128	64	16	2
128	64	16	4
128	64	16	8
128	128	16	2
128	128	16	4
128	128	16	8
256	32	16	2
256	64	16	2
256	64	16	4
256	128	16	2
256	128	16	4
256	128	16	8
256	256	16	2
256	256	16	4
256	256	16	8
512	64	16	2
512	128	16	2
512	128	16	4
512	256	16	2
512	256	16	4

La	Ls	N	R
512	512	16	2
512	512	16	4
512	512	16	8
1024	128	16	2
1024	256	16	2
1024	256	16	4
1024	512	16	2
1024	512	16	4
1024	1024	16	2
1024	1024	16	4
1024	1024	16	8
64	32	32	2
64	32	32	4
64	32	32	8
64	32	32	16
64	64	32	2
64	64	32	4
64	64	32	8
64	64	32	16
128	32	32	2
128	32	32	4
128	32	32	8
128	64	32	2
128	64	32	4
128	64	32	8
128	64	32	16
128	128	32	2
128	128	32	4
128	128	32	8
128	128	32	16
256	32	32	2
256	32	32	4
256	64	32	2
256	64	32	4
256	64	32	8
256	128	32	2
256	128	32	4
256	128	32	8
256	128	32	16
256	256	32	2
256	256	32	4

La	Ls	N	R
256	256	32	8
256	256	32	16
512	64	32	2
512	64	32	4
512	128	32	2
512	128	32	4
512	128	32	8
512	256	32	2
512	256	32	4
512	256	32	8
512	256	32	16
512	512	32	2
512	512	32	4
512	512	32	8
512	512	32	16
1024	128	32	2
1024	128	32	4
1024	256	32	2
1024	256	32	4
1024	256	32	8
1024	512	32	2
1024	512	32	4
1024	512	32	8
1024	512	32	16
1024	1024	32	2
1024	1024	32	4
1024	1024	32	8
1024	1024	32	16
128	64	64	2
128	64	64	4
128	64	64	8
128	64	64	16
128	64	64	32
128	128	64	2
128	128	64	4
128	128	64	8
128	128	64	16
128	128	64	32
256	64	64	2
256	64	64	4
256	64	64	8

La	Ls	N	R
256	64	64	16
256	128	64	2
256	128	64	4
256	128	64	8
256	128	64	16
256	128	64	32
256	256	64	2
256	256	64	4
256	256	64	8
256	256	64	16
256	256	64	32
512	64	64	2
512	64	64	4
512	64	64	8
512	128	64	2
512	128	64	4
512	128	64	8
512	128	64	16
512	256	64	2
512	256	64	4
512	256	64	8
512	256	64	16
512	256	64	32
512	512	64	2
512	512	64	4
512	512	64	8
512	512	64	16
512	512	64	32
1024	128	64	2
1024	128	64	4
1024	128	64	8
1024	256	64	2
1024	256	64	4
1024	256	64	8
1024	256	64	16
1024	512	64	2
1024	512	64	4
1024	512	64	8
1024	512	64	16
1024	512	64	32
1024	1024	64	2

La	Ls	N	R
1024	1024	64	4
1024	1024	64	8
1024	1024	64	16
1024	1024	64	32
256	128	128	4
256	128	128	8
256	128	128	16
256	128	128	32
256	128	128	64
256	256	128	4
256	256	128	8
256	256	128	16
256	256	128	32
256	256	128	64
512	128	128	4
512	128	128	8
512	128	128	16
512	128	128	32
512	256	128	4
512	256	128	8
512	256	128	16
512	256	128	32
512	256	128	64
512	512	128	4
512	512	128	8
512	512	128	16
512	512	128	32
512	512	128	64
1024	128	128	4
1024	128	128	8
1024	128	128	16
1024	256	128	4
1024	256	128	8
1024	256	128	16
1024	256	128	32
1024	512	128	4
1024	512	128	8
1024	512	128	16
1024	512	128	32
1024	512	128	64
1024	1024	128	4

La	Ls	N	R
1024	1024	128	8
1024	1024	128	16
1024	1024	128	32
1024	1024	128	64
512	256	256	8
512	256	256	16
512	256	256	32
512	256	256	64
512	256	256	128
512	512	256	8
512	512	256	16
512	512	256	32
512	512	256	64
512	512	256	128
1024	256	256	8
1024	256	256	16
1024	256	256	32
1024	256	256	64
1024	512	256	8
1024	512	256	16
1024	512	256	32
1024	512	256	64
1024	512	256	128
1024	1024	256	8
1024	1024	256	16
1024	1024	256	32
1024	1024	256	64
1024	1024	256	128
1024	512	512	16
1024	512	512	32
1024	512	512	64
1024	512	512	128
1024	512	512	256
1024	1024	512	16
1024	1024	512	32
1024	1024	512	64
1024	1024	512	128
1024	1024	512	256

APPENDIX B
Glossary

The following acronyms and terms are used in this manual:

BFP
: block floating point

CFX Digital Signal Processor
: the Ezairo 7100 processing core that is used to configure the system and coordinate the flow of signal data progressing through the system

CORDIC
: COordinate Rotational DIgital Computer

DFT
: discrete Fourier transform

FENG
: Filter Engine; one of the cores in Ezairo 7100

FFT
: fast Fourier transform

FIFO
: first-in, first-out

FIR
: finite impulse response

function chain
: a configurable sequence of microcode functions that are executed on the HEAR Configurable Accelerator in response to a command from the CFX or an event from the FIFO controller

HCT
: HEAR Configuration Tool

HEAR Configurable Accelerator
: the Ezairo 7100 core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions; designed to work with the CFX to reduce the signal processing load on the CFX

IIR
: infinite impulse response

IOC
: input/output controller

LSB
: least significant bit

LSW
: least significant word

microcode module
: a configurable block of microcode and data composed of one or more functions executable on the HEAR Configurable Accelerator

microcode function
: a signal processing operation executable on the HEAR Configurable Accelerator; part of a microcode module

MSB
: most significant bit

MSW
: most significant word

RAM
: random access memory

ROM
: read-only memory

sinc
: the mathematical function described by $\frac{\sin(x)}{x}$

WOLA
: weighted overlap add

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is a registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20798-009